

WINCS02 API Specification

Microchip Technology Inc. ©2025-2026

Revision a59011f4b, October 23, 2026: WINCS02 3.3.0 (a59011f4b)

Table of Contents

Introduction . . . 1
- Product Support . . . 1
- Definitions . . . 1
- Driver Initialization . . . 2
- Transport Initialization . . . 2
  - Synchronous Transport Initialization . . . 2
  - Asynchronous Transport Initialization . . . 3
- Event Processing . . . 4
- Asynchronous Event Codes . . . 5
- Command Functions and Responses . . . 6
- Command Responses & AECs . . . 7

Overview . . . 9
- Parameter Types . . . 9
  - WINC_TYPE_STATUS . . . 9
  - WINC_TYPE_INTEGER, WINC_TYPE_INTEGER_UNSIGNED . . . 9
  - WINC_TYPE_INTEGER_FRAC . . . 9
  - WINC_TYPE_STRING, WINC_TYPE_BYTE_ARRAY . . . 9
  - WINC_TYPE_IPV4ADDR . . . 9
  - WINC_TYPE_IPV6ADDR . . . 9
  - WINC_TYPE_BOOL . . . 9
  - WINC_TYPE_MACADDR . . . 10
  - WINC_TYPE_UTC_TIME . . . 10
- Configuration Functions . . . 11
  - Single Value Parameters: . . . 11
  - Multiple Value Parameters: . . . 12
  - Query Response Elements: . . . 14
  - Complex Value Elements: . . . 14
  - Default Configurations and Presets: . . . 15

Driver API Reference . . . 17
- Device . . . 17
  - WINC_DevInit . . . 17
  - WINC_DevDeinit . . . 18
  - WINC_DevReset . . . 19
  - WINC_DevSetDebugPrintf . . . 20
- Transport . . . 20
  - WINC_SDIODeviceInit . . . 20
- Event Functions . . . 22
  - WINC_DevHandleEvent . . . 22

WINC_DevUpdateEvent . . . 23
WINC_DevEventPending . . . 24
Command Requests . . . 25
WINC_CmdReqInit . . . 25
WINC_DevTransmitCmdReq . . . 26
WINC_CmdReqDiscard . . . 27
WINC_CmdReqSend . . . 28
Responses and AECs . . . 29
WINC_DevAECCallbackRegister . . . 29
WINC_DevAECCallbackDeregister . . . 30
WINC_DevInterceptCallbackRegister . . . 31
WINC_DevUnpackElements . . . 32
WINC_CmdReadParamElem . . . 33
Bus State and Errors . . . 34
WINC_DevBusStateGet . . . 34
WINC_DevBusStateSet . . . 35
WINC_DevGetLastError . . . 36
WINC_DevGetLastErrorEx . . . 37
WINC_DevErrorToString . . . 38
Tables . . . 39
WINC_TablesCommandIterate, WINC_TablesAECIterate, WINC_TablesStatusIterate . . . 39
WINC_TablesModuleStatusIterate . . . 40
WINC_TablesCommandLookupByID, WINC_TablesAECLookupByID, WINC_TablesStatusLookupByID . . . 41
WINC_TablesCommandLookupByString, WINC_TablesAECLookupByString . . . 42
WINC_TablesModuleIDListGet . . . 43
Driver Types . . . 44
WINC_DEV_INIT . . . 44
WINC_DEV_CMDREQ_EVENT_TYPE . . . 44
WINC_DEV_EVENT_RSP_ELEMS . . . 44
WINC_DEV_EVENT_STATUS_ARGS . . . 45
WINC_DEV_EVENT_COMPLETE_ARGS . . . 45
WINC_DEV_EVENT_SRC_CMD . . . 45
WINC_DEV_BUS_STATE_TYPE . . . 46
WINC_DEV_ERROR_TYPE . . . 46
WINC_DEV_EXTENDED_ERROR . . . 46
WINC_DEV_CMD_RSP_CB . . . 47
WINC_DEV_AEC_RSP_CB . . . 47
WINC_DEV_RX_INTERCEPT_CB . . . 47
WINC_DEV_EVENT_CHECK_FP . . . 47
WINC_SDIO_SEND_RECEIVE_FP . . . 48

WINC_SDIO_STATE_TYPE . . . 48
Status Response Codes . . . 49
SYSTEM (Module ID = 1) . . . 55
AEC Reference: . . . 55
WINC_AEC_ID_BOOT . . . 55
INTERNAL (Module ID = 2) . . . 56
Function Reference: . . . 56
WINC_CmdGMI . . . 56
WINC_CmdGMM . . . 57
WINC_CmdGMR . . . 58
CFG (Module ID = 3) . . . 59
Function Reference: . . . 59
WINC_CmdCFG . . . 59
WINC_CmdCFGCP . . . 62
Examples: . . . 66
DHCP (Module ID = 4) . . . 68
Introduction . . . 68
Address Pool . . . 68
Function Reference: . . . 69
WINC_CmdDHCPSC . . . 69
DNS (Module ID = 5) . . . 71
Introduction: . . . 71
Server Address Configuration . . . 71
Resource Queries . . . 71
Reverse Address Queries . . . 71
Connection Types . . . 72
DNSSEC Support . . . 73
Cache Control . . . 73
MDNS Support . . . 73
Function Reference: . . . 74
WINC_CmdDNSC . . . 74
WINC_CmdDNSRESOLV . . . 77
AEC Reference: . . . 78
WINC_AEC_ID_DNSRESOLV . . . 78
WINC_AEC_ID_DNSERR . . . 81
Examples: . . . 82
FS (Module ID = 7) . . . 88
Function Reference: . . . 88
WINC_CmdFSOP . . . 88
WINC_CmdFSLOAD . . . 91
WINC_CmdFSSTORE . . . 93

WINC_CmdFSTSFR . . . 95
WINC_CmdFSRECV . . . 97
AEC Reference: . . . 99
WINC_AEC_ID_FSUP . . . 99
Examples: . . . 100
MQTT (Module ID = 8) . . . 103
Introduction . . . 103
Key Features . . . 103
Connecting to a Broker . . . 104
Publishing Messages . . . 105
Publish Completion Events . . . 107
Subscribing and Receiving Messages . . . 107
Maximum Packet Size (MQTT v5.0) . . . 108
MQTT v5.0 Properties . . . 110
Typical Usage Flow . . . 111
Troubleshooting . . . 112
Function Reference: . . . 113
WINC_CmdMQTTC . . . 113
WINC_CmdMQTTCONN . . . 116
WINC_CmdMQTTSUB . . . 117
WINC_CmdMQTTSUBLST . . . 118
WINC_CmdMQTTSUBRD . . . 119
WINC_CmdMQTTUNSUB . . . 121
WINC_CmdMQTTPUB . . . 122
WINC_CmdMQTTPUBFRAG . . . 124
WINC_CmdMQTTLWT . . . 126
WINC_CmdMQTTDISCONN . . . 127
WINC_CmdMQTTPROPTX . . . 128
WINC_CmdMQTTPROPTXKV . . . 131
WINC_CmdMQTTPROPRX . . . 133
WINC_CmdMQTTPROPRXPRESET . . . 136
WINC_CmdMQTTPROPTXS . . . 138
AEC Reference: . . . 141
WINC_AEC_ID_MQTTCONN . . . 141
WINC_AEC_ID_MQTTCONNACK . . . 142
WINC_AEC_ID_MQTTPUBACK . . . 144
WINC_AEC_ID_MQTTPUBCOMP . . . 145
WINC_AEC_ID_MQTTPUBERR . . . 146
WINC_AEC_ID_MQTTSUB . . . 147
WINC_AEC_ID_MQTTUNSUB . . . 148
WINC_AEC_ID_MQTTSUBRX . . . 149

WINC_AEC_ID_MQTTPROPRX . . . 150
WINC_AEC_ID_MQTTPUBSENT . . . 151
WINC_AEC_ID_MQTTSUBRXERR . . . 152
Examples: . . . 153
NETIF (Module ID = 9) . . . 158
Introduction . . . 158
IPv4 . . . 158
IPv6 . . . 158
Layer 2 Access . . . 158
Function Reference: . . . 159
WINC_CmdNETIFC . . . 159
WINC_CmdNETIFTX . . . 162
AEC Reference: . . . 163
WINC_AEC_ID_NETIFRX . . . 163
OTA (Module ID = 10) . . . 164
Introduction . . . 164
OTA Sequence . . . 164
Timeout . . . 166
Error Handling . . . 166
Troubleshooting . . . 166
Internal HTTP/S OTA Example . . . 167
NVM OTA Example . . . 169
Function Reference: . . . 170
WINC_CmdOTAC . . . 170
WINC_CmdOTADL . . . 172
WINC_CmdOTAVFY . . . 173
WINC_CmdOTAACT . . . 174
WINC_CmdOTAINV . . . 175
AEC Reference: . . . 176
WINC_AEC_ID_OTA . . . 176
WINC_AEC_ID_OTAERR . . . 177
Examples: . . . 178
PING (Module ID = 11) . . . 181
Introduction . . . 181
Function Reference: . . . 182
WINC_CmdPING . . . 182
AEC Reference: . . . 183
WINC_AEC_ID_PING . . . 183
WINC_AEC_ID_PINGERR . . . 184
RST (Module ID = 12) . . . 185
Introduction . . . 185

Function Reference: 186
WINC_CmdRST 186
SNTP (Module ID = 13) 187
Introduction 187
Function Reference: 188
WINC_CmdSNTPC 188
AEC Reference: 190
WINC_AEC_ID_SNTPERR 190
SOCKET (Module ID = 14) 191
Introduction: 191
Socket Lifecycle 191
Reading Socket Data 194
Socket Errors 198
Function Reference: 198
WINC_CmdSOCKO 198
WINC_CmdSOCKBL 200
WINC_CmdSOCKBR 201
WINC_CmdSOCKBM 202
WINC_CmdSOCKTLS 203
WINC_CmdSOCKWR 204
WINC_CmdSOCKWRTO 205
WINC_CmdSOCKRD 207
WINC_CmdSOCKRDBUF 209
WINC_CmdSOCKCL 211
WINC_CmdSOCKLST 212
WINC_CmdSOCKC 214
AEC Reference: 217
WINC_AEC_ID_SOCKIND 217
WINC_AEC_ID_SOCKRXT 218
WINC_AEC_ID_SOCKRXU 219
WINC_AEC_ID_SOCKCL 220
WINC_AEC_ID_SOCKTLS 221
WINC_AEC_ID_SOCKERR 222
Examples: 223
TIME (Module ID = 16) 230
Introduction 230
Function Reference: 231
WINC_CmdTIME 231
WINC_CmdTIMEUTCSEC 233
WINC_CmdTIMERFC 234
AEC Reference: 235

WINC_AEC_ID_TIME . . . 235
TLS (Module ID = 17) . . . 236
Introduction . . . 236
Quick Start: TLS Client . . . 236
Troubleshooting . . . 237
TLS configuration . . . 238
TLS Cipher Suite codes . . . 241
TLS Error Codes . . . 242
Function Reference: . . . 248
WINC_CmdTLSC . . . 248
WINC_CmdTLSCDA . . . 252
WINC_CmdTLSCSC . . . 254
AEC Reference: . . . 256
WINC_AEC_ID_TLSERR . . . 256
Examples: . . . 257
WAP (Module ID = 18) . . . 258
Introduction . . . 258
Function Reference: . . . 259
WINC_CmdWAPC . . . 259
WINC_CmdWAP . . . 262
AEC Reference: . . . 263
WINC_AEC_ID_WAPAIP . . . 263
WINC_AEC_ID_WAPSC . . . 264
WINC_AEC_ID_WAPSD . . . 265
WINC_AEC_ID_WAPERR . . . 266
WSCN (Module ID = 19) . . . 267
Introduction . . . 267
Periodic Scanning . . . 267
Function Reference: . . . 268
WINC_CmdWSCNC . . . 268
WINC_CmdWSCN . . . 272
WINC_CmdWSCNPC . . . 273
WINC_CmdWSCNP . . . 277
AEC Reference: . . . 278
WINC_AEC_ID_WSCNIND . . . 278
WINC_AEC_ID_WSCNDONE . . . 279
WINC_AEC_ID_WSCNPDONE . . . 280
WINC_AEC_ID_WSCNPERR . . . 281
Examples: . . . 282
WSTA (Module ID = 20) . . . 288
Introduction . . . 288

Quick Start: Connecting to an Access Point . . . 288
Configuring the STA . . . 289
Controlling STA connection . . . 290
STA information . . . 290
WPA Enterprise . . . 291
Function Reference: . . . 299
WINC_CmdWSTAC . . . 299
WINC_CmdWSTA . . . 303
AEC Reference: . . . 305
WINC_AEC_ID_WSTAAIP . . . 305
WINC_AEC_ID_WSTALD . . . 306
WINC_AEC_ID_WSTAERR . . . 307
WINC_AEC_ID_WSTALU . . . 308
WINC_AEC_ID_WSTAROAM . . . 309
Examples: . . . 310
ASSOC (Module ID = 22) . . . 317
Introduction . . . 317
Function Reference: . . . 318
WINC_CmdASSOC . . . 318
AEC Reference: . . . 319
WINC_AEC_ID_ASSOC . . . 319
SI (Module ID = 23) . . . 320
Introduction . . . 320
Function Reference: . . . 321
WINC_CmdSI . . . 321
AEC Reference: . . . 322
WINC_AEC_ID_SIMSTAT . . . 322
WINC_AEC_ID_SIMERR . . . 323
WINC_AEC_ID_SIWPKTS . . . 324
WINC_AEC_ID_SIHEAP . . . 325
WINC_AEC_ID_SIHERR . . . 326
WINC_AEC_ID_SISTACK . . . 327
WPROV (Module ID = 25) . . . 328
Introduction . . . 328
Function Reference: . . . 329
WINC_CmdWPROVC . . . 329
WINC_CmdWPROV . . . 331
AEC Reference: . . . 332
WINC_AEC_ID_WPROVAT . . . 332
WINC_AEC_ID_WPROVDT . . . 333
DI (Module ID = 26) . . . 334

Function Reference: . . . 334
WINC_CmdDI . . . 334
Examples: . . . 336
EXTCRYPTO (Module ID = 27) . . . 337
Introduction . . . 337
Using External Crypto for Secure Signing . . . 337
Function Reference: . . . 338
WINC_CmdEXTCRYPTO . . . 338
AEC Reference: . . . 339
WINC_AEC_ID_EXTCRYPTO . . . 339
Examples: . . . 341
WIFI (Module ID = 28) . . . 342
Introduction . . . 342
Transmitting vendor-specific IEs . . . 342
Receiving vendor-specific IEs . . . 343
Function Reference: . . . 344
WINC_CmdWIFIC . . . 344
AEC Reference: . . . 348
WINC_AEC_ID_WIFIVSIERX . . . 348
Examples: . . . 349
NVM (Module ID = 29) . . . 352
Introduction . . . 352
Function Reference: . . . 353
WINC_CmdNVMC . . . 353
WINC_CmdNVMER . . . 355
WINC_CmdNVMWR . . . 356
WINC_CmdNVMCHK . . . 357
WINC_CmdNVMRD . . . 358
AEC Reference: . . . 359
WINC_AEC_ID_NVMER . . . 359
WINC_AEC_ID_NVMCHK . . . 360
WINC_AEC_ID_NVMERR . . . 361
Examples: . . . 362
DFU (Module ID = 30) . . . 363
Introduction . . . 363
Determine the NVM partition in which to program the firmware image . . . 363
Prepare the firmware image . . . 364
Boot the device in DFU mode . . . 365
Erase and program the NVM partition . . . 365
Reset the device . . . 365
Function Reference: . . . 366

WINC_CmdDFUADR . . . 366
WINC_CmdDFUSEQ . . . 367
Examples: . . . 368
PPS (Module ID = 31) . . . 369
Introduction . . . 369
PPS Operation Details . . . 369
EXtreme Deep Sleep (XDS) . . . 373
Summary . . . 373
Function Reference: . . . 374
WINC_CmdPPSC . . . 374
WINC_CmdPPS . . . 376
WINC_CmdPPSPAUSE . . . 377
AEC Reference: . . . 378
WINC_AEC_ID_PPS . . . 378
Examples: . . . 379
SYSLOG (Module ID = 32) . . . 380
Introduction . . . 380
Function Reference: . . . 381
WINC_CmdSYSLOGC . . . 381
AEC Reference: . . . 383
WINC_AEC_ID_SYSLOG . . . 383
ARB (Module ID = 33) . . . 384
Introduction . . . 384
Function Reference: . . . 385
WINC_CmdARB . . . 385
Examples: . . . 386
HTTP (Module ID = 34) . . . 387
Introduction . . . 387
Key Features . . . 387
How It Works . . . 388
Advanced Features . . . 389
Error Handling . . . 390
Function Reference: . . . 391
WINC_CmdHTTPC . . . 391
WINC_CmdHTTPGET . . . 394
WINC_CmdHTTPRD . . . 395
WINC_CmdHTTPCL . . . 396
AEC Reference: . . . 397
WINC_AEC_ID_HTTPRSPCODE . . . 397
WINC_AEC_ID_HTTPRSPFIELD . . . 398
WINC_AEC_ID_HTTPBODYRX . . . 399

WINC_AEC_ID_HTTPBODYEND . . . 400
WINC_AEC_ID_HTTPCL . . . 401
WINC_AEC_ID_HTTPERR . . . 402
Examples: . . . 403
WBCN (Module ID = 35) . . . 408
Introduction . . . 408
Configuring the beacon . . . 408
Transmitting beacons . . . 409
Detecting beacons . . . 409
Examples . . . 410
Function Reference: . . . 417
WINC_CmdWBCNC . . . 417
WINC_CmdWBCN . . . 420
AEC Reference: . . . 422
WINC_AEC_ID_WBCNDONE . . . 422
WINC_AEC_ID_WBCNERR . . . 423
Examples: . . . 424
Berkeley Sockets APIs . . . 428
Introduction . . . 428
Socket Initialisation . . . 428
Slab Memory Allocation . . . 428
Socket Event Callback . . . 430
Creating Sockets . . . 430
Server Listening Sockets . . . 431
Client Sockets . . . 433
Multicast Sockets . . . 433
Incoming Server Socket Connections . . . 435
Client Socket Connections . . . 436
Sending Data . . . 437
Receiving Data . . . 438
TCP Socket Termination . . . 439
Closing Sockets . . . 440
DNS Resolution. . . . 440
API Reference . . . 441
WINC_SOCKET_EVENT . . . 441
WINC_SOCKET_STATUS . . . 442
WINC_SOCKET_INIT_TYPE . . . 443
WINC_SOCKET_EVENT_CALLBACK . . . 444
WINC_SockDeinit . . . 445
WINC_SockInit . . . 446
accept . . . 447

bind . . . 448
connect . . . 449
inet_ntop . . . 450
inet_pton . . . 451
listen . . . 452
poll . . . 453
recv . . . 454
recvfrom . . . 455
send . . . 456
sendto . . . 457
setsockopt . . . 458
shutdown . . . 460
socket . . . 461
freeaddrinfo . . . 462
getaddrinfo . . . 463
getpeername . . . 464
getsockname . . . 465
htonl . . . 466
htons . . . 467
ntohl . . . 468
ntohs . . . 469
Examples . . . 470
Prerequisites . . . 470
DNS Lookup . . . 471
TCP Client . . . 474
TCP Server . . . 478
UDP Client . . . 482
UDP Server . . . 486
List of Examples . . . 489
Index . . . 492

Introduction

This reference manual provides information on the functions and features of the WINCS02 application programming interface (API). The API provides the functionality of a wireless LAN access device with integrated TCP/IP and TLS stack. Once the initial configuration has been performed, the device may access a Wi-Fi network and send/receive data.

Product Support

This reference manual refers to the interface API:

Name	Version	Revision
WINCS02	3.3.0	a59011f4b

Definitions

In this document, the following naming conventions are used:

- API or Interface: The software functions which are provided to utilize the functionality described in this manual and the responses they create.
- App or Application: The software not described by this manual which uses the API functionality.
- Device: The WINCS02 module.
- Command Request: A buffer holding one or more commands, built with **WINC_CmdReqInit** and **WINC_Cmd*** functions and sent with **WINC_DevTransmitCmdReq**.
- Response: Data returned by the device for a command, delivered to the command request callback.
- Status: The final result of a command, delivered as a **WINC_DEV_CMDREQ_EVENT_CMD_STATUS** event.
- AEC (Asynchronous Event Code): An unsolicited message from the device, delivered to the AEC callback.
- TLV (Type-Length-Value): The encoding of the parameter elements in commands, responses and AECs, see **WINC_DEV_PARAM_ELEM**.

Driver Initialization

The driver is initialized by calling **WINC_DevInit**. This function will initialize the driver state and prepare internal structures. The application must provide this function with a block of memory to act as a receive buffer for SPI operations, ideally this should be 2KBytes in size.

WINC_DevInit will return a **WINC_DEVICE_HANDLE** type device handle which can be used in future operations with the driver API. If initialization fails it will return a device handle value of **WINC_DEVICE_INVALID_HANDLE**.

```
WINC_DEVICE_HANDLE wincDevHandle;
WINC_DEV_INIT devInitData;
uint8_t devRcvBuffer[2048];

devInitData.pReceiveBuffer    = devRcvBuffer;
devInitData.receiveBufferSize = sizeof(devRcvBuffer);

wincDevHandle = WINC_DevInit(&devInitData);
```

Transport Initialization

The driver uses an SDIO over SPI protocol to communicate with the WINCS02 device, this transport layer must be initialized before it can be used by the driver.

Device initialization can take some time, therefore the transport initialization can be perform synchronously or asynchronously. In both cases the function **WINC_SDIODeviceInit** is called. This function accepts a pointer to a state variable to maintain progress state and a pointer to a **WINC_SDIO_SEND_RECEIVE_FP** type SPI function which can send/receive SPI data.

WINC_SDIODeviceInit returns a **WINC_SDIO_STATUS_TYPE** status value indicating the result of the current operation. Negative values are fatal errors, zero represents success while positive value are ongoing states.

Synchronous Transport Initialization

If the application wishes to block while initialization is performed it may call **WINC_SDIODeviceInit** without a state parameter. The function will perform all operations required until either initialization is complete or an error condition is encountered.

```
bool APP_WINC_SPISendReceive(void* pTransmitData, void* pReceiveData, size_t size);

WINC_SDIO_STATUS_TYPE wincSDIOStatus;

wincSDIOStatus = WINC_SDIODeviceInit(NULL, APP_WINC_SPISendReceive);
```

 ©2025-2026

Asynchronous Transport Initialization

To perform asynchronous initialization the application must provide a state parameter to **WINC_SDIODeviceInit**, this parameter will be updated by each call to **WINC_SDIODeviceInit**.

The **WINC_SDIO_STATUS_TYPE** return value of **WINC_SDIODeviceInit** indicates the status of the initialization, a positive value will be used to indicate the operation is still ongoing and the function should be called again to progress further. It is advisable to introduce a wait between calls as positive return values indicate the next stage in the process could not be run immediately.

```
bool APP_WINC_SPISendReceive(void* pTransmitData, void* pReceiveData, size_t size);

WINC_SDIO_STATUS_TYPE wincSDIOStatus;
WINC_SDIO_STATE_TYPE wincSDIOState;

wincSDIOState = WINC_SDIO_STATE_UNKNOWN;

do
{
    wincSDIOStatus = WINC_SDIODeviceInit(&wincSDIOState, APP_WINC_SPISendReceive);

    delay_ms(100);
}
while (wincSDIOStatus > 0);
```

Event Processing

The device signals that it has data for the driver, such as command responses, status or AECs, by asserting its interrupt line (active low). The application must service the driver using three functions:

- **WINC_DevHandleEvent** reads and acknowledges a new event from the device. It must only be called when no event is in progress, otherwise it fails with **WINC_DEV_ERROR_EVENT_PENDING**. The optional **WINC_DEV_EVENT_CHECK_FP** function should return true while the interrupt line is asserted; when supplied, the driver waits for the device to release the line after the event is acknowledged.
- **WINC_DevUpdateEvent** progresses the current event. Each call processes a single message, so it must be called repeatedly while **WINC_DevEventPending** returns true; no further interrupt is raised part way through an event. Calling it with no event in progress has no effect.
- **WINC_DevEventPending** returns true while an event is still being processed.

The command response and AEC callbacks are invoked from within **WINC_DevUpdateEvent**, in the context of its caller.

The interrupt is edge triggered. If the line is still asserted once the current event has completed then a further event is waiting and should be handled.

```
static volatile bool wincEventFlag = false;

/* WINCS02 interrupt line (active low, falling edge) ISR */
void APP_WINC_IntHandler(void)
{
    wincEventFlag = true;
}

/* Returns true while the WINCS02 interrupt line is asserted */
static bool APP_WINC_EventCheck(void)
{
    return APP_WINC_IntLineAsserted();
}

void APP_Tasks(void)
{
    /* Only accept a new event once the current one has been fully processed */
    if ((false == WINC_DevEventPending(wincDevHandle)) && (true == wincEventFlag))
    {
        wincEventFlag = false;

        WINC_DevHandleEvent(wincDevHandle, APP_WINC_EventCheck);
    }

    /* Progress the current event, this may invoke the application callbacks */
    if (false == WINC_DevUpdateEvent(wincDevHandle))
    {
        /* Handle error, see WINC_DevGetLastError */
    }
    else if ((false == WINC_DevEventPending(wincDevHandle)) && (true == APP_WINC_EventCheck()))
    {
        /* Interrupt still asserted with no event in progress, a new event is waiting */
        wincEventFlag = true;
    }
}
```

 ©2025-2026

Asynchronous Event Codes

In most cases any output from the API will be in response to the receipt of a function call from the application. There are cases, however, where an asynchronous event occurs that the application will need informing of (e.g. disconnection from the network). The API uses Asynchronous Event Codes (AEC) to convey such events; AECs use the same message conventions as regular responses using a different message type to distinguish them.

The application must be prepared to receive AECs at any time as it is possible for the API to send AECs resulting from a previous function or condition while the application is calling a new API function.

AECs are delivered to the application via the message callback registered by **WINC_DevAECCallbackRegister**.

Command Functions and Responses

One or more command functions are grouped into Command Requests messages which are dispatched via **WINC_DevTransmitCmdReq**. The application must provide a block of memory into which the Command Request message is constructed, this memory must persist until the **WINC_DEV_CMDREQ_EVENT_STATUS_COMPLETE** event has been received via the command response callback.

```
static void wincCmdRspCallback
(
    uintptr_t context,
    WINC_DEVICE_HANDLE devHandle,
    WINC_CMD_REQ_HANDLE cmdReqHandle,
    WINC_DEV_CMDREQ_EVENT_TYPE event,
    uintptr_t eventArg
)
{
    if (WINC_DEV_CMDREQ_EVENT_STATUS_COMPLETE == event)
    {
        free((void*)cmdReqHandle);
    }
}

WINC_CMD_REQ_HANDLE cmdReqHandle;
uint8_t *pCmdReqBuffer;

/* Allocate a command request buffer */
pCmdReqBuffer = malloc(1024);

/* Initialize a command request for three commands */
cmdReqHandle = WINC_CmdReqInit(pCmdReqBuffer, 1024, 3, wincCmdRspCallback, 0);

/* Add three commands to the request */
WINC_CmdEXAMPLE(cmdReqHandle, ARG1, ARG2);
WINC_CmdEXAMPLE(cmdReqHandle, ARG3, ARG4);
WINC_CmdEXAMPLE(cmdReqHandle, ARG5, ARG6);

/* Queue the request to be sent to the device */
WINC_DevTransmitCmdReq(wincDevHandle, cmdReqHandle);
```

Command responses are received by the callback registered when the Command Request was initialized by calling **WINC_CmdReqInit**. The callback receives the following events, in this order:

- **WINC_DEV_CMDREQ_EVENT_TX_COMPLETE** when the Command Request has been sent to the device.
- **WINC_DEV_CMDREQ_EVENT_RSP_RECEIVED** for each response to a command.
- **WINC_DEV_CMDREQ_EVENT_CMD_STATUS** when a command has completed. Once the status event has been received there will be no further direct responses for that command.
- **WINC_DEV_CMDREQ_EVENT_STATUS_COMPLETE** when all commands in the request have completed. The Command Request memory may then be released.

The structure passed in *eventArg* for each event is given in WINC_DEV_CMDREQ_EVENT_TYPE.

 ©2025-2026

Command Responses & AECs

Command response and AEC event information is carried in a **WINC_DEV_EVENT_RSP_ELEMS** structure to either the command response callback as a **WINC_DEV_CMDREQ_EVENT_RSP_RECEIVED** event or the AEC callback.

```
typedef struct
{
    uint16_t               rspCmdId;
    uint16_t               rspId;
    uint8_t                numElems;
    WINC_DEV_EVENT_SRC_CMD srcCmd;
    WINC_DEV_PARAM_ELEM    elems[12];
} WINC_DEV_EVENT_RSP_ELEMS;
```

This structure contains the IDs of the response message, number of elements contained and the elements themselves.

Optional response data is encoded in TLV type structures which describe the data received.

```
typedef struct
{
    WINC_TYPE   type;
    uint16_t    length;
    uint8_t     *pData;
} WINC_DEV_PARAM_ELEM;
```

The data should be extracted from these structures by calling **WINC_CmdReadParamElem**. This function will decode the received element into the user supplied storage location.

In addition for command responses the original command message that caused the response is provided. AECs do not provide this information.

```
typedef struct
{
    uint8_t idx;
    uint8_t numParams;
    uint8_t *pParams;
} WINC_DEV_EVENT_SRC_CMD;
```

This structure indicates position index within the original command request message, the number of parameters and the location of the packed TLV data.

The original command data can be unpacked for examination using the command **WINC_DevUnpackElements**. This creates an initialized list of **WINC_DEV_PARAM_ELEM** structures.

```
void appCmdRspCallback
(
    uintptr_t context,
    WINC_DEVICE_HANDLE devHandle,
    WINC_CMD_REQ_HANDLE cmdReqHandle,
    WINC_DEV_CMDREQ_EVENT_TYPE event,
    uintptr_t eventArg
)
{
    if (WINC_DEV_CMDREQ_EVENT_RSP_RECEIVED == event)
    {
        const WINC_DEV_EVENT_RSP_ELEMS *const pRspElems = (const WINC_DEV_EVENT_RSP_ELEMS*)eventArg;

        if (NULL != pRspElems)
        {
            if (WINC_CMD_ID_EXAMPLE == pRspElems->rspId)
            {
                WINC_DEV_PARAM_ELEM elems[10];
                uint32_t elemValue;

                if (true == WINC_DevUnpackElements(pRspElems->srcCmd.numParams, pRspElems->srcCmd.pParams, elems))
                {
                    WINC_CmdReadParamElem(&elems[2], WINC_TYPE_INTEGER, &elemValue, sizeof(elemValue));
                }
            }
        }
    }
}
```

 ©2025-2026

Overview

Parameter Types

The definition of each function specifies the data types used for any associated parameter elements.

WINC_TYPE_STATUS

Status elements are used to convey the ID of a status message. The element is a 16-bit word where the top byte represents the module category of the message while the lower byte represents the specific message ID.

See Status Response Codes

WINC_TYPE_INTEGER, WINC_TYPE_INTEGER_UNSIGNED

Integer elements represent signed and unsigned integers up to 32-bits.

WINC_TYPE_INTEGER_FRAC

Fractional integer element represents a fractional number in the form 'X.Y'. The element is a 32-bits word with a 16-bit integer value 'X' in the high word and a 16-bit fractional value 'Y' in the low word.

WINC_TYPE_STRING, WINC_TYPE_BYTE_ARRAY

The string and byte array elements carry essentially the same information, a series of bytes. The string type is not null-terminated. The purpose of the two types is simply to indicate how the information contained should be handled.

WINC_TYPE_IPV4ADDR

The IPv4 element contains an IPv4 address value. It can be either 4 bytes or 8 bytes. A 4 byte element represents a single IPv4 address, an 8 byte element represents an IPv4 address and netmask.

WINC_TYPE_IPV6ADDR

The IPv6 element contains an IPv6 address value. It can be either 16 bytes or 17 bytes. A 16 byte element represents a single IPv6 address. A 17 byte element represents an IPv6 address and scope.

WINC_TYPE_BOOL

The bool element contains a single value of either 1 or 0 to represent True or False.

WINC_TYPE_MACADDR

The MAC address element contains a 6 byte array representing a MAC address.

WINC_TYPE_UTC_TIME

The UTC time element contains a 32-bit value representing a UTC time count.

 ©2025-2026

Configuration Functions

Certain functions are used to configure or query parts of the system. These commands allow the caller to set new parameter values and get existing parameter values. The functions can query all parameter values, query a single parameter ID, or set a single parameter ID.

Each configuration parameter will consist of an ID and a value.

Functions can have preceding arguments which are specific to that particular function, but the ID and value arguments will be the last arguments.

```
uint32_t optId, WINC_TYPE typeOptVal, uintptr_t optVal, size_t lenOptVal
```

To query all parameter values, the ID field of the function must be set to a special IGNORE value. If the configuration function was **WINC_CmdEXAMPLE** then the ignore value name would be **WINC_CMDEXAMPLE_ID_IGNORE_VAL**, i.e. **WINC_CMD<NAME>_ID_IGNORE_VAL**. Using this value for `optId` will cause all parameter values to be queried and their values returned via the response callback. The value of subsequent arguments should be set to their IGNORE values as well.

```
WINC_CmdEXAMPLE(... , WINC_CMDEXAMPLE_ID_IGNORE_VAL, WINC_TYPE_INVALID, 0, 0);
```

To query a single parameter value, the ID field of the function must be set to the ID required. The name of the ID will be of the form **WINC_CFG_PARAM_ID_<NAME>_<PARAM_NAME>**.

For example, querying the parameter named **PARAM_ONE** would look like:

```
WINC_CFG_PARAM_ID_EXAMPLE_PARAM_ONE, WINC_TYPE_INVALID, 0, 0
```

To set a single parameter value, the VALUE arguments also need to be set. If the parameter value being set is an integer type the value is loaded directly into `optVal` (`lenOptVal` is ignored for these types), if the value is an array type value the address of the array is loaded into `optVal`.

For example, setting an integer value would look like:

```
WINC_CFG_PARAM_ID_EXAMPLE_PARAM_TWO, WINC_TYPE_INTEGER, newValue, 0
```

For example, setting an array value would look like:

```
WINC_CFG_PARAM_ID_EXAMPLE_PARAM_THREE, WINC_TYPE_BYTE_ARRAY, &newValue, sizeof(newValue)
```

Single Value Parameters:

These parameters may have only one value and are accessed using the parameter ID.

Multiple Value Parameters:

These parameters may have more than one value.

When reading all values, the parameter ID can be used, however when accessing a single value within the set of all values the function will use a fractional integer form of the ID consisting of <ID>.<INDEX>

A fractional ID is formed with **WINC_CmdFormFractInt(<ID>, <INDEX>)**, for example `WINC_CmdFormFractInt(5, 1)` for 5.1. The ID part must be non-zero. The examples in this document show fractional IDs in <ID>.<INDEX> notation. Fractional IDs in responses are read as **WINC_TYPE_INTEGER_FRAC** into a **WINC_DEV_FRACT_INT_TYPE**, see WINC_CmdReadParamElem.

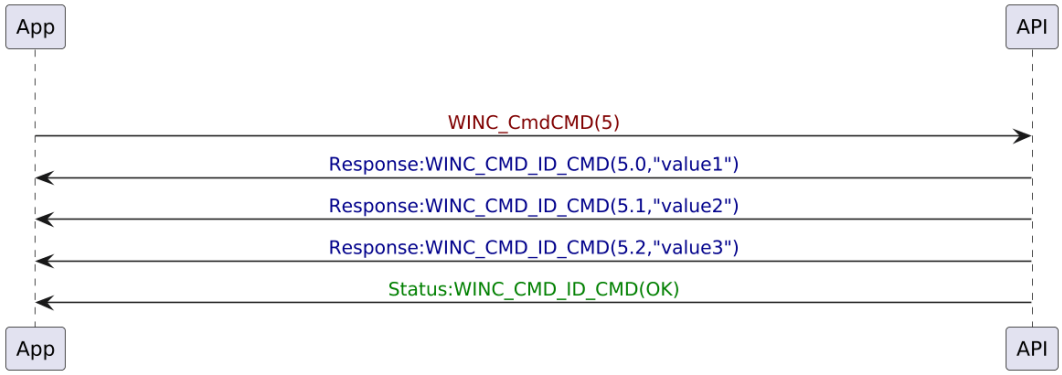


Figure 1. Reading all parameter values

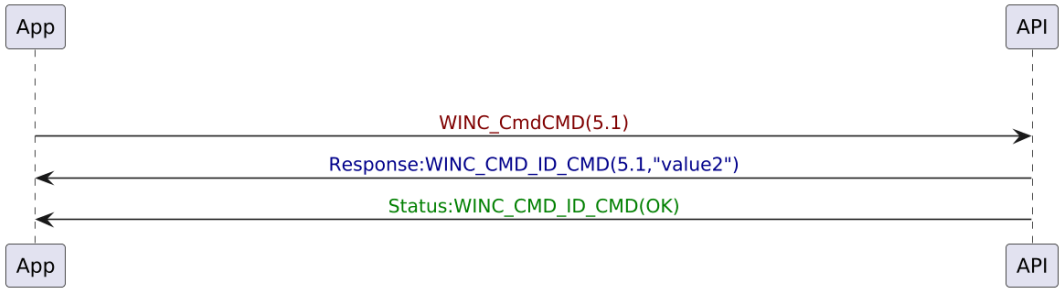


Figure 2. To read a single value

 ©2025-2026

When writing a value, the behavior will depend on the implementation of the command, however often writing a value to the parameter <ID> will cause an additional value to be appended to the set of parameter values, writing to the fractional <ID>.<INDEX> will cause that parameter index to be updated.

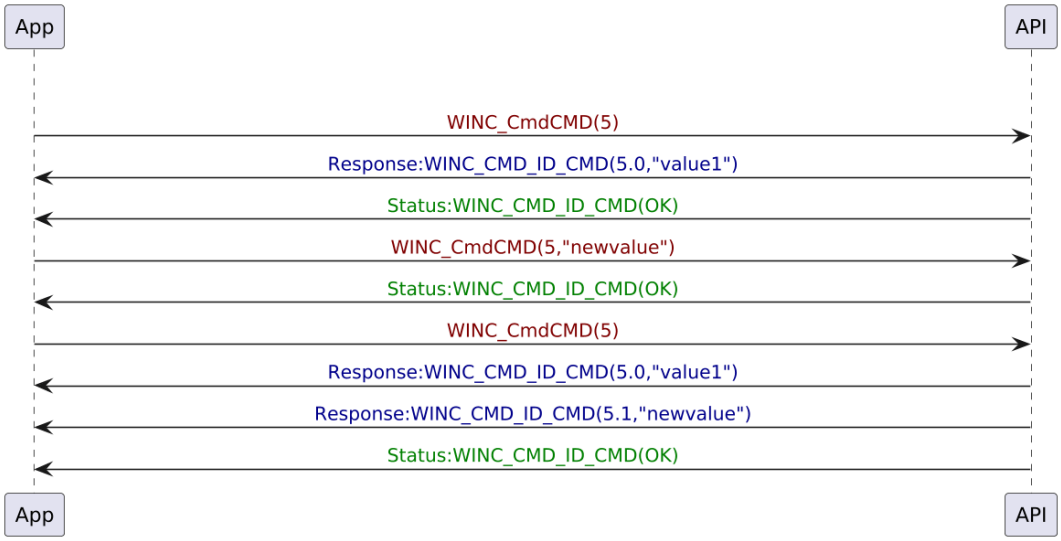


Figure 3. To set an additional value

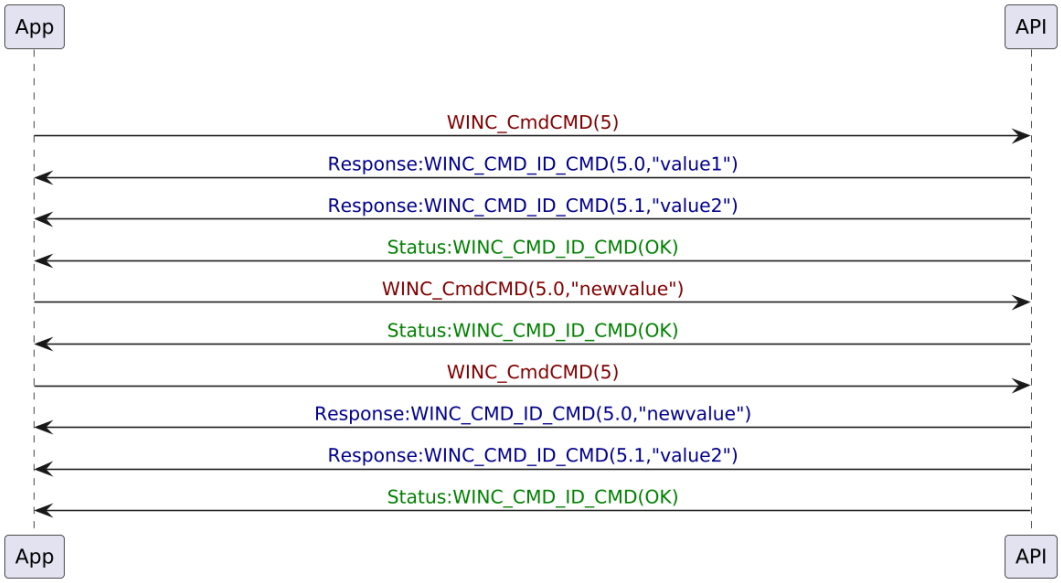


Figure 4. To replace an existing value

In addition to the fractional integer form a command response may include a range indication primarily to indicate how many valid responses will follow. This optional response will only be present if the parameter implements it and the request was to read all values, it will not be present if a request for a single value was made using a fractional integer ID.

The format of the range indication uses the non-fractional ID of the parameter and a single integer value:

This example indicates that WINC_CmdEXAMPLE parameter ID 5 will return 2 values, 5.0 and 5.1:

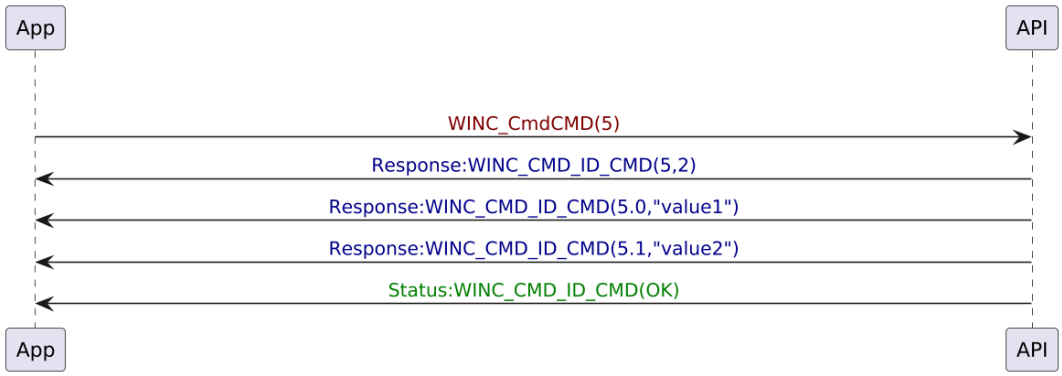


Figure 5. Range of IDs

Query Response Elements:

Standard configuration parameter queries return a simple <ID>,<VALUE> pair. For single value parameters the <ID> will be an integer, for multiple value parameters the <ID> will be a fractional integer in the form <ID>.0, <ID>.1 etc for each value.

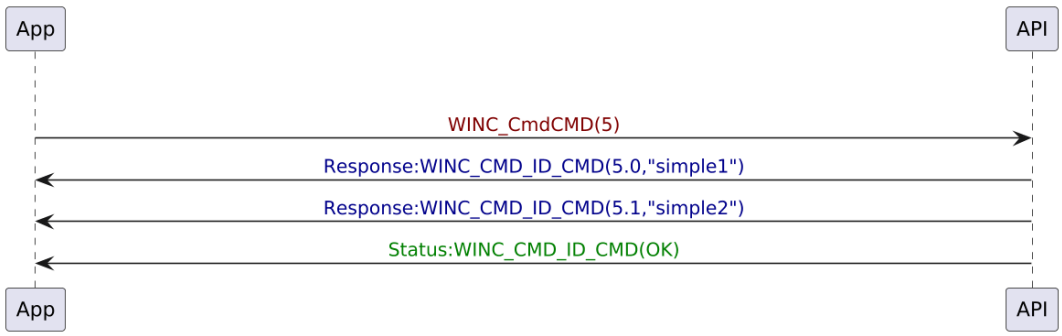


Figure 6. Simple ID/Value

Complex Value Elements:

When queried some configuration parameters will return a more complex value in place of the <VALUE> element of the response. A complex value is more than two element values.

 ©2025-2026

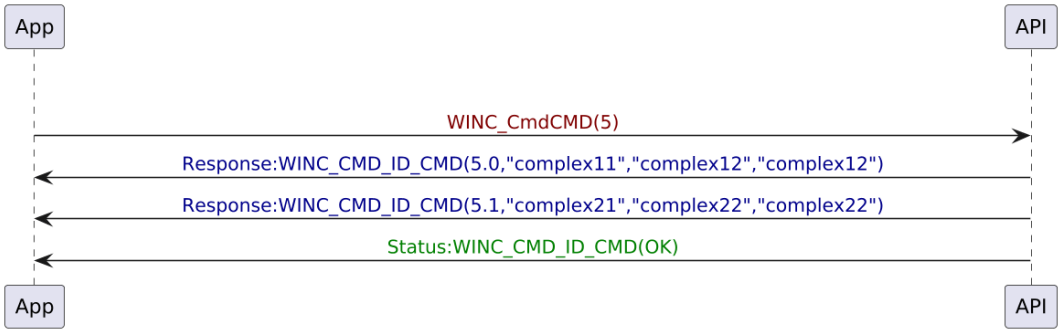


Figure 7. Complex ID/Value

Default Configurations and Presets:

Configuration commands are initially loaded with default values, also known as **preset 0**. Many configuration commands also support reloading preset values.

The reserved <ID> value 0 is used to query and load configuration command presets. Querying this <ID> will return the current base preset used to load the configuration commands values. Setting this <ID>'s value will load a preset set of values.

NOTE Currently only **preset 0** is defined to be the default system values. Other presets may be defined in future.

NOTE The queried preset indicates the preset last used to load the values in the configuration command, subsequent changes may have occurred to individual values. Therefore it does not indicate that any particular value is currently set to that preset value, only that it initially was.

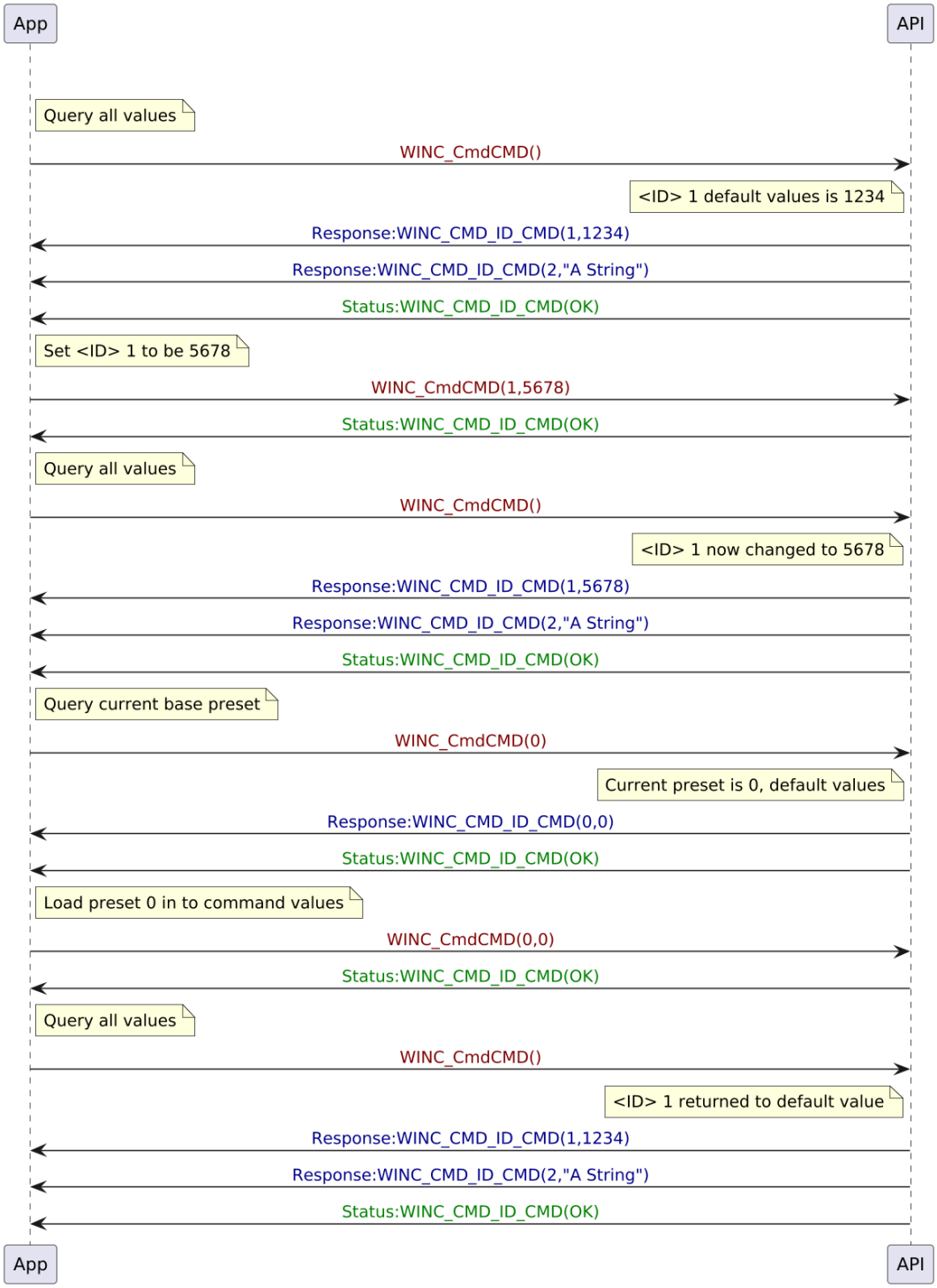


Figure 8. Using preset 0

 ©2025-2026

Driver API Reference

This section describes the driver functions used to initialize the device, send command requests and receive responses and AECs. The functions are declared in **winc_dev.h**, **winc_cmd_req.h**, **winc_cmds.h**, **winc_sdio_drv.h** and **winc_tables.h**.

Device

WINC_DevInit

Description

Initializes the driver state and returns a device handle for use with the other driver functions.

```
WINC_DEVICE_HANDLE WINC_DevInit(const WINC_DEV_INIT *pInitData)
```

Name	Type	Description
pInitData	const WINC_DEV_INIT *	Pointer to the WINC_DEV_INIT initialization data.

Returns

A device handle, or **WINC_DEVICE_INVALID_HANDLE** if *pInitData* is NULL or the receive buffer is too small.

Remarks

The receive buffer must persist for the lifetime of the driver. If **WINC_DEV_CACHE_LINE_SIZE** is defined the buffer is aligned to, and truncated to whole, cache lines.

If the driver is already initialized the existing handle is returned and *pInitData* is ignored.

On success the bus state is **WINC_DEV_BUS_STATE_IDLE**; see WINC_DevBusStateSet.

WINC_DevDeinit

Description

De-initializes the driver.

```
void WINC_DevDeinit(WINC_DEVICE_HANDLE devHandle)
```

Name	Type	Description
devHandle	WINC_DEVICE_HANDLE	Device handle obtained from WINC_DevInit.

Returns

None.

Remarks

Any queued command requests are discarded; their callbacks receive **WINC_DEV_CMDREQ_EVENT_STATUS_COMPLETE** with an *eventArg* of 0.

 ©2025-2026

WINC_DevReset

Description

Resets the driver state following a device communications failure.

```
bool WINC_DevReset(WINC_DEVICE_HANDLE devHandle)
```

Name	Type	Description
devHandle	WINC_DEVICE_HANDLE	Device handle obtained from WINC_DevInit.

Returns

true or false indicating success or failure.

Remarks

All pending command requests are completed with errors via their callbacks, and any event in progress is discarded. Registered AEC and intercept callbacks and the receive buffer are retained.

The bus is left in the **WINC_DEV_BUS_STATE_ERROR** state, so command requests submitted after the reset are completed rather than queued. To recover, set the bus state to **WINC_DEV_BUS_STATE_IDLE**, reset the device, repeat the transport initialization (WINC_SDIODeviceInit) and then set the bus state to **WINC_DEV_BUS_STATE_ACTIVE**.

WINC_DevSetDebugPrintf

Description

Sets the function used by the driver to output debug messages.

```
void WINC_DevSetDebugPrintf(WINC_DEBUG_PRINTF_FP pfPrintf)
```

Name	Type	Description
pfPrintf	WINC_DEBUG_PRINTF_FP	Pointer to a printf style function: `void (*)(const char *format, ⋯)`. NULL disables debug output.

Returns

None.

Remarks

The messages output depend on the **WINC_DEBUG_LEVEL** configuration option.

Transport

WINC_SDIODeviceInit

Description

Initializes the SDIO over SPI transport to the device.

```
WINC_SDIO_STATUS_TYPE WINC_SDIODeviceInit(WINC_SDIO_STATE_TYPE *pState, WINC_SDIO_SEND_RECEIVE_FP pfSendReceive)
```

Name	Type	Description
pState	WINC_SDIO_STATE_TYPE *	Pointer to the initialization state, or NULL. The state must be set to **WINC_SDIO_STATE_UNKNOWN** before the first call.
pfSendReceive	WINC_SDIO_SEND_RECEIVE_FP	Pointer to the SPI send/receive function, see WINC_SDIO_SEND_RECEIVE_FP.

 ©2025-2026

Returns

Value	Description
WINC_SDIO_STATUS_OK	Initialization complete.
WINC_SDIO_STATUS_RESET_WAITING	Waiting for the device reset to complete, call again.
WINC_SDIO_STATUS_OP_WAITING	Waiting for the CMD5 operation to complete, call again.
WINC_SDIO_STATUS_ERROR	An error occurred.
WINC_SDIO_STATUS_CFG_FAILED	Configuration of the device failed.
WINC_SDIO_STATUS_OP_FAILED	The CMD5 operation failed.
WINC_SDIO_STATUS_RESET_FAILED	The reset operation failed.
WINC_SDIO_STATUS_ABORT_FAILED	The abort operation failed.

Remarks

If *pState* is NULL the function blocks until initialization completes or fails. Otherwise each call performs the next step and returns a positive waiting status while initialization is in progress; see Asynchronous Transport Initialization. Negative values are errors.

Once initialization completes, set the bus state to **WINC_DEV_BUS_STATE_ACTIVE** with WINC_DevBusStateSet to allow command requests to be sent.

Event Functions

WINC_DevHandleEvent

Description

Reads and acknowledges a new event signalled by the device interrupt.

```
bool WINC_DevHandleEvent(WINC_DEVICE_HANDLE devHandle, WINC_DEV_EVENT_CHECK_FP pfEventIntCheck)
```

Name	Type	Description
devHandle	WINC_DEVICE_HANDLE	Device handle obtained from WINC_DevInit.
pfEventIntCheck	WINC_DEV_EVENT_CHECK_FP	Pointer to a function returning true while the device interrupt line is asserted, or NULL.

Returns

true or false indicating success or failure.

Remarks

Must only be called when WINC_DevEventPending returns false, otherwise it fails with **WINC_DEV_ERROR_EVENT_PENDING**.

If *pfEventIntCheck* is provided the driver waits, after acknowledging the event, for the device to release its interrupt line. If the line is not released within **WINC_DEV_INT_RELEASE_RETRY_CNT** checks the call fails with **WINC_DEV_ERROR_INT_STUCK** and the bus enters the error state.

See Event Processing.

 ©2025-2026

WINC_DevUpdateEvent

Description

Progresses the current event.

```
bool WINC_DevUpdateEvent(WINC_DEVICE_HANDLE devHandle)
```

Name	Type	Description
devHandle	WINC_DEVICE_HANDLE	Device handle obtained from WINC_DevInit.

Returns

true or false indicating success or failure. A failure caused by a bus error leaves the bus in the **WINC_DEV_BUS_STATE_ERROR** state; see WINC_DevReset.

Remarks

Each call processes a single message, so the function must be called repeatedly while WINC_DevEventPending returns true. Calling it with no event in progress has no effect.

Command response, AEC and intercept callbacks are invoked from within this function, in the context of the caller. The driver lock is not held while a callback runs, so callbacks may call other driver functions.

WINC_DevEventPending

Description

Indicates whether an event is part way through being processed.

```
bool WINC_DevEventPending(WINC_DEVICE_HANDLE devHandle)
```

Name	Type	Description
devHandle	WINC_DEVICE_HANDLE	Device handle obtained from WINC_DevInit.

Returns

true if WINC_DevUpdateEvent must be called again to complete the current event, otherwise false.

Remarks

No further interrupt is raised by the device while an event is in progress.

 ©2025-2026

Command Requests

WINC_CmdReqInit

Description

Initializes a command request in an application supplied buffer.

```
WINC_CMD_REQ_HANDLE WINC_CmdReqInit(uint8_t* pBuffer, size_t lenBuffer, int numCommands, WINC_DEV_CMD_RSP_CB
pfCmdRspCallback, uintptr_t cmdRspCallbackCtx)
```

Name	Type	Description
pBuffer	uint8_t *	Pointer to the buffer to contain the command request.
lenBuffer	size_t	Length of the buffer.
numCommands	int	Maximum number of commands to be added to the request.
pfCmdRspCallback	WINC_DEV_CMD_RSP_CB	Pointer to the callback to receive events for this request, see WINC_DEV_CMD_RSP_CB.
cmdRspCallbackCtx	uintptr_t	Context value passed to the callback.

Returns

A command request handle, or **WINC_CMD_REQ_INVALID_HANDLE** if the buffer is NULL or too small.

Remarks

Commands are added by calling the **WINC_Cmd** functions with the handle, then the request is sent with WINC_DevTransmitCmdReq.

The handle is the address of *pBuffer*. The buffer must persist until the callback receives **WINC_DEV_CMDREQ_EVENT_STATUS_COMPLETE**, after which it may be freed or reused.

WINC_DevTransmitCmdReq

Description

Queues a command request to be sent to the device.

```
bool WINC_DevTransmitCmdReq(WINC_DEVICE_HANDLE devHandle, WINC_CMD_REQ_HANDLE cmdReqHandle)
```

Name	Type	Description
devHandle	WINC_DEVICE_HANDLE	Device handle obtained from WINC_DevInit.
cmdReqHandle	WINC_CMD_REQ_HANDLE	Command request handle obtained from WINC_CmdReqInit.

Returns

true or false indicating success or failure.

Remarks

Requests are sent in order. A request is only sent while the bus state is **WINC_DEV_BUS_STATE_ACTIVE**; requests queued in the **WINC_DEV_BUS_STATE_IDLE** state are sent when the bus becomes active.

Ownership of the command request buffer depends on the result:

- true: the request is queued. The buffer is released through the callback with **WINC_DEV_CMDREQ_EVENT_STATUS_COMPLETE**; the application must not free it before then.
- false, and WINC_DevBusStateGet returns **WINC_DEV_BUS_STATE_ERROR**: the callback has already been called with **WINC_DEV_CMDREQ_EVENT_STATUS_COMPLETE**. The application must not free the buffer again.
- false, and the bus is not in the error state: the request was rejected and never queued, and the callback has not been called. The application must free the buffer.

```
if (false == WINC_DevTransmitCmdReq(wincDevHandle, cmdReqHandle))
{
    if (WINC_DEV_BUS_STATE_ERROR != WINC_DevBusStateGet(wincDevHandle))
    {
        /* Rejected before being queued, the buffer must be freed here */
        free((void*)cmdReqHandle);
    }
}
```

 ©2025-2026

WINC_CmdReqDiscard

Description

Discards a command request.

```
bool WINC_CmdReqDiscard(WINC_DEVICE_HANDLE devHandle, WINC_CMD_REQ_HANDLE cmdReqHandle)
```

Name	Type	Description
devHandle	WINC_DEVICE_HANDLE	Device handle passed to the callback, may be **WINC_DEVICE_INVALID_HANDLE** if the request has not been sent.
cmdReqHandle	WINC_CMD_REQ_HANDLE	Command request handle obtained from WINC_CmdReqInit.

Returns

true or false indicating success or failure.

Remarks

Used to abandon a request which has not been sent. The callback receives **WINC_DEV_CMDREQ_EVENT_STATUS_COMPLETE** with an *eventArg* of 0, and the buffer may then be released.

WINC_CmdReqSend

Description

Adds an already formatted command to a command request.

```
bool WINC_CmdReqSend(WINC_CMD_REQ_HANDLE handle, WINC_COMMAND_REQUEST *pCmdReq)
```

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request handle obtained from WINC_CmdReqInit.
pCmdReq	WINC_COMMAND_REQUEST *	Pointer to a formatted command message.

Returns

true or false indicating success or failure.

Remarks

This is the equivalent of a **WINC_Cmd** function for a command message built by the application, for example one forwarded from another interface. It fails if there is insufficient space in the request buffer.

 ©2025-2026

Responses and AECs

WINC_DevAECCallbackRegister

Description

Registers a callback to receive AECs.

```
bool WINC_DevAECCallbackRegister(WINC_DEVICE_HANDLE devHandle, WINC_DEV_AEC_RSP_CB pfAecRspCallback, uintptr_t
aecRspCallbackCtx)
```

Name	Type	Description
devHandle	WINC_DEVICE_HANDLE	Device handle obtained from WINC_DevInit.
pfAecRspCallback	WINC_DEV_AEC_RSP_CB	Pointer to the callback, see WINC_DEV_AEC_RSP_CB.
aecRspCallbackCtx	uintptr_t	Context value passed to the callback.

Returns

true or false indicating success or failure. Fails if **WINC_DEV_NUM_AEC_CB_ENTRIES** callbacks are already registered.

Remarks

Every registered callback receives every AEC. The callback should be registered before the transport is initialized so that the boot AEC is received.

WINC_DevAECCallbackDeregister

Description

De-registers an AEC callback.

```
bool WINC_DevAECCallbackDeregister(WINC_DEVICE_HANDLE devHandle, WINC_DEV_AEC_RSP_CB pfAecRspCallback)
```

Name	Type	Description
devHandle	WINC_DEVICE_HANDLE	Device handle obtained from WINC_DevInit.
pfAecRspCallback	WINC_DEV_AEC_RSP_CB	Pointer to the callback previously registered.

Returns

true or false indicating success or failure.

Remarks

None.

 ©2025-2026

WINC_DevInterceptCallbackRegister

Description

Registers a callback to receive each raw message from the device before it is processed.

```
bool WINC_DevInterceptCallbackRegister(WINC_DEVICE_HANDLE devHandle, WINC_DEV_RX_INTERCEPT_CB pfInterceptCallback,
uintptr_t interceptCallbackCtx)
```

Name	Type	Description
devHandle	WINC_DEVICE_HANDLE	Device handle obtained from WINC_DevInit.
pfInterceptCallback	WINC_DEV_RX_INTERCEPT_CB	Pointer to the callback, see WINC_DEV_RX_INTERCEPT_CB, or NULL to remove it.
interceptCallbackCtx	uintptr_t	Context value passed to the callback.

Returns

true or false indicating success or failure.

Remarks

Only one intercept callback can be registered. The callback observes messages, it cannot prevent their normal processing.

WINC_DevUnpackElements

Description

Unpacks packed TLV parameters into an array of element structures.

```
bool WINC_DevUnpackElements(uint8_t numTlvs, const uint8_t *pTLVBytes, WINC_DEV_PARAM_ELEM *pElems)
```

Name	Type	Description
numTlvs	uint8_t	Number of TLVs to unpack.
pTLVBytes	const uint8_t *	Pointer to the packed TLVs.
pElems	WINC_DEV_PARAM_ELEM *	Pointer to an array of at least *numTlvs* elements to receive the result.

Returns

true or false indicating success or failure.

Remarks

Used to examine the parameters of the command which caused a response or status, from the *srcCmd.numParams* and *srcCmd.pParams* fields of the event structure. The elements point into the original data, which is only valid during the callback.

 ©2025-2026

WINC_CmdReadParamElem

Description

Extracts the value of a response element or parameter.

```
size_t WINC_CmdReadParamElem(const WINC_DEV_PARAM_ELEM *const pElem, WINC_TYPE typeVal, void *pVal, size_t lenVal)
```

Name	Type	Description
pElem	const WINC_DEV_PARAM_ELEM *const	Pointer to the element.
typeVal	WINC_TYPE	Type of the value to extract.
pVal	void *	Pointer to the buffer to receive the value.
lenVal	size_t	Length of the buffer.

Returns

The length of the value written to *pVal*, or 0 for error.

Remarks

pVal is cleared before the value is written.

- Integer types: any integer element can be read into any integer type of size *lenVal*; signed values are sign extended. Reading as **WINC_TYPE_INTEGER_FRAC** fills a **WINC_DEV_FRACT_INT_TYPE**, with the fractional part set to -1 if the element was a whole integer.
- **WINC_TYPE_STRING** and **WINC_TYPE_BYTE_ARRAY**: either type can be read as the other. *lenVal* must be at least the element length; strings are not null-terminated.
- **WINC_TYPE_IPV4ADDR** and **WINC_TYPE_IPV6ADDR**: *lenVal* may be the short (4 or 16 bytes) or long (8 or 17 bytes) form; any part not present in the element is left zero.
- Other types: the type must match and *lenVal* must equal the element length.

Bus State and Errors

WINC_DevBusStateGet

Description

Returns the current bus state.

```
WINC_DEV_BUS_STATE_TYPE WINC_DevBusStateGet(WINC_DEVICE_HANDLE devHandle)
```

Name	Type	Description
devHandle	WINC_DEVICE_HANDLE	Device handle obtained from WINC_DevInit.

Returns

The current bus state, see WINC_DEV_BUS_STATE_TYPE.

Remarks

None.

 ©2025-2026

WINC_DevBusStateSet

Description

Sets the bus state.

```
bool WINC_DevBusStateSet(WINC_DEVICE_HANDLE devHandle, WINC_DEV_BUS_STATE_TYPE busState)
```

Name	Type	Description
devHandle	WINC_DEVICE_HANDLE	Device handle obtained from WINC_DevInit.
busState	WINC_DEV_BUS_STATE_TYPE	**WINC_DEV_BUS_STATE_IDLE** or **WINC_DEV_BUS_STATE_ACTIVE**.

Returns

true or false indicating success or failure. Other states are rejected.

Remarks

Setting **WINC_DEV_BUS_STATE_ACTIVE** starts sending any queued command requests. Setting **WINC_DEV_BUS_STATE_IDLE** stops new requests being sent, for example while the device is being reset or is in a low power state, and clears the **WINC_DEV_BUS_STATE_ERROR** state.

WINC_DevGetLastError

Description

Returns the code of the last driver error.

```
WINC_DEV_ERROR_TYPE WINC_DevGetLastError(WINC_DEVICE_HANDLE devHandle)
```

Name	Type	Description
devHandle	WINC_DEVICE_HANDLE	Device handle obtained from WINC_DevInit.

Returns

The last error code, or **WINC_DEV_ERROR_NONE** if no error has occurred since the last call. See WINC_DEV_ERROR_TYPE.

Remarks

The error is cleared when read. Use after a driver function returns false.

 ©2025-2026

WINC_DevGetLastErrorEx

Description

Returns extended information about the last driver error.

```
bool WINC_DevGetLastErrorEx(WINC_DEVICE_HANDLE devHandle, WINC_DEV_EXTENDED_ERROR *pError)
```

Name	Type	Description
devHandle	WINC_DEVICE_HANDLE	Device handle obtained from WINC_DevInit.
pError	WINC_DEV_EXTENDED_ERROR *	Pointer to a WINC_DEV_EXTENDED_ERROR structure to receive the error information.

Returns

true if error information was available, false if no error has occurred since the last call.

Remarks

Only available when the **WINC_DEV_EXTENDED_ERROR_INFO** configuration option is defined. The error is cleared when read.

WINC_DevErrorToString

Description

Converts an error code to a descriptive string.

```
const char* WINC_DevErrorToString(WINC_DEV_ERROR_TYPE error)
```

Name	Type	Description
error	WINC_DEV_ERROR_TYPE	Error code.

Returns

Pointer to a static string describing the error.

Remarks

None.

 ©2025-2026

Tables

The table functions translate command, AEC and status IDs to and from their names, for example for logging.

WINC_TablesCommandIterate, WINC_TablesAECIterate, WINC_TablesStatusIterate

Description

Iterate through the command, AEC or status tables.

```
uintptr_t WINC_TablesCommandIterate(uintptr_t iterState, uint16_t *pId, const char **ppName)
uintptr_t WINC_TablesAECIterate(uintptr_t iterState, uint16_t *pId, const char **ppName)
uintptr_t WINC_TablesStatusIterate(uintptr_t iterState, uint16_t *pId, const char **ppName)
```

Name	Type	Description
iterState	uintptr_t	0 to start at the first entry, otherwise the value returned by the previous call.
pId	uint16_t *	Pointer to receive the ID of the entry, or NULL.
ppName	const char **	Pointer to receive the name of the entry, or NULL.

Returns

The iteration state to pass to the next call.

Remarks

The tables contain **WINC_CMD_ID_TABLE_SZ**, **WINC_AEC_ID_TABLE_SZ** and **WINC_STATUS_ID_TABLE_SZ** entries respectively. The entry after the last has a NULL name.

WINC_TablesModuleStatusIterate

Description

Iterates through the status table, returning only the statuses of one module.

```
uintptr_t WINC_TablesModuleStatusIterate(uintptr_t iterState, uint16_t modId, uint16_t *pId, const char **ppName)
```

Name	Type	Description
iterState	uintptr_t	0 to start at the first entry, otherwise the value returned by the previous call.
modId	uint16_t	Module ID.
pId	uint16_t *	Pointer to receive the status ID, or NULL.
ppName	const char **	Pointer to receive the status name, or NULL.

Returns

The iteration state to pass to the next call.

Remarks

The module ID is the upper byte of the status ID; see Status Response Codes.

 ©2025-2026

WINC_TablesCommandLookupByID, WINC_TablesAECLookupByID, WINC_TablesStatusLookupByID

Description

Look up the name of a command, AEC or status ID.

```
const char* WINC_TablesCommandLookupByID(uint16_t id)
const char* WINC_TablesAECLookupByID(uint16_t id)
const char* WINC_TablesStatusLookupByID(uint16_t id)
```

Name	Type	Description
id	uint16_t	Command, AEC or status ID.

Returns

Pointer to the name, or NULL if the ID is not found.

Remarks

None.

WINC_TablesCommandLookupByString, WINC_TablesAECLookupByString

Description

Look up the ID of a command or AEC name.

```
uint16_t WINC_TablesCommandLookupByString(const char *pName)
uint16_t WINC_TablesAECLookupByString(const char *pName)
```

Name	Type	Description
pName	const char *	Pointer to the name.

Returns

The ID, or 0 if the name is not found.

Remarks

The whole name must match; *pName* may be followed by any non-alphanumeric character.

 ©2025-2026

WINC_TablesModuleIDListGet

Description

Returns the ID of a module from the module list.

```
int WINC_TablesModuleIDListGet(uint8_t idx)
```

Name	Type	Description
idx	uint8_t	Index into the module list, 0 to **WINC_NUM_MODULES** - 1.

Returns

The module ID, or -1 if *idx* is out of range.

Remarks

None.

Driver Types

WINC_DEV_INIT

Initialization data for WINC_DevInit.

```
typedef struct
{
    uint8_t *pReceiveBuffer;
    size_t  receiveBufferSize;
} WINC_DEV_INIT;
```

Member	Description
pReceiveBuffer	Buffer used to receive messages from the device, ideally 2KB.
receiveBufferSize	Size of the receive buffer in bytes.

WINC_DEV_CMDREQ_EVENT_TYPE

Events delivered to the command response callback, WINC_DEV_CMD_RSP_CB, and the structure pointed to by its *eventArg* parameter.

Event	eventArg	Description
WINC_DEV_CMDREQ_EVENT_TX_COMPLETE	0	The request has been sent to the device.
WINC_DEV_CMDREQ_EVENT_RSP_RECEIVED	WINC_DEV_EVENT_RSP_ELEMS	A response has been received for a command in the request.
WINC_DEV_CMDREQ_EVENT_CMD_STATUS	WINC_DEV_EVENT_STATUS_ARGS	A command in the request has completed; no further responses will be received for it.
WINC_DEV_CMDREQ_EVENT_STATUS_COMPLETE	WINC_DEV_EVENT_COMPLETE_ARGS, or 0 if discarded	All commands in the request have completed. The request buffer may now be released.

WINC_DEV_EVENT_RSP_ELEMS

A command response or AEC, see Command Responses & AECs.

Member	Description
rspCmdId	ID of the command which caused the response.
rspId	ID of the response or AEC.
numElems	Number of valid entries in *elems*.
srcCmd	Source command, see WINC_DEV_EVENT_SRC_CMD. Not provided for AECs.
elems	Response elements, read with WINC_CmdReadParamElem.

 ©2025-2026

WINC_DEV_EVENT_STATUS_ARGS

```
typedef struct
{
    uint16_t                rspCmdId;
    uint16_t                status;
    uint16_t                seqNum;
    WINC_DEV_EVENT_SRC_CMD  srcCmd;
} WINC_DEV_EVENT_STATUS_ARGS;
```

Member	Description
rspCmdId	ID of the command.
status	Status of the command, see Status Response Codes.
seqNum	Sequence number of the command message.
srcCmd	Source command, see WINC_DEV_EVENT_SRC_CMD.

WINC_DEV_EVENT_COMPLETE_ARGS

```
typedef struct
{
    uint8_t                 numCmds;
    uint8_t                 numErrors;
} WINC_DEV_EVENT_COMPLETE_ARGS;
```

Member	Description
numCmds	Number of commands in the request.
numErrors	Number of commands which did not complete successfully. Equal to *numCmds* if the request failed as a whole.

WINC_DEV_EVENT_SRC_CMD

The command which caused a response or status.

Member	Description
idx	Index of the command within the request.
numParams	Number of parameters of the command.
pParams	Packed TLV parameters, unpack with WINC_DevUnpackElements.

WINC_DEV_BUS_STATE_TYPE

Value	Description
WINC_DEV_BUS_STATE_UNKNOWN	The driver is not initialized.
WINC_DEV_BUS_STATE_ERROR	A communications error has occurred. Requests are completed with errors and not sent; see WINC_DevReset.
WINC_DEV_BUS_STATE_IDLE	Requests are queued but not sent.
WINC_DEV_BUS_STATE_ACTIVE	Requests are sent to the device.

WINC_DEV_ERROR_TYPE

Error codes returned by WINC_DevGetLastError.

Value	Description
WINC_DEV_ERROR_NONE	No error.
WINC_DEV_ERROR_INVALID_HANDLE	Invalid handle.
WINC_DEV_ERROR_NOT_INITIALIZED	The driver is not initialized.
WINC_DEV_ERROR_INVALID_PARAM	Invalid parameter.
WINC_DEV_ERROR_BUS_ALREADY_ERROR	The bus is in the error state.
WINC_DEV_ERROR_EVENT_PENDING	An event is already in progress.
WINC_DEV_ERROR_NULL_EVENT_HEADER	Internal event error.
WINC_DEV_ERROR_ZERO_LENGTH_EVENT	A zero length event was received.
WINC_DEV_ERROR_UNKNOWN_EVENT_TYPE	An unknown event type was received.
WINC_DEV_ERROR_MUTEX_FAILED	The lock could not be acquired.
WINC_DEV_ERROR_SPI_CMD52_FAILED	An SPI CMD52 transfer failed.
WINC_DEV_ERROR_SPI_CMD53_READ_FAILED	An SPI CMD53 read failed.
WINC_DEV_ERROR_SPI_CMD53_WRITE_FAILED	An SPI CMD53 write failed.
WINC_DEV_ERROR_BUFFER_OVERFLOW	A message did not fit in the receive buffer.
WINC_DEV_ERROR_INT_STUCK	The device did not release its interrupt line.

WINC_DEV_EXTENDED_ERROR

Only available when **WINC_DEV_EXTENDED_ERROR_INFO** is defined.

Member	Description
code	Error code, see WINC_DEV_ERROR_TYPE.
spiStatus	SPI status for SPI errors, otherwise 0.
file	Source file in which the error was detected.
line	Source line at which the error was detected.

 ©2025-2026

WINC_DEV_CMD_RSP_CB

Command response callback, registered with WINC_CmdReqInit.

```
void (*WINC_DEV_CMD_RSP_CB)(uintptr_t context, WINC_DEVICE_HANDLE devHandle, WINC_CMD_REQ_HANDLE cmdReqHandle,
WINC_DEV_CMDREQ_EVENT_TYPE event, uintptr_t eventArg)
```

event and *eventArg* are described in WINC_DEV_CMDREQ_EVENT_TYPE. The structure pointed to by *eventArg* is only valid during the callback.

WINC_DEV_AEC_RSP_CB

AEC callback, registered with WINC_DevAECCallbackRegister.

```
void (*WINC_DEV_AEC_RSP_CB)(uintptr_t context, WINC_DEVICE_HANDLE devHandle, const WINC_DEV_EVENT_RSP_ELEMS *const
pElems)
```

pElems is only valid during the callback.

WINC_DEV_RX_INTERCEPT_CB

Receive intercept callback, registered with WINC_DevInterceptCallbackRegister.

```
void (*WINC_DEV_RX_INTERCEPT_CB)(uintptr_t context, WINC_DEVICE_HANDLE devHandle, WINC_COMMAND_MSG_TYPE msgType,
uint8_t *pMsg, size_t msgLength)
```

msgType is the message type (status, response or AEC), *pMsg* and *msgLength* the raw message. *pMsg* is only valid during the callback.

WINC_DEV_EVENT_CHECK_FP

Interrupt line check function, passed to WINC_DevHandleEvent.

```
bool (*WINC_DEV_EVENT_CHECK_FP)(void)
```

Returns true while the device interrupt line is asserted.

WINC_SDIO_SEND_RECEIVE_FP

SPI transfer function, passed to WINC_SDIODeviceInit.

```
bool (*WINC_SDIO_SEND_RECEIVE_FP)(void* pTransmitData, void* pReceiveData, size_t size)
```

Performs a full duplex transfer of *size* bytes, asserting the device chip select for the duration of the call, and returns true on success. Either buffer may be NULL: if *pTransmitData* is NULL 0xFF bytes must be transmitted (see the **WINC_CONF_SPI_MOSI_IDLE_LEVEL** configuration option); if *pReceiveData* is NULL the received data is discarded.

WINC_SDIO_STATE_TYPE

Transport initialization state used by WINC_SDIODeviceInit. The application sets it to **WINC_SDIO_STATE_UNKNOWN** to start initialization and must not otherwise modify it.

 ©2025-2026

Status Response Codes

CORE Status Codes (Module ID = 0)

Value	Name	Description
0x0000	WINC_STATUS_OK	OK
0x0001	WINC_STATUS_ERROR	General Error
0x0002	WINC_STATUS_INVALID_CMD	Invalid Command
0x0003	WINC_STATUS_UNKNOWN_CMD	Unknown Command
0x0004	WINC_STATUS_INVALID_PARAMETER	Invalid Parameter
0x0005	WINC_STATUS_INCORRECT_NUM_PARAMS	Incorrect Number of Parameters
0x0006	WINC_STATUS_STORE_UPDATE_BLOCKED	Configuration Update Blocked
0x0007	WINC_STATUS_STORE_ACCESS_FAILED	Configuration Access Failed
0x0008	WINC_STATUS_TIMEOUT	Command Timed Out
0x0009	WINC_STATUS_HOST_INTERFACE_FAILED	Host Interface Failed
0x000a	WINC_STATUS_ACCESS_DENIED	Access Denied
0x000b	WINC_STATUS_CONFIG_CONFLICT	Configuration Conflict

SYSTEM Status Codes (Module ID = 1)

Value	Name	Description
0x0100	WINC_STATUS_WIFI_API_REQUEST_FAILED	Wi-Fi Request Failed
0x0101	WINC_STATUS_STA_NOT_CONNECTED	STA Not Connected
0x0102	WINC_STATUS_NETWORK_ERROR	Network Error
0x0103	WINC_STATUS_FILE_SYSTEM_ERROR	File System Error

CFG Status Codes (Module ID = 3)

Value	Name	Description
0x0300	WINC_STATUS_CFG_NOT_PRESENT	Configuration not present

DNS Status Codes (Module ID = 5)

Value	Name	Description
0x0500	WINC_STATUS_DNS_TYPE_NOT_SUPPORTED	DNS Type Not Supported
0x0501	WINC_STATUS_DNS_TIMEOUT	DNS Query Timeout
0x0502	WINC_STATUS_DNS_ERROR	DNS Error
0x0503	WINC_STATUS_DNS_RECORD_NOT_FOUND	DNS Record Not Found
0x0504	WINC_STATUS_DNS_NON_EXISTENT_DOMAIN	DNS Non-Existent Domain
0x0505	WINC_STATUS_DNS_AUTH_DATA_ERROR	DNS Data Authentication Error
0x0506	WINC_STATUS_DNS_TRUNCATED_RESPONSE	DNS Response Truncated

FS Status Codes (Module ID = 7)

Value	Name	Description
0x0700	WINC_STATUS_TSFR_PROTOCOL_NOT_SUPPORTED	Unsupported File Transfer Protocol
0x0701	WINC_STATUS_FILE_EXISTS	File Exists
0x0702	WINC_STATUS_FILE_NOT_FOUND	File Not Found
0x0703	WINC_STATUS_INVALID_FILE_TYPE	Invalid File Type
0x0704	WINC_STATUS_FILE_CREATE_FAILED	File create failed
0x0705	WINC_STATUS_FILE_WRITE_FAILED	File write failed

MQTT Status Codes (Module ID = 8)

Value	Name	Description
0x0800	WINC_STATUS_MQTT_ERROR	MQTT Error

NETIF Status Codes (Module ID = 9)

Value	Name	Description
0x0900	WINC_STATUS_NETWORK_INTF_DOWN	Network interface down

 ©2025-2026

OTA Status Codes (Module ID = 10)

Value	Name	Description
0x0a00	WINC_STATUS_ERASE_DONE	Erase Done
0x0a01	WINC_STATUS_WRITE_DONE	Write Done
0x0a02	WINC_STATUS_VERIFY_DONE	Verify Done
0x0a03	WINC_STATUS_ACTIVATE_DONE	Activate Done
0x0a04	WINC_STATUS_INVALIDATE_DONE	Invalidate Done
0x0a05	WINC_STATUS_OTA_ERROR	OTA Error
0x0a06	WINC_STATUS_OTA_NO_STA_CONN	No STA Connection
0x0a07	WINC_STATUS_OTA_PROTOCOL_ERROR	Protocol Error
0x0a08	WINC_STATUS_OTA_TLS_ERROR	TLS Error
0x0a09	WINC_STATUS_OTA_IMG_TOO_LARGE	Image Too Large
0x0a0a	WINC_STATUS_OTA_TIMEOUT	Timeout
0x0a0b	WINC_STATUS_OTA_VERIFY_FAILED	Image Verify Failed

PING Status Codes (Module ID = 11)

Value	Name	Description
0x0b00	WINC_STATUS_PING_FAILED	Ping Failed

SNTP Status Codes (Module ID = 13)

Value	Name	Description
0x0d00	WINC_STATUS_SNTP_SERVER_TMO	NTP Server Timeout
0x0d01	WINC_STATUS_SNTP_PROTOCOL_ERROR	NTP Protocol Error

SOCKET Status Codes (Module ID = 14)

Value	Name	Description
0x0e00	WINC_STATUS_SOCKET_ID_NOT_FOUND	Socket ID Not Found
0x0e01	WINC_STATUS_LENGTH_MISMATCH	Length Mismatch
0x0e02	WINC_STATUS_NO_FREE_SOCKETS	No Free Sockets
0x0e03	WINC_STATUS_SOCKET_INVALID_PROTOCOL	Invalid Socket Protocol
0x0e04	WINC_STATUS_SOCKET_CLOSE_FAILED	Socket Close Failed
0x0e05	WINC_STATUS_SOCKET_BIND_FAILED	Socket Bind Failed
0x0e06	WINC_STATUS_SOCKET_TLS_FAILED	Socket TLS Failed
0x0e07	WINC_STATUS_SOCKET_CONNECT_FAILED	Socket Connect Failed
0x0e08	WINC_STATUS_SOCKET_SEND_FAILED	Socket Send Failed
0x0e09	WINC_STATUS_SOCKET_SET_OPT_FAILED	Socket Set Option Failed
0x0e0a	WINC_STATUS_SOCKET_REMOTE_NOT_SET	Socket Destination Not Set
0x0e0b	WINC_STATUS_MULTICAST_ERROR	Multicast Error
0x0e0c	WINC_STATUS_SOCKET_NOT_READY	Socket Not Ready
0x0e0d	WINC_STATUS_SOCKET_SEQUENCE_ERROR	Socket Sequence Error

TIME Status Codes (Module ID = 16)

Value	Name	Description
0x1000	WINC_STATUS_TIME_ERROR	Time Error

TLS Status Codes (Module ID = 17)

Value	Name	Description
0x1100	WINC_STATUS_TLS_CA_CERT_MISSING	CA Cert Missing
0x1101	WINC_STATUS_TLS_CA_CERT_VALIDATION	CA Cert Validation
0x1102	WINC_STATUS_TLS_CA_CERT_DATE_VALIDATION	CA Cert Date Validation
0x1103	WINC_STATUS_TLS_KEY_PAIR_INCOMPLETE	Key Pair Incomplete
0x1104	WINC_STATUS_TLS_PEER_DOMAIN_MISSING	Peer Domain Missing

WAP Status Codes (Module ID = 18)

Value	Name	Description
0x1200	WINC_STATUS_WAP_STOP_REFUSED	Soft AP Stop Not Permitted
0x1201	WINC_STATUS_WAP_STOP_FAILED	Soft AP Stop Failed
0x1202	WINC_STATUS_WAP_START_REFUSED	Soft AP Start Not Permitted
0x1203	WINC_STATUS_WAP_START_FAILED	Soft AP Start Failed
0x1204	WINC_STATUS_UNSUPPORTED_SEC_TYPE	Unsupported Security Type

 ©2025-2026

WSCN Status Codes (Module ID = 19)

Value	Name	Description
0x1300	WINC_STATUS_WSCN_SCAN_IN_PROGRESS	Scan In Progress
0x1301	WINC_STATUS_WSCN_PERIODIC_SCAN_IS_RUNNING	Periodic Scan is Running
0x1302	WINC_STATUS_WSCN_INTERVAL_TOO_SHORT	Scan Interval Too Short
0x1303	WINC_STATUS_WSCN_AP_MODE_ACTIVE	Soft AP Active
0x1304	WINC_STATUS_WSCN_NO_CHANNELS	No Channels To Scan
0x1305	WINC_STATUS_WSCN_STA_TRANSITIONING	STA transition In Progress

WSTA Status Codes (Module ID = 20)

Value	Name	Description
0x1400	WINC_STATUS_STA_DISCONN_REFUSED	STA Disconnect Not Permitted
0x1401	WINC_STATUS_STA_DISCONN_FAILED	STA Disconnect Failed
0x1402	WINC_STATUS_STA_CONN_REFUSED	STA Connection Not Permitted
0x1403	WINC_STATUS_STA_CONN_FAILED	STA Connection Failed

ASSOC Status Codes (Module ID = 22)

Value	Name	Description
0x1600	WINC_STATUS_ASSOC_NOT_FOUND	Association Not Found

NVM Status Codes (Module ID = 29)

Value	Name	Description
0x1d00	WINC_STATUS_NVM_LOCKED	NVM Locked Until Reset

DFU Status Codes (Module ID = 30)

Value	Name	Description
0x1e00	WINC_STATUS_DFU_ADDRESS_WARNING	No Bootable Image In Other Partition

PPS Status Codes (Module ID = 31)

Value	Name	Description
0x1f00	WINC_STATUS_PPS_WIFI_PS_NOT_ENABLED	Wi-Fi PS Not Enabled
0x1f01	WINC_STATUS_PPS_TIMEOUT	PPS Timeout
0x1f02	WINC_STATUS_PPS_PAUSE_EXPIRED	PPS Pause Expired

ARB Status Codes (Module ID = 33)

Value	Name	Description
0x2100	WINC_STATUS_ARB_NO_INCREASE	The Value Provided Would Not Increase The ARB
0x2101	WINC_STATUS_ARB_REJECTED	The Value Provided Would Invalidate The Current Image

HTTP Status Codes (Module ID = 34)

Value	Name	Description
0x2200	WINC_STATUS_HTTP_DNS_FAILED	HTTP DNS failed
0x2201	WINC_STATUS_HTTP_TLS_FAILED	HTTP TLS failed
0x2202	WINC_STATUS_HTTP_CONNECT_FAILED	HTTP connect failed
0x2203	WINC_STATUS_HTTP_MEMORY	HTTP buffer allocation
0x2204	WINC_STATUS_HTTP_RESPONSE_HEADERS	HTTP response format
0x2205	WINC_STATUS_HTTP_RECV_FAILED	HTTP receive failed
0x2206	WINC_STATUS_HTTP_SEND_FAILED	HTTP send failed
0x2207	WINC_STATUS_HTTP_TIMEOUT	HTTP timeout

WBCN Status Codes (Module ID = 35)

Value	Name	Description
0x2300	WINC_STATUS_WBCN_IN_PROGRESS	Beaconing In Progress
0x2301	WINC_STATUS_WBCN_AP_MODE_ACTIVE	Soft AP Active
0x2302	WINC_STATUS_WBCN_TX_ERROR	Beacon Tx Error
0x2303	WINC_STATUS_WBCN_STA_TRANSITIONING	STA Connection In Progress
0x2304	WINC_STATUS_WBCN_NO_CHANNELS	No Beacon Channels Selected
0x2305	WINC_STATUS_WBCN_INTERVAL_TOO_SHORT	Beacon Interval Too Short

 ©2025-2026

SYSTEM (Module ID = 1)

AEC Reference:

WINC_AEC_ID_BOOT

Description

System has booted.

AEC Syntax

AEC	Description
No Elements	

INTERNAL (Module ID = 2)

Function Reference:

WINC_CmdGMI

Description

This function requests manufacturer identification.

```
bool WINC_CmdGMI(WINC_CMD_REQ_HANDLE handle)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<MAN_ID>	Information Response

Response Element Description

Element Name	Type	Description
<MAN_ID>	STRING	Manufacturers ID/name

 ©2025-2026

WINC_CmdGMM

Description

This function requests model identification.

```
bool WINC_CmdGMM(WINC_CMD_REQ_HANDLE handle)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<MODEL_ID>	Information Response

Response Element Description

Element Name	Type	Description
<MODEL_ID>	STRING	Model information

WINC_CmdGMR

Description

This function requests revision identification.

```
bool WINC_CmdGMR(WINC_CMD_REQ_HANDLE handle)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<VERSION_INFO>	Information Response

Response Element Description

Element Name	Type	Description
<VERSION_INFO>	STRING	Version information

 ©2025-2026

CFG (Module ID = 3)

Function Reference:

WINC_CmdCFG

Description

This function is used to read or set the system configuration.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdCFG
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDCFG_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

 ©2025-2026

Configuration Parameters (WINC_CFG_PARAM_ID_CFG_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset **Value** / **Label** / **Description** 0 / DEFAULT / Default.
1	DEVICE_NAME	STRING	The device name Maximum length of string is 32
10	VERSION	INTEGER_FRAC (Read Only)	Software version number
11	PATCH	INTEGER (Read Only)	Software patch number
12	SECURITY	INTEGER (Read Only)	Security value
13	BUILD_HASH	STRING (Read Only)	Build ID Maximum length of string is 5
14	BUILD_TIME	UTC_TIME (Read Only)	Build time
20	CMD_PORT	INTEGER (Read Only)	Command port **Value** / **Label** / **Description** 1 / UART1 / UART1.
21	CMD_BAUD	INTEGER_UNSIGNED	Command baud rate. Default is 230400
22	CMD_TIMEOUT	INTEGER_UNSIGNED	Command baud rate change timeout (ms), 0 disables fallback. Default is 0
50	ARCHIVE_FILTER	STRING INTEGER_UNSIGNED	Archive filter Maximum length of string is 32 This is a multiple value parameter with an ID range 50.0 to 50.7 The filter is a list of command IDs either in string form ("+WSTAC") or integer form (0x1400).
51	ARCHIVE_SLOTS	UTC_TIME	Archive slots This is a multiple value parameter with an ID range 51.1 to 51.2
52	AUTOEXEC_STATUS	STATUS (Read Only)	Auto-execute status
100	DEBUG_PORT	INTEGER_UNSIGNED	Debug port **Value** / **Label** / **Description** 0 / OFF / Off. 1 / UART1 / UART1. 2 / UART2 / UART2.
101	DEBUG_BAUD	INTEGER_UNSIGNED	Debug baud rate

WINC_CmdCFGCP

Description

This function is used to copy configurations to/from storage.

Configurations managed by WINC_CmdCFGCP are groups of configuration command parameters. The command allows these parameters to be saved to either flash files or archive slots in memory for later retrieval.

WINC_CmdCFGCP supports:

- Saving active configurations to flash files
- Saving active configurations to in memory archive slots
- Restoring flash files to active configuration
- Restoring flash files to in memory archive slots
- Restoring in memory archive slots to active configurations
- Duplicating in memory archive slots

NOTE	Only parameter tables of configuration commands which have been changed are stored to flash files or archive slots. If no parameters have been changed, nothing is stored; the command still succeeds but the destination slot or file is not created.

Table 1. Configuration Persistence

Location	After reset
Active configuration (slot 0)	Lost, returns to the defaults (preset 0)
Archive slots 1 and 2, and their timestamps	Lost
Configuration files (file type 20)	Kept
The 'autoexec' configuration file	Kept, and loaded at boot (see Auto-Execute Configuration)

When a configuration is stored:

- The whole parameter table of a configuration command is stored once any of its elements has been set, not only the elements which were changed.
- Loading preset 0 (<ID> 0, see Default Configurations and Presets) marks the table as unchanged again, so it is not stored.
- Only configuration elements are stored; read-only and state elements, such as ARCHIVE_SLOTS and AUTOEXEC_STATUS, are not.
- The configurations of WINC_CmdSOCKC and WINC_CmdSYSLOGC are never stored.
- The parameters of WINC_CmdCFG, including ARCHIVE_FILTER, are stored like any other configuration command.

 ©2025-2026

When a configuration is restored to the active configuration:

- The stored values are applied on top of the current configuration. Tables which are not in the stored configuration keep their current values; to restore a clean configuration, load preset 0 for each command first, or reset the WINCS02.
- The restore stops at the first error, and values already applied are kept.
- A value which is rejected as an invalid parameter is retried, to allow for dependencies between commands; if it is still rejected after 100 ms the restore fails with TIMEOUT.

Active Configuration

The active configuration is considered to occupy slot 0 for the purposes of the source and destination of WINC_CmdCFGCP.

Archive Slots

There are two archive slots, numbered 1 and 2; they represent in memory holding locations for configuration sets. The parameters in these slots are not active and not maintained through reset.

Slots can be queried through ARCHIVE_SLOTS, these parameters contain the timestamp of the configuration set when stored from an active configuration. The timestamp is the UTC time, so it is close to 1970 if the time has not been set (see WINC_CmdTIME). Querying an empty slot fails. A configuration copied from a file to a slot keeps the timestamp of the file.

A slot can be emptied by writing a zero value to the appropriate index within ARCHIVE_SLOTS. Writing a zero value to ARCHIVE_SLOTS without an index empties all slots.

Archive Filters

The ARCHIVE_FILTER contains a list of command IDs which are filtered out when storing or loading a configuration set. This allows a partial configuration to be stored or restored.

- Up to 8 command IDs can be added. Each write adds one ID to the list; adding an ID which is already in the list, or a ninth ID, fails.
- An ID can be given as a number or as the command name, which must match exactly including the leading `+` (for example `"+WSTAC"`). Queries return the numeric IDs.
- Writing 0 clears the list.
- The filter is applied both when storing and when restoring, using the filter which is active at that time. A command filtered out when the configuration was stored cannot be restored from it.

See example Storing a partial configuration.

Archive Files

Configurations are stored in 'Configuration Files' types which can be queried and deleted through WINC_CmdFSOP.

Source and Destination

The source and destination may each be a slot number (0 for the active configuration, or an archive slot) or a configuration file name. Copying from one file to another is not supported. Copying to a file that already exists fails; the existing file must first be deleted using WINC_CmdFSOP. Copying from an empty archive slot, or from a file that does not exist, fails.

Configuration Summary

The response to WINC_CmdCFGCP is a summary list of command IDs with the number of command elements being stored or restored and the total size of the memory used for that command. The command ID is reported in hexadecimal. The summary is only reported when the source or destination is the active configuration.

Auto-Execute Configuration

If a configuration is stored to an archive file called 'autoexec' it is automatically loaded during device boot, once the file system has been mounted. The command interface may already be available when it is loaded, so the host should not rely on the configuration being active as soon as the WINC_AEC_ID_BOOT AEC is received.

After WINC_AEC_ID_BOOT, query AUTOEXEC_STATUS to check the result:

- CFG_NOT_PRESENT: there is no 'autoexec' file, or it has not been loaded yet.
- Any other status: the status of loading the file; OK if every stored value was applied.

To replace the auto-execute configuration, first delete the 'autoexec' file (file type 20) using WINC_CmdFSOP, then store the new configuration to it.

 ©2025-2026

```
bool WINC_CmdCFGCP
(
    WINC_CMD_REQ_HANDLE handle
    WINC_TYPE typeCfgsrc
    uintptr_t cfgsrc
    size_t lenCfgsrc
    WINC_TYPE typeCfgdst
    uintptr_t cfgdst
    size_t lenCfgdst
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
typeCfgsrc	WINC_TYPE	Type of cfgsrc, cfgsrc is not sent if the type is WINC_TYPE_INVALID.
cfgsrc	uintptr_t	Configuration source: slot number (0 = active) or file name. For integer types cfgsrc holds the value, for other types cfgsrc is a pointer to the data.
lenCfgsrc	size_t	Length in bytes of the data cfgsrc points to, ignored for integer types.
typeCfgdst	WINC_TYPE	Type of cfgdst, cfgdst is not sent if the type is WINC_TYPE_INVALID.
cfgdst	uintptr_t	Configuration destination: slot number (0 = active) or file name. For integer types cfgdst holds the value, for other types cfgdst is a pointer to the data.
lenCfgdst	size_t	Length in bytes of the data cfgdst points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<CMD_ID>,<NUM_ITEMS>,<ITEM_LEN>	copy event

Response Element Description

Element Name	Type	Description
<CMD_ID>	INTEGER	Configuration command ID (hexadecimal)
<NUM_ITEMS>	INTEGER	Number of items
<ITEM_LEN>	INTEGER	Length of items

Examples:

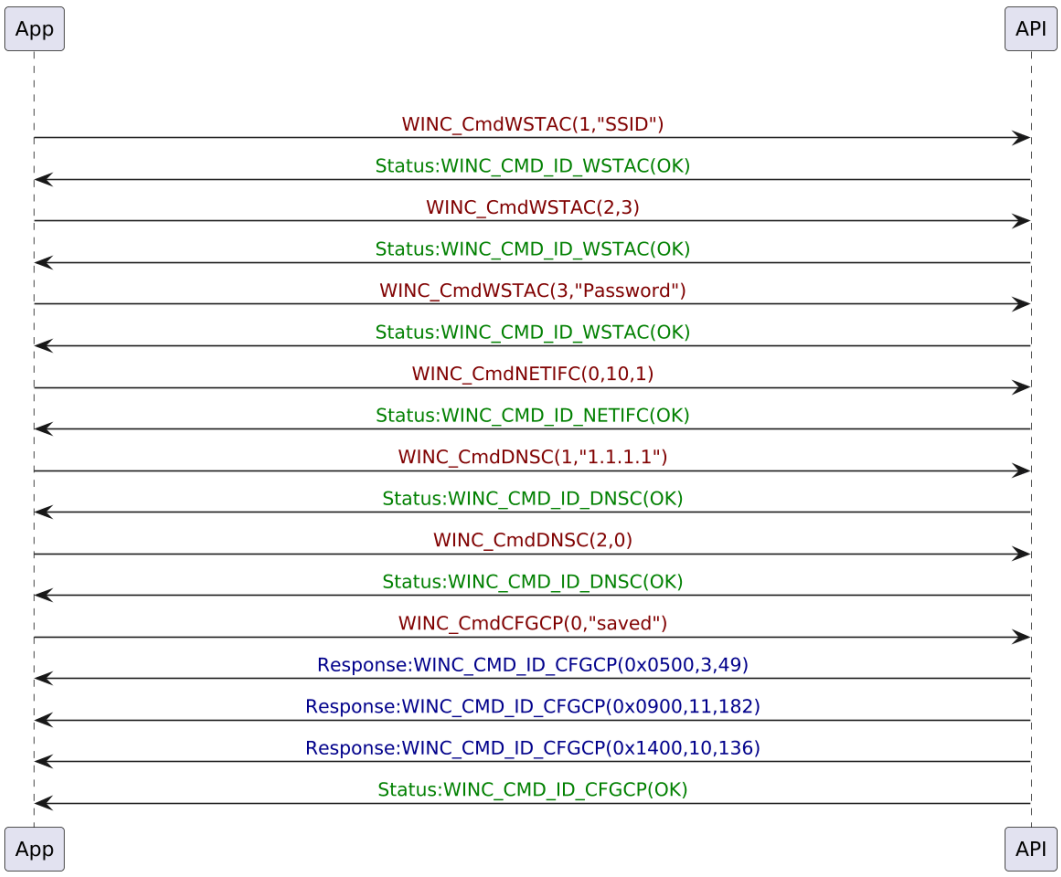


Figure 9. Saving active configuration to storage

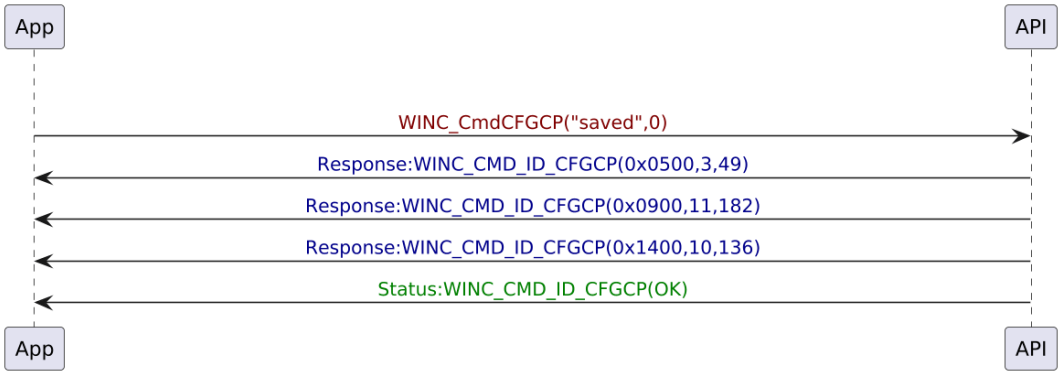


Figure 10. Restoring configuration from storage

 ©2025-2026

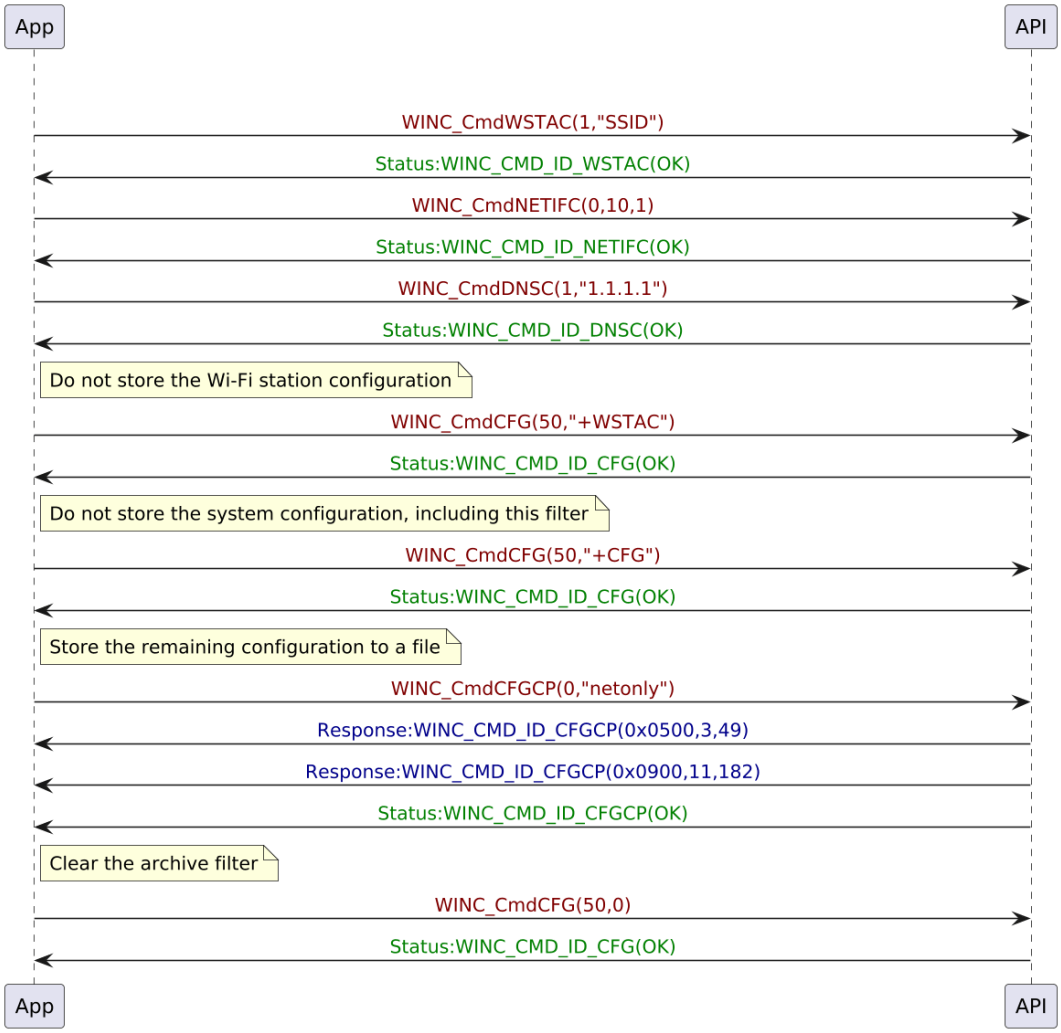


Figure 11. Storing a partial configuration

DHCP (Module ID = 4)

Introduction

The WINCS02 provides a DHCP server for clients connected to its soft AP.

The server is configured with WINC_CmdDHCPSC and is started and stopped together with the soft AP by WINC_CmdWAP, if ENABLED is set. While the server is running, its configuration cannot be changed.

Address Pool

If no pool start address is configured, the pool is placed in the half of the soft AP subnet that does not contain the WINCS02's own address. The pool holds at most 32 leases. The soft AP subnet must be /29 or larger, and the pool start address must not be the network, broadcast or WINCS02 address, otherwise the soft AP fails to start.

When the server is running, reading the configuration reports the active pool start, pool end and number of leases. When it is stopped, the configured pool start is reported.

The gateway and DNS server addresses, if configured, are provided to clients in the DHCP offer.

 ©2025-2026

Function Reference:

WINC_CmdDHCPSC

Description

This function is used to read or set the DHCP server configuration.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdDHCPSC
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optIdx
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optIdx	int32_t	Pool index. optIdx will be ignored if its value is WINC_CMDDHCPSC_IDX_IGNORE_VAL.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDDHCPSC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<MIN_IDX>,<MAX_IDX>	Valid index range
<ID>,<VAL>	Read response

Response Element Description

Element Name	Type	Description
<MIN_IDX>	INTEGER	Lowest valid index
<MAX_IDX>	INTEGER	Highest valid index
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_DHCP_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset **Value** / **Label** / **Description** 0 / DEFAULT / Default.
1	ENABLED	BOOL	DHCP server enabled when the soft AP is started **Value** / **Label** / **Description** 0 / DISABLED / Disabled. 1 / ENABLED / Enabled.
2	POOL_START	IPV4ADDR	Start address of DHCP server pool (0 = automatic) Unsigned 32-bit value Format of IPv4 address is A1|A2|A3|A4 = A1.A2.A3.A4
3	POOL_END	IPV4ADDR (Read Only)	End address of the active DHCP server pool (read only, while running) Unsigned 32-bit value Format of IPv4 address is A1|A2|A3|A4 = A1.A2.A3.A4
4	POOL_LEASES	INTEGER (Read Only)	Number of leases in the active pool (read only, while running) Unsigned 16-bit value
5	NETIF_IDX	INTEGER	Network interface index Valid range is 0 to 1
10	GATEWAY	IPV4ADDR	The address of the default gateway Unsigned 32-bit value Format of IPv4 address is A1|A2|A3|A4 = A1.A2.A3.A4

 ©2025-2026

DNS (Module ID = 5)

Introduction:

Server Address Configuration

There are three methods available to configure the DNS servers used by the device.

1. Manually configured via WINC_CmdDNSC option DNS_SVR.
2. Automatically configured via IPv4 DHCP option 6.
3. Automatically configured via IPv6 Router Advertisement RNDSS.

Switching between manual and automatic configuration is performed by WINC_CmdDNSC option DNS_AUTO.

Resource Queries

The supported query types are listed in WINC_CmdDNSRESOLV parameter type, other types may be specified as well. For unknown type the WINC_AEC_ID_DNSRESOLV response will simply include the undecoded RR data.

The DNS query ANY is indirectly supported, it has generally been deprecated as a DNS query type and some servers do not respond to it. If the ANY query is specified the device will generate two separate queries for A and AAAA records. When generating the queries for ANY, the device will use the configuration of WINC_CmdDNSC option DNS_PROTO_PREF to determine selection and order of A and AAAA queries.

NOTE	DNS_PROTO_PREF specifies the order of queries made, it does not control the order of responses received.

Reverse Address Queries

IP address to name queries can be made using a **PTR** (12) query in the form:

- <IPv4 Addr>.in-addr.arpa
- <IPv6 Addr>.ip6.arpa

Connection Types

The DNS modules supports UDP, TCP, TLS and Multicast-UDP for performing DNS queries.

The default connection method is to use UDP to port 53 of the servers configured either manually or through DHCP/RA.

DNS over UDP

For queries to be sent via UDP, WINC_CmdDNSC option DNS_CONN_PREF must be set to 0 (UDP/TLS) and TLS_CONF must be 0. These are the defaults.

The UDP port on the server used for queries will be set by DNS_SRV_PORT which defaults to 53.

DNS over TCP

For queries to be sent via TCP, WINC_CmdDNSC option DNS_CONN_PREF must be set to 1 (TCP/TLS) and TLS_CONF must be 0.

The TCP port on the server used for queries will be set by DNS_SRV_PORT which defaults to 53.

Option DNS_CONN_IDLE_TIMEOUT can be used to set an idle timeout, the DNS module will request the IDLE timeout from the DNS server and honour that time period.

DNS over TLS

For queries to be sent via TLS, WINC_CmdDNSC option DNS_CONN_PREF must be set to 1 (TCP/TLS) and TLS_CONF must be set to a TLS configuration index (see WINC_CmdTLSC). DTLS (DNS_CONN_PREF set to 0 with a TLS configuration) is not supported.

The TLS configuration must allow the DNS server to be authenticated. By default this requires a CA certificate and the server's domain name to be configured; see TLS.

When TLS is used, queries are sent only to the first configured DNS server.

The TLS port on the server used for queries will be set by DNS_SRV_SECURE_PORT which defaults to 853.

Option DNS_CONN_IDLE_TIMEOUT can be used to set an idle timeout, the DNS module will request the IDLE timeout from the DNS server and honour that time period.

 ©2025-2026

DNSSEC Support

Support for DNSSEC is configured by WINC_CmdDNSC option DNS_DNSSEC_MODE which offers three possible modes of operation:

1. No support, the DO bit of EDNS0 extended flags is not set indicating that the device is unaware of DNSSEC operations.
2. DNSSEC aware, this sets the DO bit of EDNS0 extended flags to indicate that DNSSEC RRs should be sent in response to queries. These RRs will be passed to the application for further processing.
3. DNSSEC authenticated answer only. In addition to the previous state the DNS module will not process any query responses which do not have either the AA bit set (indicating the DNS server is an authority for the domain) or the AD bit set (indicating the server has authenticated the response).

Cache Control

Query responses are held in a small answer cache to speed up further queries. Records are held in the cache based on the TTL value specified in the records. In addition the application can set a cache timeout via WINC_CmdDNSC option DNS_CACHE_TTL which, if lower than a response TTL, will override it causing records to be expired from the cache sooner.

MDNS Support

In addition to unicast DNS queries the DNS module supports multicast queries for those records in the **.local** domain.

WINC_CmdDNSC option DNS_MC_SRV_PREF configures if either or both IPv4 and IPv6 multicast queries should be made. If DNS_MC_SRV_PREF is non-zero and the query domain is **.local** a multicast DNS query will be made.

Function Reference:

WINC_CmdDNSC

Description

This function is used to read or set the DNS configuration.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdDNSC
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDDNSC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response

 ©2025-2026

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_DNS_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset **Value / Label / Description** 0 / DEFAULT / Default.
1	DNS_SVR	IPV4ADDR IPV6ADDR	DNS server IP address. 1.0-1.1 configured (writable), 1.0-1.3 active (read only) Unsigned 32-bit value Format of IPv4 address is A1|A2|A3|A4 = A1.A2.A3.A4 Format of IPv6 address is A1|...|A16 = A1A2::A15A16 This is a multiple value parameter with an ID range 1.0 to 1.3 See DNS Servers below.
2	DNS_AUTO	BOOL	DNS server auto-assignment **Value / Label / Description** 0 / MANUAL / Manual - use DNS_SVR. 1 / AUTO / Auto - through DHCP etc..
3	DNS_TIMEOUT	INTEGER_UNSIGNED	DNS timeout in milliseconds Unsigned 16-bit value
4	DNS_PROTO_PREF	INTEGER_UNSIGNED	Result protocol preference **Value / Label / Description** 1 / A_AAAA / A preferred over AAAA records. 2 / AAAA_A / AAAA preferred over A records. 3 / A / A only. 4 / AAAA / AAAA only.
6	DNS_CACHE_TTL	INTEGER_UNSIGNED	DNS cache maximum TTL Unsigned 16-bit value
7	TLS_CONF	INTEGER	TLS configuration index (see WINC_CmdTLSCDA). 0 disables TLS Valid range is 0 to 4

<table>
<tr><th>ID</th><th>Name</th><th>Type</th><th>Description</th></tr>
<tr><td>8</td><td>DNS_DNSSEC_MODE</td><td>INTEGER_UNSIGNED</td><td>DNSSEC mode

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>OFF</td><td>DNSSEC awareness off.</td></tr><tr><td>1</td><td>CD</td><td>DNSSEC aware, checking disabled.</td></tr><tr><td>2</td><td>AD</td><td>DNSSEC authenticated answers only.</td></tr></table></td></tr>
<tr><td>10</td><td>DNS_SRV_PORT</td><td>INTEGER_UNSIGNED</td><td>DNS server UDP/TCP port

Positive unsigned 16-bit value</td></tr>
<tr><td>11</td><td>DNS_SRV_SECURE_PORT</td><td>INTEGER_UNSIGNED</td><td>DNS server TLS port

Positive unsigned 16-bit value</td></tr>
<tr><td>12</td><td>DNS_CONN_PREF</td><td>INTEGER_UNSIGNED</td><td>Connection preference. TLS is used if TLS_CONF is set

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>UDP_TLS</td><td>UDP or TLS.</td></tr><tr><td>1</td><td>TCP_TLS</td><td>TCP or TLS.</td></tr></table></td></tr>
<tr><td>13</td><td>DNS_MC_SRV_PREF</td><td>INTEGER_UNSIGNED</td><td>Multicast server preference

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>OFF</td><td>Not used.</td></tr><tr><td>1</td><td>IPv4</td><td>IPv4 servers only.</td></tr><tr><td>2</td><td>IPv6</td><td>IPv6 servers only.</td></tr><tr><td>3</td><td>IPANY</td><td>IPv4 & IPv6 servers.</td></tr></table></td></tr>
<tr><td>14</td><td>DNS_CONN_IDLE_TIMEOUT</td><td>INTEGER_UNSIGNED</td><td>DNS connection idle timeout (in milliseconds)

Unsigned 16-bit value</td></tr>
</table>

DNS Servers

DNS_SVR holds two lists:

- **Configured servers**, indices 1.0 and 1.1. These can be set, and are reported while the network interface is down. They are used when DNS_AUTO is 0.
- **Active servers**, indices 1.0 to 1.3. These are read only and reported while the network interface is up. They contain the servers currently in use, from the configured list or from DHCP / router advertisements.

Setting DNS_SVR without an index adds the address to the next free configured slot; this fails if both slots are in use. Setting it to 0.0.0.0 without an index removes all configured servers.

 ©2025-2026

WINC_CmdDNSRESOLV

Description

This function is used to resolve domain names via DNS.

```
bool WINC_CmdDNSRESOLV
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t type
    const uint8_t* pQueryName
    size_t lenQueryName
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
type	uint8_t	Type of record.
pQueryName	const uint8_t*	Query name to resolve. Maximum length of string is 256
lenQueryName	size_t	Length of pQueryName in bytes, strings are not NUL terminated so the length must not include a terminator.

Return Values

Value	Description
true	Success.
false	Failure.

AEC Reference:

WINC_AEC_ID_DNSRESOLV

Description

Resolve results.

AEC Syntax

AEC	Description
<TYPE>,<QUERY_NAME>,<ADDRESS>	A & AAAA response
<TYPE>,<QUERY_NAME>,<NSDNAME>	NS response
<TYPE>,<QUERY_NAME>,<CNAME>	CNAME response
<TYPE>,<QUERY_NAME>,<MNAME>,<SERIAL>,<REFRESH>,<RETRY>,<EXPIRE>,<MINIMUM>,<RNAME>	SOA response
<TYPE>,<QUERY_NAME>,<PTRDNAME>	PTR response
<TYPE>,<QUERY_NAME>,<EXCHANGE>,<PREFERENCE>	MX response
<TYPE>,<QUERY_NAME>,<TXT_DATA>	TXT response
<TYPE>,<QUERY_NAME>,<TARGET>,<PRIORITY>,<WEIGHT>,<PORT>	SRV response
<TYPE>,<QUERY_NAME>,<KEY_TAG>,<ALGORITHM>,<DIGEST_TYPE>,<DIGEST>	DS response
<TYPE>,<QUERY_NAME>,<SIGNER_NAME>,<TYPE>,<ALGORITHM>,<LABELS>,<ORIG_TTL>,<SIG_EXPIRE>,<SIG_INCEPT>,<KEY_TAG>,<SIGNATURE>	RRSIG response
<TYPE>,<QUERY_NAME>,<NEXT_DOMAIN>,<TYPE_BIT_MAPS>	NSEC response
<TYPE>,<QUERY_NAME>,<FLAGS>,<PROTOCOL>,<ALGORITHM>,<PUBLIC_KEY>	DNSKEY response

 ©2025-2026

AEC Element Syntax

<table>
<tr><th>Element Name</th><th>Type</th><th>Description</th></tr>
<tr><td><TYPE></td><td>INTEGER</td><td>Type of record

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>1</td><td>A</td><td>A.</td></tr><tr><td>2</td><td>NS</td><td>NS.</td></tr><tr><td>5</td><td>CNAME</td><td>CNAME.</td></tr><tr><td>6</td><td>SOA</td><td>SOA.</td></tr><tr><td>12</td><td>PTR</td><td>PTR.</td></tr><tr><td>15</td><td>MX</td><td>MX.</td></tr><tr><td>16</td><td>TXT</td><td>TXT.</td></tr><tr><td>28</td><td>AAAA</td><td>AAAA.</td></tr><tr><td>33</td><td>SRV</td><td>SRV.</td></tr><tr><td>43</td><td>DS</td><td>DS.</td></tr><tr><td>46</td><td>RRSIG</td><td>RRSIG.</td></tr><tr><td>47</td><td>NSEC</td><td>NSEC.</td></tr><tr><td>48</td><td>DNSKEY</td><td>DNSKEY.</td></tr><tr><td>255</td><td>ANY</td><td>A and/or AAAA (DNSERR only).</td></tr></table>
Positive unsigned 8-bit value</td></tr>
<tr><td><QUERY_NAME></td><td>STRING</td><td>Original query name requested</td></tr>
<tr><td><ADDRESS></td><td>IP_ADDRESS</td><td>IP address</td></tr>
<tr><td><NSDNAME></td><td>STRING</td><td>Name server domain name</td></tr>
<tr><td><CNAME></td><td>STRING</td><td>Canonical alias domain name</td></tr>
<tr><td><MNAME></td><td>STRING</td><td>Primary name server domain name</td></tr>
<tr><td><SERIAL></td><td>INTEGER</td><td>Serial number

Unsigned 32-bit value</td></tr>
<tr><td><REFRESH></td><td>INTEGER</td><td>Zone refresh interval

Unsigned 32-bit value</td></tr>
<tr><td><RETRY></td><td>INTEGER</td><td>Zone refresh retry interval

Unsigned 32-bit value</td></tr>
<tr><td><EXPIRE></td><td>INTEGER</td><td>Zone expiry interval

Unsigned 32-bit value</td></tr>
<tr><td><MINIMUM></td><td>INTEGER</td><td>Minimum exported RR TTL

Unsigned 32-bit value</td></tr>
<tr><td><RNAME></td><td>STRING</td><td>Mailbox domain name</td></tr>
<tr><td><PTRDNAME></td><td>STRING</td><td>Pointer domain name</td></tr>
<tr><td><EXCHANGE></td><td>STRING</td><td>Exchange domain name</td></tr>
</table>

Element Name	Type	Description
<PREFERENCE>	INTEGER	Preference Unsigned 16-bit value
<TXT_DATA>	STRING	Descriptive text
<TARGET>	STRING	Target domain name
<PRIORITY>	INTEGER	Host priority Unsigned 16-bit value
<WEIGHT>	INTEGER	Relative service weight Unsigned 16-bit value
<PORT>	INTEGER	Service port Unsigned 16-bit value
<KEY_TAG>	INTEGER	Key tag Unsigned 16-bit value
<ALGORITHM>	INTEGER	Algorithm Positive unsigned 8-bit value
<DIGEST_TYPE>	INTEGER	Digest type Positive unsigned 8-bit value
<DIGEST>	BYTE_ARRAY	Digest
<SIGNER_NAME>	STRING	Signer’s name
<LABELS>	INTEGER	Number of labels Unsigned 8-bit value
<ORIG_TTL>	INTEGER	Original TTL Unsigned 32-bit value
<SIG_EXPIRE>	INTEGER	Signature expiration Unsigned 32-bit value
<SIG_INCEPT>	INTEGER	Signature inception Unsigned 32-bit value
<SIGNATURE>	BYTE_ARRAY	Signature
<NEXT_DOMAIN>	STRING	Next domain name
<TYPE_BIT_MAPS>	BYTE_ARRAY	Type bit maps
<FLAGS>	INTEGER	Flags Unsigned 16-bit value
<PROTOCOL>	INTEGER	Protocol **Value** / **Label** / **Description** 3 / DNSSEC / DNSSEC.
<PUBLIC_KEY>	BYTE_ARRAY	Public key

 ©2025-2026

WINC_AEC_ID_DNSERR

Description

Resolve failure.

AEC Syntax

AEC	Description
<ERROR_CODE>,<TYPE>,<QUERY_NAME>	Resolve failure

AEC Element Syntax

Element Name	Type	Description
<ERROR_CODE>	INTEGER	Error code See Status Response Codes
<TYPE>	INTEGER	Type of record **Value / Label / Description** 1 / A / A. 2 / NS / NS. 5 / CNAME / CNAME. 6 / SOA / SOA. 12 / PTR / PTR. 15 / MX / MX. 16 / TXT / TXT. 28 / AAAA / AAAA. 33 / SRV / SRV. 43 / DS / DS. 46 / RRSIG / RRSIG. 47 / NSEC / NSEC. 48 / DNSKEY / DNSKEY. 255 / ANY / A and/or AAAA (DNSERR only). Positive unsigned 8-bit value
<QUERY_NAME>	STRING	Original query name requested

Examples:



Figure 12. A record query



Figure 13. NS record query



Figure 14. CNAME record query

 ©2025-2026



Figure 15. SOA record query

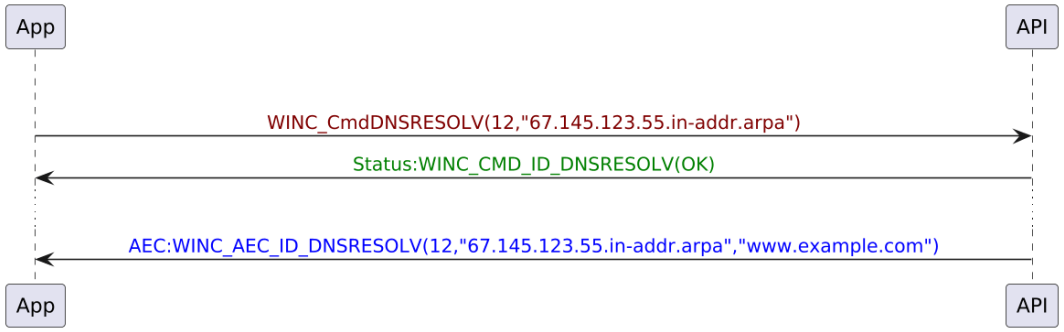


Figure 16. PTR record query

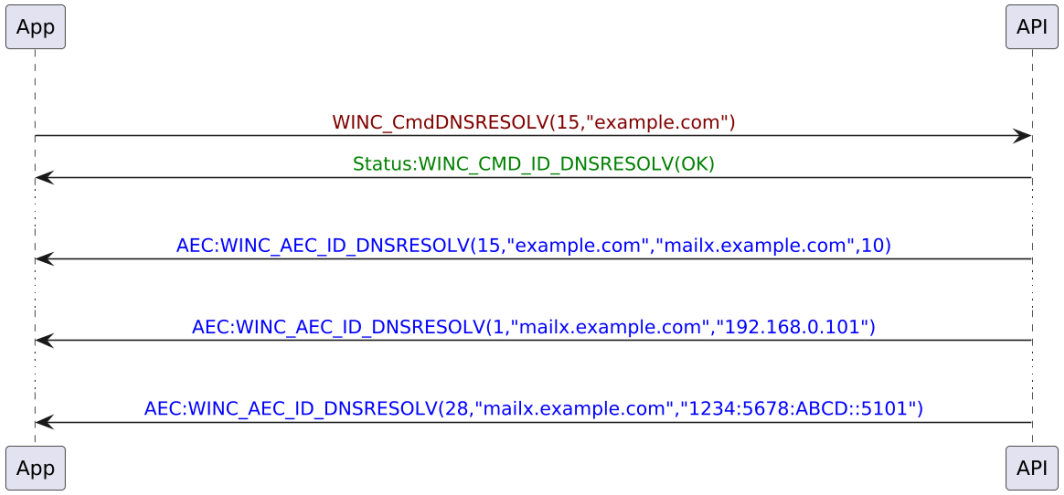


Figure 17. MX record query

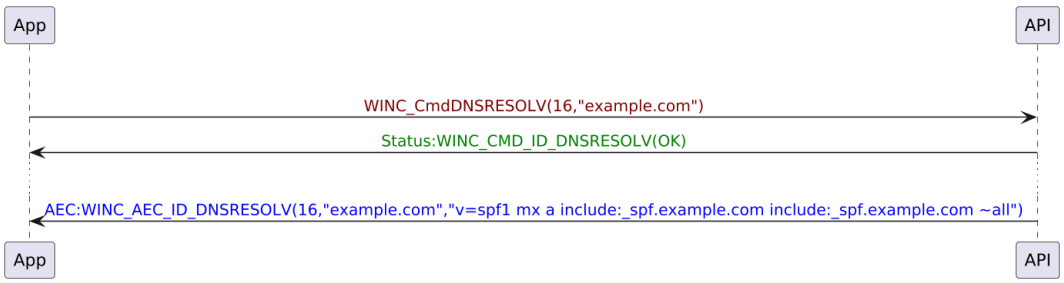


Figure 18. TXT record query



Figure 19. AAAA record query

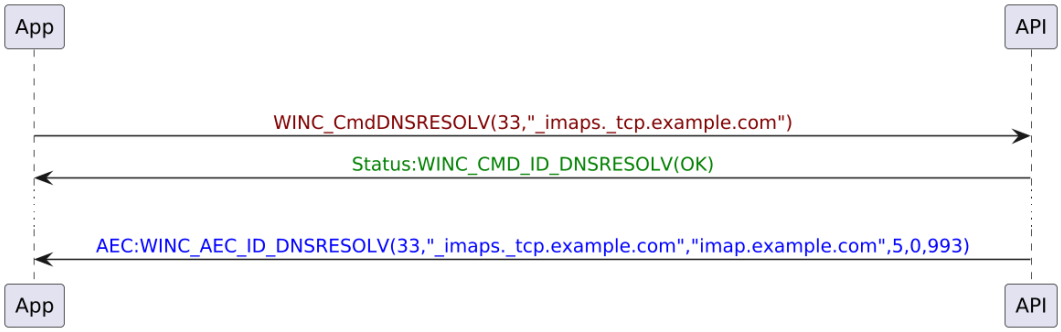


Figure 20. SRV record query



Figure 21. DS record query

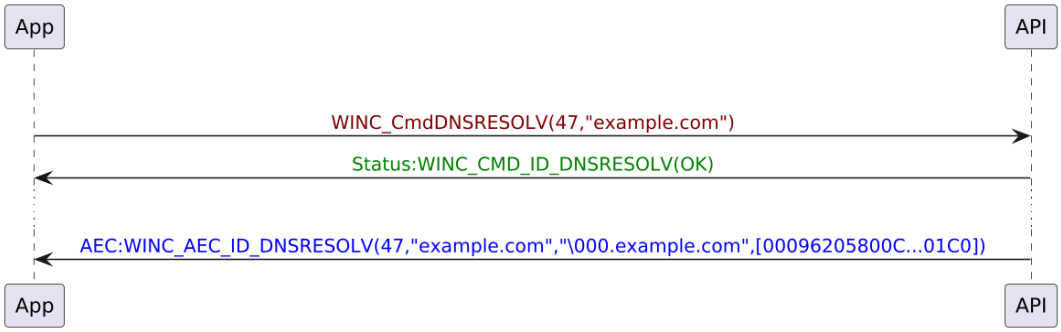


Figure 22. NSEC record query

 ©2025-2026



Figure 23. DNSKEY record query



Figure 24. ANY record query

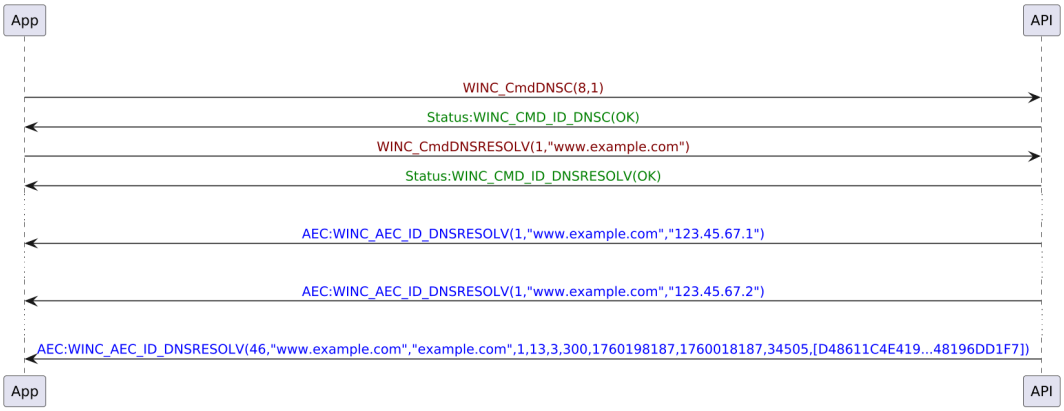


Figure 25. A record query With DNSSEC awareness

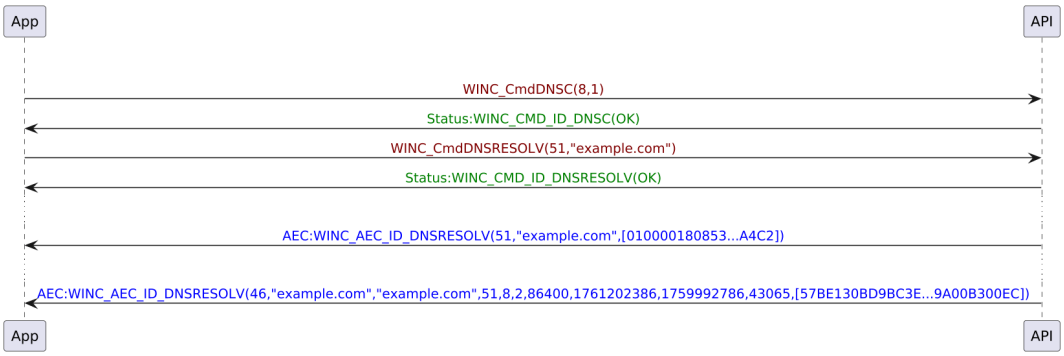


Figure 26. NSEC3PARAM (unknown type) record query

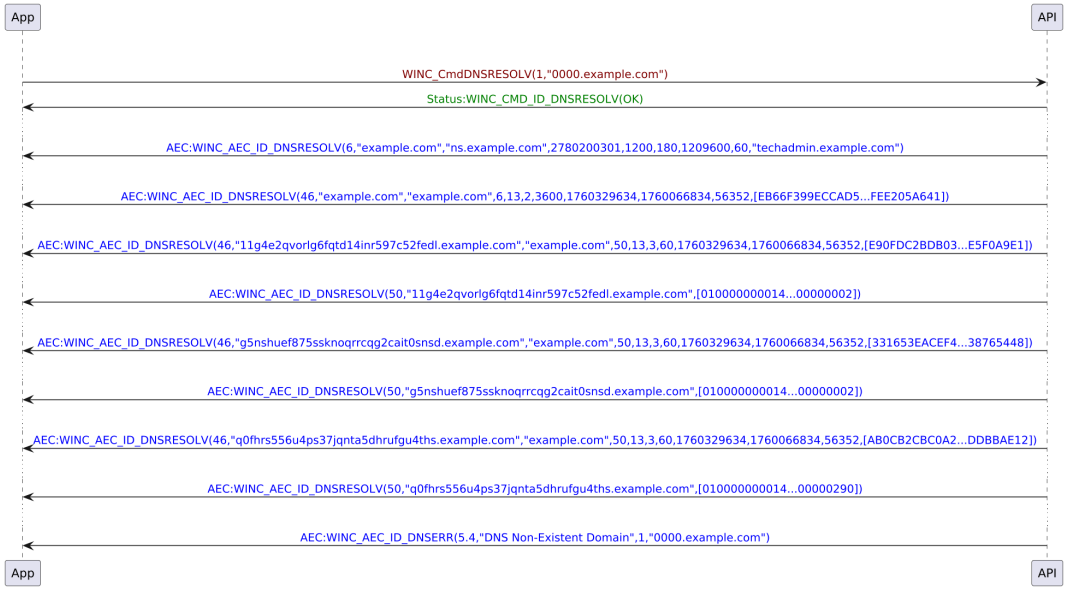


Figure 27. NSEC3 (unknown type) response to invalid A record query

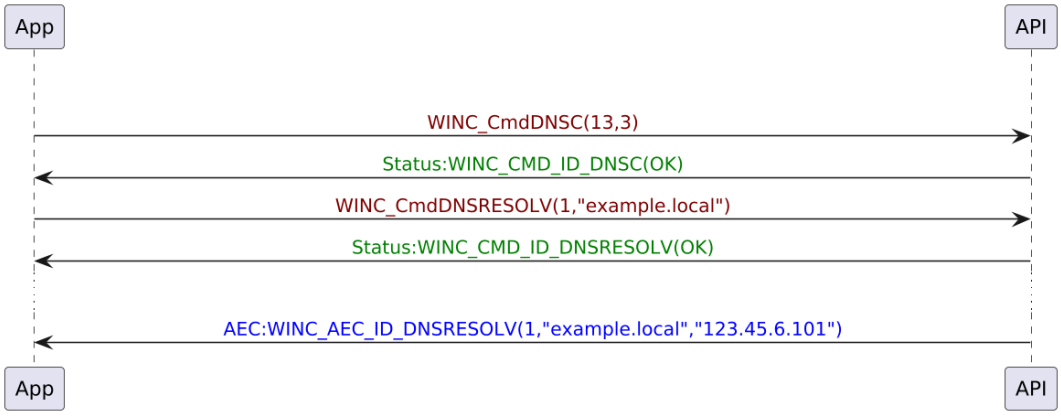


Figure 28. Multicast A record query

 ©2025-2026

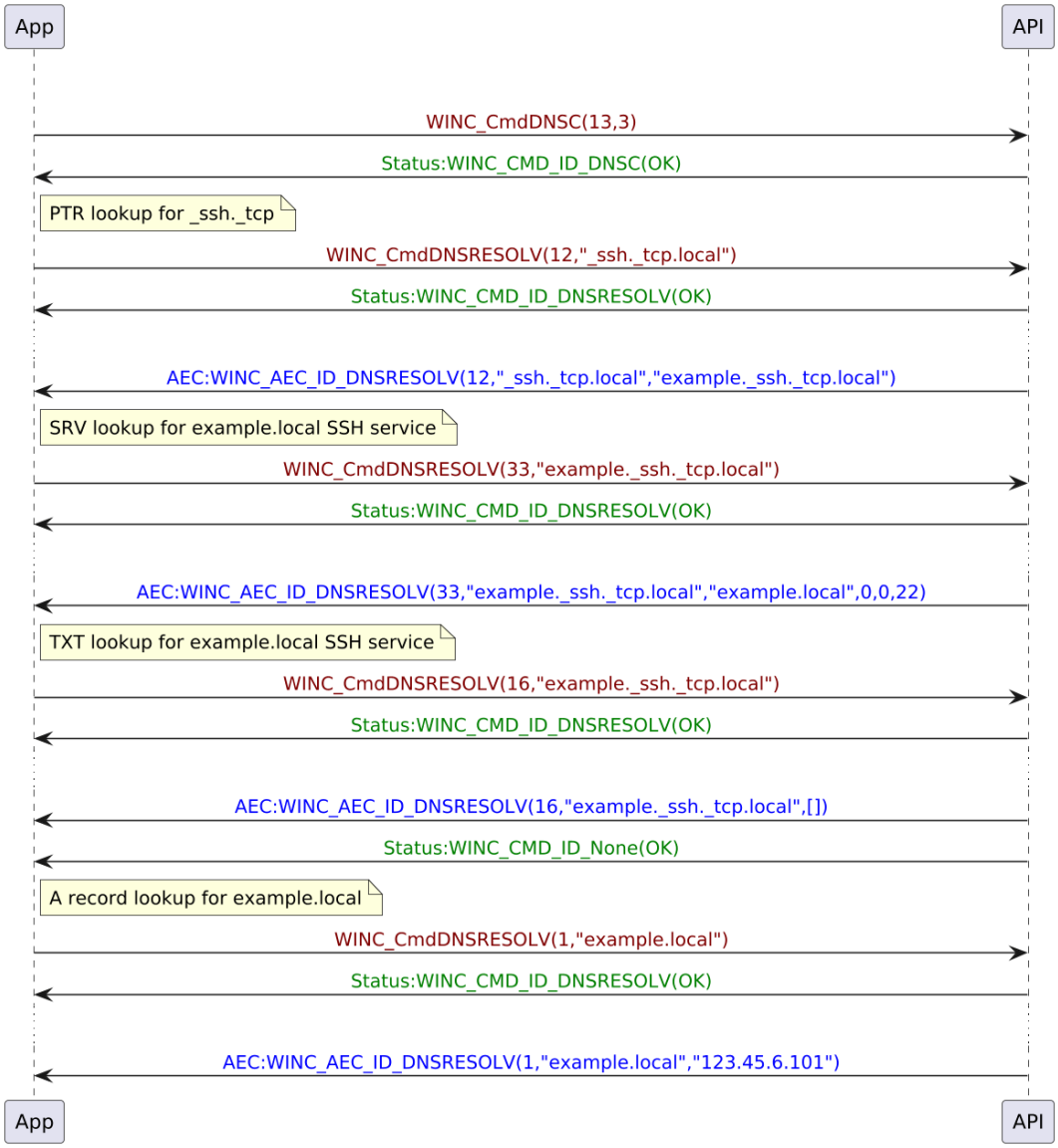


Figure 29. MDNS-SD of local SSH services

FS (Module ID = 7)

Function Reference:

WINC_CmdFSOP

Description

This function performs a filesystem operation.

These operations use the WINC_CMD_ID_FS command request, therefore all status and response messages received will be using the WINC_CMD_ID_FS command ID. Messages are further identified by their <OP> response value indicating the type of operation involved.

The filesystem operation command is split into several sub-commands:

- List
- Delete
- Info

List Sub-Command:

The list sub-command produces a list of files present in the filesystem based on the file type specified.

Delete Sub-Command:

The delete sub-command deletes a single file object of the type specified from the filesystem.

Info Sub-Command:

The info sub-command returns information on the filesystem.

```
bool WINC_CmdFSOP
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t op
    int32_t optFiletype
    const uint8_t* pOptFilename
    size_t lenOptFilename
)
```

 ©2025-2026

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
op	uint8_t	Operation. **Value / Label / Description** 2 / LIST / List. 3 / DEL / Delete. 4 / INFO / Information.
optFiletype	int32_t	File type. **Value / Label / Description** 0 / USER / User. 1 / CERT / Certificate. 2 / PRIKEY / Private Key. 3 / DHPARAM / Diffie-Hellman parameters. 20 / CFG / Configuration File. optFiletype will be ignored if its value is WINC_CMDFSOP_FILETYPE_IGNORE_VAL.
pOptFilename	const uint8_t*	The name of the file. Maximum length of string is 32 pOptFilename will be ignored if its value is NULL.
lenOptFilename	size_t	Length of pOptFilename in bytes, strings are not NUL terminated so the length must not include a terminator.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<OP>,<FILETYPE>,<FILENAME>	List operation response
<OP>,<FREE_SPACE>,<FREE_HANDLES>	Information operation response

Response Element Description

<table>
<tr><th>Element Name</th><th>Type</th><th>Description</th></tr>
<tr><td><OP></td><td>INTEGER</td><td>Operation

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>1</td><td>LOAD</td><td>Load.</td></tr><tr><td>2</td><td>LIST</td><td>List.</td></tr><tr><td>3</td><td>DEL</td><td>Delete.</td></tr><tr><td>4</td><td>INFO</td><td>Information.</td></tr><tr><td>5</td><td>STORE</td><td>Store.</td></tr></table></td></tr>
<tr><td><FILETYPE></td><td>INTEGER</td><td>File type

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>USER</td><td>User.</td></tr><tr><td>1</td><td>CERT</td><td>Certificate.</td></tr><tr><td>2</td><td>PRIKEY</td><td>Private Key.</td></tr><tr><td>3</td><td>DHPARAM</td><td>Diffie-Hellman parameters.</td></tr><tr><td>20</td><td>CFG</td><td>Configuration File.</td></tr></table></td></tr>
<tr><td><FILENAME></td><td>STRING</td><td>The name of the file</td></tr>
<tr><td><FREE_SPACE></td><td>INTEGER</td><td>Free space

Positive unsigned 16-bit value</td></tr>
<tr><td><FREE_HANDLES></td><td>INTEGER</td><td>Free file handles

Positive unsigned 16-bit value</td></tr>
</table>

 ©2025-2026

WINC_CmdFSLOAD

Description

This function performs a filesystem operation.

This operation uses the WINC_CMD_ID_FS command request, therefore all status and response messages received will be using the WINC_CMD_ID_FS command ID. Messages are further identified by their <OP> response value indicating a value of 1.

```
bool WINC_CmdFSLOAD
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t filetype
    const uint8_t* pOptFilename
    size_t lenOptFilename
    uint16_t optFilelength
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
filetype	uint8_t	File type. **Value** / **Label** / **Description** 0 / USER / User. 1 / CERT / Certificate. 2 / PRIKEY / Private Key. 3 / DHPARAM / Diffie-Hellman parameters. 20 / CFG / Configuration File.
pOptFilename	const uint8_t*	The name of the file. Maximum length of string is 32 pOptFilename will be ignored if its value is NULL.
lenOptFilename	size_t	Length of pOptFilename in bytes, strings are not NUL terminated so the length must not include a terminator.
optFilelength	uint16_t	File length. optFilelength will be ignored if its value is WINC_CMDFSLOAD_FILELENGTH_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<OP>,<TSFR_HANDLE>	FS-TSFR response

Response Element Description

Element Name	Type	Description
<OP>	INTEGER	Operation **Value** / **Label** / **Description** 1 / LOAD / Load. 2 / LIST / List. 3 / DEL / Delete. 4 / INFO / Information. 5 / STORE / Store.
<TSFR_HANDLE>	INTEGER	Transfer handle Positive unsigned 16-bit value

 ©2025-2026

WINC_CmdFSSTORE

Description

This function performs a filesystem operation.

This operation uses the WINC_CMD_ID_FS command request, therefore all status and response messages received will be using the WINC_CMD_ID_FS command ID. Messages are further identified by their <OP> response value indicating a value of 5.

```
bool WINC_CmdFSSTORE
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t filetype
    const uint8_t* pFilename
    size_t lenFilename
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
filetype	uint8_t	File type. **Value** / **Label** / **Description** 0 / USER / User. 1 / CERT / Certificate. 2 / PRIKEY / Private Key. 3 / DHPARAM / Diffie-Hellman parameters. 20 / CFG / Configuration File.
pFilename	const uint8_t*	The name of the file. Maximum length of string is 32
lenFilename	size_t	Length of pFilename in bytes, strings are not NUL terminated so the length must not include a terminator.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<OP>,<TSFR_HANDLE>	FS-TSFR response

Response Element Description

Element Name	Type	Description
<OP>	INTEGER	Operation **Value** / **Label** / **Description** 1 / LOAD / Load. 2 / LIST / List. 3 / DEL / Delete. 4 / INFO / Information. 5 / STORE / Store.
<TSFR_HANDLE>	INTEGER	Transfer handle Positive unsigned 16-bit value

 ©2025-2026

WINC_CmdFSTSFR

Description

This function performs a filesystem transfer operation.

```
bool WINC_CmdFSTSFR
(
    WINC_CMD_REQ_HANDLE handle
    uint16_t optTsfrHandle
    int32_t optBlockNum
    const uint8_t* pOptData
    size_t lenOptData
    int32_t optCrc
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optTsfrHandle	uint16_t	Transfer handle. optTsfrHandle will be ignored if its value is WINC_CMDFSTSFR_TSFR_HANDLE_IGNORE_VAL.
optBlockNum	int32_t	Block number. optBlockNum will be ignored if its value is WINC_CMDFSTSFR_BLOCK_NUM_IGNORE_VAL.
pOptData	const uint8_t*	Transfer data. pOptData will be ignored if its value is NULL.
lenOptData	size_t	Length of pOptData in bytes.
optCrc	int32_t	Transfer CRC-16. optCrc will be ignored if its value is WINC_CMDFSTSFR_CRC_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<TSFR_HANDLE>,<BLOCK_NUM>,<BYTES_REMAIN>	FS-TSFR load response
<TSFR_HANDLE>,<BLOCK_NUM>,<BYTES_REMAIN>,<DATA>	FS-TSFR store response

Response Element Description

Element Name	Type	Description
<TSFR_HANDLE>	INTEGER	Transfer handle Positive unsigned 16-bit value
<BLOCK_NUM>	INTEGER	Block number Unsigned 8-bit value
<BYTES_REMAIN>	INTEGER	Bytes remaining Positive unsigned 16-bit value
<DATA>	STRING BYTE_ARRAY	Transfer data

 ©2025-2026

WINC_CmdFSRECV

Description

This function performs a filesystem transfer operation.

This transfer operation uses the WINC_CMD_ID_FSTSFR command request, therefore all status and response messages received will be using the WINC_CMD_ID_FSTSFR command ID. Messages are further identified by their <TSFR_HANDLE> response value indicating transfer context used.

```
bool WINC_CmdFSRECV
(
    WINC_CMD_REQ_HANDLE handle
    uint16_t tsfrHandle
    uint8_t blockNum
    uint16_t optDataLength
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
tsfrHandle	uint16_t	Transfer handle.
blockNum	uint8_t	Block number.
optDataLength	uint16_t	Data length. optDataLength will be ignored if its value is WINC_CMDFSRECV_DATA_LENGTH_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<TSFR_HANDLE>,<BLOCK_NUM>,<BYTES_REMAIN>	FS-TSFR load response
<TSFR_HANDLE>,<BLOCK_NUM>,<BYTES_REMAIN>,<DATA>	FS-TSFR store response

Response Element Description

Element Name	Type	Description
<TSFR_HANDLE>	INTEGER	Transfer handle Positive unsigned 16-bit value
<BLOCK_NUM>	INTEGER	Block number Unsigned 8-bit value
<BYTES_REMAIN>	INTEGER	Bytes remaining Positive unsigned 16-bit value
<DATA>	STRING BYTE_ARRAY	Transfer data

 ©2025-2026

AEC Reference:

WINC_AEC_ID_FSUP

Description

Filesystem update.

AEC Syntax

AEC	Description
<OP>,<FILETYPE>,<FILENAME>,<FILELENGTH>	Filesystem load

AEC Element Syntax

Element Name	Type	Description
<OP>	INTEGER	Operation **Value** / **Label** / **Description** 1 / LOAD / Load. 2 / LIST / List. 3 / DEL / Delete. 4 / INFO / Information. 5 / STORE / Store.
<FILETYPE>	INTEGER	File type **Value** / **Label** / **Description** 0 / USER / User. 1 / CERT / Certificate. 2 / PRIKEY / Private Key. 3 / DHPARAM / Diffie-Hellman parameters. 20 / CFG / Configuration File.
<FILENAME>	STRING	The name of the file
<FILELENGTH>	INTEGER	File length Positive unsigned 16-bit value

Examples:

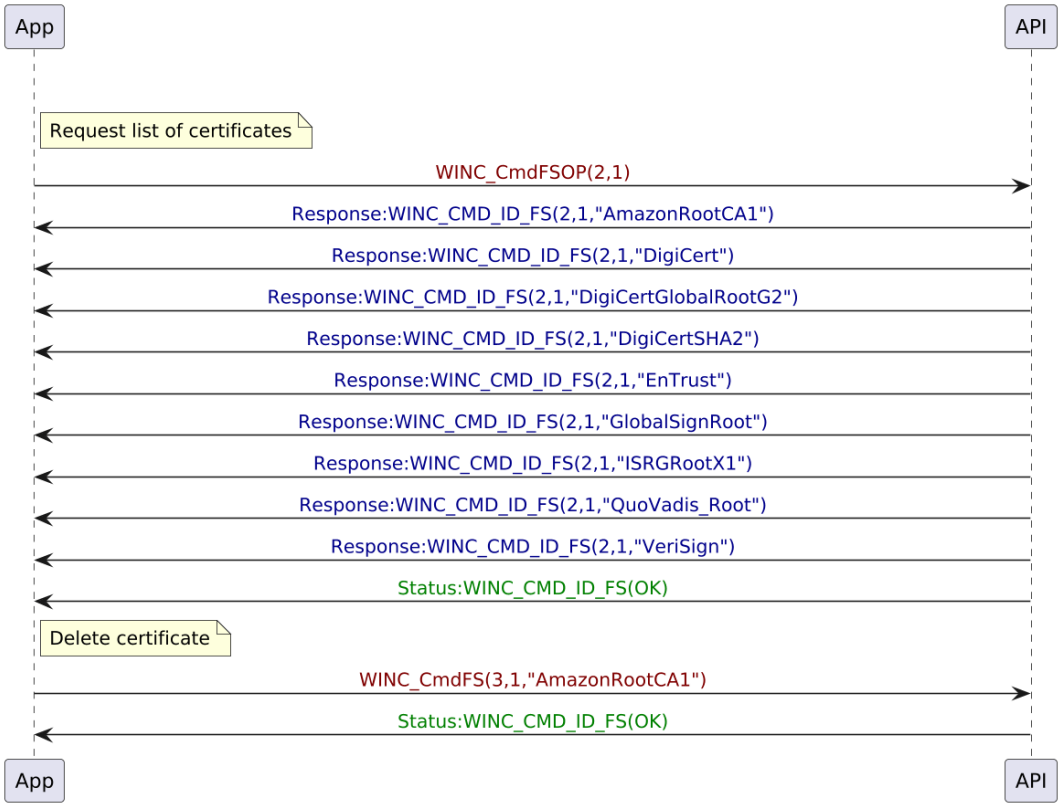


Figure 30. List and delete a file

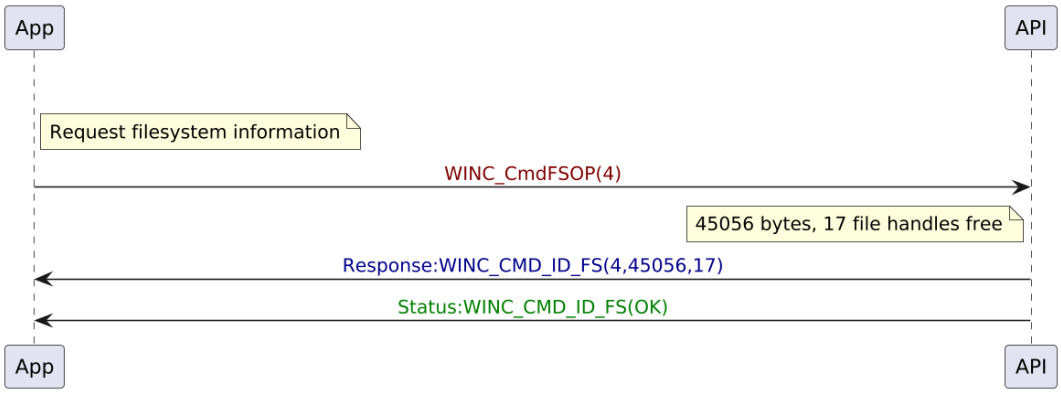


Figure 31. Get filesystem information

 ©2025-2026

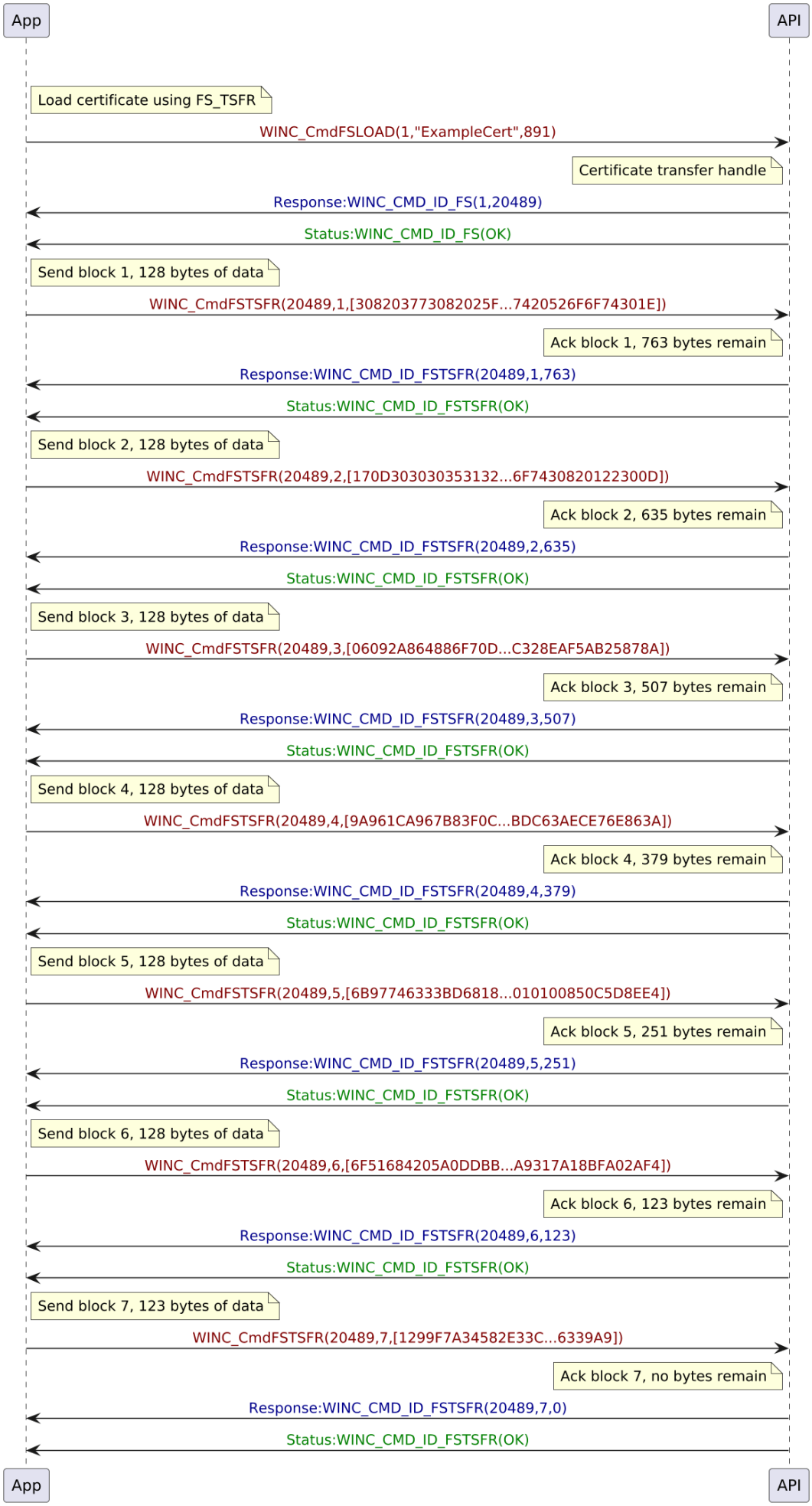


Figure 32. File load using FS-TSFR protocol

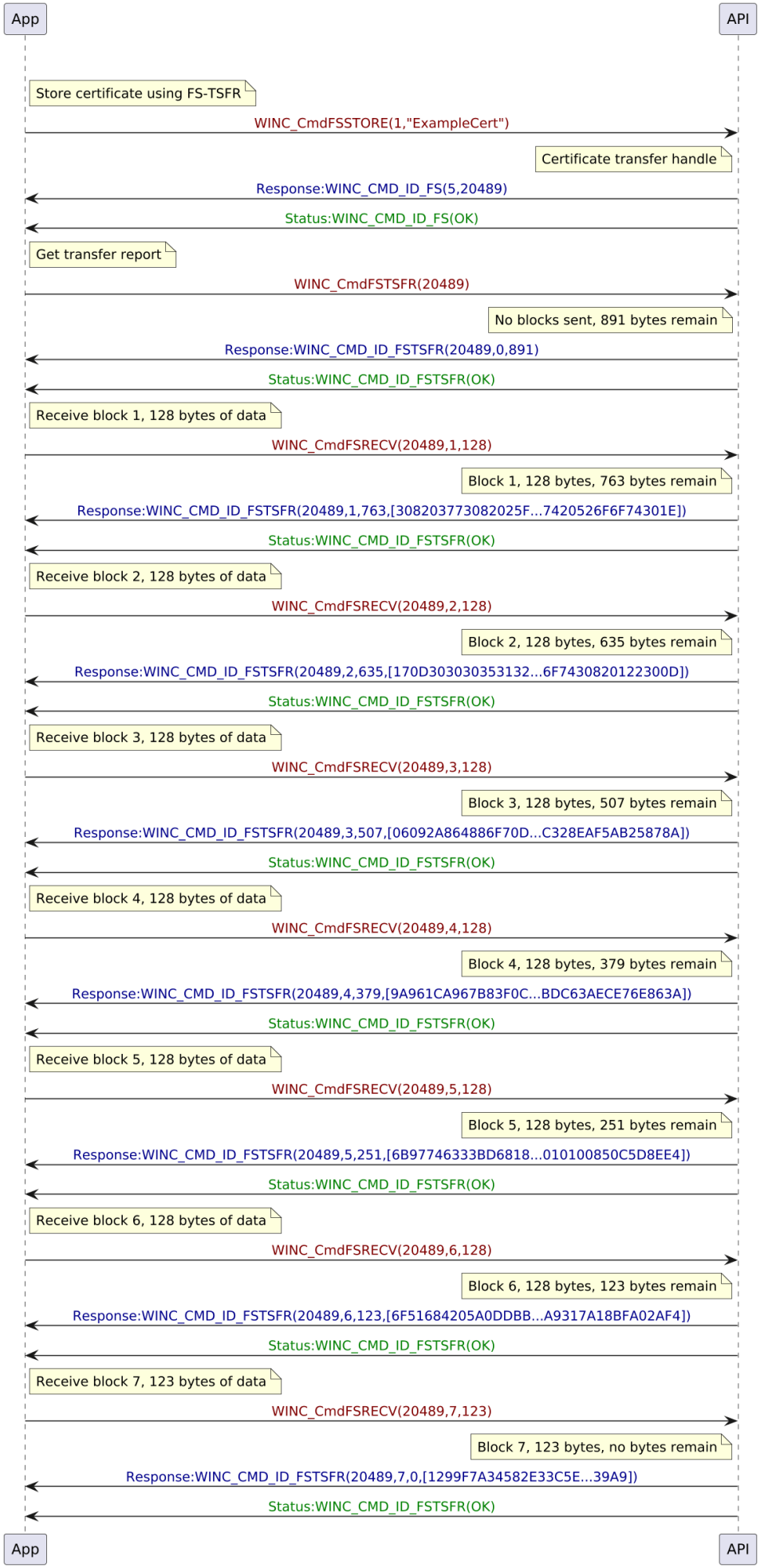


Figure 33. File store using FS-TSFR protocol

 ©2025-2026

MQTT (Module ID = 8)

Introduction

The WINCS02 supports MQTT v3.1.1 and MQTT v5.0 client functionality via its API, enabling communication with MQTT brokers for IoT applications, telemetry, messaging, and cloud integration.

Key Features

Protocol Support

Both MQTT v3.1.1 and MQTT v5.0 protocols are supported. MQTT v5.0 includes enhanced features such as properties, reason codes, and improved error reporting.

Quality of Service (QoS)

All three QoS levels are supported:

- QoS 0 (At most once) - Fire and forget, no acknowledgment
- QoS 1 (At least once) - Guaranteed delivery with acknowledgment
- QoS 2 (Exactly once) - Guaranteed single delivery with four-way handshake

TLS Security

Secure MQTT connections (MQTTS) over TLS are supported by configuring a TLS context before connecting.

Large Message Handling

The device can send and receive messages larger than its internal buffers through message fragmentation. The MQTT stack handles reassembly of multi-chunk messages automatically.

Connecting to a Broker

To establish an MQTT connection:

1. Configure the broker parameters using WINC_CmdMQTTC:
   - BROKER_ADDR - Broker hostname or IP address
   - BROKER_PORT - Broker port (typically 1883 for MQTT, 8883 for MQTTS)
   - CLIENT_ID - Unique client identifier
   - USERNAME and PASSWORD - Authentication credentials (if required)
2. For secure connections, configure TLS using WINC_CmdMQTTC parameter TLS_CONF to reference a TLS context configured via TLSC.
3. Connect to the broker using WINC_CmdMQTTCONN.
4. Monitor the WINC_AEC_ID_MQTTCONN AEC for connection status.

 ©2025-2026

Publishing Messages

Publish messages to topics using WINC_CmdMQTTPUB. The function accepts the topic name, QoS level, retain flag, and message payload.

Fragmented Publish Mode

For payloads exceeding the internal buffer limit, use fragmented publish to send large messages in multiple parts. The firmware reassembles fragments before transmitting a single MQTT PUBLISH packet.

Starting a Fragment Sequence (First Fragment)

Call WINC_CmdMQTTPUB with frag=1:

- frag=1 indicates the first fragment of a new sequence
- qos, retain, and topicNameId are specified only in the first fragment
- Topic can be a string (`WINC_TYPE_STRING`) or an integer topic alias (`WINC_TYPE_INTEGER`, MQTT v5.0)
- pTopicPayload contains the first fragment of the payload

Continuation Fragments

Call WINC_CmdMQTTPUBFRAG for all subsequent fragments:

- optFrag = fragment number (must match expected sequence, 2-64)
- optComp = 0 for continuation, 1 for final fragment
- pOptTopicPayload contains the fragment payload

Query Fragment State

Call WINC_CmdMQTTPUBFRAG with optFrag set to `WINC_CMDMQTTPUBFRAG_FRAG_IGNORE_VAL` to query the current fragment accumulation state.

Publish Responses

Responses to WINC_CmdMQTTPUB and WINC_CmdMQTTPUBFRAG are received with the response ID of WINC_CmdMQTTPUB:

- A single-message publish, or the final fragment, with QoS 0 produces no response; only the command status is received.
- With QoS 1 or 2, the packet ID is returned. The same packet ID appears in the subsequent WINC_AEC_ID_MQTTPUBACK or WINC_AEC_ID_MQTTPUBCOMP AEC.
- Each non-final fragment returns the next expected fragment number and the total bytes accumulated.
- A fragment state query returns the next expected fragment number, the total bytes accumulated and the topic name, or no response if no fragment sequence is in progress.

Fragmentation Limits

Constraint	Value	Notes
Maximum fragments	64	Per message
Maximum packet size	16,384 bytes (16KB)	Advertised protocol limit
Timeout	30 seconds	Between fragments

NOTE The 16KB limit applies to the total MQTT packet size, which includes the topic name, MQTT v5 properties (if enabled), and payload. With a short topic and no properties, payloads up to ~16KB are possible. Longer topics or additional properties reduce the available payload space.

Error Handling

- Out-of-sequence fragments are rejected and the context is cleaned up
- Starting a new sequence (frag=1) while one is in progress is rejected
- Continuation fragments (optFrag>1) without a prior first fragment are rejected
- Invalid QoS values (>2) are rejected before allocating context
- Exceeding the maximum fragment count or payload size returns an error

 ©2025-2026

Publish Completion Events

When a publish operation completes, the device emits an Asynchronous Event Code (AEC) to signal completion. This provides reliable synchronization for back-to-back publishes without requiring arbitrary delays.

QoS	AEC	Meaning
0	WINC_AEC_ID_MQTTPUBSENT	Publish sent to broker (fire-and-forget complete)
1	WINC_AEC_ID_MQTTPUBACK	PUBACK received from broker
2	WINC_AEC_ID_MQTTPUBCOMP	PUBCOMP received from broker (4-way handshake complete)

If a publish fails, WINC_AEC_ID_MQTTPUBERR is emitted.

IMPORTANT	Wait for the completion AEC before sending the next publish command. The device can only process one publish at a time. Sending a new publish while the previous one is in the PUBLISHING state will result in an error.

Subscribing and Receiving Messages

Subscribe to topics using WINC_CmdMQTTSUB. When messages arrive on subscribed topics, the WINC_AEC_ID_MQTTSUBRX AEC is emitted.

Inline vs Buffered Message Delivery

The READ_THRESHOLD parameter sets the largest message, in bytes, that is delivered inline. The default is 128 bytes.

- **Inline delivery**: a received message whose length is less than or equal to the threshold is included directly in the WINC_AEC_ID_MQTTSUBRX AEC.
- **Buffered delivery**: a received message longer than the threshold is held by the device. The WINC_AEC_ID_MQTTSUBRX AEC reports only the message ID and length, and the application must use WINC_CmdMQTTSUBRD to retrieve the data.

Messages received in more than one part are always buffered, whatever the threshold. Messages larger than approximately 1 KB will therefore always require WINC_CmdMQTTSUBRD.

NOTE	Setting the threshold to 0 causes all received messages to be buffered regardless of size, which is useful when consistent handling of all messages is required.

Maximum Packet Size (MQTT v5.0)

MQTT v5.0 introduces the Maximum Packet Size property (ID 39), which informs the broker of the largest MQTT packet the client can receive. This property is configured via WINC_CmdMQTTPROPTX and controlled by WINC_CmdMQTTPROPTXS.

Default Behaviour

- The Maximum Packet Size property defaults to 16384 bytes (16KB).
- The property is **enabled by default** and will be included in the CONNECT packet.
- The broker will respect this limit and will not send packets larger than the specified size.

Configuration Options

Action	Function Call
Query current value	`MQTTPROPTX(39)`
Set to 8KB	`MQTTPROPTX(39, 8192)`
Disable property in CONNECT	`MQTTPROPTXS(39, 0)`
Re-enable property in CONNECT	`MQTTPROPTXS(39, 1)`

Validation Rules

- The value cannot be set to 0 (zero).
- The value cannot exceed the system maximum of 16384 bytes.

Internal Enforcement

The device enforces the packet size limit internally regardless of broker behaviour:

- When the property is **enabled** in CONNECT: The configured value is enforced. The broker knows the limit and should not exceed it, but the device will reject oversized packets as a safety measure.
- When the property is **disabled** in CONNECT: The system maximum (16384 bytes) is enforced. Since the broker was not informed of any limit, the device uses its maximum capability.
- For **MQTT v3.1.1** connections: The system maximum (16384 bytes) is always enforced, as v3.1.1 does not support the Maximum Packet Size property.

NOTE If you configure a lower Maximum Packet Size value, then disable the property and connect to a different broker, the device will use the system maximum (16KB) for enforcement - not your previously configured value. This ensures the device accepts the largest packets possible when the broker has not been informed of any limit.

 ©2025-2026

Oversized Received Messages

A broker that does not honour the Maximum Packet Size property - or that was never told the limit because the property was disabled in CONNECT - may still forward a message larger than the device can buffer. Such a message cannot be delivered, but it also cannot simply be abandoned: the remainder of the payload would be left in the socket, leaving the MQTT byte stream out of frame.

The device therefore handles an undeliverable received message as follows:

1. The payload is read from the connection and discarded, keeping the MQTT stream framed and allowing the QoS 1/QoS 2 acknowledgement to be sent.
2. The WINC_AEC_ID_MQTTSUBRXERR AEC is emitted, reporting why the message was dropped along with the topic, packet identifier and total message length.
3. Once the payload has been fully drained, the session is closed cleanly with an MQTT v5.0 DISCONNECT carrying reason code `0x95` (Packet too large) for an oversized message, or `0x83` (Implementation specific error) for the other drop reasons.

The reported reasons are:

REASON	Meaning
1	The message exceeds the maximum packet size.
2	There was insufficient memory to buffer the message.
3	The message did not match any active subscription.
4	The message length was inconsistent with the receive buffer.

No data from a dropped message is delivered, and subsequent messages are unaffected. The session must be re-established after the disconnection, and any subscriptions re-created, as the subscription list is cleared when the session closes.

NOTE `MQTTPROPTXS(39, 1)` (the default) is the way to avoid this situation altogether: when the broker is told the limit, it will not forward an oversized message in the first place. `MQTTPROPTX(39, <VALUE>)` lowers the advertised limit below the 16KB system maximum if the application cannot handle messages that large.

MQTT v5.0 Properties

MQTT v5.0 supports additional properties that can be included in various packet types. Properties are managed using:

- WINC_CmdMQTTPROPTX - Configure property values to transmit
- WINC_CmdMQTTPROPTXS - Select which properties to include in transmitted packets
- WINC_CmdMQTTPROPRX - Query properties received from the broker

Properties are only added to packets where they are applicable according to the MQTT v5.0 specification.

 ©2025-2026

Typical Usage Flow

A typical MQTT session follows this pattern:

1. Configure Wi-Fi and establish network connectivity
2. (Optional) Configure SNTP for accurate timestamps
3. Configure TLS context if using secure connection
4. Configure MQTT client parameters
5. (Optional) Adjust MQTT v5.0 properties
6. Connect to the broker
7. Subscribe to topics of interest
8. Publish messages and/or receive subscribed messages
9. Disconnect when finished

Troubleshooting

Connection Failures

Ensure the broker URL, port, and credentials are correct. For TLS connections, verify the CA certificate is loaded and the TLS context is properly configured.

Message Size Errors

If publishing fails, check that the hex-encoded payload plus command overhead does not exceed the command line limit. For received messages that appear truncated, check the Maximum Packet Size configuration.

QoS 2 Timeouts

QoS 2 requires a complete four-way handshake. Ensure network connectivity is stable and the broker supports QoS 2.

 ©2025-2026

Function Reference:

WINC_CmdMQTTC

Description

This function is used to read or set the MQTT configuration.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdMQTTC
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDMQTTC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

 ©2025-2026

Configuration Parameters (WINC_CFG_PARAM_ID_MQTT_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset **Value** / **Label** / **Description** 0 / DEFAULT / Default.
1	BROKER_ADDR	STRING IPV4ADDR IPV6ADDR	Broker domain name or IP address Maximum length of string is 256 Format of IPv4 address is A1|A2|A3|A4 = A1.A2.A3.A4 Format of IPv6 address is A1|…|A16 = A1A2::A15A16
2	BROKER_PORT	INTEGER	Broker listening TCP port Unsigned 16-bit value
3	CLIENT_ID	STRING	MQTT client ID Maximum length of string is 64
4	USERNAME	STRING	Username Maximum length of string is 128
5	PASSWORD	STRING	Password Maximum length of string is 256
6	KEEP_ALIVE	INTEGER	Keep alive time (in seconds) Valid range is 0 to 0x7FFF
7	TLS_CONF	INTEGER	TLS configuration index (see WINC_CmdTLSCDA). 0 disables TLS Valid range is 0 to 4 This parameter will only be visible to read if not zero. Setting a value greater than zero enables TLS and makes this parameter visible. Setting a value of zero disables TLS and makes this parameter hidden.
8	PROTO_VER	INTEGER_UNSIGNED	MQTT protocol version, either 3 or 5 **Value** / **Label** / **Description** 3 / V3_1_1 / V3.1.1. 5 / V5 / V5.
9	READ_THRESHOLD	INTEGER_UNSIGNED	Largest received message size (in bytes) delivered inline, larger messages are buffered. Default is 128 Unsigned 16-bit value
10	SOCKET_ID	INTEGER_UNSIGNED	Socket ID

WINC_CmdMQTTCONN

Description

This function is used to connect to an MQTT broker.

When called with the clean parameter set to WINC_CMDMQTTCONN_CLEAN_IGNORE_VAL, the current connection state is reported asynchronously using the WINC_AEC_ID_MQTTCONN AEC, rather than as a command response.

```
bool WINC_CmdMQTTCONN
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optClean
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optClean	int32_t	Clean Session. **Value** / **Label** / **Description** 0 / USE_EXIST / Use existing session, if available. 1 / USE_NEW / Use new session. optClean will be ignored if its value is WINC_CMDMQTTCONN_CLEAN_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

WINC_CmdMQTTSUB

Description

This function is used to subscribe to an MQTT topic.

```
bool WINC_CmdMQTTSUB
(
    WINC_CMD_REQ_HANDLE handle
    const uint8_t* pTopicName
    size_t lenTopicName
    uint8_t maxQos
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
pTopicName	const uint8_t*	Topic Name.
lenTopicName	size_t	Length of pTopicName in bytes, strings are not NUL terminated so the length must not include a terminator.
maxQos	uint8_t	Maximum QoS. **Value** / **Label** / **Description** 0 / QOS0 / QoS 0. 1 / QOS1 / QoS 1. 2 / QOS2 / QoS 2.

Return Values

Value	Description
true	Success.
false	Failure.

WINC_CmdMQTTSUBLST

Description

This function is used to list MQTT topic subscriptions.

```
bool WINC_CmdMQTTSUBLST(WINC_CMD_REQ_HANDLE handle)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<TOPIC_NAME>,<QOS>	Subscription report

Response Element Description

Element Name	Type	Description
<TOPIC_NAME>	STRING	Topic Name
<QOS>	INTEGER	QoS **Value** / **Label** / **Description** 0 / QOS0 / QoS 0. 1 / QOS1 / QoS 1. 2 / QOS2 / QoS 2.

 ©2025-2026

WINC_CmdMQTTSUBRD

Description

This function is used to receive data from subscriptions.

The WINC_CmdMQTTSUBRD function is used in conjunction with the setting of READ_THRESHOLD.

When a message is received the API will emit a WINC_AEC_ID_MQTTSUBRX AEC, this will normally include the received data, however when a message is received which would exceed READ_THRESHOLD the API will emit a WINC_AEC_ID_MQTTSUBRX AEC which includes a message ID and data length. This avoids saturating the application with data it may not be able to handle. The application must use this function to read the buffered message.

Setting READ_THRESHOLD to zero will cause all received messages to be buffered for reading using this function.

Buffered messages are read in the order they were received; only the oldest buffered message for the topic can be read, and requesting a different message ID returns an error.

A message may be read in several parts. If <LENGTH> is less than the remaining message data, only that many bytes are returned, and the next read of the same message ID continues from that point. The message is released once all of its data has been read. If no message is buffered for the topic, no data is returned.

The length element of the response is the number of bytes returned by this read.

NOTE The message ID is only relevant for QoS1 and QoS2 subscriptions.

```
bool WINC_CmdMQTTSUBRD
(
    WINC_CMD_REQ_HANDLE handle
    const uint8_t* pTopicName
    size_t lenTopicName
    uint16_t msgId
    uint16_t length
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
pTopicName	const uint8_t*	Topic Name.
lenTopicName	size_t	Length of pTopicName in bytes, strings are not NUL terminated so the length must not include a terminator.
msgId	uint16_t	Message Identifier.
length	uint16_t	The number of bytes to receive.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<MSG_LENGTH>,<TOPIC_PAYLOAD>	Read subscribed data

Response Element Description

Element Name	Type	Description
<MSG_LENGTH>	INTEGER	Length of message
<TOPIC_PAYLOAD>	STRING	Topic Payload

 ©2025-2026

WINC_CmdMQTTUNSUB

Description

This function is used to unsubscribe from an MQTT topic.

```
bool WINC_CmdMQTTUNSUB
(
    WINC_CMD_REQ_HANDLE handle
    const uint8_t* pTopicName
    size_t lenTopicName
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
pTopicName	const uint8_t*	Topic Name.
lenTopicName	size_t	Length of pTopicName in bytes, strings are not NUL terminated so the length must not include a terminator.

Return Values

Value	Description
true	Success.
false	Failure.

WINC_CmdMQTTPUB

Description

Publish a message (single or fragmented).

```
bool WINC_CmdMQTTPUB
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t frag
    uint8_t qos
    uint8_t retain
    WINC_TYPE typeTopicNameId
    uintptr_t topicNameId
    size_t lenTopicNameId
    const uint8_t* pTopicPayload
    size_t lenTopicPayload
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
frag	uint8_t	Fragment Number (0=complete, 1=first, 2+=continuation).
qos	uint8_t	QoS. **Value** / **Label** / **Description** 0 / QOS0 / QoS 0. 1 / QOS1 / QoS 1. 2 / QOS2 / QoS 2.
retain	uint8_t	Retain Message. **Value** / **Label** / **Description** 0 / NOT_RETAINED / Not retained on the server. 1 / RETAINED / Retained on the server.
typeTopicNameId	WINC_TYPE	Type of topicNameId, topicNameId is not sent if the type is WINC_TYPE_INVALID.
topicNameId	uintptr_t	Topic Name or Alias. For integer types topicNameId holds the value, for other types topicNameId is a pointer to the data.
lenTopicNameId	size_t	Length in bytes of the data topicNameId points to, ignored for integer types.
pTopicPayload	const uint8_t*	Topic Payload.
lenTopicPayload	size_t	Length of pTopicPayload in bytes.

 ©2025-2026

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<PACKET_ID>	Publish (QoS 1 & 2)
<FRAG>,<TOTAL_LEN>	Fragment accumulation feedback
<FRAG>,<TOTAL_LEN>,<TOPIC_NAME>	Fragment state query

Response Element Description

Element Name	Type	Description
<PACKET_ID>	INTEGER	Packet ID
<FRAG>	INTEGER	Next expected fragment number
<TOTAL_LEN>	INTEGER	Total accumulated bytes
<TOPIC_NAME>	STRING	Topic Name

WINC_CmdMQTTPUBFRAG

Description

Publish a message (single or fragmented).

```
bool WINC_CmdMQTTPUBFRAG
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optFrag
    int32_t optComp
    const uint8_t* pOptTopicPayload
    size_t lenOptTopicPayload
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optFrag	int32_t	Fragment Number (0=complete, 1=first, 2+=continuation). optFrag will be ignored if its value is WINC_CMDMQTTPUBFRAG_FRAG_IGNORE_VAL.
optComp	int32_t	Completion Flag. **Value** / **Label** / **Description** 0 / MORE / More fragments follow. 1 / FINAL / Final fragment, publish now. optComp will be ignored if its value is WINC_CMDMQTTPUBFRAG_COMP_IGNORE_VAL.
pOptTopicPayload	const uint8_t*	Topic Payload. pOptTopicPayload will be ignored if its value is NULL.
lenOptTopicPayload	size_t	Length of pOptTopicPayload in bytes.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

Response Format

Response	Description
<PACKET_ID>	Publish (QoS 1 & 2)
<FRAG>,<TOTAL_LEN>	Fragment accumulation feedback
<FRAG>,<TOTAL_LEN>,<TOPIC_NAME>	Fragment state query

Response Element Description

Element Name	Type	Description
<PACKET_ID>	INTEGER	Packet ID
<FRAG>	INTEGER	Next expected fragment number
<TOTAL_LEN>	INTEGER	Total accumulated bytes
<TOPIC_NAME>	STRING	Topic Name

WINC_CmdMQTTLWT

Description

This function is used to define a last will message.

```
bool WINC_CmdMQTTLWT
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t qos
    uint8_t retain
    const uint8_t* pTopicName
    size_t lenTopicName
    const uint8_t* pTopicPayload
    size_t lenTopicPayload
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
qos	uint8_t	QoS. **Value** / **Label** / **Description** 0 / QOS0 / QoS 0. 1 / QOS1 / QoS 1. 2 / QOS2 / QoS 2.
retain	uint8_t	Retain Message. **Value** / **Label** / **Description** 0 / NOT_RETAINED / Not retained on the server. 1 / RETAINED / Retained on the server.
pTopicName	const uint8_t*	Topic Name.
lenTopicName	size_t	Length of pTopicName in bytes, strings are not NUL terminated so the length must not include a terminator.
pTopicPayload	const uint8_t*	Topic Payload.
lenTopicPayload	size_t	Length of pTopicPayload in bytes.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

WINC_CmdMQTTDISCONN

Description

This function is used to disconnect from a broker.

```
bool WINC_CmdMQTTDISCONN
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optReasonCode
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optReasonCode	int32_t	Reason Code. optReasonCode will be ignored if its value is WINC_CMDMQTTDISCONN_REASON_CODE_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

WINC_CmdMQTTPROPTX

Description

This function is used to read or set the MQTT transmit properties.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

Any defined and selected properties are added to transmitted MQTT packets. Only properties applicable to the MQTT packet type are added. Properties must be selected via the WINC_CmdMQTTPROPTXS function before they are included.

Property names and identifiers are as defined in the MQTT Version 5.0 specification (OASIS), section 2.2.2.2.

NOTE Not all properties listed below may be available in any implementation of this command set.

User property (38) supports multiple entries in the form of key/value pairs. If a key/value pair is specified for the command it will be added, if only a key is specified it will be removed.

```
bool WINC_CmdMQTTPROPTX
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDMQTTPROPTX_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response
<ID>,<KEY>,<VALS>	Read response (key/value)

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value
<KEY>	STRING	Parameter key
<VALS>	STRING	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_MQTT_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset **Value** / **Label** / **Description** 0 / DEFAULT / Default.
1	PAYLOAD_FORMAT_IND	INTEGER_UNSIGNED	Payload Format Indicator
2	MSG_EXPIRY_INTERVAL	INTEGER_UNSIGNED	Message Expiry Interval
3	CONTENT_TYPE	STRING	Content Type Maximum length of string is 64
8	RESP_TOPIC	STRING	Response Topic Maximum length of string is 64
9	CORRELATION_DATA	BYTE_ARRAY	Correlation Data
11	SUBSCRIPTION_ID	INTEGER_UNSIGNED	Subscription Identifier
17	SESSION_EXPIRY_INTERVAL	INTEGER_UNSIGNED	Session Expiry Interval
23	REQ_PROB_INFO	INTEGER_UNSIGNED	Request Problem Information
24	WILL_DELAY_INTERVAL	INTEGER_UNSIGNED	Will Delay Interval
25	REQ_RESP_INFO	INTEGER_UNSIGNED	Request Response Information
31	REASON_STR	STRING	Reason String Maximum length of string is 64
33	RECEIVE_MAX	INTEGER_UNSIGNED	Receive Maximum Unsigned 16-bit value
34	TOPIC_ALIAS_MAX	INTEGER_UNSIGNED	Topic Alias Maximum Unsigned 16-bit value
35	TOPIC_ALIAS	INTEGER_UNSIGNED	Topic Alias Unsigned 16-bit value
39	MAX_PACKET_SZ	INTEGER_UNSIGNED	Maximum Packet Size

 ©2025-2026

WINC_CmdMQTTPROPTXKV

Description

This function is used to read or set the MQTT transmit properties.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdMQTTPROPTXKV
(
    WINC_CMD_REQ_HANDLE handle
    const uint8_t* pKey
    size_t lenKey
    const uint8_t* pOptVals
    size_t lenOptVals
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
pKey	const uint8_t*	Parameter key.
lenKey	size_t	Length of pKey in bytes, strings are not NUL terminated so the length must not include a terminator.
pOptVals	const uint8_t*	Parameter value. pOptVals will be ignored if its value is NULL.
lenOptVals	size_t	Length of pOptVals in bytes, strings are not NUL terminated so the length must not include a terminator.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response
<ID>,<KEY>,<VALS>	Read response (key/value)

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value
<KEY>	STRING	Parameter key
<VALS>	STRING	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_MQTT_*)

ID	Name	Type	Description
38	USER_PROP	BYTE_ARRAY	User Property This is a multiple value parameter with an ID range 38.0 to 38.9

 ©2025-2026

WINC_CmdMQTTPROPRX

Description

This function is used to read or reset the MQTT receive properties.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

Any defined properties are added to transmitted MQTT packets. Only properties applicable to the MQTT packet type are added.

Property names and identifiers are as defined in the MQTT Version 5.0 specification (OASIS), section 2.2.2.2.

Any properties received are added to the list of available receive properties which can be queried by this command. Updates to the received properties are announced using the WINC_AEC_ID_MQTTPROPRX AEC and can be read via this command.

NOTE	Not all properties listed below may be available in any implementation of this command set.

User property (38) supports multiple entries in the form of key/value pairs.

```
bool WINC_CmdMQTTPROPRX
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optId
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDMQTTPROPRX_ID_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response
<ID>,<KEY>,<VALS>	Read response (key/value)

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value
<KEY>	STRING	Parameter key
<VALS>	STRING	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_MQTT_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset **Value** / **Label** / **Description** 0 / DEFAULT / Default.
1	PAYLOAD_FORMAT_IND	INTEGER_UNSIGNED (Read Only)	Payload Format Indicator
2	MSG_EXPIRY_INTERVAL	INTEGER_UNSIGNED (Read Only)	Message Expiry Interval
3	CONTENT_TYPE	STRING (Read Only)	Content Type Maximum length of string is 64
8	RESP_TOPIC	STRING (Read Only)	Response Topic Maximum length of string is 64
9	CORRELATION_DATA	BYTE_ARRAY (Read Only)	Correlation Data
11	SUBSCRIPTION_ID	INTEGER_UNSIGNED (Read Only)	Subscription Identifier
17	SESSION_EXPIRY_INTERVAL	INTEGER_UNSIGNED (Read Only)	Session Expiry Interval
18	ASSIGNED_CLIENT_ID	STRING (Read Only)	Assigned Client Identifier Maximum length of string is 64
19	SERVER_KEEP_ALIVE	INTEGER_UNSIGNED (Read Only)	Server Keep Alive
23	REQ_PROB_INFO	INTEGER_UNSIGNED (Read Only)	Request Problem Information
25	REQ_RESP_INFO	INTEGER_UNSIGNED (Read Only)	Request Response Information
26	RESP_INFO	STRING (Read Only)	Response Information Maximum length of string is 64
28	SERVER_REF	STRING (Read Only)	Server Reference Maximum length of string is 64
31	REASON_STR	STRING (Read Only)	Reason String Maximum length of string is 64

 ©2025-2026

ID	Name	Type	Description
33	RECEIVE_MAX	INTEGER_UNSIGNED (Read Only)	Receive Maximum Unsigned 16-bit value
34	TOPIC_ALIAS_MAX	INTEGER_UNSIGNED (Read Only)	Topic Alias Maximum Unsigned 16-bit value
35	TOPIC_ALIAS	INTEGER_UNSIGNED (Read Only)	Topic Alias Unsigned 16-bit value
36	MAX_QOS	INTEGER_UNSIGNED (Read Only)	Maximum QoS Valid range is 0 to 2
37	RETAIN_AVAIL	INTEGER_UNSIGNED (Read Only)	Retain Available
38	USER_PROP	BYTE_ARRAY (Read Only)	User Property This is a multiple value parameter with an ID range 38.0 to 38.9
39	MAX_PACKET_SZ	INTEGER_UNSIGNED (Read Only)	Maximum Packet Size
40	WILDCARD_SUB_AVAIL	INTEGER_UNSIGNED (Read Only)	Wildcard Subscription Available
41	SUBSCRIPTION_ID_AVAIL	INTEGER_UNSIGNED (Read Only)	Subscription Identifier Available
42	SHARED_SUBSCRIPTION_AVAIL	INTEGER_UNSIGNED (Read Only)	Shared Subscription Available

WINC_CmdMQTTPROPRXPRESET

Description

This function is used to read or reset the MQTT receive properties.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdMQTTPROPRXPRESET
(
    WINC_CMD_REQ_HANDLE handle
    WINC_TYPE typeVal
    uintptr_t val
    size_t lenVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
typeVal	WINC_TYPE	Type of val, val is not sent if the type is WINC_TYPE_INVALID.
val	uintptr_t	Parameter value. For integer types val holds the value, for other types val is a pointer to the data.
lenVal	size_t	Length in bytes of the data val points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response
<ID>,<KEY>,<VALS>	Read response (key/value)

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value
<KEY>	STRING	Parameter key
<VALS>	STRING	Parameter value

 ©2025-2026

Configuration Parameters (WINC_CFG_PARAM_ID_MQTT_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset **Value** / **Label** / **Description** 0 / DEFAULT / Default.

WINC_CmdMQTTPROPTXS

Description

This function is used to define which transmit properties are selected.

```
bool WINC_CmdMQTTPROPTXS
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optPropId
    int32_t optPropSel
)
```

 ©2025-2026

Function Parameter Syntax

<table>
<tr><th>Name</th><th>Type</th><th>Description</th></tr>
<tr><td>handle</td><td>WINC_CMD_REQ_HANDLE</td><td>Command request session handle.</td></tr>
<tr><td>optPropId</td><td>int32_t</td><td>Property Identifier.

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>ALL</td><td>All properties.</td></tr><tr><td>1</td><td>PAYLOAD_FORMAT_IND</td><td>Payload Format Indicator.</td></tr><tr><td>2</td><td>MSG_EXPIRY_INTERVAL</td><td>Message Expiry Interval.</td></tr><tr><td>3</td><td>CONTENT_TYPE</td><td>Content Type.</td></tr><tr><td>8</td><td>RESP_TOPIC</td><td>Response Topic.</td></tr><tr><td>9</td><td>CORRELATION_DATA</td><td>Correlation Data.</td></tr><tr><td>11</td><td>SUBSCRIPTION_ID</td><td>Subscription Identifier.</td></tr><tr><td>17</td><td>SESSION_EXPIRY_INTERVAL</td><td>Session Expiry Interval.</td></tr><tr><td>23</td><td>REQ_PROB_INFO</td><td>Request Problem Information.</td></tr><tr><td>24</td><td>WILL_DELAY_INTERVAL</td><td>Will Delay Interval.</td></tr><tr><td>25</td><td>REQ_RESP_INFO</td><td>Request Response Information.</td></tr><tr><td>31</td><td>REASON_STR</td><td>Reason String.</td></tr><tr><td>33</td><td>RECEIVE_MAX</td><td>Receive Maximum.</td></tr><tr><td>34</td><td>TOPIC_ALIAS_MAX</td><td>Topic Alias Maximum.</td></tr><tr><td>35</td><td>TOPIC_ALIAS</td><td>Topic Alias.</td></tr><tr><td>38</td><td>USER_PROP</td><td>User Property.</td></tr><tr><td>39</td><td>MAX_PACKET_SZ</td><td>Maximum Packet Size.</td></tr></table>
optPropId will be ignored if its value is WINC_CMDMQTTPROPTXS_PROP_ID_IGNORE_VAL.</td></tr>
<tr><td>optPropSel</td><td>int32_t</td><td>Property Selected.

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>PROP_NOT_SEL</td><td>Property not selected.</td></tr><tr><td>1</td><td>PROP_SEL</td><td>Property is selected.</td></tr></table>
optPropSel will be ignored if its value is WINC_CMDMQTTPROPTXS_PROP_SEL_IGNORE_VAL.</td></tr>
</table>

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<PROP_ID>,<PROP_SEL>	Read response

Response Element Description

Element Name	Type	Description
<PROP_ID>	INTEGER	Property Identifier
<PROP_SEL>	INTEGER	Property Selected Value / Label / Description 0 / NOT_SELECTED / Property not selected. 1 / SELECTED / Property is selected.

 ©2025-2026

AEC Reference:

WINC_AEC_ID_MQTTCONN

Description

Connection state.

AEC Syntax

AEC	Description
<CONN_STATE>	Connection state

AEC Element Syntax

Element Name	Type	Description
<CONN_STATE>	INTEGER	MQTT connected state **Value** / **Label** / **Description** 0 / NOT_CONNECTED / Not connected. 1 / CONNECTED / Connected.

WINC_AEC_ID_MQTTCONNACK

Description

Connection acknowledge.

AEC Syntax

AEC	Description
<CONNACK_FLAGS>,<CONN_REASON_CODE>	Connection state

 ©2025-2026

AEC Element Syntax

<table>
<tr><th>Element Name</th><th>Type</th><th>Description</th></tr>
<tr><td><CONNACK_FLAGS></td><td>INTEGER</td><td>Connect acknowledge flags

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>NO_SESSION</td><td>No session.</td></tr><tr><td>1</td><td>SESSION_PRESENT</td><td>Session present.</td></tr></table></td></tr>
<tr><td><CONN_REASON_CODE></td><td>INTEGER</td><td>Connect reason code

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>SUCCESS</td><td>Success.</td></tr><tr><td>128</td><td>ERROR</td><td>Unspecified error.</td></tr><tr><td>129</td><td>MALFORM_PKT</td><td>Malformed packet.</td></tr><tr><td>130</td><td>PROT_ERR</td><td>Protocol error.</td></tr><tr><td>131</td><td>IMP_SPEC_ERR</td><td>Implementation specific error.</td></tr><tr><td>132</td><td>PROT_NA</td><td>Unsupported protocol version.</td></tr><tr><td>133</td><td>INVALID_CLIENT_ID</td><td>Client identifier not valid.</td></tr><tr><td>134</td><td>BAD_USER_PASS</td><td>Bad username or password.</td></tr><tr><td>135</td><td>NOT_AUTH</td><td>Not authorized.</td></tr><tr><td>136</td><td>SRV_NA</td><td>Server unavailable.</td></tr><tr><td>137</td><td>SRV_BUSY</td><td>Server busy.</td></tr><tr><td>138</td><td>BANNED</td><td>Banned.</td></tr><tr><td>140</td><td>BAD_AUTH</td><td>Bad authentication method.</td></tr><tr><td>144</td><td>TOP_INVALID</td><td>Topic name invalid.</td></tr><tr><td>149</td><td>PKT_TOO_LARGE</td><td>Packet too large.</td></tr><tr><td>151</td><td>QUOTA_EXCEED</td><td>Quota exceeded.</td></tr><tr><td>153</td><td>PAYLOAD_INVALID</td><td>Payload format invalid.</td></tr><tr><td>154</td><td>RETAIN_NA</td><td>Retain not supported.</td></tr><tr><td>155</td><td>QOS_NA</td><td>QoS not supported.</td></tr><tr><td>156</td><td>USE_DIFF_SRV</td><td>Use another server.</td></tr><tr><td>157</td><td>SRV_MOVED</td><td>Server moved.</td></tr><tr><td>159</td><td>CONN_RATE_EXCEED</td><td>Connection rate exceeded.</td></tr></table></td></tr>
</table>

WINC_AEC_ID_MQTTPUBACK

Description

Publish acknowledge.

AEC Syntax

AEC	Description
<PACKET_ID>	Publish acknowledge

AEC Element Syntax

Element Name	Type	Description
<PACKET_ID>	INTEGER	Packet ID

 ©2025-2026

WINC_AEC_ID_MQTTPUBCOMP

Description

Publish complete.

AEC Syntax

AEC	Description
<PACKET_ID>	Publish complete

AEC Element Syntax

Element Name	Type	Description
<PACKET_ID>	INTEGER	Packet ID

WINC_AEC_ID_MQTTPUBERR

Description

Publish error.

AEC Syntax

AEC	Description
<PACKET_ID>	Publish error

AEC Element Syntax

Element Name	Type	Description
<PACKET_ID>	INTEGER	Packet ID

 ©2025-2026

WINC_AEC_ID_MQTTSUB

Description

Subscribe response.

AEC Syntax

AEC	Description
<REASON_CODE>	Subscribe response

AEC Element Syntax

Element Name	Type	Description
<REASON_CODE>	INTEGER	SUBACK return code

Value	Label	Description
0	QOS0_GRANTED	Granted QoS 0.
1	QOS1_GRANTED	Granted QoS 1.
2	QOS2_GRANTED	Granted QoS 2.
128	ERROR	Unspecified error.
131	IMP_SPEC_ERR	Implementation specific error.
135	NOT_AUTH	Not authorized.
143	TOP_FILT_INVALID	Topic filter invalid.
145	PKT_ID_IN_USE	Packet identifier in use.
151	QUOTA_EXCEED	Quota exceeded.
158	SHARED_SUB_NOT_SUP	Shared Subscriptions not supported.
161	SUB_IDENT_NA	Subscription identifiers not supported.
162	WILDCARD_SUB_NA	Wildcard subscriptions not supported.

WINC_AEC_ID_MQTTUNSUB

Description

Unsubscribe response.

AEC Syntax

AEC	Description
<RESULT>	Unsubscribe response

AEC Element Syntax

Element Name	Type	Description
<RESULT>	INTEGER	Result of unsubscribe request **Value** / **Label** / **Description** 0 / SUCCESS / Success. 15 / NO_SUB / No subscription existed. 128 / ERROR / Unspecified error. 131 / IMP_SPEC_ERR / Implementation specific error. 135 / NOT_AUTH / Not authorized. 143 / TOP_FILT_INVALID / Topic filter invalid. 145 / PKT_ID_IN_USE / Packet identifier in use.

 ©2025-2026

WINC_AEC_ID_MQTTSUBRX

Description

Receive subscribed data.

AEC Syntax

AEC	Description
<DUP>,<QOS>,<RETAIN>,<TOPIC_NAME>,<TOPIC_PAYLOAD>	Received subscribed data
<DUP>,<QOS>,<RETAIN>,<TOPIC_NAME>,<MSG_ID>,<MSG_LENGTH>	Received subscribed data notification

AEC Element Syntax

Element Name	Type	Description
<DUP>	INTEGER	Duplicate Message **Value** / **Label** / **Description** 0 / NEW_MSG / New message. 1 / DUP_MSG / Duplicate message.
<QOS>	INTEGER	QoS **Value** / **Label** / **Description** 0 / QOS0 / QoS 0. 1 / QOS1 / QoS 1. 2 / QOS2 / QoS 2.
<RETAIN>	INTEGER	Retain Message **Value** / **Label** / **Description** 0 / NOT_RETAINED / Not retained on the server. 1 / RETAINED / Retained on the server.
<TOPIC_NAME>	STRING	Topic Name
<TOPIC_PAYLOAD>	STRING	Topic Payload
<MSG_ID>	INTEGER	Message ID
<MSG_LENGTH>	INTEGER	Length of message

WINC_AEC_ID_MQTTPROPRX

Description

Indicates property has been updated.

AEC Syntax

AEC	Description
<PROP_ID>	Property ID updated
<PROP_ID>,<VAL>	Property ID updated, with value
<PROP_ID>,<KEY>,<VALS>	Property ID updated, with key/value pair

AEC Element Syntax

Element Name	Type	Description
<PROP_ID>	INTEGER	Property Identifier
<VAL>	Any	Parameter value
<KEY>	STRING	Parameter key
<VALS>	STRING	Parameter value

 ©2025-2026

WINC_AEC_ID_MQTTPUBSENT

Description

Publish sent (QoS 0).

AEC Syntax

AEC	Description
No Elements	Publish sent (QoS 0)

WINC_AEC_ID_MQTTSUBRXERR

Description

Received subscribed data was dropped.

AEC Syntax

AEC	Description
<REASON>,<TOPIC_NAME>,<MSG_ID>,<MSG_LENGTH>	Received message dropped

AEC Element Syntax

Element Name	Type	Description
<REASON>	INTEGER	Reason the received message was dropped Value / Label / Description: 1 / TOO_LARGE / Message exceeds the maximum packet size. 2 / NO_MEMORY / Insufficient memory to buffer the message. 3 / NO_SUB / No matching subscription. 4 / BAD_LENGTH / Message length inconsistent with buffer.
<TOPIC_NAME>	STRING	Topic Name
<MSG_ID>	INTEGER	Message ID
<MSG_LENGTH>	INTEGER	Length of message

 ©2025-2026

Examples:

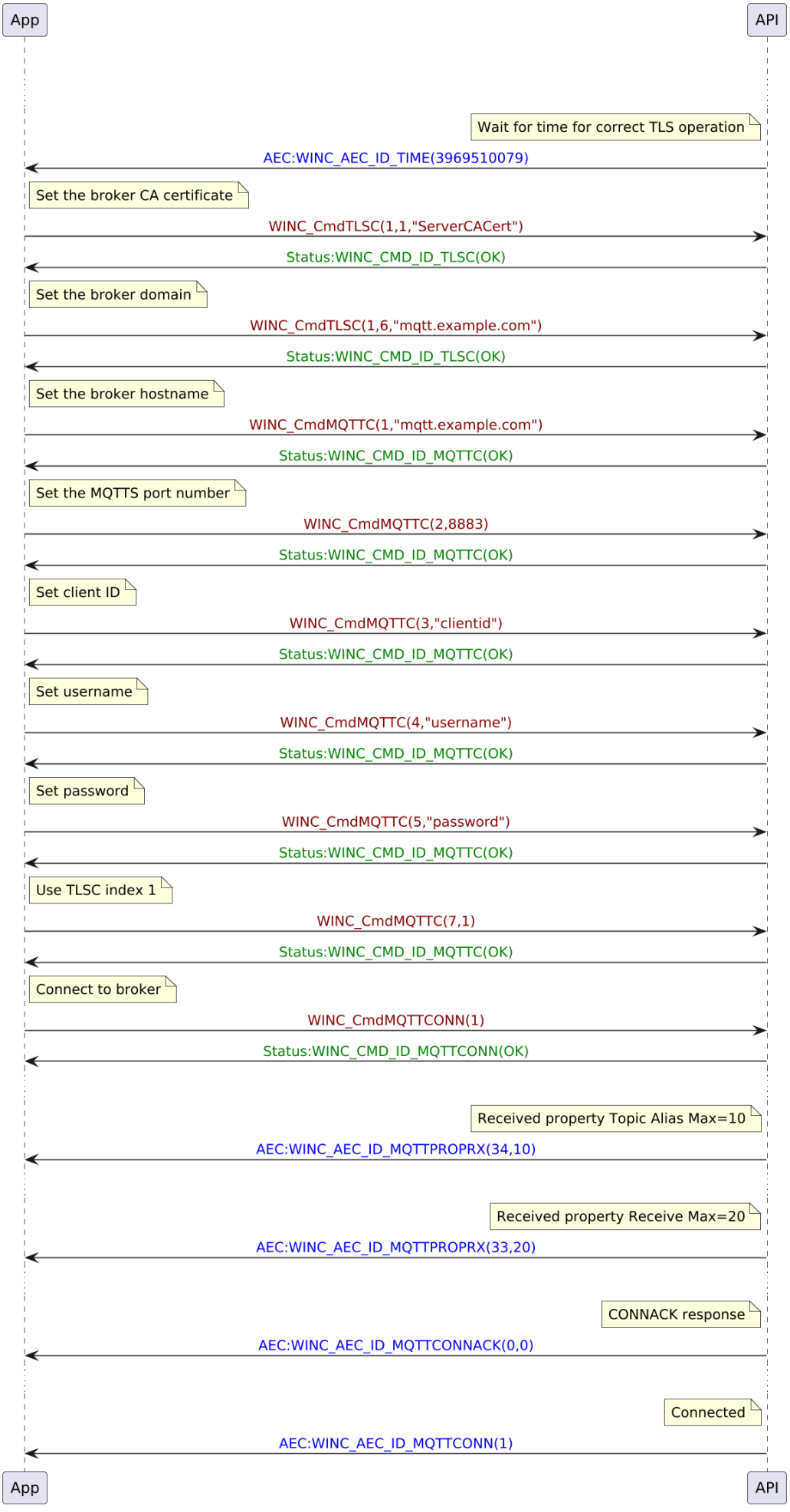


Figure 34. MQTT connection directly over TLS

 ©2025-2026

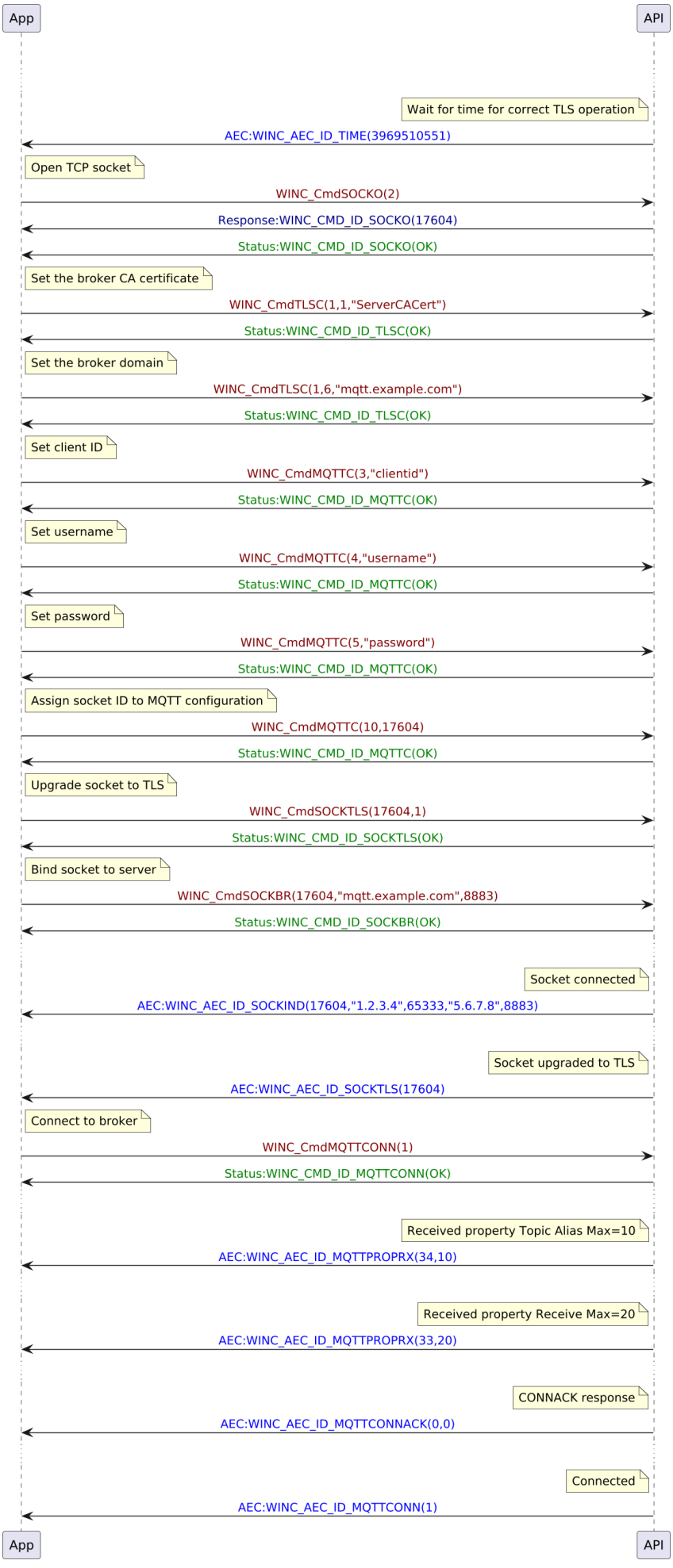


Figure 35. MQTT connection over TLS socket

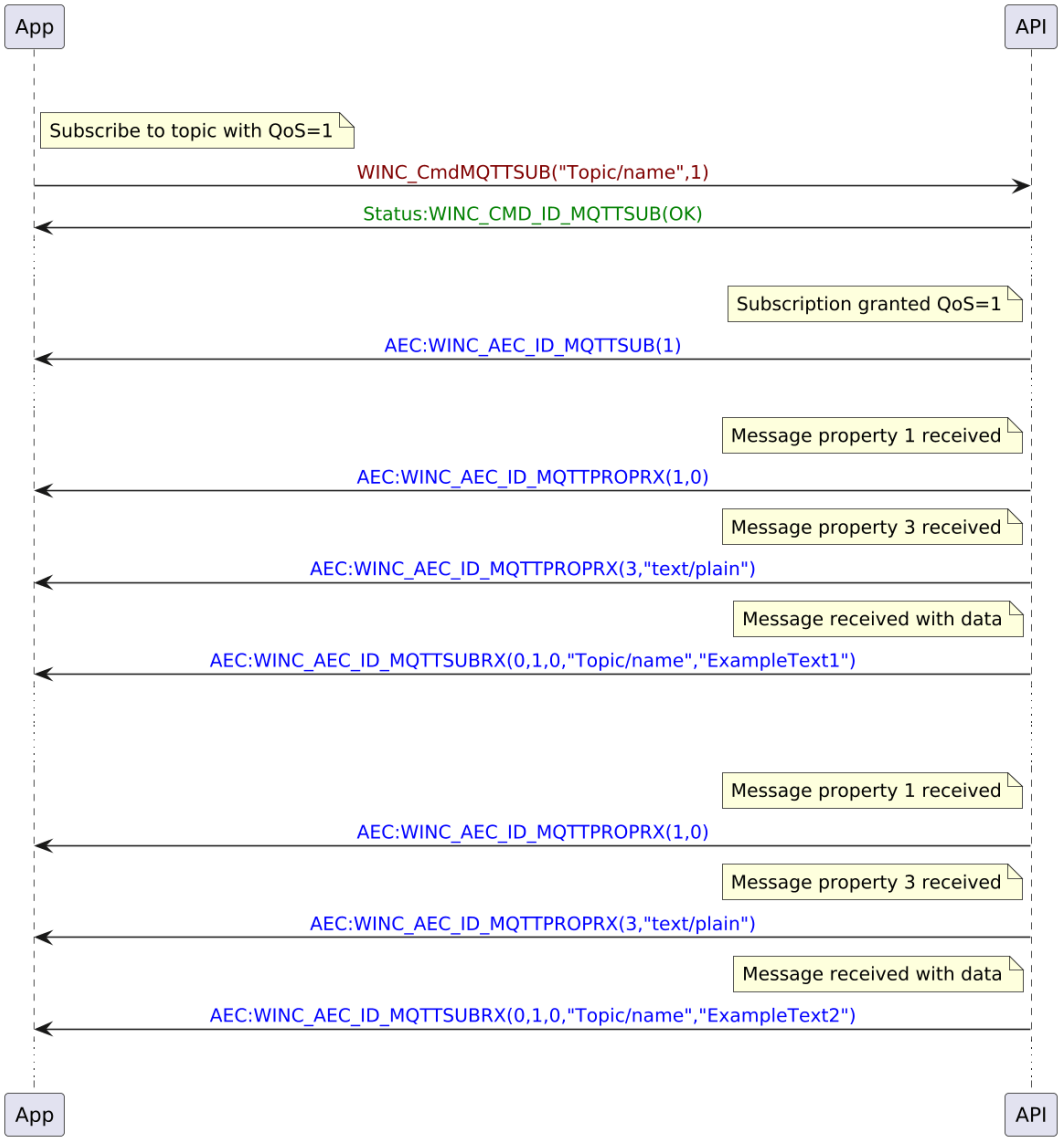


Figure 36. Subscribe to MQTT topic and receive data (asynchronously)

 ©2025-2026

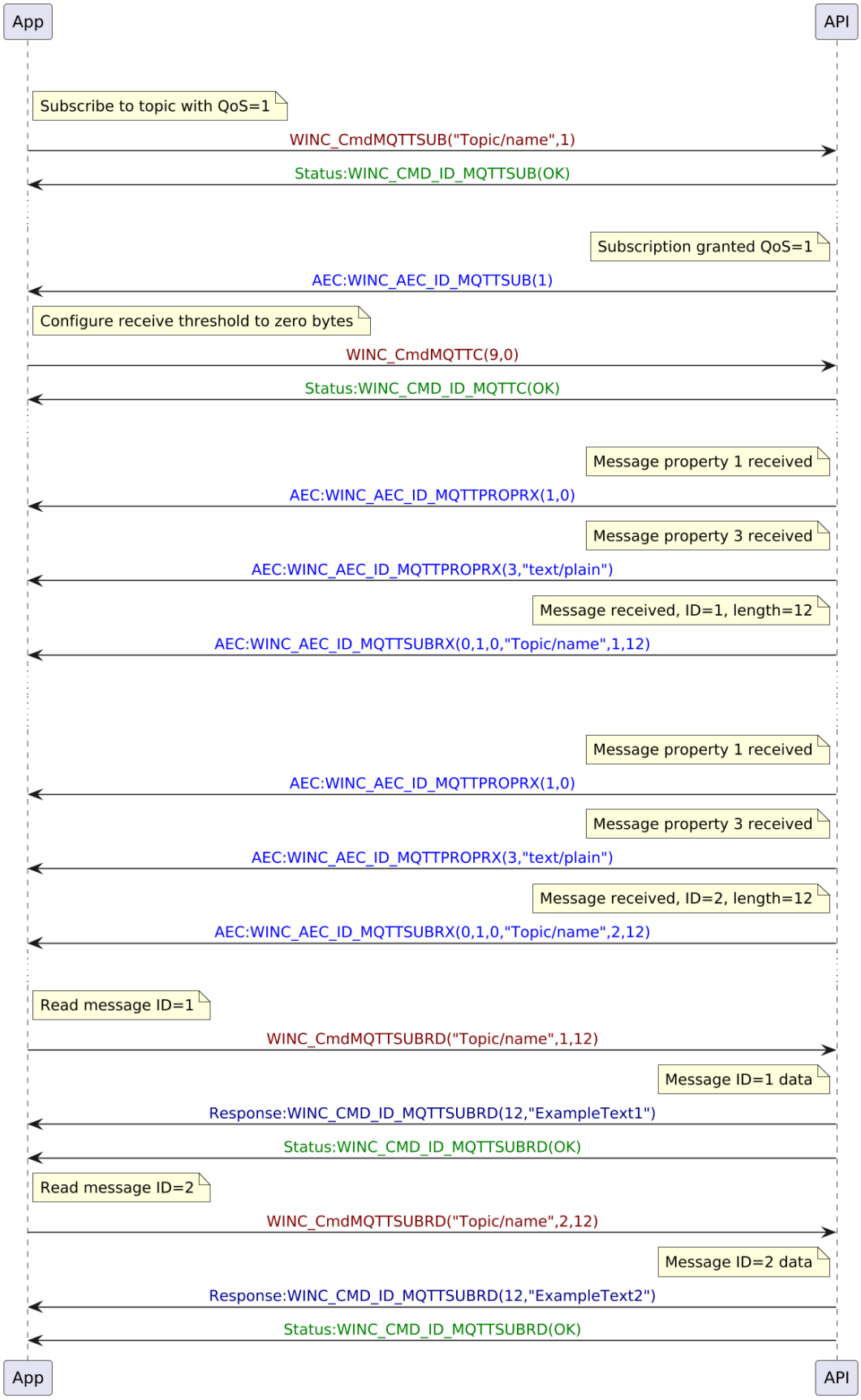


Figure 37. Subscribe to MQTT topic and receive data (polled)

NETIF (Module ID = 9)

Introduction

The NETIF module configures and reports the settings of the network interfaces. Two interface configurations are provided, with indices 0 and 1. The WSTA, WAP and DHCP modules each select the interface they use with their NETIF_IDX setting, which defaults to 0. WINC_CmdNETIFC with no parameters reports the range of valid indices.

An interface's configuration can only be written while the interface is down, except L2_MONITOR_MODE and L2_MONITOR_MAX_LEN. Where an address has both a configured and a live value, the configured value is reported while the interface is down and the live value while it is up.

IPv4

The DHCP client (DHCPC_EN) is enabled by default and applies when the interface is used by the station. If DHCP is disabled and no static address is configured, a link-local (ZeroConf) address is used.

IPv6

Stateless address autoconfiguration (IPV6_SLAAC) is enabled by default. The link-local address, global addresses and the gateway learned from router advertisements are reported once the interface is up.

Layer 2 Access

If L2_ONLY is set the interface is brought up without the TCP/IP stack.

L2_MONITOR_MODE selects which received frames (unicast, broadcast, multicast) are reported by the WINC_AEC_ID_NETIFRX AEC, optionally truncated to L2_MONITOR_MAX_LEN bytes. Monitored frames are still passed to the TCP/IP stack.

WINC_CmdNETIFTX transmits a complete Ethernet frame, including its header, of up to 1596 bytes. The interface must be up.

 ©2025-2026

Function Reference:

WINC_CmdNETIFC

Description

This function is used to read or set the network interface configuration.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdNETIFC
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optIntf
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optIntf	int32_t	Interface number. optIntf will be ignored if its value is WINC_CMDNETIFC_INTF_IGNORE_VAL.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDNETIFC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<MIN_IDX>,<MAX_IDX>	Valid index range
<ID>,<VAL>	Read response

Response Element Description

Element Name	Type	Description
<MIN_IDX>	INTEGER	Lowest valid index
<MAX_IDX>	INTEGER	Highest valid index
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_NETIF_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset Value: 0 — Label: DEFAULT — Description: Default.
1	NAME	STRING (Read Only)	Network interface name Maximum length of string is 16
2	ETHER	MACADDR (Read Only)	Ethernet MAC address of interface
3	HOSTNAME	STRING	Host name of this interface (NetBIOS name), up to 16 characters Maximum length of string is 16
10	DHCPC_EN	BOOL	DHCP client enable Value: 0 — Label: DISABLED — Description: Disabled - use static configuration. Value: 1 — Label: ENABLED — Description: Enabled - use automatic settings.
11	DHCPC_LEASE_START	UTC_TIME (Read Only)	DHCP lease start time, displayed in system time format
12	DHCPC_LEASE_ENDS	UTC_TIME (Read Only)	DHCP lease end time, displayed in system time format
13	DHCPC_SRV_ADDR	IPV4ADDR (Read Only)	IP address of DHCP server Unsigned 32-bit value Format of IPv4 address is A1|A2|A3|A4 = A1.A2.A3.A4

 ©2025-2026

<table>
<tr><th>ID</th><th>Name</th><th>Type</th><th>Description</th></tr>
<tr><td>20</td><td>L2_ONLY</td><td>BOOL</td><td>Layer 2 only flag
<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>LAYER3</td><td>Layer 3 active.</td></tr><tr><td>1</td><td>LAYER2</td><td>Layer 3 not active.</td></tr></table></td></tr>
<tr><td>21</td><td>L2_MONITOR_MODE</td><td>INTEGER_UNSIGNED</td><td>L2 monitor mode
<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>OFF</td><td>Off.</td></tr><tr><td>1</td><td>UNICAST</td><td>Unicast.</td></tr><tr><td>2</td><td>BCAST</td><td>Broadcast.</td></tr><tr><td>3</td><td>UNI_BCAST</td><td>Unicast + Broadcast.</td></tr><tr><td>4</td><td>MCAST</td><td>Multicast.</td></tr><tr><td>5</td><td>UNI_MCAST</td><td>Unicast + Multicast.</td></tr><tr><td>6</td><td>B_MCAST</td><td>Broadcast + Multicast.</td></tr><tr><td>7</td><td>UNI_B_MCAST</td><td>Unicast + Broadcast + Multicast.</td></tr></table></td></tr>
<tr><td>22</td><td>L2_MONITOR_MAX_LEN</td><td>INTEGER_UNSIGNED</td><td>Maximum number of bytes of each frame reported in NETIFRX AECs, 0 for the whole frame</td></tr>
<tr><td>40</td><td>IP_MASK</td><td>IPV4ADDR</td><td>IP address and net mask
Format of IPv4 address and netmask is
A1|A2|A3|A4|M1|M2|M3|M4 = A1.A2.A3.A4 + M1.M2.M3.M4</td></tr>
<tr><td>41</td><td>GATEWAY</td><td>IPV4ADDR</td><td>IP address of gateway
Format of IPv4 address is
A1|A2|A3|A4 = A1.A2.A3.A4</td></tr>
<tr><td>60</td><td>IPV6_GLO_ADDR</td><td>IPV6ADDR</td><td>IPv6 global address
Format of IPv6 address and scope is
A1|...|A16 | M1 = A1A2::A15A16/M1
This is a multiple value parameter
with an ID range 60.0 to 60.1</td></tr>
<tr><td>61</td><td>IPV6_LL_ADDR</td><td>IPV6ADDR
(Read Only)</td><td>IPv6 link-local address
Format of IPv6 address and scope is
A1|...|A16 | M1 = A1A2::A15A16/M1</td></tr>
<tr><td>62</td><td>IPV6_GATEWAY</td><td>IPV6ADDR</td><td>IPv6 gateway address
Format of IPv6 address is
A1|...|A16 = A1A2::A15A16</td></tr>
<tr><td>63</td><td>IPV6_SLAAC</td><td>BOOL</td><td>IPv6 SLAAC enable
<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>DISABLED</td><td>Disabled - use static address only.</td></tr><tr><td>1</td><td>ENABLED</td><td>Enabled - allow autoconfiguration (default).</td></tr></table></td></tr>
</table>

WINC_CmdNETIFTX

Description

This function is used to send L2 data frames.

```
bool WINC_CmdNETIFTX
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t intf
    const uint8_t* pData
    size_t lenData
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
intf	uint8_t	Interface number.
pData	const uint8_t*	Data frame data.
lenData	size_t	Length of pData in bytes.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

AEC Reference:

WINC_AEC_ID_NETIFRX

Description

Received L2 data frame.

AEC Syntax

AEC	Description
<INTF>,<DATA_LENGTH>,<DATA>	L2 data frame

AEC Element Syntax

Element Name	Type	Description
<INTF>	INTEGER	Interface number Valid range is 0 to 1
<DATA_LENGTH>	INTEGER	Length of data
<DATA>	BYTE_ARRAY	Data frame

OTA (Module ID = 10)

Introduction

Over-the-air (OTA) allows the firmware of the WINC to be upgraded. There are two OTA methods: an internal HTTP/HTTPS download based method using OTA (Module ID = 10) functions; or a host controlled flash write based method using NVM (Module ID = 29) functions.

Both methods make use of an alternate firmware image partition in non-volatile memory. The new image is placed in the alternate partition, then verified and activated. Once the new image has been activated, the current image continues to run, but the new image will have higher priority to run at the next boot.

After a successful OTA, activation and reboot, the alternate firmware image partition will contain the previously run image. It is possible to rollback to this previous image using the WINC_CmdOTAINV function. This function verifies the previous image, then invalidates the current running firmware image, so that it cannot be booted again. The current image continues to run, but the previous image will run at the next boot.

NOTE	If the OTA feature is not being utilised, the alternate firmware image partition on the WINC can be used for data storage. The partition supports up to 960kB of data. Data can be erased, written, read and checked via the NVM functions.

OTA Sequence

Each of WINC_CmdOTADL, WINC_CmdOTAVFY, WINC_CmdOTAACT and WINC_CmdOTAINV returns a new operation ID in its response, and every WINC_AEC_ID_OTA and WINC_AEC_ID_OTAERR AEC which follows carries that ID. Only one OTA operation can run at a time; a new operation is refused until the previous one has completed.

Download

WINC_CmdOTADL downloads the image from the server configured with WINC_CmdOTAC into the alternate partition. It is refused if:

- WINC_CmdOTAINV has been used since the last reset: NVM_LOCKED;
- the STA is not connected: OTA_NO_STA_CONN;
- a download is already running, HOST is not set, or the HTTP client is in use by the HTTP module.

On success the WINC_AEC_ID_OTA AEC reports, in order:

1. Erase Done: the whole alternate partition has been erased.
2. Write Done: the whole image has been received and written.
3. Verify Done: the image signature has been verified.

The downloaded image is latent: it does not run until it is activated. A download cannot be cancelled.

 ©2025-2026

Verify and Activate

WINC_CmdOTAVFY verifies the image in the alternate partition, reported by the WINC_AEC_ID_OTA AEC with Verify Done. It is only needed for the NVM method; a download is verified automatically.

WINC_CmdOTAACT verifies the image again and then activates it, reported with Activate Done. The current image continues to run; the activated image runs from the next reset.

If the image fails verification, WINC_CmdOTAVFY and WINC_CmdOTAACT fail with OTA_VERIFY_FAILED and no AEC is sent. WINC_CmdOTAACT fails with an error if the image has already been activated.

NOTE	A download started after WINC_CmdOTAACT and before the next reset erases the activated image.

Rollback

WINC_CmdOTAINV verifies the image in the alternate partition, which must be bootable: the previous (rollback) image, or an image activated since the last reset, but not a latent image. It then invalidates the current image, reported with Invalidate Done, and the alternate image runs from the next reset.

After WINC_CmdOTAINV, WINC_CmdNVMER, WINC_CmdNVMWR and WINC_CmdOTADL fail with NVM_LOCKED until the next reset.

After an Error

When the WINC_AEC_ID_OTAERR AEC is sent the download stops, and the alternate partition is left partly erased or written. Correct the cause and start the download again; the partition is erased again first. For the NVM method, erase and rewrite the partition.

- OTA_ERROR: an erase or flash write failed, or the connection closed before the whole image was received. The server must send a Content-Length header.
- OTA_PROTOCOL_ERROR: the server returned an HTTP status other than 200, which is included in the AEC (see Error Handling), or a DNS, connection or request error occurred.
- OTA_TLS_ERROR: the TLS session could not be created or the handshake failed, see TLS.
- OTA_IMG_TOO_LARGE: the image is larger than the partition.
- OTA_TIMEOUT: no progress was made within the timeout, see Timeout. Loss of the STA link, or the server closing the connection, during the download is also reported as a timeout.
- OTA_VERIFY_FAILED: the downloaded image failed verification.

After Reset

The WINC_AEC_ID_BOOT AEC reports the version of the running firmware. WINC_CmdDI lists the image in each partition; after a successful update the new image is CURRENT and the previous image is ROLLBACK. The WINCS02 does not roll back automatically; use WINC_CmdOTAINV to return to the previous image.

Timeout

The internal OTA download has a timeout set by TIMEOUT, 20 seconds by default. The download is stopped with OTA_TIMEOUT if no progress is made within the timeout. The timer starts once the partition has been erased, and covers the DNS lookup, the connection, the TLS handshake and each gap between received packets. If the server sends a Keep-Alive header with a timeout, that value is used instead.

NOTE	It may be necessary to use a timeout greater than the default if the network conditions are poor, or the server has a high latency.

Error Handling

The OTA module reports errors via the WINC_AEC_ID_OTAERR AEC. For protocol-specific errors (such as HTTP status codes indicating authentication failures, missing resources, or server errors), the extended error format includes the protocol type and the protocol-specific error code.

When the error code is `WINC_STATUS_OTA_PROTOCOL_ERROR`, the AEC includes additional fields:

- `PROTOCOL` - Protocol type (1 = HTTP)
- `PROTOCOL_STATUS` - Protocol-specific error code (HTTP status code)

For example, if the OTA server returns HTTP 401 (Unauthorized), the error callback will receive:

- `OTA_OP_ID` = operation ID
- `ERROR_CODE` = `WINC_STATUS_OTA_PROTOCOL_ERROR`
- `PROTOCOL` = 1 (HTTP)
- `PROTOCOL_STATUS` = 401

Common HTTP status codes that may be reported:

- `401` - Unauthorized (authentication required)
- `403` - Forbidden (access denied)
- `404` - Not Found (firmware file does not exist)
- `500` - Internal Server Error
- `503` - Service Unavailable

Errors detected when an OTA command is issued, such as OTA_NO_STA_CONN, or a failed verification by WINC_CmdOTAVFY, WINC_CmdOTAACT or WINC_CmdOTAINV, are returned as the command's status rather than by an AEC.

Troubleshooting

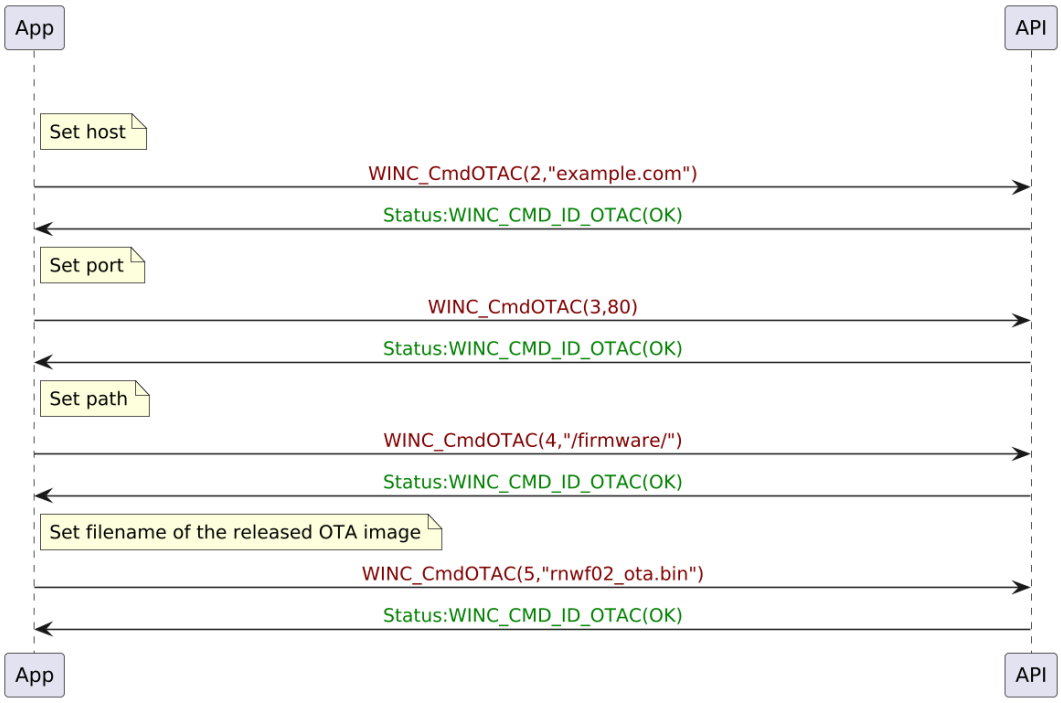
Further firmware image information can be gathered with the WINC_CmdDI function which can help with debugging available images on the WINC.

 ©2025-2026

Internal HTTP/S OTA Example

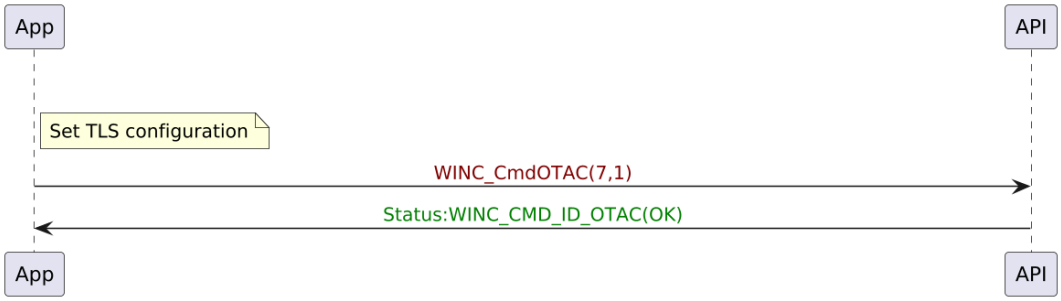
For OTA using the internal HTTP/HTTPS download based method, a HTTP server is required to serve the firmware image file. The host configures the download, then downloads and activates the new image. Image verification is done internally when the download completes.

HTTP Configuration

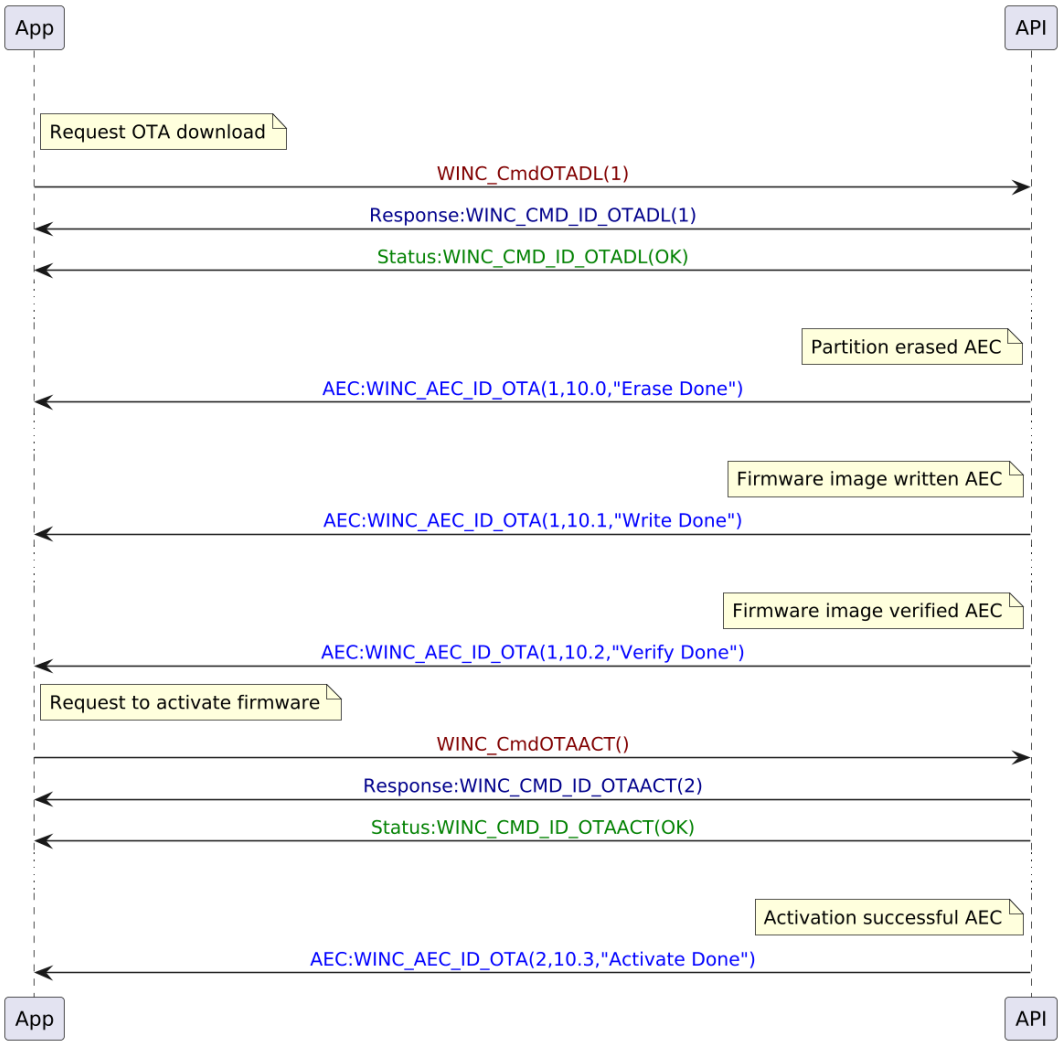


HTTPS Configuration

To use HTTPS a TLS configuration is required.



Download and Activation

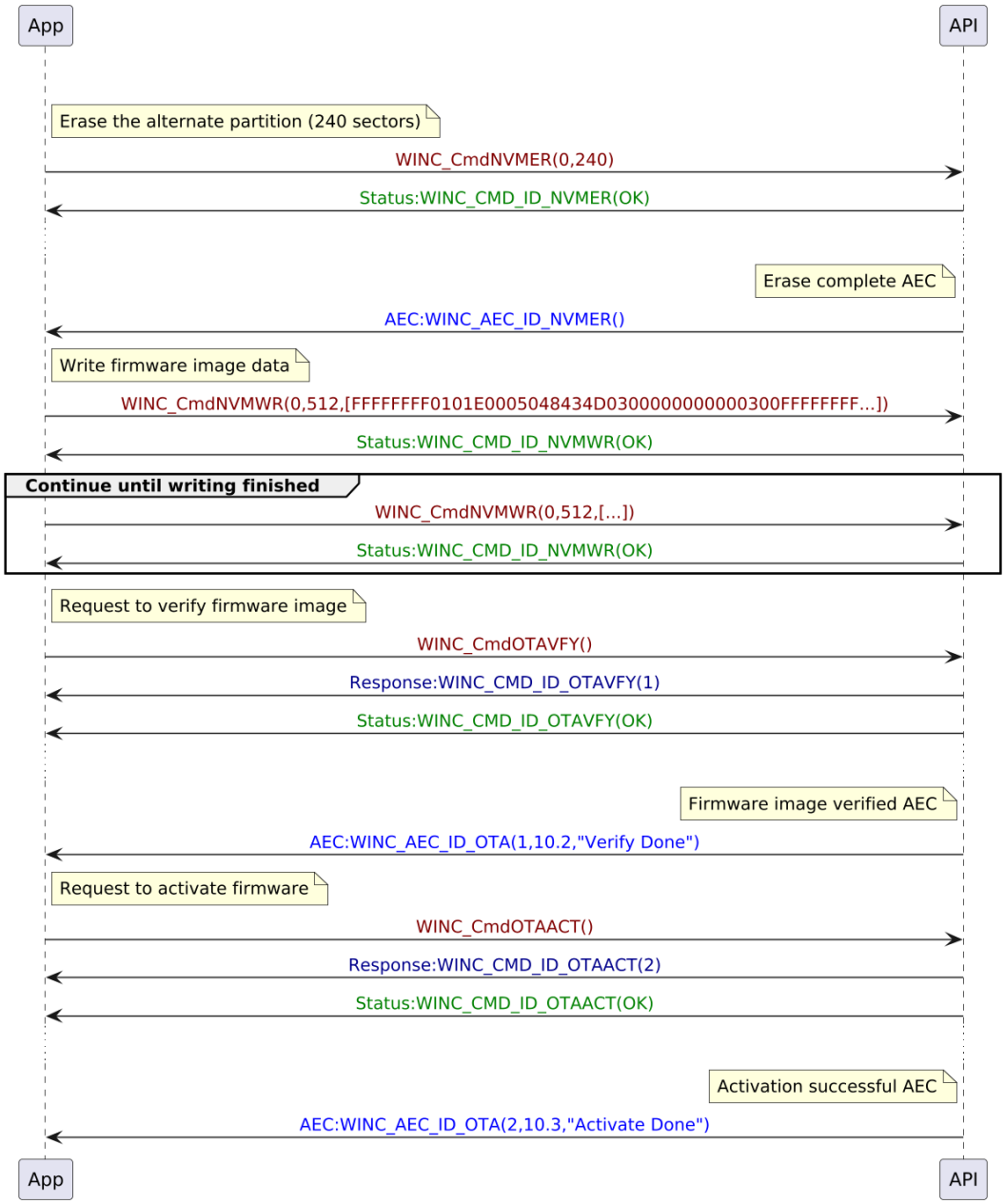


 ©2025-2026

NVM OTA Example

For OTA using the host controlled flash write based method, the host accesses the alternate firmware image partition of the WINC with flash read/write/erase functions. This feature allows extra flexibility with the data transport method, and ability to implement more advanced OTA logic if required - such as piecemeal updating with persistence between power cycles.

Once the host has written the image to the WINC using NVM (Module ID = 29) functions, the host should verify and activate the new image using OTA (Module ID = 10) functions.



Function Reference:

WINC_CmdOTAC

Description

This function is used to read or set the OTA configuration.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdOTAC
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDOTAC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response

 ©2025-2026

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_OTA_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset **Value** **Label** **Description** 0 DEFAULT Default.
1	PROTOCOL	INTEGER (Read Only)	Protocol type **Value** **Label** **Description** 1 HTTP HTTP.
2	HOST	STRING IPV4ADDR IPV6ADDR	Host domain name or IP address Maximum length of string is 255 Format of IPv4 address is A1|A2|A3|A4 = A1.A2.A3.A4 Format of IPv6 address is A1|...|A16 = A1A2::A15A16
3	PORT	INTEGER_UNSIGNED	Server port (e.g. 80 for HTTP, 443 for HTTPS); must be set
4	PATH	STRING	Path to firmware Maximum length of string is 1024
5	FILE	STRING	Firmware file Maximum length of string is 1024
6	TIMEOUT	INTEGER_UNSIGNED	Timeout in seconds Positive unsigned 8-bit value
7	TLS_CONF	INTEGER	TLS configuration index (see WINC_CmdTLSCDA). 0 disables TLS Valid range is 0 to 4
10	SOCKET_ID	INTEGER_UNSIGNED	Socket ID

WINC_CmdOTADL

Description

This function allows the downloading of firmware to the device.

```
bool WINC_CmdOTADL
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t state
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
state	uint8_t	Enable the OTA download feature. **Value** / **Label** / **Description** 1 / ENABLE / Use configuration from WINC_CmdOTAC command.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<OTA_OP_ID>	OTA download response

Response Element Description

Element Name	Type	Description
<OTA_OP_ID>	INTEGER	Operation ID Positive unsigned 8-bit value

 ©2025-2026

WINC_CmdOTAVFY

Description

This function verifies the OTA firmware image.

```
bool WINC_CmdOTAVFY(WINC_CMD_REQ_HANDLE handle)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<OTA_OP_ID>	OTA verify response

Response Element Description

Element Name	Type	Description
<OTA_OP_ID>	INTEGER	Operation ID Positive unsigned 8-bit value

WINC_CmdOTAACT

Description

This function activates the OTA firmware image.

```
bool WINC_CmdOTAACT(WINC_CMD_REQ_HANDLE handle)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<OTA_OP_ID>	OTA activate response

Response Element Description

Element Name	Type	Description
<OTA_OP_ID>	INTEGER	Operation ID Positive unsigned 8-bit value

 ©2025-2026

WINC_CmdOTAINV

Description

This function will invalidate the current running firmware image. Invalidation does not delete the current running firmware image.

```
bool WINC_CmdOTAINV(WINC_CMD_REQ_HANDLE handle)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<OTA_OP_ID>	OTA invalidate response

Response Element Description

Element Name	Type	Description
<OTA_OP_ID>	INTEGER	Operation ID Positive unsigned 8-bit value

AEC Reference:

WINC_AEC_ID_OTA

Description

OTA state.

AEC Syntax

AEC	Description
<OTA_OP_ID>,<OTA_STATE>	OTA status

AEC Element Syntax

Element Name	Type	Description
<OTA_OP_ID>	INTEGER	Operation ID Positive unsigned 8-bit value
<OTA_STATE>	INTEGER	OTA status code One of the OTA module status codes: WINC_STATUS_ERASE_DONE WINC_STATUS_WRITE_DONE WINC_STATUS_VERIFY_DONE WINC_STATUS_ACTIVATE_DONE WINC_STATUS_INVALIDATE_DONE See Status Response Codes.

 ©2025-2026

WINC_AEC_ID_OTAERR

Description

OTA error.

AEC Syntax

AEC	Description
<OTA_OP_ID>,<ERROR_CODE>	OTA error
<OTA_OP_ID>,<ERROR_CODE>,<PROTOCOL>,<PROTOCOL_STATUS>	OTA protocol status

AEC Element Syntax

Element Name	Type	Description
<OTA_OP_ID>	INTEGER	Operation ID Positive unsigned 8-bit value
<ERROR_CODE>	INTEGER	Error code See Status Response Codes
<PROTOCOL>	INTEGER	Protocol type **Value** / **Label** / **Description** 1 / HTTP / HTTP protocol.
<PROTOCOL_STATUS>	INTEGER	Protocol-specific status code For HTTP protocol this is the server response code, valid range is 100 to 599

Examples:

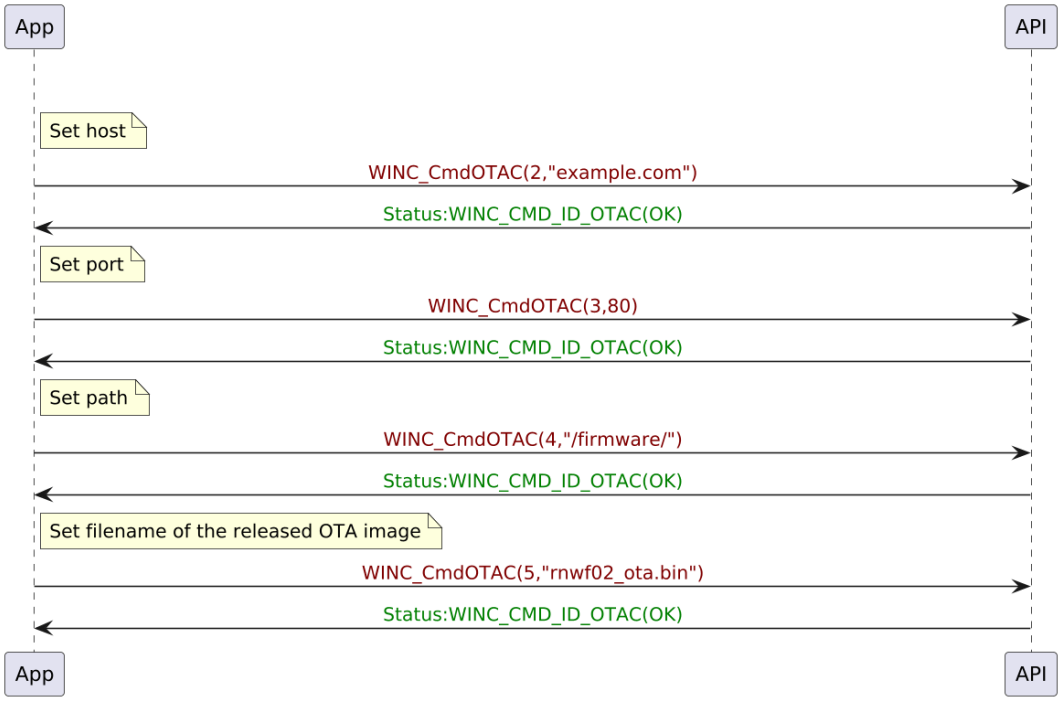


Figure 38. Internal OTA HTTP Configuration

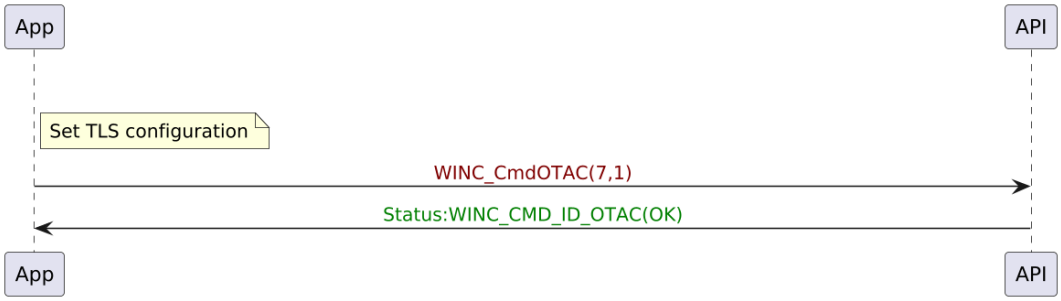


Figure 39. Internal OTA HTTPS Configuration

 ©2025-2026

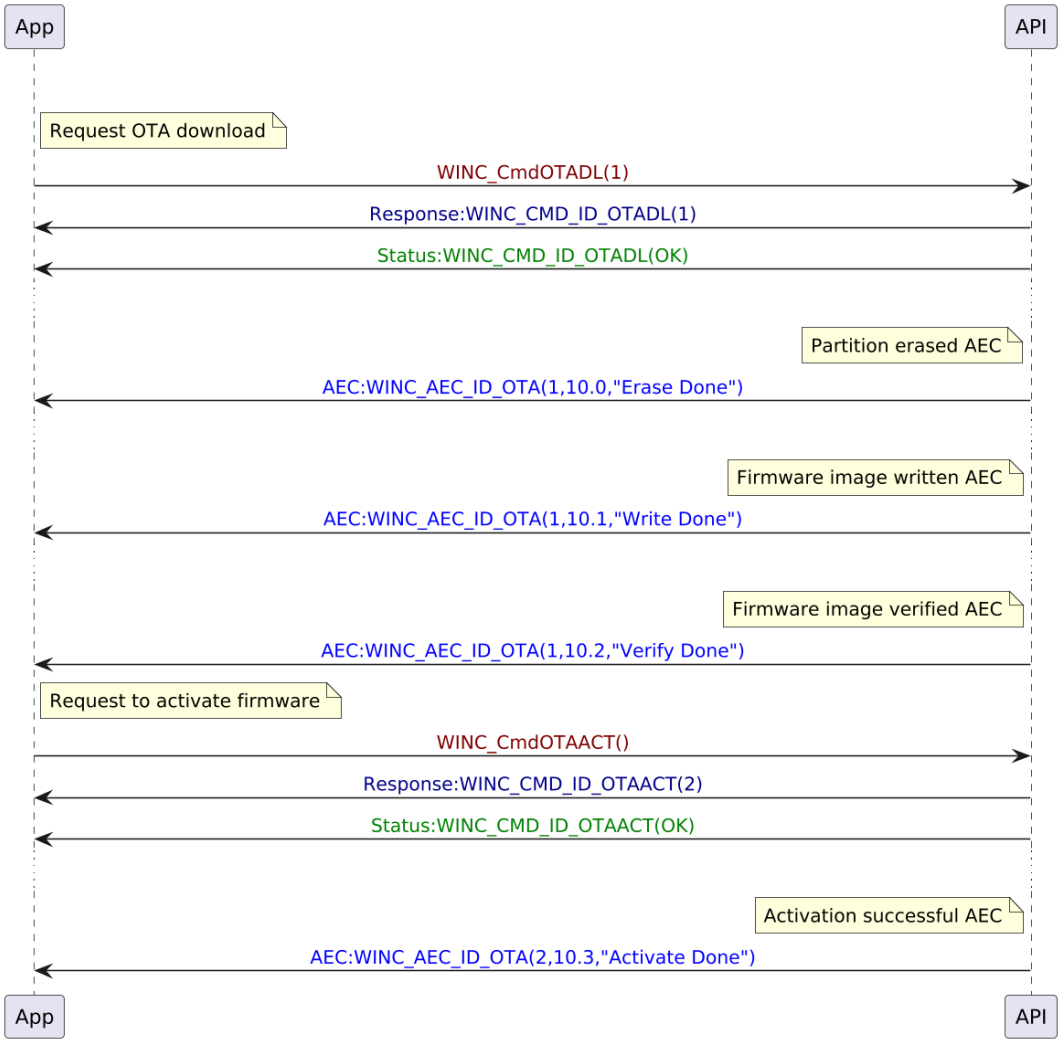


Figure 40. Internal OTA

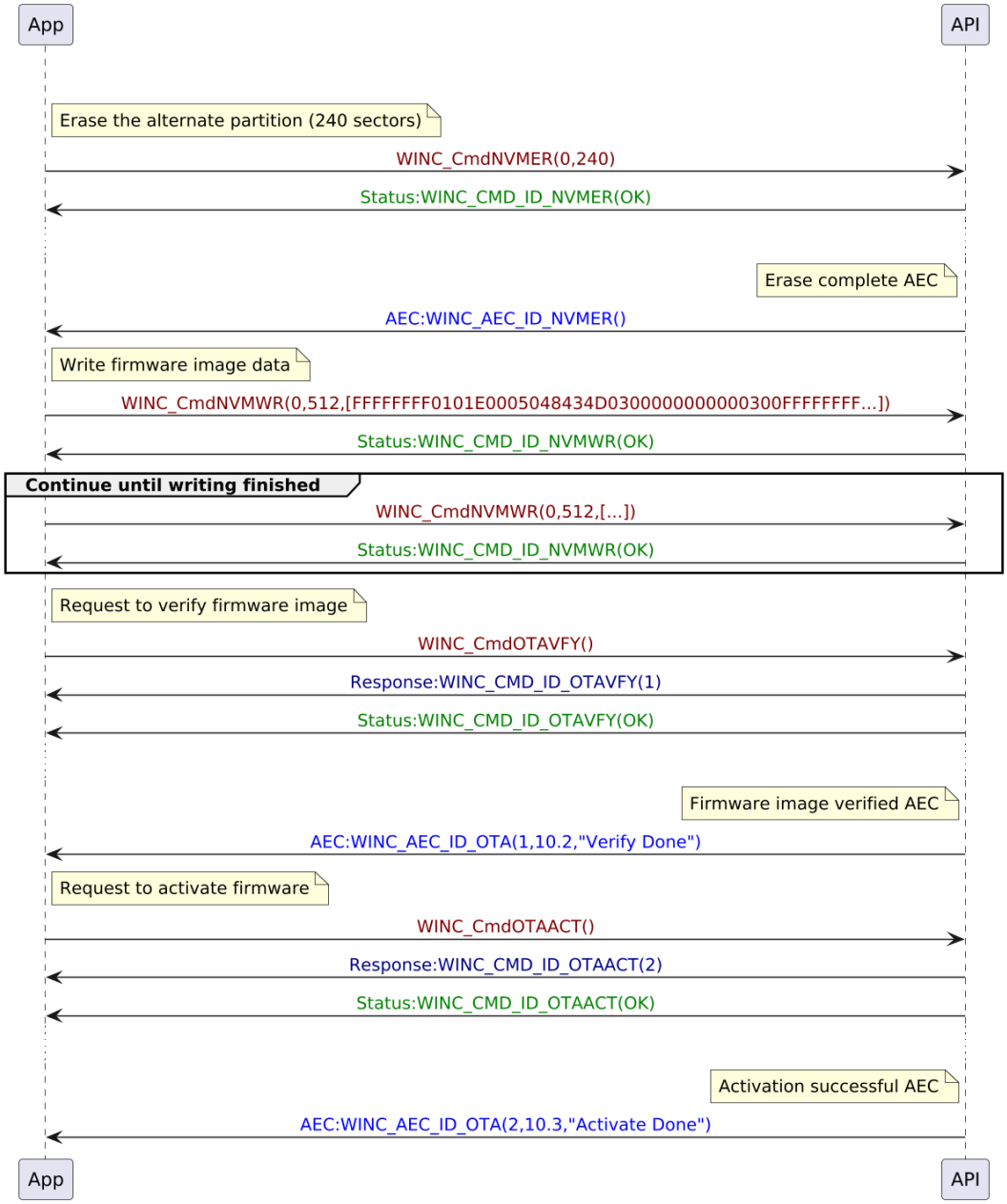


Figure 41. NVM OTA

 ©2025-2026

PING (Module ID = 11)

Introduction

The PING module sends a single ICMP echo request to a remote host and reports the round trip time.

The target may be an IPv4 or IPv6 address, or a host name which is first resolved using DNS. The WINC_CmdPING function completes once the request has been sent; the result is reported later by the WINC_AEC_ID_PING AEC, which carries the address and round trip time, or by the WINC_AEC_ID_PINGERR AEC if no reply is received within one second.

More than one request may be outstanding at a time.

Function Reference:

WINC_CmdPING

Description

This function sends a ping (ICMP Echo Request) to the target address.

```
bool WINC_CmdPING
(
    WINC_CMD_REQ_HANDLE handle
    WINC_TYPE typeTargetAddr
    uintptr_t targetAddr
    size_t lenTargetAddr
    uint8_t optProtocolVersion
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
typeTargetAddr	WINC_TYPE	Type of targetAddr, targetAddr is not sent if the type is WINC_TYPE_INVALID.
targetAddr	uintptr_t	IP address or host name of target. For integer types targetAddr holds the value, for other types targetAddr is a pointer to the data.
lenTargetAddr	size_t	Length in bytes of the data targetAddr points to, ignored for integer types.
optProtocolVersion	uint8_t	IP protocol version. **Value** / **Label** / **Description** 4 / IPV4 / IPv4. 6 / IPV6 / IPv6. optProtocolVersion will be ignored if its value is WINC_CMDPING_PROTOCOL_VERSION_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

AEC Reference:

WINC_AEC_ID_PING

Description

Successful ping.

AEC Syntax

AEC	Description
<IP_ADDRESS>,<RTT>	Ping success response

AEC Element Syntax

Element Name	Type	Description
<IP_ADDRESS>	STRING	IP address of the target
<RTT>	INTEGER	Round trip time (in milliseconds)

WINC_AEC_ID_PINGERR

Description

Error.

AEC Syntax

AEC	Description
<ERROR_CODE>	Ping error

AEC Element Syntax

Element Name	Type	Description
<ERROR_CODE>	INTEGER	Error code See Status Response Codes

 ©2025-2026

RST (Module ID = 12)

Introduction

The WINC_CmdRST function performs a full software reset of the WINCS02. The command status is sent before the reset takes place.

All runtime state and configuration is lost, including network connections, the system time and scan results. After the reset the configuration is restored from the configuration archive file "autoexec" if one has been stored, see CFG.

Function Reference:

WINC_CmdRST

Description

This function is used to reset the device.

```
bool WINC_CmdRST(WINC_CMD_REQ_HANDLE handle)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

SNTP (Module ID = 13)

Introduction

The SNTP module configures the NTP client which sets the system time. The client is enabled by default.

The source of the NTP server is selected by NTP_STATIC:

- AUTO: the NTP server supplied by the DHCP server is used. If none is supplied the client does not start.
- MANUAL: the server configured by NTP_SVR is used, this may be a host name or an IPv4 or IPv6 address.

The client starts when the station interface obtains an IP address. Changes made with WINC_CmdSNTPC while connected are applied immediately.

Each successful synchronisation sets the system time and sends a WINC_AEC_ID_TIME AEC. Server timeouts and protocol errors are reported by the WINC_AEC_ID_SNTPERR AEC.

While the station is connected and the client is enabled, reading NTP_SVR returns the address of the server in use rather than the configured value.

Function Reference:

WINC_CmdSNTPC

Description

This function is used to read or set the SNTP configuration.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdSNTPC
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDSNTPC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response

 ©2025-2026

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_SNTP_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset **Value** / **Label** / **Description** 0 / DEFAULT / Default.
1	NTP_ENABLED	BOOL	Network Time (NTP) client function **Value** / **Label** / **Description** 0 / DISABLED / Disabled. 1 / ENABLED / Enabled.
2	NTP_STATIC	BOOL	NTP configuration mode **Value** / **Label** / **Description** 0 / AUTO / Auto-assigned by network. 1 / MANUAL / Manually configured by user.
3	NTP_SVR	STRING IPV4ADDR IPV6ADDR	The address/name of NTP server Maximum length of string is 128 Format of IPv4 address is A1|A2|A3|A4 = A1.A2.A3.A4 Format of IPv6 address is A1|...|A16 = A1A2::A15A16

AEC Reference:

WINC_AEC_ID_SNTPERR

Description

SNTP error.

AEC Syntax

AEC	Description
<ERROR_CODE>	SNTP error

AEC Element Syntax

Element Name	Type	Description
<ERROR_CODE>	INTEGER	Error code See Status Response Codes

 ©2025-2026

SOCKET (Module ID = 14)

Introduction:

Sockets can be used in two ways:

- Directly, through the WINC_CmdSOCK* functions and the WINC_AEC_ID_SOCK* AECs described in this section.
- Through the Berkeley Sockets API. This layer is built on the same functions and manages socket creation, configuration and data transfer itself; it is the recommended way to use sockets.

NOTE	A socket created through the Berkeley Sockets API must not also be operated with the WINC_CmdSOCK* functions. AEC callbacks registered by the application also receive the socket AECs for those sockets, and should ignore them.

Socket Lifecycle

Opening

WINC_CmdSOCKO opens a UDP or TCP socket and returns its socket ID. The IP version (4 or 6) is optional; if it is not given IPv4 is used when the socket is bound. Socket IDs are allocated sequentially and are not reused until the 16-bit ID wraps.

A network connection is not needed to open a socket, but a TCP connection needs a local IP address. Up to 30 sockets can be open at once, after which WINC_CmdSOCKO fails with NO_FREE_SOCKETS. The TCP/IP stack provides 10 TCP and 10 UDP sockets, which are shared with the MQTT and HTTP modules.

TCP Client

1. WINC_CmdSOCKBR connects the socket to a remote IP address or host name. A successful command status indicates the connection has started.
2. The WINC_AEC_ID_SOCKIND AEC reports that the connection is established, with the local and remote addresses and ports.

If the connection is not established within 10 seconds, including when it is refused by the remote host, the WINC_AEC_ID_SOCKERR AEC reports Socket Bind Failed. If the host name cannot be resolved, WINC_AEC_ID_SOCKERR reports the DNS error. The socket ID remains allocated in both cases and must be closed with WINC_CmdSOCKCL.

TCP Server

1. WINC_CmdSOCKBL listens on a local port. The number of pending sockets (default 1) sets the number of connections which can be open at once; each one reserves a TCP stack socket when the socket starts listening.
2. Each accepted connection is given a new socket ID, reported by the WINC_AEC_ID_SOCKIND AEC. The new socket inherits the listening socket's WINC_CmdSOCKC and TLS settings. Data is read and written using the new socket ID.

Closing an accepted socket frees its connection slot for the next connection. Closing the listening socket stops new connections; sockets which are already connected stay open.

See example Opening a TCP server socket.

UDP

- WINC_CmdSOCKBL binds the socket to a local port, and datagrams from any remote host are received.
- WINC_CmdSOCKBR sets the default destination used by WINC_CmdSOCKWR.
- WINC_CmdSOCKWRTO sends to the given address. It binds the socket if needed, and sets the default destination if none is set.
- WINC_CmdSOCKBM joins a multicast group.

TLS

WINC_CmdSOCKTLS applies a TLS configuration (1 to 4, see WINC_CmdTLSC) to a TCP socket:

- if it is used before WINC_CmdSOCKBR or WINC_CmdSOCKBL, the handshake starts once the connection is established;
- if it is used on a connected socket, the connection is upgraded to TLS.

The WINC_AEC_ID_SOCKTLS AEC reports that the handshake has completed; data cannot be written until then. On a server, each accepted socket performs its own handshake and reports WINC_AEC_ID_SOCKTLS with its own socket ID. If the handshake fails, the WINC_AEC_ID_SOCKERR AEC reports Socket TLS Failed.

 ©2025-2026

Writing

- WINC_CmdSOCKWR writes to a TCP socket, or to a UDP socket with a default destination, up to 1460 bytes.
- WINC_CmdSOCKWRTO writes to a UDP socket, up to 1472 bytes.
- If there is not enough transmit buffer space the command fails with SOCKET_NOT_READY; retry later.

The optional sequence number (optSeqNum, optSeqNum) detects lost or repeated writes:

- the first write which includes it sets the expected sequence number;
- every later write must include the expected sequence number, which advances by the number of bytes written, otherwise it fails with SOCKET_SEQUENCE_ERROR.

Reading

Received data is announced by the WINC_AEC_ID_SOCKRXT (TCP) or WINC_AEC_ID_SOCKRXU (UDP) AEC. The four methods of reading it are described in Reading Socket Data:

Method	Enabled by	Data delivered by	Flow control	UDP datagram larger than read
Basic	Default	WINC_CmdSOCKRD output mode 1	The application reads when ready	The rest of the datagram is discarded
In-band	WINC_CmdSOCKRD output mode 3	WINC_CmdSOCKRD, with the pending data information in the response	The application reads when ready; no AECs until all data is read	The rest of the datagram is discarded
Simple asynchronous	ASYNC_MAX_FRM_SZ	WINC_AEC_ID_SOCKRXT/ WINC_AEC_ID_SOCKRXU with the data	None, data is sent as it arrives	The rest of the datagram is lost
Acknowledged asynchronous	ASYNC_MAX_FRM_SZ and ASYNC_WIN_SZ	WINC_AEC_ID_SOCKRXT/ WINC_AEC_ID_SOCKRXU with the data and a sequence number	Window advanced by ASYNC_NEXT_SN	The rest of the datagram is lost

WINC_CmdSOCKRDBUF reads part of a UDP datagram without discarding the rest, see example Reading partial datagram.

Closing

WINC_CmdSOCKCL closes a socket and releases its socket ID.

The WINC_AEC_ID_SOCKCL AEC reports that a connected TCP socket was closed by the remote host or timed out. It is not sent when the application closes the socket with WINC_CmdSOCKCL. The socket ID remains allocated until WINC_CmdSOCKCL is used. After the remote host closes the connection, data which has not been read can still be read; after a reset it is lost.

When the network interface goes down, for example on WINC_AEC_ID_WSTALD or when the STA is disconnected with WINC_CmdWSTA, sockets are not closed and no AEC is sent. The application should close them with WINC_CmdSOCKCL.

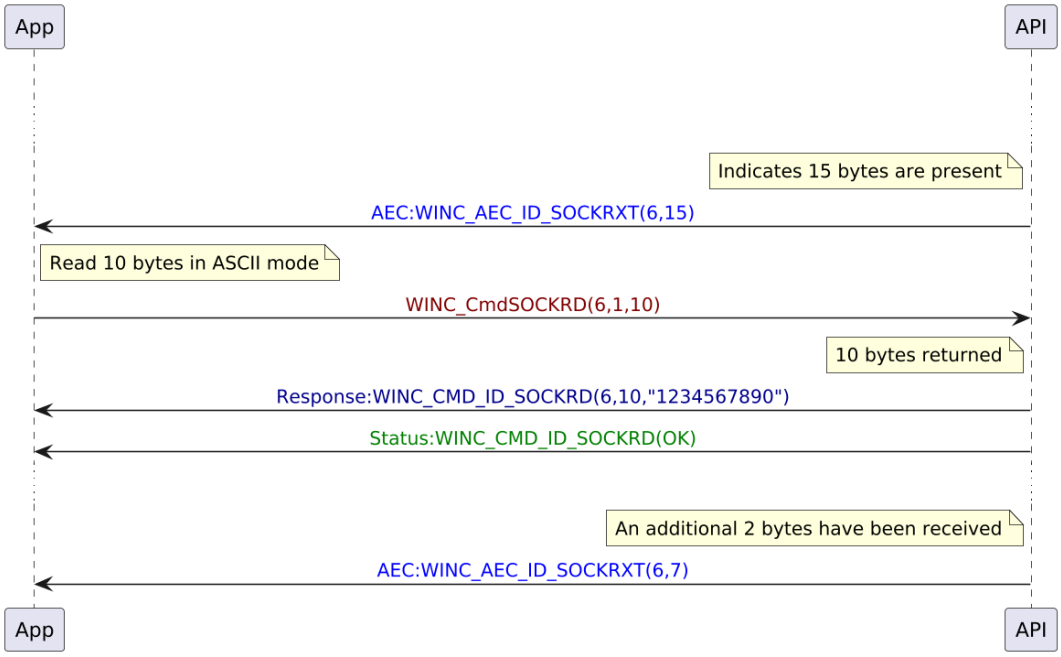
Reading Socket Data

There are four methods of reading data from a socket, two polled and two asynchronous methods. Polling involves the WINC_CmdSOCKRD function and WINC_AEC_ID_SOCKRXT /WINC_AEC_ID_SOCKRXU AECs. Asynchronous methods involve the WINC_AEC_ID_SOCKRXT /WINC_AEC_ID_SOCKRXU AECs. The Berkeley Sockets API uses the acknowledged asynchronous method.

Basic SOCKRD/AEC method:

WINC_AEC_ID_SOCKRXT or WINC_AEC_ID_SOCKRXU AECs from the device notify the application that data is available to be read. The application then calls WINC_CmdSOCKRD with an output mode of 1 to read a number of bytes from the device. After the WINC_CmdSOCKRD response further WINC_AEC_ID_SOCKRXT or WINC_AEC_ID_SOCKRXU AECs will update the available pending data.

See example Basic socket read

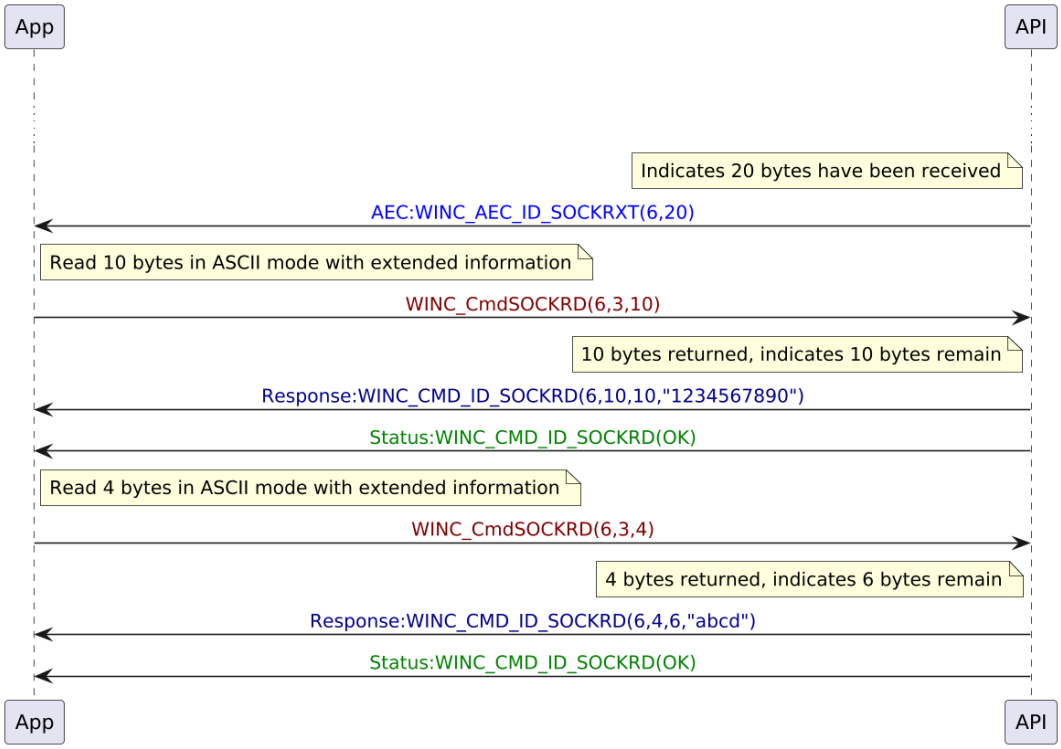


In-band SOCKRD/AEC method:

As with the basic WINC_CmdSOCKRD method, WINC_AEC_ID_SOCKRXT or WINC_AEC_ID_SOCKRXU announce the presence of data to be read from a socket. If WINC_CmdSOCKRD output mode 3 is used the resulting WINC_CmdSOCKRD response will include the pending data information and no further AECs will be sent until all data has been consumed.

See example In-band socket read

 ©2025-2026


App
API
Indicates 20 bytes have been received
AEC:WINC_AEC_ID_SOCKRXT(6,20)
Read 10 bytes in ASCII mode with extended information
WINC_CmdSOCKRD(6,3,10)
10 bytes returned, indicates 10 bytes remain
Response:WINC_CMD_ID_SOCKRD(6,10,10,"1234567890")
Status:WINC_CMD_ID_SOCKRD(OK)
Read 4 bytes in ASCII mode with extended information
WINC_CmdSOCKRD(6,3,4)
4 bytes returned, indicates 6 bytes remain
Response:WINC_CMD_ID_SOCKRD(6,4,6,"abcd")
Status:WINC_CMD_ID_SOCKRD(OK)
App
API

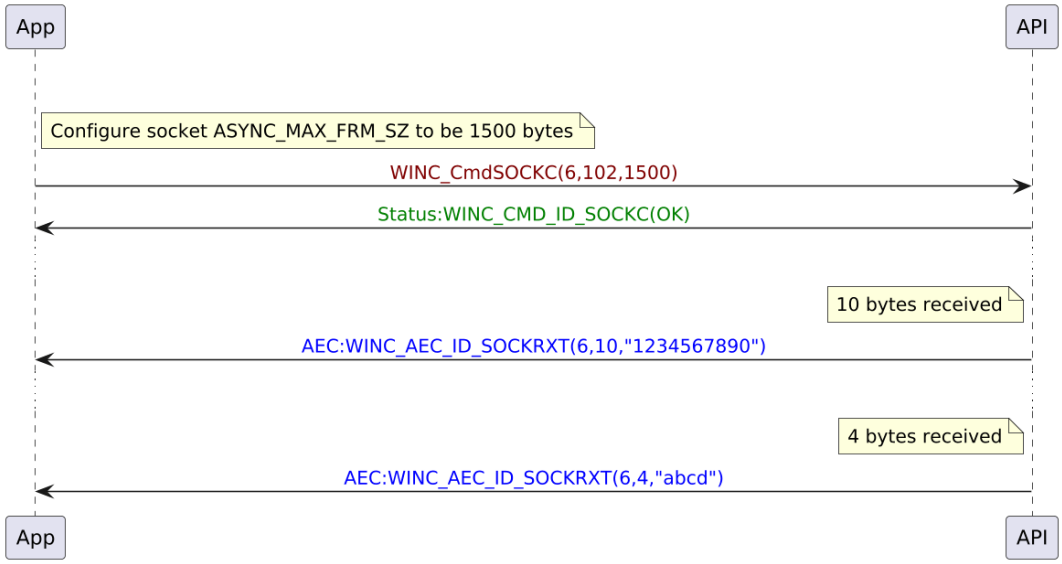
Simple Asynchronous method:

By configuring the WINC_CmdSOCKC ASYNC_MAX_FRM_SZ parameter of a socket the application can set the maximum frame size of data to be received asynchronously from the device.

NOTE	For UDP sockets, if the ASYNC_MAX_FRM_SZ value is smaller than the size of a datagram, the remaining data from the datagram will be lost.

When the ASYNC_MAX_FRM_SZ parameter is set, the device will send received socket data to the application without polling from WINC_CmdSOCKRD, the data is included in an expanded form of the WINC_AEC_ID_SOCKRXT or WINC_AEC_ID_SOCKRXU AEC.

See example Simple asynchronous socket read



 ©2025-2026

Acknowledged Asynchronous method:

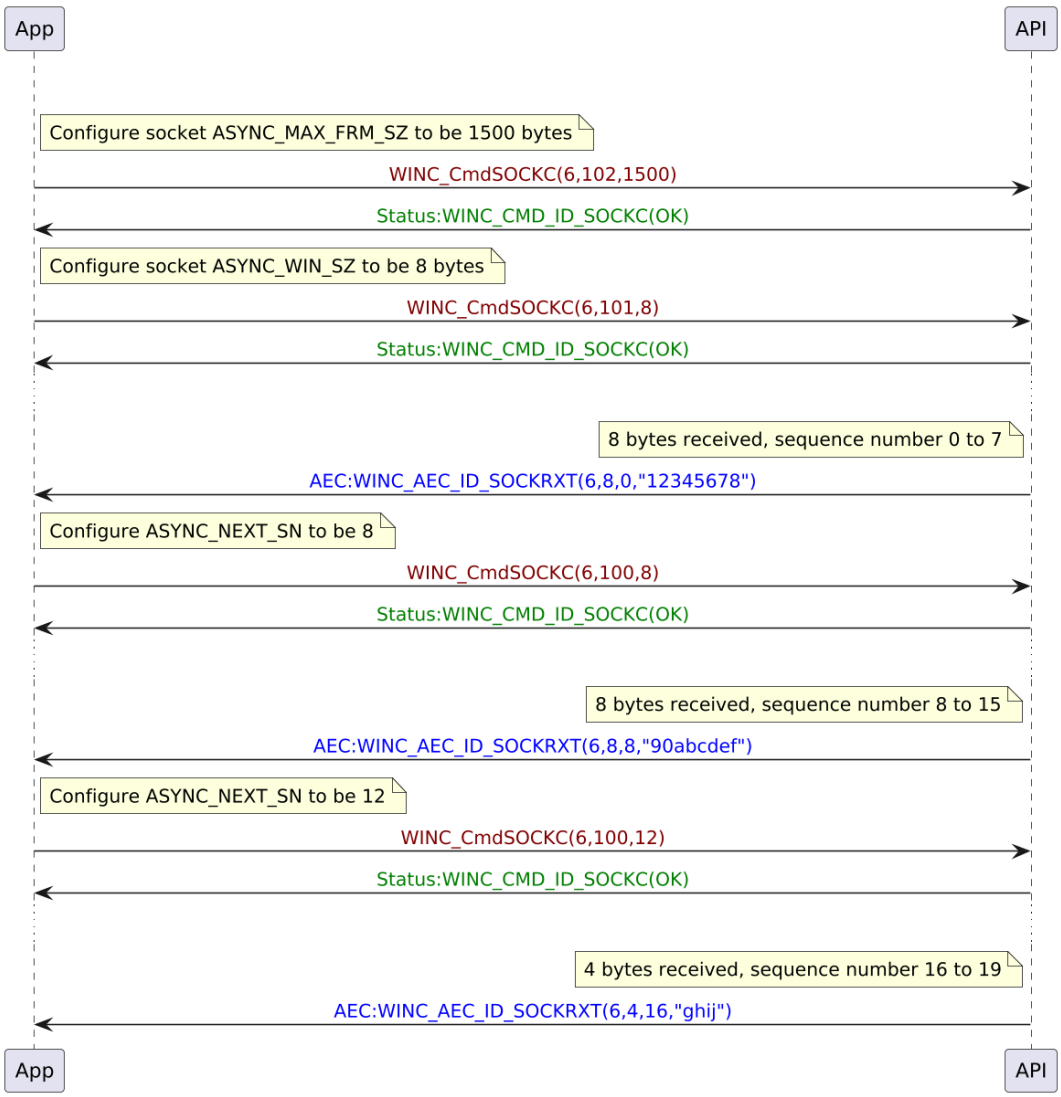
Extending the simple asynchronous method, if the ASYNC_WIN_SZ parameter of a socket is also set the device will use a sliding window protocol to send data to the application. In addition to the data included in the WINC_AEC_ID_SOCKRXT or WINC_AEC_ID_SOCKRXU AEC a sequence number is added, this represents the sequence number of the first byte of the data provided.

The device will continue to send data via AEC until ASYNC_WIN_SZ bytes have been transmitted, after which transmission will cease. To resume transmission the application must update the value of WINC_CmdSOCKC parameter ASYNC_NEXT_SN to acknowledge reception of data from the device, once this value has been updated the device will continue to send more data until ASYNC_NEXT_SN + ASYNC_WIN_SZ bytes have been sent.

The application does not need to wait until transmission has ceased before updating ASYNC_NEXT_SN, this can be done at any time and will ensure the transmission window remains open for subsequent WINC_AEC_ID_SOCKRXT or WINC_AEC_ID_SOCKRXU AECs.

Using this method the application can ensure that buffer space is available for receiving data transmission while still receiving data as it arrives at the device.

See example Acknowledged asynchronous socket read



Socket Errors

Errors which occur after a command has completed are reported by the WINC_AEC_ID_SOCKERR AEC. The error code identifies the cause:

- Socket Bind Failed: a TCP connection was not established within 10 seconds of binding to the remote address, or the bind itself failed.
- Socket TLS Failed: the TLS handshake failed or could not be started.
- A DNS error code: the remote host name could not be resolved.

Function Reference:

WINC_CmdSOCKO

Description

This function is used to open a new socket.

```
bool WINC_CmdSOCKO
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t protocol
    uint8_t optProtocolVersion
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
protocol	uint8_t	The protocol to use. **Value** / **Label** / **Description** 1 / UDP / UDP. 2 / TCP / TCP.
optProtocolVersion	uint8_t	IP protocol version. **Value** / **Label** / **Description** 4 / IPV4 / IPv4. 6 / IPV6 / IPv6. optProtocolVersion will be ignored if its value is WINC_CMDSOCKO_PROTOCOL_VERSION_IGNORE_VAL.

 ©2025-2026

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<SOCK_ID>	Socket open response

Response Element Description

Element Name	Type	Description
<SOCK_ID>	INTEGER	Socket ID Positive unsigned 16-bit value

WINC_CmdSOCKBL

Description

This function is used to bind a socket to a local port.

```
bool WINC_CmdSOCKBL
(
    WINC_CMD_REQ_HANDLE handle
    uint16_t sockId
    uint16_t lclPort
    uint8_t optPendSkts
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
sockId	uint16_t	Socket ID, as returned by SOCKO.
lclPort	uint16_t	The local port number to use.
optPendSkts	uint8_t	Maximum number of simultaneous connections, limited by the number of free TCP sockets. optPendSkts will be ignored if its value is WINC_CMDSOCKBL_PEND_SKTS_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

WINC_CmdSOCKBR

Description

This function is used to bind a socket to a remote address.

```
bool WINC_CmdSOCKBR
(
    WINC_CMD_REQ_HANDLE handle
    uint16_t sockId
    WINC_TYPE typeRmtAddr
    uintptr_t rmtAddr
    size_t lenRmtAddr
    uint16_t rmtPort
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
sockId	uint16_t	Socket ID, as returned by SOCKO.
typeRmtAddr	WINC_TYPE	Type of rmtAddr, rmtAddr is not sent if the type is WINC_TYPE_INVALID.
rmtAddr	uintptr_t	The address of the remote device. For integer types rmtAddr holds the value, for other types rmtAddr is a pointer to the data.
lenRmtAddr	size_t	Length in bytes of the data rmtAddr points to, ignored for integer types.
rmtPort	uint16_t	The port number on the remote device.

Return Values

Value	Description
true	Success.
false	Failure.

WINC_CmdSOCKBM

Description

This function is used to bind a socket to a multicast group.

```
bool WINC_CmdSOCKBM
(
    WINC_CMD_REQ_HANDLE handle
    uint16_t sockId
    WINC_TYPE typeMcastAddr
    uintptr_t mcastAddr
    size_t lenMcastAddr
    uint16_t mcastPort
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
sockId	uint16_t	Socket ID, as returned by SOCKO.
typeMcastAddr	WINC_TYPE	Type of mcastAddr, mcastAddr is not sent if the type is WINC_TYPE_INVALID.
mcastAddr	uintptr_t	The address of the multicast group. For integer types mcastAddr holds the value, for other types mcastAddr is a pointer to the data.
lenMcastAddr	size_t	Length in bytes of the data mcastAddr points to, ignored for integer types.
mcastPort	uint16_t	The port number of the multicast group.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

WINC_CmdSOCKTLS

Description

This function is used to enable TLS on a socket.

```
bool WINC_CmdSOCKTLS
(
    WINC_CMD_REQ_HANDLE handle
    uint16_t sockId
    uint8_t tlsConf
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
sockId	uint16_t	Socket ID, as returned by SOCKO.
tlsConf	uint8_t	TLS configuration index (see WINC_CmdTLSCDA).

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<SOCK_ID>	TLS success

Response Element Description

Element Name	Type	Description
<SOCK_ID>	INTEGER	Socket ID Positive unsigned 16-bit value

WINC_CmdSOCKWR

Description

This function is used to send data over a socket that is bound to a remote address and port number.

Write Sequence Numbers

The first write that includes a sequence number sets the starting sequence number for the socket. After that, every write on the socket must include the sequence number, equal to the previous sequence number plus the previous data length (modulo 65536). Otherwise the write fails with Socket Sequence Error. Sockets on which a sequence number has never been used are not affected.

```
bool WINC_CmdSOCKWR
(
    WINC_CMD_REQ_HANDLE handle
    uint16_t sockId
    uint16_t length
    int32_t optSeqNum
    const uint8_t* pData
    size_t lenData
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
sockId	uint16_t	Socket ID, as returned by SOCKO.
length	uint16_t	The length of the data to send.
optSeqNum	int32_t	Sequence number of first byte. Once used, required on every subsequent ASCII or hex write. optSeqNum will be ignored if its value is WINC_CMDSOCKWR_SEQ_NUM_IGNORE_VAL.
pData	const uint8_t*	The data to send. Maximum length of string is 1460
lenData	size_t	Length of pData in bytes.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

WINC_CmdSOCKWRTO

Description

This function is used to send data to an arbitrary destination using the connectionless UDP protocol.

Write Sequence Numbers

The first write that includes a sequence number sets the starting sequence number for the socket. After that, every write on the socket must include the sequence number, equal to the previous sequence number plus the previous data length (modulo 65536). Otherwise the write fails with Socket Sequence Error. Sockets on which a sequence number has never been used are not affected.

```
bool WINC_CmdSOCKWRTO
(
    WINC_CMD_REQ_HANDLE handle
    uint16_t sockId
    WINC_TYPE typeRmtAddr
    uintptr_t rmtAddr
    size_t lenRmtAddr
    uint16_t rmtPort
    uint16_t length
    int32_t optSeqNum
    const uint8_t* pData
    size_t lenData
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
sockId	uint16_t	Socket ID, as returned by SOCKO.
typeRmtAddr	WINC_TYPE	Type of rmtAddr, rmtAddr is not sent if the type is WINC_TYPE_INVALID.
rmtAddr	uintptr_t	The address of the remote device. For integer types rmtAddr holds the value, for other types rmtAddr is a pointer to the data.
lenRmtAddr	size_t	Length in bytes of the data rmtAddr points to, ignored for integer types.
rmtPort	uint16_t	The port number on the remote device.
length	uint16_t	The length of the data to send.
optSeqNum	int32_t	Sequence number of first byte. Once used, required on every subsequent ASCII or hex write. optSeqNum will be ignored if its value is WINC_CMDSOCKWRTO_SEQ_NUM_IGNORE_VAL.
pData	const uint8_t*	The data to send. Maximum length of string is 1472
lenData	size_t	Length of pData in bytes.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

WINC_CmdSOCKRD

Description

This function is used to read data from a socket.

Two AECs present notification of data received by the device:

- WINC_AEC_ID_SOCKRXU indicates UDP data has been received.
- WINC_AEC_ID_SOCKRXT indicates TCP data has been received.

The application is responsible for retrieving the datagram/stream data via the WINC_CmdSOCKRD function.

For TCP sockets the device will indicate, via WINC_AEC_ID_SOCKRXT, the number of bytes of data which are currently available for reading via the WINC_CmdSOCKRD function. The device may issue multiple WINC_AEC_ID_SOCKRXT AECs as data is received. When requesting data via the WINC_CmdSOCKRD function the application may receive less data than request, the number of bytes provided by the device will be declared in the WINC_CmdSOCKRD response before the data is presented. The application may request less data than that declared by the WINC_AEC_ID_SOCKRXT AEC as being available, the remaining data will be available for subsequent reading.

For UDP sockets the device will indicate, via WINC_AEC_ID_SOCKRXU, the number of bytes of data which were received in the oldest datagram received by the device. Only a single WINC_AEC_ID_SOCKRXU will be issued by the device even if subsequent UDP datagrams are received. Reading data from the UDP socket via the WINC_CmdSOCKRD function will read and discard the current datagram, if less data is requested than was indicated by the WINC_AEC_ID_SOCKRXU AEC the remaining unread data in the datagram will be discarded.

Output Modes

- **1**: the data is returned in the response.
- **3**: as mode 1, with the remaining pending data (and for UDP, the number of pending datagrams and the sender's address and port) included in the response. No further receive AECs are sent until the pending data has been read.

Binary output mode (2) is not available through this interface.

The length is limited to the amount of data available. For TCP sockets a length of 0 returns no data; for UDP sockets it reads a zero-length datagram.

```
bool WINC_CmdSOCKRD
(
    WINC_CMD_REQ_HANDLE handle
    uint16_t sockId
    uint8_t outputMode
    int32_t length
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
sockId	uint16_t	Socket ID, as returned by SOCKO.
outputMode	uint8_t	Output format of the data. **Value** / **Label** / **Description** 1 / ASCII / Data returned in the response. 2 / BINARY / Raw binary (AT interface only). 3 / ASCII_EXT / Data returned in the response, with pending data information.
length	int32_t	Maximum number of bytes to read.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<SOCK_ID>,<LENGTH>,<DATA>	Socket read response
<SOCK_ID>,<LENGTH>,<PEND_DATA>,<DATA>	Socket read response (TCP extended)
<SOCK_ID>,<LENGTH>,<PEND_DATA>,<PEND_DATAGRAM>,<RMT_ADDR>,<RMT_PORT>,<DATA>	Socket read response (UDP extended)

Response Element Description

Element Name	Type	Description
<SOCK_ID>	INTEGER	Socket ID Positive unsigned 16-bit value
<LENGTH>	INTEGER	Number of bytes
<DATA>	BYTE_ARRAY	Socket data
<PEND_DATA>	INTEGER	Number of pending bytes
<PEND_DATAGRAM>	INTEGER	Number of pending datagrams
<RMT_ADDR>	INTEGER STRING BYTE_ARRAY IP_ADDRESS	Remote address
<RMT_PORT>	INTEGER	Remote port Positive unsigned 16-bit value

 ©2025-2026

WINC_CmdSOCKRDBUF

Description

This function is used to read data from a socket buffer (UDP only).

```
bool WINC_CmdSOCKRDBUF
(
    WINC_CMD_REQ_HANDLE handle
    uint16_t sockId
    uint8_t outputMode
    int32_t length
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
sockId	uint16_t	Socket ID, as returned by SOCKO.
outputMode	uint8_t	Output format of the data. **Value** / **Label** / **Description** 1 / ASCII / Data returned in the response. 2 / BINARY / Raw binary (AT interface only). 3 / ASCII_EXT / Data returned in the response, with pending data information.
length	int32_t	Maximum number of bytes to read.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<SOCK_ID>,<LENGTH>,<DATA>	Socket read response
<SOCK_ID>,<LENGTH>,<PEND_DATA>,<PEND_DATAGRAM>,<RMT_ADDR>,<RMT_PORT>,<DATA>	Socket read response (UDP extended)

Response Element Description

Element Name	Type	Description
<SOCK_ID>	INTEGER	Socket ID Positive unsigned 16-bit value
<LENGTH>	INTEGER	Number of bytes
<DATA>	BYTE_ARRAY	Socket data
<PEND_DATA>	INTEGER	Number of pending bytes
<PEND_DATAGRAM>	INTEGER	Number of pending datagrams
<RMT_ADDR>	INTEGER STRING BYTE_ARRAY IP_ADDRESS	Remote address
<RMT_PORT>	INTEGER	Remote port Positive unsigned 16-bit value

 ©2025-2026

WINC_CmdSOCKCL

Description

This function is used to close a socket.

```
bool WINC_CmdSOCKCL
(
    WINC_CMD_REQ_HANDLE handle
    uint16_t sockId
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
sockId	uint16_t	Socket ID, as returned by SOCKO.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<SOCK_ID>	Socket close response

Response Element Description

Element Name	Type	Description
<SOCK_ID>	INTEGER	Socket ID Positive unsigned 16-bit value

WINC_CmdSOCKLST

Description

This function is used to present a list of the device's open sockets/connections.

```
bool WINC_CmdSOCKLST
(
    WINC_CMD_REQ_HANDLE handle
    uint16_t optSockId
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optSockId	uint16_t	Socket ID, as returned by SOCKO. optSockId will be ignored if its value is WINC_CMDSOCKLST_SOCK_ID_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<SOCK_ID>,<PROTOCOL>,<RMT_ADDR>,<RMT_PORT>,<LCL_PORT>	Socket list response

 ©2025-2026

Response Element Description

Element Name	Type	Description
<SOCK_ID>	INTEGER	Socket ID Positive unsigned 16-bit value
<PROTOCOL>	INTEGER	The protocol in use **Value** / **Label** / **Description** 1 / UDP / UDP. 2 / TCP / TCP.
<RMT_ADDR>	INTEGER STRING BYTE_ARRAY IP_ADDRESS	Remote address
<RMT_PORT>	INTEGER	Remote port Positive unsigned 16-bit value
<LCL_PORT>	INTEGER	Local port Positive unsigned 16-bit value

WINC_CmdSOCKC

Description

This function is used to read or set the socket configuration.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdSOCKC
(
    WINC_CMD_REQ_HANDLE handle
    uint16_t optSockId
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optSockId	uint16_t	Socket ID, as returned by SOCKO. optSockId will be ignored if its value is WINC_CMDSOCKC_SOCK_ID_IGNORE_VAL.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDSOCKC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<MIN_IDX>,<MAX_IDX>	Valid index range
<ID>,<VAL>	Read response

 ©2025-2026

Response Element Description

Element Name	Type	Description
<MIN_IDX>	INTEGER	Lowest valid index
<MAX_IDX>	INTEGER	Highest valid index
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_SOCKET_*)

ID	Name	Type	Description
1	SO_LINGER	BOOL	Socket linger enable/disable (TCP) **Value** / **Label** / **Description** 0 / DISABLE / Disable. 1 / ENABLE / Enable.
2	SO_NODELAY	BOOL	Socket no delay (TCP) **Value** / **Label** / **Description** 0 / DISABLE / Disable. 1 / ENABLE / Enable.
3	SO_KEEPALIVE	INTEGER_UNSIGNED	Socket keep alive timeout in milliseconds (TCP), 0 disables Unsigned 16-bit value
4	SO_RCVBUF	INTEGER_UNSIGNED	Socket receive buffer size (TCP) Valid range is 1460 to 32768
5	SO_SNDBUF	INTEGER_UNSIGNED	Socket transmit buffer size (TCP) Valid range is 1460 to 32768
32	IP_TOS	INTEGER_UNSIGNED	Socket Type of Service (TCP/UDP) **Value** / **Label** / **Description** 0x00 / NORM / Normal. 0x04 / REL / Reliability. 0x08 / THRPUT / Throughput. 0x10 / LOW_DELAY / Low delay.
100	ASYNC_NEXT_SN	INTEGER_UNSIGNED	Async next sequence number (acknowledgement) Acknowledges received data up to this sequence number and reopens the receive window. Must lie within the current window.
101	ASYNC_WIN_SZ	INTEGER_UNSIGNED	Async window size in bytes, 0 for no window. Default is 0 When non-zero, asynchronously delivered data carries a sequence number and delivery stops once this many bytes are unacknowledged.
102	ASYNC_MAX_FRM_SZ	INTEGER_UNSIGNED	Async maximum frame size in bytes, 0 disables async delivery. Default is 0 When non-zero, received data is delivered asynchronously in the socket receive AEC, up to this many bytes per AEC.

 ©2025-2026

AEC Reference:

WINC_AEC_ID_SOCKIND

Description

Socket established.

AEC Syntax

AEC	Description
<SOCK_ID>,<LCL_ADDR>,<LCL_PORT>,<RMT_ADDR>,<RMT_PORT>	TCP connection established

AEC Element Syntax

Element Name	Type	Description
<SOCK_ID>	INTEGER	Socket ID Positive unsigned 16-bit value
<LCL_ADDR>	INTEGER STRING BYTE_ARRAY IP_ADDRESS	Local address
<LCL_PORT>	INTEGER	Local port Positive unsigned 16-bit value
<RMT_ADDR>	INTEGER STRING BYTE_ARRAY IP_ADDRESS	Remote address
<RMT_PORT>	INTEGER	Remote port Positive unsigned 16-bit value

WINC_AEC_ID_SOCKRXT

Description

TCP receive.

AEC Syntax

AEC	Description
<SOCK_ID>,<PEND_DATA>	TCP receive (polled)
<SOCK_ID>,<LENGTH>,<DATA>	TCP receive (async)
<SOCK_ID>,<LENGTH>,<SEQ_NUM>,<DATA>	TCP receive (async reliable)

AEC Element Syntax

Element Name	Type	Description
<SOCK_ID>	INTEGER	Socket ID Positive unsigned 16-bit value
<PEND_DATA>	INTEGER	Number of pending bytes
<LENGTH>	INTEGER	Number of bytes
<DATA>	BYTE_ARRAY	Socket data
<SEQ_NUM>	INTEGER	Sequence number of first byte Unsigned 16-bit value

 ©2025-2026

WINC_AEC_ID_SOCKRXU

Description

UDP receive.

AEC Syntax

AEC	Description
<SOCK_ID>,<RMT_ADDR>,<RMT_PORT>,<PEND_DATA>,<PEND_DATAGRAM>	UDP receive (polled)
<SOCK_ID>,<RMT_ADDR>,<RMT_PORT>,<LENGTH>,<DATA>	UDP receive (async)
<SOCK_ID>,<RMT_ADDR>,<RMT_PORT>,<LENGTH>,<SEQ_NUM>,<DATA>	UDP receive (async reliable)

AEC Element Syntax

Element Name	Type	Description
<SOCK_ID>	INTEGER	Socket ID Positive unsigned 16-bit value
<RMT_ADDR>	INTEGER STRING BYTE_ARRAY IP_ADDRESS	Remote address
<RMT_PORT>	INTEGER	Remote port Positive unsigned 16-bit value
<PEND_DATA>	INTEGER	Number of pending bytes
<PEND_DATAGRAM>	INTEGER	Number of pending datagrams
<LENGTH>	INTEGER	Number of bytes
<DATA>	BYTE_ARRAY	Socket data
<SEQ_NUM>	INTEGER	Sequence number of first byte Unsigned 16-bit value

WINC_AEC_ID_SOCKCL

Description

Socket close.

AEC Syntax

AEC	Description
<SOCK_ID>	Socket close response

AEC Element Syntax

Element Name	Type	Description
<SOCK_ID>	INTEGER	Socket ID Positive unsigned 16-bit value

 ©2025-2026

WINC_AEC_ID_SOCKTLS

Description

TLS success.

AEC Syntax

AEC	Description
<SOCK_ID>	TLS success

AEC Element Syntax

Element Name	Type	Description
<SOCK_ID>	INTEGER	Socket ID Positive unsigned 16-bit value

WINC_AEC_ID_SOCKERR

Description

Socket error.

AEC Syntax

AEC	Description
<SOCK_ID>,<ERROR_CODE>	Socket error, see error code

AEC Element Syntax

Element Name	Type	Description
<SOCK_ID>	INTEGER	Socket ID Positive unsigned 16-bit value
<ERROR_CODE>	INTEGER	Error code See Status Response Codes

 ©2025-2026

Examples:

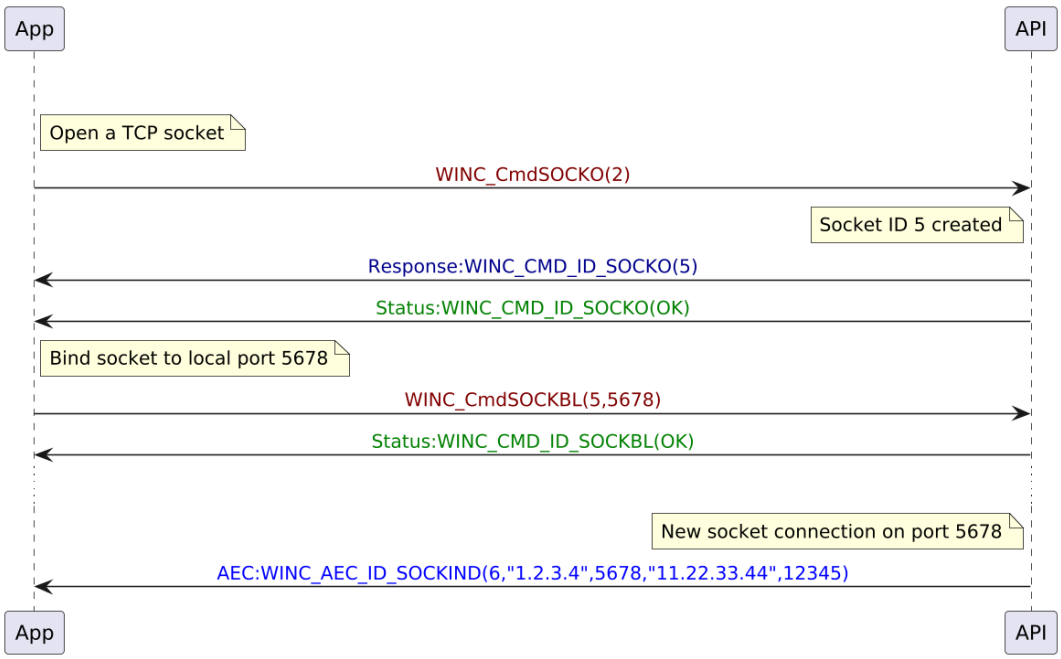


Figure 42. Opening a TCP server socket

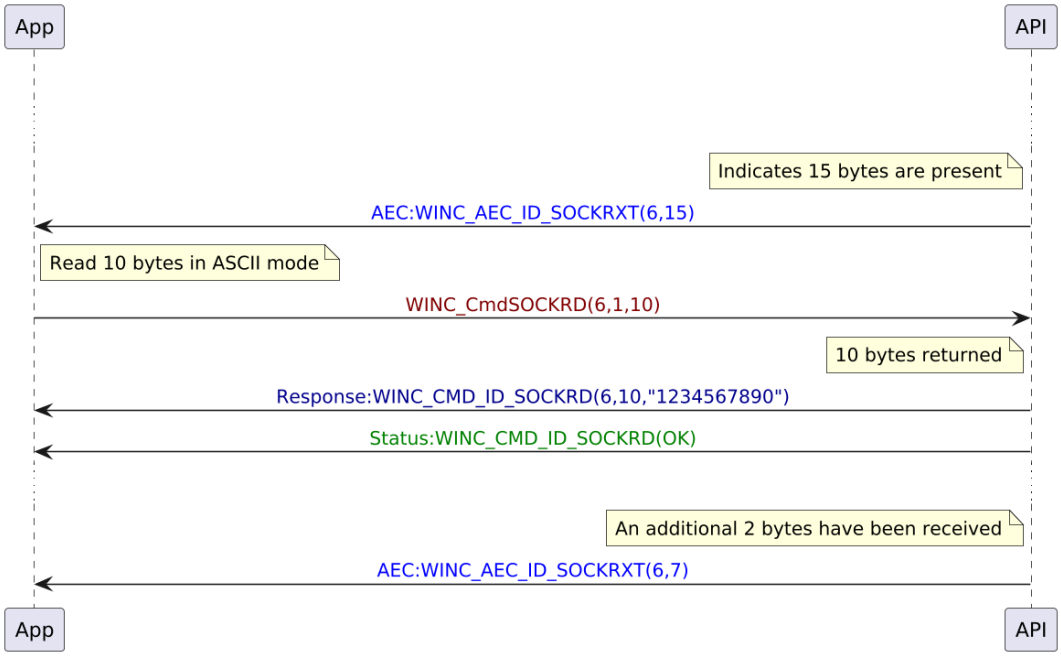


Figure 43. Basic socket read

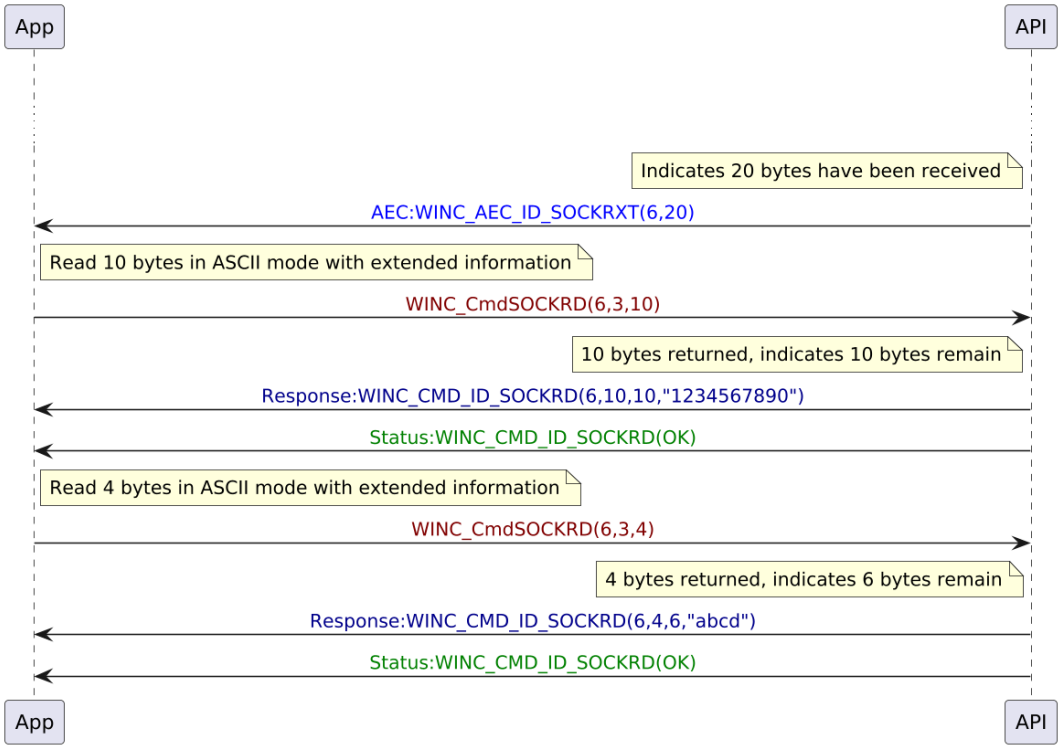


Figure 44. In-band socket read

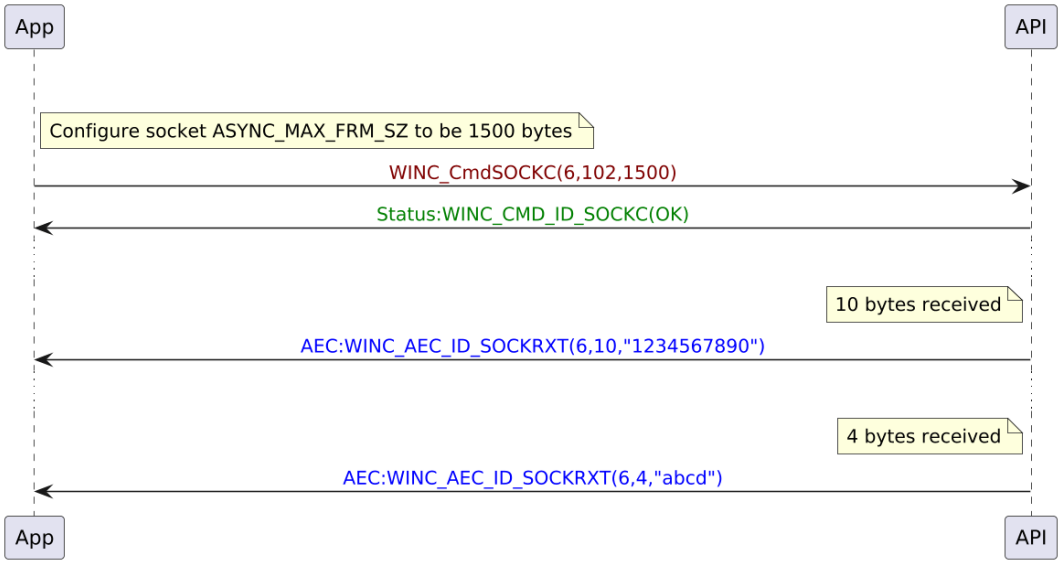


Figure 45. Simple asynchronous socket read

 ©2025-2026

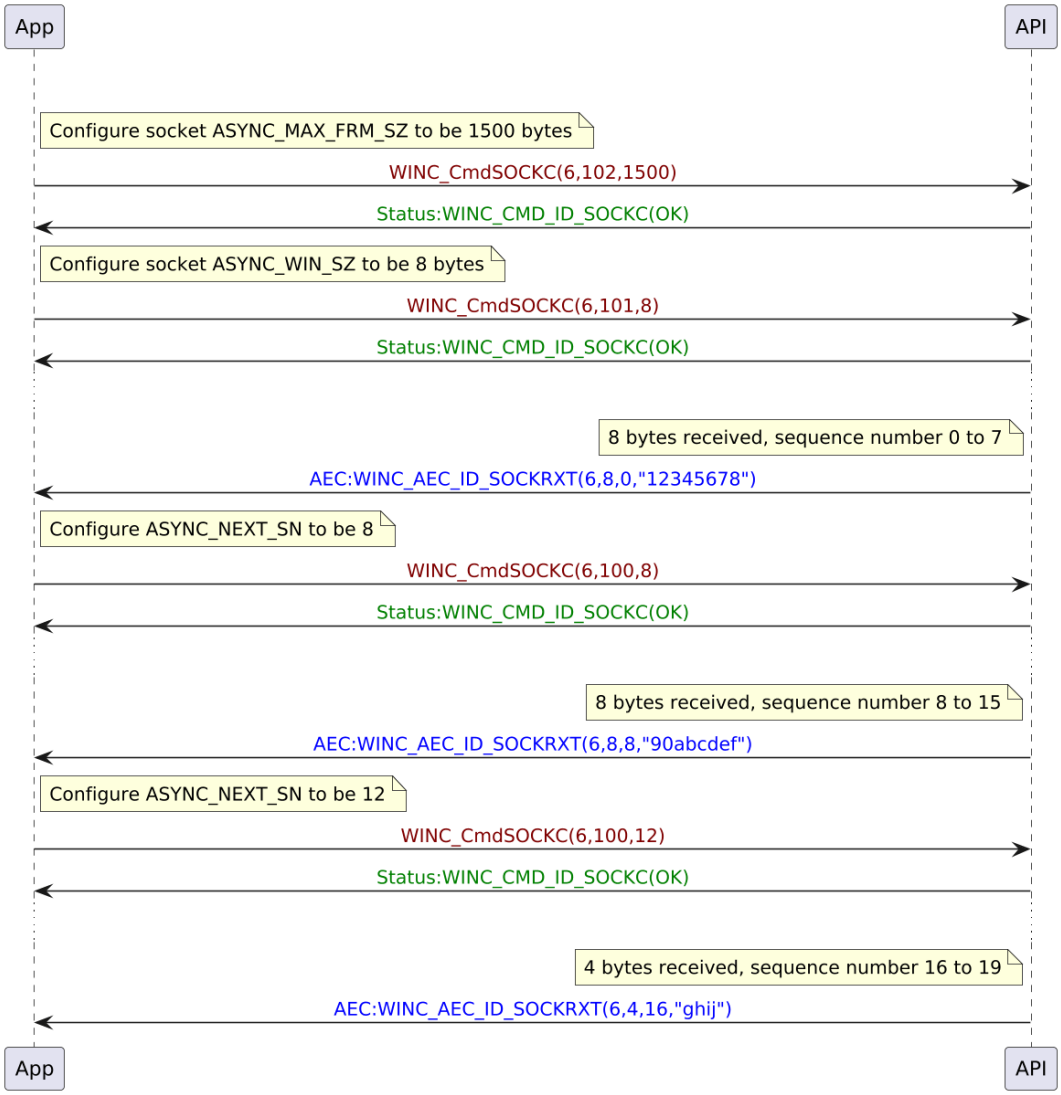


Figure 46. Reliable asynchronous socket read



Figure 47. Set TLS configuration 1 for TLS client

 ©2025-2026

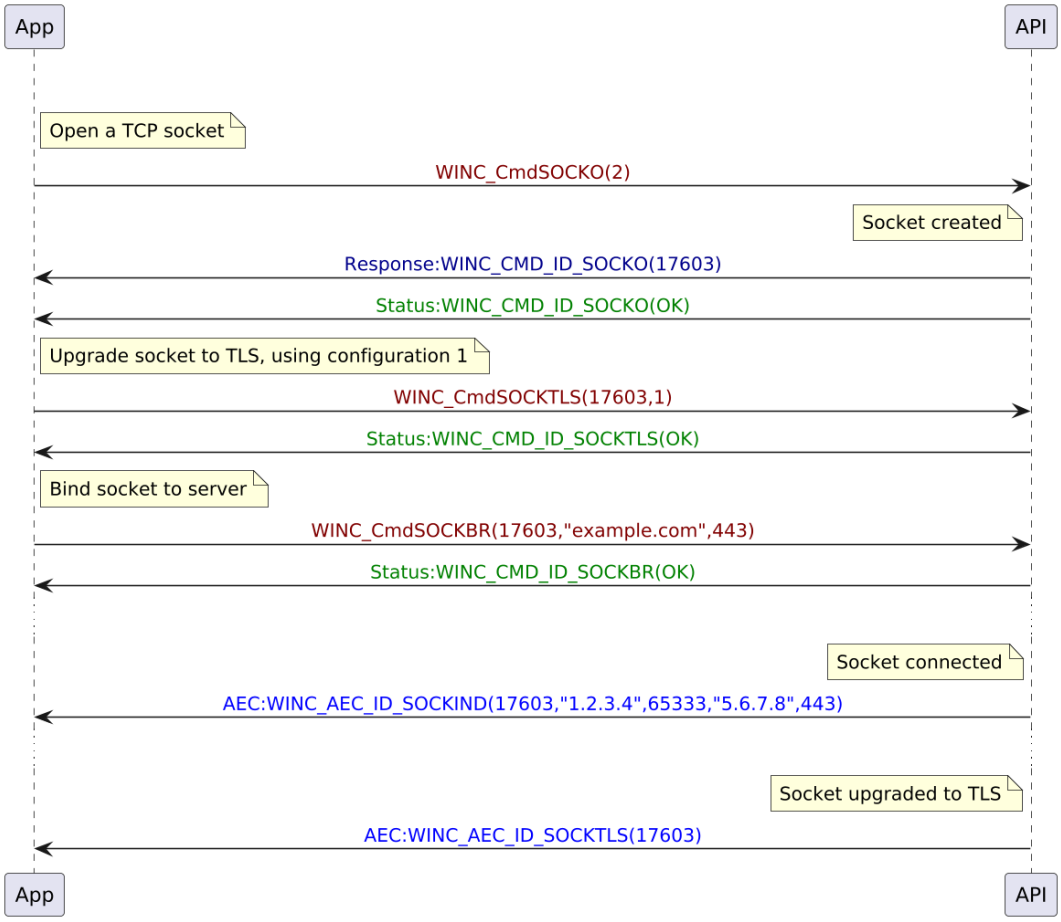


Figure 48. Open a TLS client socket

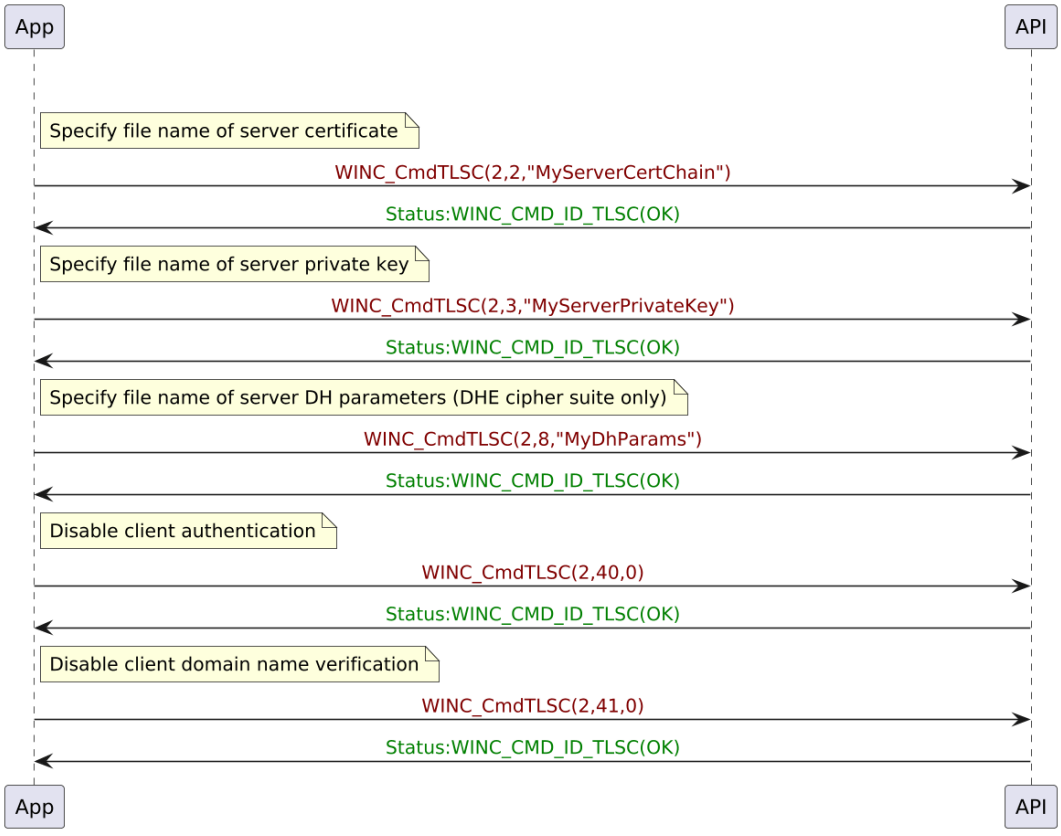


Figure 49. Set TLS configuration 2 for TLS server

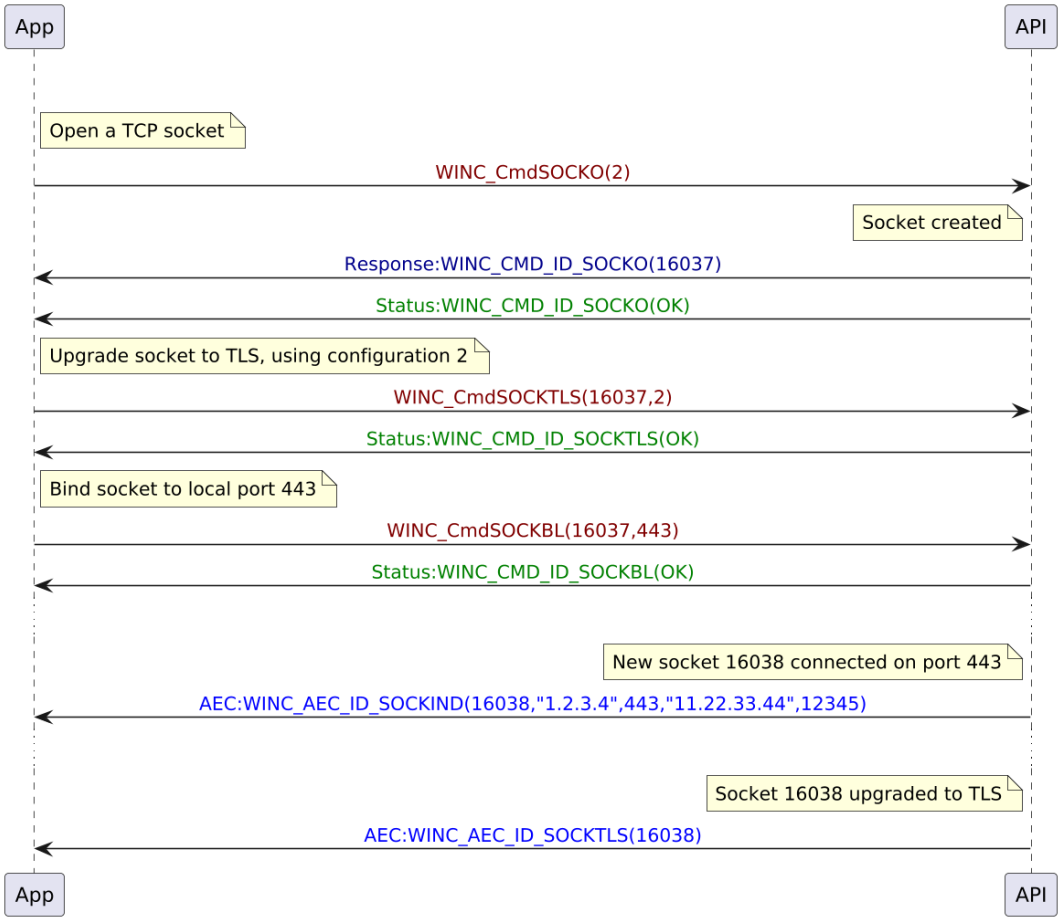


Figure 50. Open a TLS server socket

 ©2025-2026

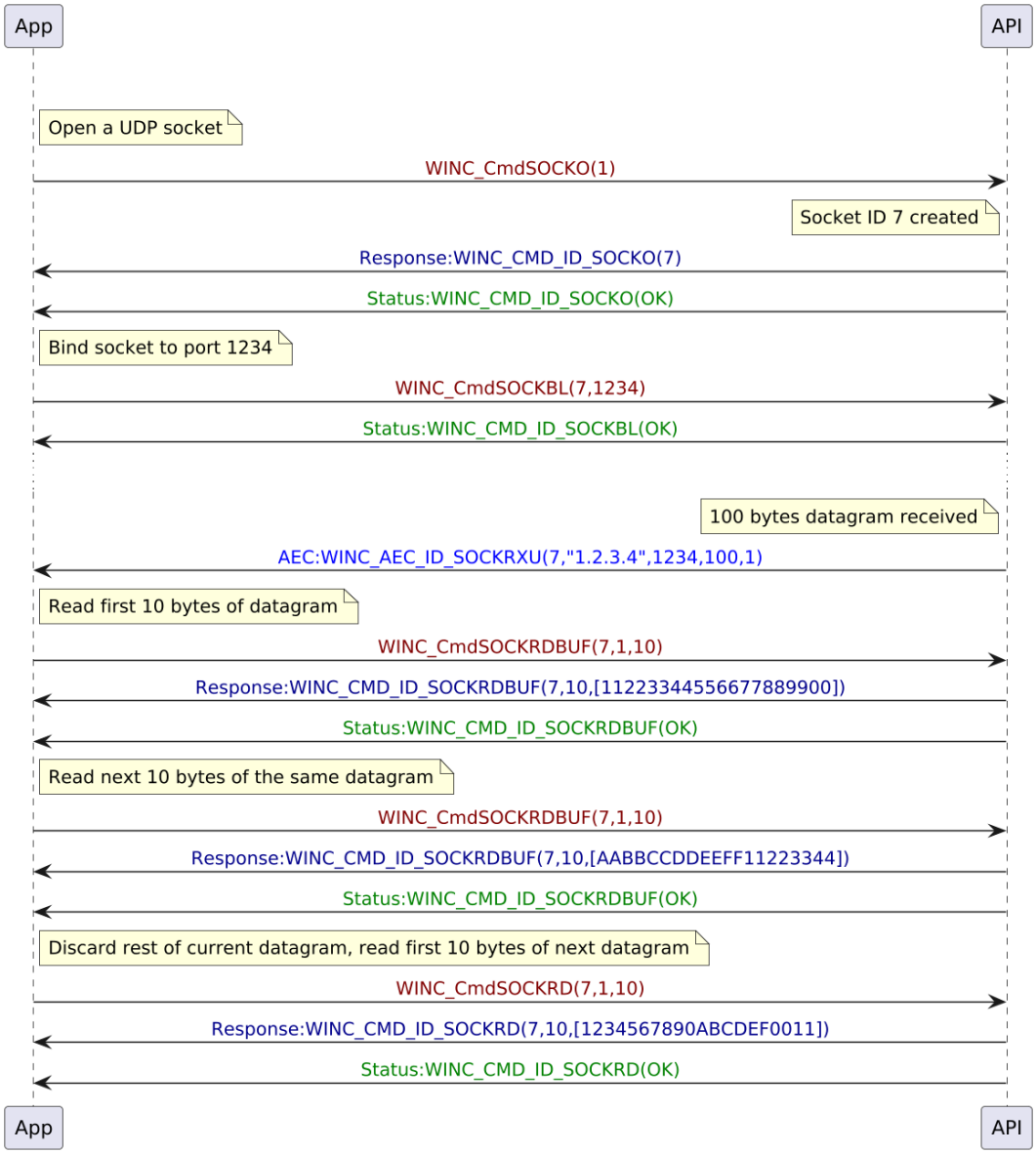


Figure 51. Reading partial datagram

TIME (Module ID = 16)

Introduction

The TIME module sets and reads the system time. Three formats are supported:

- UNIX: seconds since 1970-01-01.
- NTP: seconds since 1900-01-01.
- RFC3339: a string of the form YYYY-MM-DDTHH:MM:SS.00Z.

A query with WINC_CmdTIME reports the time in a WINC_AEC_ID_TIME AEC followed by the command status. The time is set with WINC_CmdTIMEUTCSEC or WINC_CmdTIMERFC.

Until the time is set it counts from 1970-01-01 at boot. When the SNTP client is enabled, each synchronisation overwrites a manually set time. The time is not retained across a reset.

 ©2025-2026

Function Reference:

WINC_CmdTIME

Description

This function is used to set or query the system time.

Three formats of time are supported, two are in UTC seconds and one is a string based on RFC3339 / ISO-8601 format:

1. UNIX timestamp - UTC seconds since 1st January 1970
2. NTP time - UTC seconds since 1st January 1900
3. RFC3339 / ISO-8601 - String representation of date/time

RFC3339 / ISO-8601 format represents the time as:

```
YYYY-MM-DDTHH:MM:SS.00Z
```

The two fractional-second digits must be present but are ignored; times are reported with ".00".

For example:

```
2019-02-28T12:34:56.00Z - 28th February 2019, 12:34 PM 56 seconds (UTC+0)
```

```
bool WINC_CmdTIME
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t optFormat
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optFormat	uint8_t	Format of time. **Value / Label / Description** 1 / UTC_UNIX / UTC seconds (epoch 01/01/1970 - UNIX timestamp). 2 / UTC_NTP / UTC seconds (epoch 01/01/1900 - NTP time). 3 / RFC3339 / RFC3339 / ISO-8601 format. optFormat will be ignored if its value is WINC_CMDTIME_FORMAT_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

WINC_CmdTIMEUTCSEC

Description

This function is used to set or query the system time.

```
bool WINC_CmdTIMEUTCSEC
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t format
    uint32_t utcSec
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
format	uint8_t	Format of time. Value / Label / Description 1 / UTC_UNIX / UTC seconds (epoch 01/01/1970 - UNIX timestamp). 2 / UTC_NTP / UTC seconds (epoch 01/01/1900 - NTP time). 3 / RFC3339 / RFC3339 / ISO-8601 format.
utcSec	uint32_t	UTC seconds.

Return Values

Value	Description
true	Success.
false	Failure.

WINC_CmdTIMERFC

Description

This function is used to set or query the system time.

```
bool WINC_CmdTIMERFC
(
    WINC_CMD_REQ_HANDLE handle
    const uint8_t* pDateTime
    size_t lenDateTime
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
pDateTime	const uint8_t*	Date/time in format YYYY-MM-DDTHH:MM:SS.00Z.
lenDateTime	size_t	Length of pDateTime in bytes, strings are not NUL terminated so the length must not include a terminator.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

AEC Reference:

WINC_AEC_ID_TIME

Description

Time report.

AEC Syntax

AEC	Description
<TIME>	Time report

AEC Element Syntax

Element Name	Type	Description
<TIME>	INTEGER STRING BYTE_ARRAY	Current time

TLS (Module ID = 17)

Introduction

Transport Layer Security (TLS) establishes an encrypted communication channel between a client and server, ensuring confidentiality and integrity of exchanged data.

The WINCS02 can establish a TLS session in either client mode or server mode. The TLS configuration parameters described in this module are used in conjunction with the SOCK, MQTT, HTTP, OTA, DNS and WSTA modules to enable secure connections.

Quick Start: TLS Client

1. Load the CA certificate which signed the server's certificate using the FS module, with file type 1 (certificate), see WINC_CmdFSLOAD. The certificate may be in PEM or DER format; a PEM file may contain several certificates. File names are limited to 32 characters. The file system supplied with the WINCS02 includes a set of root CA certificates, list them with WINC_CmdFSOP.
2. Make sure the system time is valid. Certificates are checked against the system time, which counts from 1970-01-01 until it is set, so the CA certificate fails date validation until the time is set. Wait for the WINC_AEC_ID_TIME AEC from the SNTP client, or set the time with WINC_CmdTIMEUTCSEC or WINC_CmdTIMERFC. By default the SNTP client only uses an NTP server supplied by DHCP; set NTP_STATIC and NTP_SVR to use a specific server.
3. Configure one of the four TLS configurations with WINC_CmdTLSC:
   - CA_CERT_NAME: the CA certificate file name, with or without its extension. The file must already be loaded.
   - DOMAIN_NAME: the server's domain name, used for domain name verification. It is not taken from the host name used to connect.
   - SERVER_NAME: the name sent in the Server Name Indication extension. It is not set automatically; many cloud services require it.
   - CERT_NAME and PRI_KEY_NAME: only needed if the server authenticates the client.
4. Use the configuration with WINC_CmdSOCKTLS, or with the TLS configuration element of MQTTC, HTTPC, OTAC, DNSC or WSTAC.

See example Basic TLS client configuration.

Only TLS 1.2 is supported. A TLS configuration cannot be changed while it is in use; WINC_CmdTLSC fails with STORE_UPDATE_BLOCKED.

 ©2025-2026

Server Mode

A new TLS configuration has peer authentication and domain name verification enabled, in server mode as well as client mode. A server which does not authenticate its clients must disable both with PEER_AUTH and PEER_DOMAIN_VERIFY, otherwise the WINC_AEC_ID_TLSERR AEC reports CA Cert Missing or Peer Domain Missing. A server needs CERT_NAME and PRI_KEY_NAME, and DH_PARAM_NAME if DHE cipher suites are used.

See example Basic TLS server configuration.

Troubleshooting

Errors in a TLS configuration are reported by the WINC_AEC_ID_TLSERR AEC when a connection using it starts:

- CA Cert Missing: peer authentication is enabled and no CA certificate is set.
- CA Cert Validation: the CA certificate file could not be parsed.
- CA Cert Date Validation: the system time has not been set, or the CA certificate has expired.
- Key Pair Incomplete: only one of the certificate and private key is set.
- Peer Domain Missing: domain name verification is enabled and no domain name is set.

TLS handshake failures are reported by the module using the configuration:

- SOCK: WINC_AEC_ID_SOCKERR with Socket TLS Failed.
- HTTP: WINC_AEC_ID_HTTPERR with HTTP TLS Failed.
- OTA: WINC_AEC_ID_OTAERR with TLS Error.
- MQTT: WINC_AEC_ID_MQTTCONN reporting disconnected.
- WSTA (Enterprise): WINC_AEC_ID_WSTAERR with STA Connection Failed.

The wolfSSL error code is only reported through the SYSLOG module. Enable it with WINC_CmdSYSLOGC and look for a WINC_AEC_ID_SYSLOG AEC from the TLS module containing "wolfSSL error: <code>". Common codes, see TLS Error Codes for the full list:

- -188 ASN_NO_SIGNER_E: the CA certificate did not sign the server's certificate.
- -322 DOMAIN_NAME_MISMATCH: the domain name does not match the server's certificate.
- -150 ASN_BEFORE_DATE_E or -151 ASN_AFTER_DATE_E: the server's certificate is outside its validity period, or the system time has not been set.
- -155 ASN_SIG_CONFIRM_E: a certificate signature could not be verified.

TLS configuration

Set TLS parameters using the WINC_CmdTLSC and WINC_CmdTLSCSC functions.

A new TLS configuration has the following security options enabled by default:

- Peer authentication (PEER_AUTH): a CA certificate must be provided via CA_CERT_NAME.
- Peer domain name verification (PEER_DOMAIN_VERIFY): the expected domain name must be provided via DOMAIN_NAME.
- Server renegotiation info required (SERVER_RENEG_INFO, client mode only).
- Session caching (SESSION_CACHING, client mode only).

If a required certificate or domain name is not configured, the TLS handshake fails. Peer authentication and domain name verification apply in both client and server mode; a server that does not authenticate its clients must disable them.

See examples Basic TLS client configuration and Basic TLS server configuration.

Certificate and Private Key

The X.509 certificate and the corresponding private key are required for the peer to authenticate the device.

The certificate file name can be specified via CERT_NAME and must be in PEM or DER format. If the file contains multiple certificates, it must be in PEM format. The private key file name can be specified via PRI_KEY_NAME and must be in PEM or DER format. If the private key is encrypted, the password must also be provided via PRI_KEY_PASSWORD.

Certificate and private key are always required for TLS server mode. For client mode, they are required if the server authenticates the client.

Server Name

The TLS Server Name Indication (SNI) extension allows a client to specify the hostname it is connecting to during the TLS handshake. This enables a server hosting multiple domains on the same IP address to present the correct certificate.

The server name is configured via SERVER_NAME. This parameter only applies to client mode.

Peer authentication

Peer authentication validates the peer's certificate chain against trusted CA certificates provided in the CA certificate file.

Peer authentication is enabled by default and is controlled by PEER_AUTH. When it is enabled, a valid CA certificate file name must be provided via CA_CERT_NAME. If the file contains multiple certificates, it must be in PEM format.

Peer authentication is recommended in client mode.

 ©2025-2026

Domain Name Verification

Domain name verification confirms the peer's certificate was issued for the expected server by matching the configured domain name against the certificate's Subject Alternative Name (SAN), or Common Name (CN) if no SAN is present.

Domain name verification is enabled by default and is controlled by PEER_DOMAIN_VERIFY. When it is enabled, the expected domain name must be provided via DOMAIN_NAME.

Peer domain verification is recommended in client mode.

Cipher Suites

A cipher suite defines the algorithms used for key exchange, data encryption, and message authentication during a TLS session. The cipher suites index (CIPHER_SUITES_IDX) references a cipher suite list configured using the WINC_CmdTLSCSC function.

Diffie-Hellman Parameters

Diffie-Hellman (DH) parameters define the prime number and generator used to establish a shared secret between client and server without transmitting the secret itself. The file containing the Diffie-Hellman parameters must be specified via DH_PARAM_NAME if DHE key exchange cipher suites are enabled. Diffie-Hellman parameters are only required for server mode.

Certificate Pinning

Certificate pinning allows specifying a list of expected hash digest/hash algorithm pairs. During the TLS handshake, the peer's leaf certificate is hashed and compared against the configured digests. If none match, the connection is rejected.

Certificate pinning provides additional security beyond CA-based authentication by ensuring connections are only made to servers with specific known certificates, protecting against compromised CAs or misissued certificates.

See example configuring certificate digests

External crypto operations

TLS client authentication allows a server to verify the client's identity by requesting the client to sign a message with its private key, which is verified against the public key in the X.509 certificate. Signing operations can be delegated to an external crypto device via the EXTCRYPTO module.

When signing is enabled via EXTCRYPTO_OPS, only the certificate file must be provided via CERT_NAME. The private key is not required as it resides in the external crypto device.

See example TLS client configuration with external crypto device

Server Renegotiation Extension

The TLS renegotiation info extension was introduced to address the TLS renegotiation vulnerability (CVE-2009-3555). Servers that are patched against this vulnerability include the renegotiation info extension during the TLS handshake.

When the server renegotiation info (SERVER_RENEG_INFO) is configured as "required", the client rejects connections to servers that do not provide it. Otherwise, the client will connect even if the server does not send the extension. "required" is the recommended setting unless connecting to legacy servers that do not support the security patch. This option is only applicable to client mode.

Session Caching

Session caching enables TLS session resumption using cached session data. When a TLS handshake completes, both client and server retain session state (including the master secret and negotiated cipher suite). On subsequent connections to the same server, this cached session can be reused to perform an abbreviated handshake that skips computationally expensive public-key operations. Session caching significantly reduces connection establishment time.

When a TLS configuration parameter is modified, the previously cached session is discarded.

 ©2025-2026

TLS Cipher Suite codes

The supported cipher suites are listed below.

Code	Cipher Suite Name
0x002F	TLS_RSA_WITH_AES_128_CBC_SHA
0x0035	TLS_RSA_WITH_AES_256_CBC_SHA
0x0033	TLS_DHE_RSA_WITH_AES_128_CBC_SHA
0x0039	TLS_DHE_RSA_WITH_AES_256_CBC_SHA
0x003C	TLS_RSA_WITH_AES_128_CBC_SHA256
0x003D	TLS_RSA_WITH_AES_256_CBC_SHA256
0x0067	TLS_DHE_RSA_WITH_AES_128_CBC_SHA256
0x006B	TLS_DHE_RSA_WITH_AES_256_CBC_SHA256
0x009C	TLS_RSA_WITH_AES_128_GCM_SHA256
0x009D	TLS_RSA_WITH_AES_256_GCM_SHA384
0x009E	TLS_DHE_RSA_WITH_AES_128_GCM_SHA256
0x009F	TLS_DHE_RSA_WITH_AES_256_GCM_SHA384
0xC009	TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA
0xC00A	TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA
0xC013	TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA
0xC014	TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA
0xC023	TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256
0xC024	TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384
0xC027	TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256
0xC028	TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384
0xC02B	TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256
0xC02C	TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384
0xC02F	TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
0xC030	TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384

TLS Error Codes

To assist with debugging TLS issues, error codes from wolfSSL can be obtained by enabling logging via WINC_CmdSYSLOGC.

Error Code Enum	Error Code	Error Description
OPEN_RAN_E	-101	opening random device error
READ_RAN_E	-102	reading random device error
WINCRYPT_E	-103	windows crypt init error
CRYPTGEN_E	-104	windows crypt generation error
RAN_BLOCK_E	-105	reading random device would block
BAD_MUTEX_E	-106	Bad mutex operation
MP_INIT_E	-110	mp_init error state
MP_READ_E	-111	mp_read error state
MP_EXPTMOD_E	-112	mp_exptmod error state
MP_TO_E	-113	mp_to_xxx error state, can't convert
MP_SUB_E	-114	mp_sub error state, can't subtract
MP_ADD_E	-115	mp_add error state, can't add
MP_MUL_E	-116	mp_mul error state, can't multiply
MP_MULMOD_E	-117	mp_mulmod error state, can't multiply mod
MP_MOD_E	-118	mp_mod error state, can't mod
MP_INVMOD_E	-119	mp_invmod error state, can't inv mod
MP_CMP_E	-120	mp_cmp error state
MP_ZERO_E	-121	got a mp zero result, not expected
MEMORY_E	-125	out of memory error
RSA_WRONG_TYPE_E	-130	RSA wrong block type for RSA function
RSA_BUFFER_E	-131	RSA buffer error, output too small or input too large
BUFFER_E	-132	output buffer too small or input too large
ALGO_ID_E	-133	setting algo id error
PUBLIC_KEY_E	-134	setting public key error
DATE_E	-135	setting date validity error
SUBJECT_E	-136	setting subject name error
ISSUER_E	-137	setting issuer name error
CA_TRUE_E	-138	setting CA basic constraint true error
EXTENSIONS_E	-139	setting extensions error
ASN_PARSE_E	-140	ASN parsing error, invalid input
ASN_VERSION_E	-141	ASN version error, invalid number
ASN_GETINT_E	-142	ASN get big int error, invalid data
ASN_RSA_KEY_E	-143	ASN key init error, invalid input
ASN_OBJECT_ID_E	-144	ASN object id error, invalid id
ASN_TAG_NULL_E	-145	ASN tag error, not null
ASN_EXPECT_0_E	-146	ASN expect error, not zero

 ©2025-2026

ASN_BITSTR_E	-147	ASN bit string error, wrong id
ASN_UNKNOWN_OID_E	-148	ASN oid error, unknown sum id
ASN_DATE_SZ_E	-149	ASN date error, bad size
ASN_BEFORE_DATE_E	-150	ASN date error, current date before
ASN_AFTER_DATE_E	-151	ASN date error, current date after
ASN_SIG_OID_E	-152	ASN signature error, mismatched oid
ASN_TIME_E	-153	ASN time error, unknown time type
ASN_INPUT_E	-154	ASN input error, not enough data
ASN_SIG_CONFIRM_E	-155	ASN sig error, confirm failure
ASN_SIG_HASH_E	-156	ASN sig error, unsupported hash type
ASN_SIG_KEY_E	-157	ASN sig error, unsupported key type
ASN_DH_KEY_E	-158	ASN key init error, invalid input
ASN_CRIT_EXT_E	-160	ASN unsupported critical extension
ECC_BAD_ARG_E	-170	ECC input argument of wrong type
ASN_ECC_KEY_E	-171	ASN ECC bad input
ECC_CURVE_OID_E	-172	Unsupported ECC OID curve type
BAD_FUNC_ARG	-173	Bad function argument provided
NOT_COMPILED_IN	-174	Feature not compiled in
UNICODE_SIZE_E	-175	Unicode password too big
NO_PASSWORD	-176	no password provided by user
ALT_NAME_E	-177	alt name size problem, too big
AES_GCM_AUTH_E	-180	AES-GCM Authentication check failure
AES_CCM_AUTH_E	-181	AES-CCM Authentication check failure
CAVIUM_INIT_E	-182	Cavium Init type error
COMPRESS_INIT_E	-183	Compress init error
COMPRESS_E	-184	Compress error
DECOMPRESS_INIT_E	-185	DeCompress init error
DECOMPRESS_E	-186	DeCompress error
BAD_ALIGN_E	-187	Bad alignment for operation, no alloc
ASN_NO_SIGNER_E	-188	ASN sig error, no CA signer to verify certificate
ASN_CRL_CONFIRM_E	-189	ASN CRL no signer to confirm failure
ASN_CRL_NO_SIGNER_E	-190	ASN CRL no signer to confirm failure
ASN_OCSP_CONFIRM_E	-191	ASN OCSP signature confirm failure
BAD_ENC_STATE_E	-192	Bad ecc enc state operation
BAD_PADDING_E	-193	Bad padding, msg not correct length
REQ_ATTRIBUTE_E	-194	Setting cert request attributes error
PKCS7_OID_E	-195	PKCS#7, mismatched OID error
PKCS7_RECIP_E	-196	PKCS#7, recipient error
FIPS_NOT_ALLOWED_E	-197	FIPS not allowed error
ASN_NAME_INVALID_E	-198	ASN name constraint error
RNG_FAILURE_E	-199	RNG Failed, Reinitialize
HMAC_MIN_KEYLEN_E	-200	FIPS Mode HMAC Minimum Key Length error

RSA_PAD_E	-201	RSA Padding Error
LENGTH_ONLY_E	-202	Returning output length only
IN_CORE_FIPS_E	-203	In Core Integrity check failure
AES_KAT_FIPS_E	-204	AES KAT failure
DES3_KAT_FIPS_E	-205	DES3 KAT failure
HMAC_KAT_FIPS_E	-206	HMAC KAT failure
RSA_KAT_FIPS_E	-207	RSA KAT failure
DRBG_KAT_FIPS_E	-208	HASH DRBG KAT failure
DRBG_CONT_FIPS_E	-209	HASH DRBG Continuous test failure
AESGCM_KAT_FIPS_E	-210	AESGCM KAT failure
THREAD_STORE_KEY_E	-211	Thread local storage key create failure
THREAD_STORE_SET_E	-212	Thread local storage key set failure
MAC_CMP_FAILED_E	-213	MAC comparison failed
IS_POINT_E	-214	ECC is point on curve failed
ECC_INF_E	-215	ECC point infinity error
ECC_PRIV_KEY_E	-216	ECC private key not valid error
SRP_CALL_ORDER_E	-217	SRP function called in the wrong order
SRP_VERIFY_E	-218	SRP proof verification failed
SRP_BAD_KEY_E	-219	SRP bad ephemeral values
ASN_NO_SKID	-220	ASN no Subject Key Identifier found
ASN_NO_AKID	-221	ASN no Authority Key Identifier found
ASN_NO_KEYUSAGE	-223	ASN no Key Usage found
SKID_E	-224	Setting Subject Key Identifier error
AKID_E	-225	Setting Authority Key Identifier error
KEYUSAGE_E	-226	Bad Key Usage value
CERTPOLICIES_E	-227	Setting Certificate Policies error
WC_INIT_E	-228	wolfCrypt failed to initialize
SIG_VERIFY_E	-229	wolfCrypt signature verify error
BAD_PKCS7_SIGNEEDS_CHECKCOND_E	-230	Bad condition variable operation
SIG_TYPE_E	-231	Signature Type not enabled/available
HASH_TYPE_E	-232	Hash Type not enabled/available
WC_KEY_SIZE_E	-234	Key size error, either too small or large
ASN_COUNTRY_SIZE_E	-235	ASN Cert Gen, invalid country code size
MISSING_RNG_E	-236	RNG required but not provided
ASN_PATHLEN_SIZE_E	-237	ASN CA path length too large error
ASN_PATHLEN_INV_E	-238	ASN CA path length inversion error
BAD_KEYWRAP_ALG_E	-239	Algorithm error with keywrap
BAD_KEYWRAP_IV_E	-240	Decrypted AES key wrap IV incorrect
WC_CLEANUP_E	-241	wolfCrypt cleanup failed
ECC_CDH_KAT_FIPS_E	-242	ECC CDH known answer test failure
DH_CHECK_PUB_E	-243	DH check public key error
BAD_PATH_ERROR	-244	Bad path for opendir

 ©2025-2026

ASYNC_OP_E	-245	Async operation error
ECC_PRIVATEONLY_E	-246	Invalid use of private only ECC key
EXTKEYUSAGE_E	-247	Bad extended key usage value
WC_HW_E	-248	Error with hardware crypto use
WC_HW_WAIT_E	-249	Hardware waiting on resource
PSS_SALTLEN_E	-250	PSS length of salt is too long for hash
PRIME_GEN_E	-251	Failure finding a prime
BER_INDEF_E	-252	Cannot decode indefinite length BER
RSA_OUT_OF_RANGE_E	-253	Ciphertext to decrypt out of range
RSAPSS_PAT_FIPS_E	-254	RSA-PSS PAT failure
ECDSA_PAT_FIPS_E	-255	ECDSA PAT failure
DH_KAT_FIPS_E	-256	DH KAT failure
AESCCM_KAT_FIPS_E	-257	AESCCM KAT failure
SHA3_KAT_FIPS_E	-258	SHA-3 KAT failure
ECDHE_KAT_FIPS_E	-259	ECDHE KAT failure
AES_GCM_OVERFLOW_E	-260	AES-GCM invocation counter overflow
AES_CCM_OVERFLOW_E	-261	AES-CCM invocation counter overflow
RSA_KEY_PAIR_E	-262	RSA Key Pair-Wise consistency check fail
DH_CHECK_PRIVE_E	-263	DH check private key error
WC_AFALG_SOCK_E	-264	AF_ALG socket error
WC_DEVCRYPTO_E	-265	/dev/crypto error
ZLIB_INIT_ERROR	-266	Zlib init error
ZLIB_COMPRESS_ERROR	-267	Zlib compression error
ZLIB_DECOMPRESS_ERROR	-268	Zlib decompression error
PKCS7_NO_SIGNER_E	-269	No signer in PKCS7 signed data msg
WC_PKCS7_WANT_READ_E	-270	PKCS7 stream operation wants more input
CRYPTOCB_UNAVAILABLE	-271	Crypto callback unavailable
PKCS7_SIGNEEDS_CHECK	-272	Signature needs verified by caller
ASN_SELF_SIGNED_E	-275	ASN self-signed certificate error
MIN_CODE_E	-300	errors -101 - -299
INPUT_CASE_ERROR	-301	process input state error
PREFIX_ERROR	-302	bad index to key rounds
MEMORY_ERROR	-303	out of memory
VERIFY_FINISHED_ERROR	-304	verify problem on finished
VERIFY_MAC_ERROR	-305	verify mac problem
PARSE_ERROR	-306	parse error on header
UNKNOWN_HANDSHAKE_TYPE	-307	weird handshake type
SOCKET_ERROR_E	-308	error state on socket
SOCKET_NODATA	-309	expected data, not there
INCOMPLETE_DATA	-310	don’t have enough data to complete task
UNKNOWN_RECORD_TYPE	-311	unknown type in record hdr
DECRYPT_ERROR	-312	error during decryption

FATAL_ERROR	-313	revcd alert fatal error
ENCRYPT_ERROR	-314	error during encryption
FREAD_ERROR	-315	fread problem
NO_PEER_KEY	-316	need peer’s key
NO_PRIVATE_KEY	-317	need the private key
RSA_PRIVATE_ERROR	-318	error during rsa priv op
NO_DH_PARAMS	-319	server missing DH params
BUILD_MSG_ERROR	-320	build message failure
BAD_HELLO	-321	client hello malformed
DOMAIN_NAME_MISMATCH	-322	peer subject name mismatch
WANT_READ	-323	want read, call again
NOT_READY_ERROR	-324	handshake layer not ready
VERSION_ERROR	-326	record layer version error
WANT_WRITE	-327	want write, call again
BUFFER_ERROR	-328	malformed buffer input
VERIFY_CERT_ERROR	-329	verify cert error
VERIFY_SIGN_ERROR	-330	verify sign error
CLIENT_ID_ERROR	-331	psk client identity error
SERVER_HINT_ERROR	-332	psk server hint error
PSK_KEY_ERROR	-333	psk key error
GETTIME_ERROR	-337	gettimeofday() error
GETITIMER_ERROR	-338	getitimer() error
SIGACT_ERROR	-339	sigaction() error
SETITIMER_ERROR	-340	setitimer() error
LENGTH_ERROR	-341	record layer length error
PEER_KEY_ERROR	-342	cant decode peer key
ZERO_RETURN	-343	peer sent close notify
SIDE_ERROR	-344	wrong client/server type
NO_PEER_CERT	-345	peer didn’t send key
ECC_CURVETYPE_ERROR	-350	Bad ECC Curve Type
ECC_CURVE_ERROR	-351	Bad ECC Curve
ECC_PEERKEY_ERROR	-352	Bad Peer ECC Key
ECC_MAKEKEY_ERROR	-353	Bad Make ECC Key
ECC_EXPORT_ERROR	-354	Bad ECC Export Key
ECC_SHARED_ERROR	-355	Bad ECC Shared Secret
NOT_CA_ERROR	-357	Not CA cert error
BAD_CERT_MANAGER_ERROR	-359	Bad Cert Manager
OCSP_CERT_REVOKED	-360	OCSP Certificate revoked
CRL_CERT_REVOKED	-361	CRL Certificate revoked
CRL_MISSING	-362	CRL Not loaded
MONITOR_SETUP_E	-363	CRL Monitor setup error
THREAD_CREATE_E	-364	Thread Create Error

 ©2025-2026

OCSP_NEED_URL	-365	OCSP need an URL for lookup
OCSP_CERT_UNKNOWN	-366	OCSP responder doesn’t know
OCSP_LOOKUP_FAIL	-367	OCSP lookup not successful
MAX_CHAIN_ERROR	-368	max chain depth exceeded
COOKIE_ERROR	-369	dtls cookie error
SEQUENCE_ERROR	-370	dtls sequence error
SUITES_ERROR	-371	suites pointer error
OUT_OF_ORDER_E	-373	out of order message
BAD_KEA_TYPE_E	-374	bad KEA type found
SANITY_CIPHER_E	-375	sanity check on cipher error
RECV_OVERFLOW_E	-376	RXCB returned more than rqed
GEN_COOKIE_E	-377	Generate Cookie Error
NO_PEER_VERIFY	-378	Need peer cert verify Error
FWRITE_ERROR	-379	fwrite problem
CACHE_MATCH_ERROR	-380	cache hrd match error
UNKNOWN_SNI_HOST_NAME_E	-381	Unrecognized host name Error
UNKNOWN_MAX_FRAG_LEN_E	-382	Unrecognized max frag len Error
KEYUSE_SIGNATURE_E	-383	KeyUse digSignature error
KEYUSE_ENCIPHER_E	-385	KeyUse KeyEncipher error
EXTKEYUSE_AUTH_E	-386	Extended Key Usage server/client auth not set
SEND_OOB_READ_E	-387	Send Cb out of bounds read
SECURE_RENEGOTIATION_E	-388	Invalid renegotiation info
SESSION_TICKET_LEN_E	-389	Session Ticket too large
SESSION_TICKET_EXPECT_E	-390	Session Ticket missing
SCR_DIFFERENT_CERT_E	-391	SCR Different cert error
SESSION_SECRET_CB_E	-392	Session secret CB fcn failure
NO_CHANGE_CIPHER_E	-393	Finished before change cipher
SANITY_MSG_E	-394	Sanity check on msg order error
DUPLICATE_MST_E	-395	Duplicate message error
SNI_UNSUPPORTED	-396	SSL 3.0 does not support SNI
SOCKET_PEER_CLOSED_E	-397	Underlying transport closed
BAD_TICKET_KEY_CB_SZ	-398	Bad session ticket key cb size
BAD_TICKET_MSG_SZ	-399	Bad session ticket msg size
BAD_TICKET_ENCRYPT	-400	Bad user ticket encrypt
DH_KEY_SIZE_E	-401	DH key too small
SNI_ABSENT_ERROR	-402	No SNI request
RSA_SIGN_FAULT	-403	RSA sign fault
HANDSHAKE_SIZE_ERROR	-404	Handshake message too large
UNKNOWN_ALPN_PROTOCOL_NAME_E	-405	Unrecognized protocol name error
BAD_CERTIFICATE_STATUS_ERROR	-406	Bad certificate status message
OCSP_INVALID_STATUS	-407	Invalid OCSP status
OCSP_WANT_READ	-408	OCSP callback response

RSA_KEY_SIZE_E	-409	RSA key too small
ECC_KEY_SIZE_E	-410	ECC key too small
DTLS_EXPORT_VER_E	-411	Export version error
INPUT_SIZE_E	-412	Input size too big error
CTX_INIT_MUTEX_E	-413	Initialize ctx mutex error
EXT_MASTER_SECRET_NEEDED_E	-414	Need EMS enabled to resume
DTLS_POOL_SZ_E	-415	Exceeded DTLS pool size
DECODE_E	-416	Decode handshake message error
HTTP_TIMEOUT	-417	HTTP timeout for OCSP or CRL req
WRITE_DUP_READ_E	-418	Write dup write side can’t read
WRITE_DUP_WRITE_E	-419	Write dup read side can’t write
INVALID_CERT_CTX_E	-420	TLS cert ctx not matching
BAD_KEY_SHARE_DATA	-421	Key share data invalid
MISSING_HANDSHAKE_DATA	-422	Handshake message missing data
BAD_BINDER	-423	Binder does not match
EXT_NOT_ALLOWED	-424	Extension not allowed in msg
INVALID_PARAMETER	-425	Security parameter invalid
MCAST_HIGHWATER_CB_E	-426	Multicast highwater cb error
ALERT_COUNT_E	-427	Alert count exceeded error
EXT_MISSING	-428	Required extension not found
UNSUPPORTED_EXTENSION	-429	TLS extension not requested by the client
PRF_MISSING	-430	PRF not compiled in
DTLS_RETX_OVER_TX	-431	Retransmit DTLS flight over
DH_PARAMS_NOT_FFDHE_E	-432	DH params from server not FFDHE
TCA_INVALID_ID_TYPE	-433	TLS extension Trusted CA ID type invalid
TCA_ABSENT_ERROR	-434	TLS extension Trusted CA ID response absent
UNSUPPORTED_SUITE	-500	Unsupported cipher suite
MATCH_SUITE_ERROR	-501	Can’t match cipher suite
COMPRESSION_ERROR	-502	Compression mismatch
KEY_SHARE_ERROR	-503	Key share mismatch
POST_HAND_AUTH_ERROR	-504	Client won’t do post-hand auth
HRR_COOKIE_ERROR	-505	HRR msg cookie mismatch

Function Reference:

WINC_CmdTLSC

Description

This function is used to read or set the TLS configuration.

 ©2025-2026

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdTLSC
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t optConf
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optConf	uint8_t	Configuration number. optConf will be ignored if its value is WINC_CMDTLSC_CONF_IGNORE_VAL.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDTLSC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<MIN_IDX>,<MAX_IDX>	Valid index range
<ID>,<VAL>	Read response

Response Element Description

Element Name	Type	Description
<MIN_IDX>	INTEGER	Lowest valid index
<MAX_IDX>	INTEGER	Highest valid index
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_TLS_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset **Value** / **Label** / **Description** 0 / DEFAULT / Default.
1	CA_CERT_NAME	STRING	CA certificate name Maximum length of string is 32
2	CERT_NAME	STRING	Certificate name Maximum length of string is 32
3	PRI_KEY_NAME	STRING	Private key name Maximum length of string is 32
4	PRI_KEY_PASSWORD	STRING	Private key password Maximum length of string is 64
5	SERVER_NAME	STRING	Server name Maximum length of string is 256
6	DOMAIN_NAME	STRING	Domain name Maximum length of string is 256
7	CIPHER_SUITES_IDX	INTEGER	Cipher suite index Valid range is 1 to 2
8	DH_PARAM_NAME	STRING	Server Diffie-Hellman parameters identifier Maximum length of string is 32
20	EXTCRYPTO_OPS	INTEGER	Crypto operations to be done externally **Value** / **Label** / **Description** 0 / NONE / None. 1 / SIGN / Signing.
21	SIGN_TYPES	INTEGER (Read Only)	Types of signing to be done externally **Value** / **Label** / **Description** 1 / ECDSA / ECDSA.
40	PEER_AUTH	BOOL	Peer authentication. Default is enabled **Value** / **Label** / **Description** 0 / DISABLED / Peer authentication is disabled. 1 / ENABLED / Peer authentication is enabled.

 ©2025-2026

<table>
<tr><th>ID</th><th>Name</th><th>Type</th><th>Description</th></tr>
<tr><td>41</td><td>PEER_DOMAIN_VERIFY</td><td>BOOL</td><td>Peer domain verification. Default is enabled

Value | Label | Description
0 | DISABLED | Peer domain name verification is disabled.
1 | ENABLED | Peer domain name verification is enabled.</td></tr>
<tr><td>42</td><td>SERVER_RENEG_INFO</td><td>BOOL</td><td>Client mode: require renegotiation info from server. Default is required

Value | Label | Description
0 | NOT REQUIRED | Do not require renegotiation info from server.
1 | REQUIRED | Require renegotiation info from server.</td></tr>
<tr><td>43</td><td>SESSION_CACHING</td><td>BOOL</td><td>Client mode: cache and reuse session IDs. Default is enabled

Value | Label | Description
0 | DISABLED | Do not reuse session IDs.
1 | ENABLED | Attempt to reuse session IDs.</td></tr>
</table>

WINC_CmdTLSCDA

Description

This function is used to read or set the TLS configuration.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdTLSCDA
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t conf
    const uint8_t* pDigest
    size_t lenDigest
    uint8_t optAlgo
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
conf	uint8_t	Configuration number.
pDigest	const uint8_t*	Hash digest.
lenDigest	size_t	Length of pDigest in bytes.
optAlgo	uint8_t	Hash algorithm. optAlgo will be ignored if its value is WINC_CMDTLSCDA_ALGO_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<MIN_IDX>,<MAX_IDX>	Valid index range
<ID>,<VAL>	Read response

 ©2025-2026

Response Element Description

Element Name	Type	Description
<MIN_IDX>	INTEGER	Lowest valid index
<MAX_IDX>	INTEGER	Highest valid index
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_TLS_*)

ID	Name	Type	Description
9	PEER_CERT_DIGESTS	Complex Value[1]	List of hash digest/algorithm pairs This is a multiple value parameter with an ID range 9.0 to 9.3 See Certificate Digests below.

Complex Value Syntax

No.	Complex Value	Description
1	<DIGEST>,<ALGO>	Hash digest/algorithm pair

Complex Value Element Syntax

Element Name	Type	Description
<DIGEST>	Byte Array	Hash digest
<ALGO>	Integer	Hash algorithm

Certificate Digests

Up to four digest/algorithm pairs can be stored. WINC_CmdTLSCDA with a digest and algorithm adds the pair after the existing entries; this fails if four are already stored. WINC_CmdTLSC with ID 9.<n> replaces that entry.

The digest length must match the algorithm: SHA1 20 bytes, SHA2-224 28 bytes, SHA2-256 32 bytes, SHA2-384 48 bytes, SHA2-512 64 bytes.

WINC_CmdTLSCDA with an empty digest removes all pairs. WINC_CmdTLSC with ID 9.<n> and an empty digest removes that entry only, and later entries move down one place. A pair cannot be removed by specifying its digest.

WINC_CmdTLSCSC

Description

This function is used to read or set the TLS cipher suite configuration.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdTLSCSC
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t optCslIdx
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optCslIdx	uint8_t	Cipher suite list index. optCslIdx will be ignored if its value is WINC_CMDTLSCSC_CSL_IDX_IGNORE_VAL.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDTLSCSC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<MIN_IDX>,<MAX_IDX>	Valid index range
<ID>,<VAL>	Read response

 ©2025-2026

Response Element Description

Element Name	Type	Description
<MIN_IDX>	INTEGER	Lowest valid index
<MAX_IDX>	INTEGER	Highest valid index
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_TLS_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset Value: 0 — Label: DEFAULT — Description: Default.
1	CIPHER_SUITES	INTEGER_UNSIGNED	Cipher suites Positive unsigned 16-bit value This is a multiple value parameter with an ID range 1.0 to 1.31
2	CIPHER_SUITES_AVAIL	INTEGER_UNSIGNED (Read Only)	Cipher suites available Positive unsigned 16-bit value This is a multiple value parameter with an ID range 2.0 to 2.31

AEC Reference:

WINC_AEC_ID_TLSERR

Description

TLS error.

AEC Syntax

AEC	Description
<CONF>,<ERROR_CODE>	TLS error

AEC Element Syntax

Element Name	Type	Description
<CONF>	INTEGER	Configuration number Valid range is 1 to 4
<ERROR_CODE>	INTEGER	Error code See Status Response Codes

 ©2025-2026

Examples:

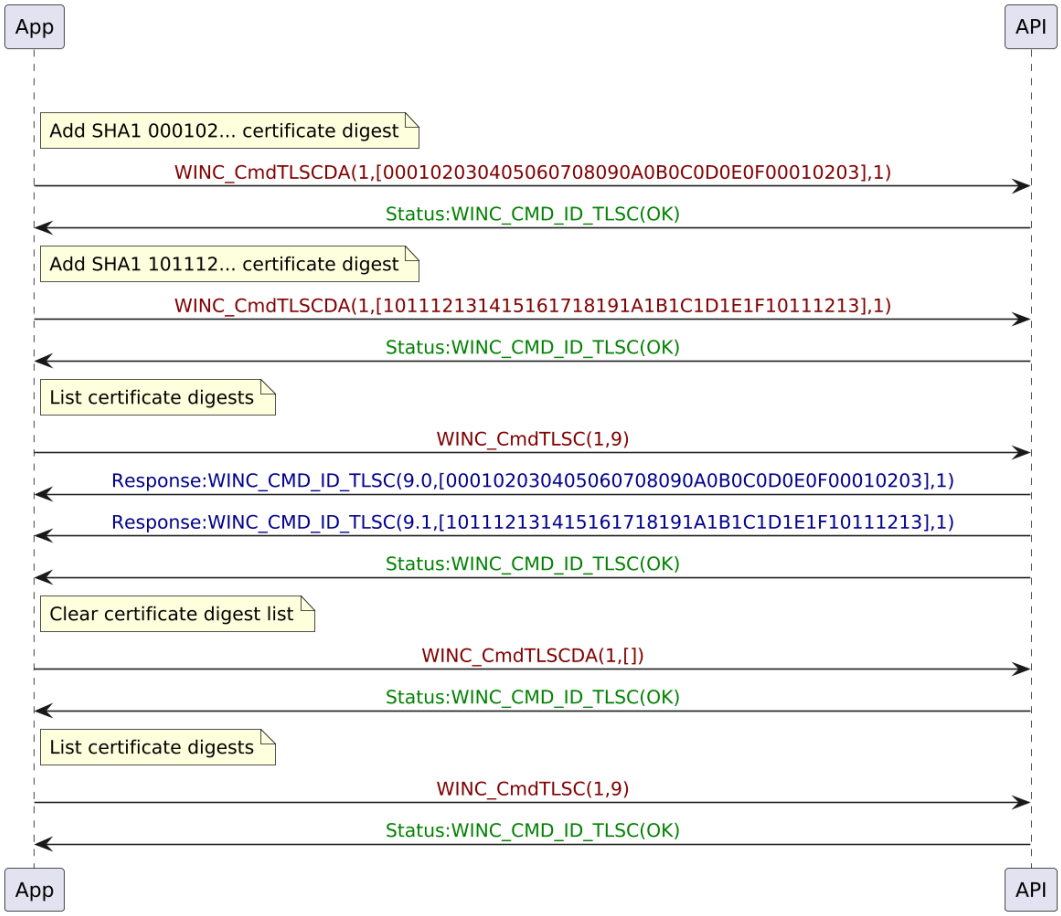


Figure 52. Peer certificate digest pinning with WINC_CmdTLSCDA

WAP (Module ID = 18)

Introduction

The WAP module operates the WINCS02 as a soft access point.

Configure the soft AP with WINC_CmdWAPC and then start it with WINC_CmdWAP. While the soft AP is running only MAC_BLOCKLIST may be changed.

The soft AP cannot be started while the station is connected, or while a scan, periodic scan or the beacon service is running.

Unless the network interface is configured as layer 2 only, starting the soft AP sets the interface address and starts the DHCP server (see DHCP) and the DNS server.

Clients are reported by the WINC_AEC_ID_WAPSC AEC when they connect, WINC_AEC_ID_WAPAIP when they are assigned an address and WINC_AEC_ID_WAPSD when they disconnect. Up to 10 clients are supported; the association IDs are the same as those used by the ASSOC module.

Stopping the soft AP completes asynchronously, the state continues to read as 1 until the soft AP has fully stopped.

 ©2025-2026

Function Reference:

WINC_CmdWAPC

Description

This function is used to read or set the device's hotspot access point configuration.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdWAPC
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDWAPC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_WAP_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset **Value / Label / Description** 0 / DEFAULT / Default.
1	SSID	STRING	Network Name Maximum length of string is 32
2	SEC_TYPE	INTEGER	Security Type **Value / Label / Description** 0 / OPEN / Open. 2 / WPA2_PERS_MIXED / WPA2 Personal Mixed Mode. 3 / WPA2_PERS / WPA2 Personal. 4 / WPA3_PERS_TRANS / WPA3 Personal Transition Mode. 5 / WPA3_PERS / WPA3 Personal.
3	CREDENTIALS	STRING	Credentials required for connecting to the network of the security type specified Maximum length of string is 128
4	CHANNEL	INTEGER	The channel on which to set up the network Valid range is 1 to 13
5	HIDDEN	BOOL	Visibility of the network **Value / Label / Description** 0 / VISIBLE / Not hidden, SSID is broadcast in beacons. 1 / HIDDEN / Hidden, SSID is not broadcast.
8	NETIF_IDX	INTEGER_UNSIGNED	Network interface index Valid range is 0 to 1
9	REKEY_INTERVAL	INTEGER_UNSIGNED	Specify the time in seconds that must pass before each re-key attempt Valid range is 60 to 86400

 ©2025-2026

<table>
<tr><th>ID</th><th>Name</th><th>Type</th><th>Description</th></tr>
<tr><td>20</td><td>MAC_BLOCKLIST</td><td>MACADDR</td><td>MAC address blocklist entry

Valid range is 0 to 3
This is a multiple value parameter
with an ID range 20.0 to 20.3</td></tr>
<tr><td>50</td><td>MFP_TYPE</td><td>INTEGER</td><td>Management Frame Protection configuration

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>DISABLED</td><td>MFP Disabled.</td></tr><tr><td>1</td><td>ENABLED</td><td>MFP Enabled.</td></tr><tr><td>2</td><td>REQUIRED</td><td>MFP Required.</td></tr></table></td></tr>
</table>

WINC_CmdWAP

Description

This function is used to enable the device's hotspot access point functionality.

```
bool WINC_CmdWAP
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optState
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optState	int32_t	State of the hotspot feature. **Value** / **Label** / **Description** 0 / DISABLE / Disable. 1 / ENABLE / Enable. optState will be ignored if its value is WINC_CMDWAP_STATE_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<STATE>	Query state

Response Element Description

Element Name	Type	Description
<STATE>	INTEGER	Connected state **Value** / **Label** / **Description** 0 / DISABLED / Disabled. 1 / ENABLED / Enabled.

 ©2025-2026

AEC Reference:

WINC_AEC_ID_WAPAIP

Description

STA IP address assignment.

AEC Syntax

AEC	Description
<ASSOC_ID>,<IP_ADDRESS>	Assignment of STA IP address

AEC Element Syntax

Element Name	Type	Description
<ASSOC_ID>	INTEGER	Association ID
<IP_ADDRESS>	STRING	IP address assigned

WINC_AEC_ID_WAPSC

Description

STA connected.

AEC Syntax

AEC	Description
<ASSOC_ID>,<MAC_ADDRESS>	Station connect

AEC Element Syntax

Element Name	Type	Description
<ASSOC_ID>	INTEGER	Association ID
<MAC_ADDRESS>	STRING	MAC address of the STA

 ©2025-2026

WINC_AEC_ID_WAPSD

Description

STA disconnected.

AEC Syntax

AEC	Description
<ASSOC_ID>,<MAC_ADDRESS>	Station disconnect

AEC Element Syntax

Element Name	Type	Description
<ASSOC_ID>	INTEGER	Association ID
<MAC_ADDRESS>	STRING	MAC address of the STA

WINC_AEC_ID_WAPERR

Description

AP error.

AEC Syntax

AEC	Description
<ERROR_CODE>	AP error

AEC Element Syntax

Element Name	Type	Description
<ERROR_CODE>	INTEGER	Error code See Status Response Codes

 ©2025-2026

WSCN (Module ID = 19)

Introduction

The WSCN module scans for access points. Two independent scan types are provided, each with its own configuration:

- One-shot active or passive scans, configured with WINC_CmdWSCNC and started with WINC_CmdWSCN.
- Periodic passive scans, configured with WINC_CmdWSCNPC and started and stopped with WINC_CmdWSCNP.

Each access point found is reported by a WINC_AEC_ID_WSCNIND AEC if ASYNC_NOTIFY is set, followed by a WINC_AEC_ID_WSCNDONE AEC with the number of results. The 8 results with the strongest RSSI may be read from SCAN_RESULTS after the scan.

Results may be filtered by minimum RSSI (RSSI_THRESH), by security type (SEC_FILTER) and by up to 4 SSIDs (FILT_LIST), an SSID ending in * matches any SSID beginning with the preceding characters.

Scanning is allowed while the station is connected, but not while it is connecting or disconnecting.

Periodic Scanning

A periodic scan sweeps the configured channels once every INTERVAL milliseconds. The duration of a sweep is the passive slot time multiplied by the number of slots and the number of channels; the interval must be at least twice the sweep duration and at least the sweep duration plus 50 ms, otherwise the scan will not start.

Periodic scanning requires the soft AP to be stopped. The WINC_CmdWSCNPC configuration cannot be changed while a periodic scan is running.

Stopping a periodic scan allows the current sweep to complete and then sends the WINC_AEC_ID_WSCNPDONE AEC. If a sweep fails to start the periodic scan ends with the WINC_AEC_ID_WSCNPERR AEC.

Function Reference:

WINC_CmdWSCNC

Description

This function is used to modify or query the behavior of the active scanning function.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdWSCNC
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDWSCNC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response

 ©2025-2026

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_WSCN_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset **Value** / **Label** / **Description** 0 / DEFAULT / Default.
1	CHANNEL	INTEGER_UNSIGNED	Channel to scan (or 0 for all channels) Valid range is 0 to 13
2	ACT_SLOT_TIME	INTEGER_UNSIGNED	The time in milliseconds to wait for probe responses Valid range is 10 to 1500
3	PASV_SLOT_TIME	INTEGER_UNSIGNED	The time in milliseconds to wait for beacons Valid range is 10 to 1500
4	NUM_SLOTS	INTEGER_UNSIGNED	Number of scan slots Positive unsigned 8-bit value
5	PROBES_PER_SLOT	INTEGER_UNSIGNED	Number of probes sent per active slot Valid range is 1 to 2
6	RSSI_THRESH	INTEGER	RSSI threshold (or 0 for none) Signed 8-bit value
7	SEC_FILTER	INTEGER_UNSIGNED	Security filter, bitmask of: **Value** / **Label** / **Description** 0x01 / PERS / Personal. 0x02 / ENT / Enterprise. 0x04 / OPEN / Open. 0x08 / NA / Unknown or unsupported. 0x10 / WPA / WPA or WPA2 Mixed Mode. 0x20 / WPA2 / WPA2. 0x40 / WPA3 / WPA3 or WPA3 Transition Mode.

<table>
<tr><th>ID</th><th>Name</th><th>Type</th><th>Description</th></tr>
<tr><td>8</td><td>ASYNC_NOTIFY</td><td>BOOL</td><td>Asynchronous notification

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>NO_AECS</td><td>No scan indication AECs are generated.</td></tr><tr><td>1</td><td>USE_AECS</td><td>Indication AECs are generated.</td></tr></table></td></tr>
<tr><td>10</td><td>FILT_LIST</td><td>BYTE_ARRAY</td><td>SSID filter list, an entry ending with '*' matches any SSID with the preceding prefix
This is a multiple value parameter
with an ID range 10.0 to 10.3</td></tr>
<tr><td>11</td><td>CHANMASK24</td><td>INTEGER_UNSIGNED</td><td>Scan-all channel mask (2.4 GHz)

<table><tr><td>15
-</td><td>14
-</td><td>13
CH14</td><td>12
CH13</td><td>11
CH12</td><td>10
CH11</td><td>9
CH10</td><td>8
CH9</td></tr><tr><td>7
CH8</td><td>6
CH7</td><td>5
CH6</td><td>4
CH5</td><td>3
CH4</td><td>2
CH3</td><td>1
CH2</td><td>0
CH1</td></tr></table>
<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0x0001</td><td>CH1</td><td>Channel 1 - 2412 MHz.</td></tr><tr><td>0x0002</td><td>CH2</td><td>Channel 2 - 2417 MHz.</td></tr><tr><td>0x0004</td><td>CH3</td><td>Channel 3 - 2422 MHz.</td></tr><tr><td>0x0008</td><td>CH4</td><td>Channel 4 - 2427 MHz.</td></tr><tr><td>0x0010</td><td>CH5</td><td>Channel 5 - 2432 MHz.</td></tr><tr><td>0x0020</td><td>CH6</td><td>Channel 6 - 2437 MHz.</td></tr><tr><td>0x0040</td><td>CH7</td><td>Channel 7 - 2442 MHz.</td></tr><tr><td>0x0080</td><td>CH8</td><td>Channel 8 - 2447 MHz.</td></tr><tr><td>0x0100</td><td>CH9</td><td>Channel 9 - 2452 MHz.</td></tr><tr><td>0x0200</td><td>CH10</td><td>Channel 10 - 2457 MHz.</td></tr><tr><td>0x0400</td><td>CH11</td><td>Channel 11 - 2462 MHz.</td></tr><tr><td>0x0800</td><td>CH12</td><td>Channel 12 - 2467 MHz.</td></tr><tr><td>0x1000</td><td>CH13</td><td>Channel 13 - 2472 MHz.</td></tr><tr><td>0x2000</td><td>CH14</td><td>Channel 14 - 2485 MHz.</td></tr></table></td></tr>
<tr><td>20</td><td>SCAN_RESULTS</td><td>Complex Value[1]
(Read Only)</td><td>Top scan results from previous scan, ordered by RSSI
This is a multiple value parameter
with an ID range 20.0 to 20.7</td></tr>
</table>

Complex Value Syntax

No.	Complex Value	Description
1	<RSSI>,<SEC_TYPE>,<CHANNEL>,<BSSID>,<SSID>	Scan indicator

 ©2025-2026

Complex Value Element Syntax

<table>
<tr><th>Element Name</th><th>Type</th><th>Description</th></tr>
<tr><td><RSSI></td><td>Integer</td><td>Received signal strength</td></tr>
<tr><td><SEC_TYPE></td><td>Integer</td><td>Recommended security type to use when connecting to this AP

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>-1</td><td>NA</td><td>Unknown or unsupported.</td></tr><tr><td>0</td><td>OPEN</td><td>Open.</td></tr><tr><td>2</td><td>WPA2_PER_MM</td><td>WPA2-Personal Mixed Mode.</td></tr><tr><td>3</td><td>WPA2_PER</td><td>WPA2-Personal.</td></tr><tr><td>4</td><td>WPA3_PER_TM</td><td>WPA3-Personal Transition Mode.</td></tr><tr><td>5</td><td>WPA3_PER</td><td>WPA3-Personal.</td></tr><tr><td>6</td><td>WPA2_ENT_MM</td><td>WPA2-Enterprise Mixed Mode.</td></tr><tr><td>7</td><td>WPA2_ENT</td><td>WPA2-Enterprise.</td></tr><tr><td>8</td><td>WPA3_ENT_TM</td><td>WPA3-Enterprise Transition Mode.</td></tr><tr><td>9</td><td>WPA3_ENT</td><td>WPA3-Enterprise.</td></tr></table></td></tr>
<tr><td><CHANNEL></td><td>Integer</td><td>The channel number of the detected device</td></tr>
<tr><td><BSSID></td><td>String</td><td>The BSSID of detected device</td></tr>
<tr><td><SSID></td><td>String</td><td>SSID of detected device</td></tr>
</table>

WINC_CmdWSCN

Description

This function is used to scan for infrastructure networks in range of the device.

```
bool WINC_CmdWSCN
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t actPasv
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
actPasv	uint8_t	Flag indicating active or passive scanning. **Value** / **Label** / **Description** 0 / PASSIVE / Passive scanning. 1 / ACTIVE / Active scanning.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

WINC_CmdWSCNPC

Description

This function is used to modify or query the behavior of the periodic passive scanning function.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdWSCNPC
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDWSCNPC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_WSCN_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset **Value** / **Label** / **Description** 0 / DEFAULT / Default.
1	CHANNEL	INTEGER_UNSIGNED	Channel to scan (or 0 for all channels) Valid range is 0 to 13
3	PASV_SLOT_TIME	INTEGER_UNSIGNED	The time in milliseconds to wait for beacons Valid range is 10 to 1500
4	NUM_SLOTS	INTEGER_UNSIGNED	Number of scan slots Positive unsigned 8-bit value
6	RSSI_THRESH	INTEGER	RSSI threshold (or 0 for none) Signed 8-bit value
7	SEC_FILTER	INTEGER_UNSIGNED	Security filter, bitmask of: **Value** / **Label** / **Description** 0x01 / PERS / Personal. 0x02 / ENT / Enterprise. 0x04 / OPEN / Open. 0x08 / NA / Unknown or unsupported. 0x10 / WPA / WPA or WPA2 Mixed Mode. 0x20 / WPA2 / WPA2. 0x40 / WPA3 / WPA3 or WPA3 Transition Mode.
8	ASYNC_NOTIFY	BOOL	Asynchronous notification **Value** / **Label** / **Description** 0 / NO_AECS / No scan indication AECs are generated. 1 / USE_AECS / Indication AECs are generated.
10	FILT_LIST	BYTE_ARRAY	SSID filter list, an entry ending with '*' matches any SSID with the preceding prefix This is a multiple value parameter with an ID range 10.0 to 10.3

 ©2025-2026

<table>
<tr><th>ID</th><th>Name</th><th>Type</th><th>Description</th></tr>
<tr><td>11</td><td>CHANMASK24</td><td>INTEGER_UNSIGNED</td><td>Scan-all channel mask (2.4 GHz)

<table>
<tr><td>15
-</td><td>14
-</td><td>13
CH14</td><td>12
CH13</td><td>11
CH12</td><td>10
CH11</td><td>9
CH10</td><td>8
CH9</td></tr>
<tr><td>7
CH8</td><td>6
CH7</td><td>5
CH6</td><td>4
CH5</td><td>3
CH4</td><td>2
CH3</td><td>1
CH2</td><td>0
CH1</td></tr>
</table>

<table>
<tr><th>Value</th><th>Label</th><th>Description</th></tr>
<tr><td>0x0001</td><td>CH1</td><td>Channel 1 - 2412 MHz.</td></tr>
<tr><td>0x0002</td><td>CH2</td><td>Channel 2 - 2417 MHz.</td></tr>
<tr><td>0x0004</td><td>CH3</td><td>Channel 3 - 2422 MHz.</td></tr>
<tr><td>0x0008</td><td>CH4</td><td>Channel 4 - 2427 MHz.</td></tr>
<tr><td>0x0010</td><td>CH5</td><td>Channel 5 - 2432 MHz.</td></tr>
<tr><td>0x0020</td><td>CH6</td><td>Channel 6 - 2437 MHz.</td></tr>
<tr><td>0x0040</td><td>CH7</td><td>Channel 7 - 2442 MHz.</td></tr>
<tr><td>0x0080</td><td>CH8</td><td>Channel 8 - 2447 MHz.</td></tr>
<tr><td>0x0100</td><td>CH9</td><td>Channel 9 - 2452 MHz.</td></tr>
<tr><td>0x0200</td><td>CH10</td><td>Channel 10 - 2457 MHz.</td></tr>
<tr><td>0x0400</td><td>CH11</td><td>Channel 11 - 2462 MHz.</td></tr>
<tr><td>0x0800</td><td>CH12</td><td>Channel 12 - 2467 MHz.</td></tr>
<tr><td>0x1000</td><td>CH13</td><td>Channel 13 - 2472 MHz.</td></tr>
<tr><td>0x2000</td><td>CH14</td><td>Channel 14 - 2485 MHz.</td></tr>
</table>
</td></tr>
<tr><td>12</td><td>INTERVAL</td><td>INTEGER_UNSIGNED</td><td>Interval in milliseconds between the start of one periodic sweep and the start of the next. A sweep must fit within half of it, with a further small margin for result retrieval

Valid range is 500 to 600000</td></tr>
<tr><td>20</td><td>SCAN_RESULTS</td><td>Complex Value[1]
(Read Only)</td><td>Top scan results from previous scan, ordered by RSSI
This is a multiple value parameter
with an ID range 20.0 to 20.7</td></tr>
</table>

Complex Value Syntax

No.	Complex Value	Description
1	<RSSI>,<SEC_TYPE>,<CHANNEL>,<BSSID>,<SSID>	Scan indicator

Complex Value Element Syntax

<table>
<tr><th>Element Name</th><th>Type</th><th>Description</th></tr>
<tr><td><RSSI></td><td>Integer</td><td>Received signal strength</td></tr>
<tr><td><SEC_TYPE></td><td>Integer</td><td>Recommended security type to use when connecting to this AP

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>-1</td><td>NA</td><td>Unknown or unsupported.</td></tr><tr><td>0</td><td>OPEN</td><td>Open.</td></tr><tr><td>2</td><td>WPA2_PER_MM</td><td>WPA2-Personal Mixed Mode.</td></tr><tr><td>3</td><td>WPA2_PER</td><td>WPA2-Personal.</td></tr><tr><td>4</td><td>WPA3_PER_TM</td><td>WPA3-Personal Transition Mode.</td></tr><tr><td>5</td><td>WPA3_PER</td><td>WPA3-Personal.</td></tr><tr><td>6</td><td>WPA2_ENT_MM</td><td>WPA2-Enterprise Mixed Mode.</td></tr><tr><td>7</td><td>WPA2_ENT</td><td>WPA2-Enterprise.</td></tr><tr><td>8</td><td>WPA3_ENT_TM</td><td>WPA3-Enterprise Transition Mode.</td></tr><tr><td>9</td><td>WPA3_ENT</td><td>WPA3-Enterprise.</td></tr></table></td></tr>
<tr><td><CHANNEL></td><td>Integer</td><td>The channel number of the detected device</td></tr>
<tr><td><BSSID></td><td>String</td><td>The BSSID of detected device</td></tr>
<tr><td><SSID></td><td>String</td><td>SSID of detected device</td></tr>
</table>

 ©2025-2026

WINC_CmdWSCNP

Description

This function is used to start or stop the periodic passive scanning function.

```
bool WINC_CmdWSCNP
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optState
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optState	int32_t	State of the periodic scan function. **Value** / **Label** / **Description** 0 / DISABLE / Disable. 1 / ENABLE / Enable. optState will be ignored if its value is WINC_CMDWSCNP_STATE_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<STATE>	Query state

Response Element Description

Element Name	Type	Description
<STATE>	INTEGER	State of the periodic scan function **Value** / **Label** / **Description** 0 / DISABLE / Disabled. 1 / ENABLE / Enabled.

AEC Reference:

WINC_AEC_ID_WSCNIND

Description

Scan results.

AEC Syntax

AEC	Description
<RSSI>,<SEC_TYPE>,<CHANNEL>,<BSSID>,<SSID>	Scan indicator

AEC Element Syntax

Element Name	Type	Description
<RSSI>	INTEGER	Received signal strength
<SEC_TYPE>	INTEGER	Recommended security type to use when connecting to this AP **Value** / **Label** / **Description** -1 / NA / Unknown or unsupported. 0 / OPEN / Open. 2 / WPA2_PER_MM / WPA2-Personal Mixed Mode. 3 / WPA2_PER / WPA2-Personal. 4 / WPA3_PER_TM / WPA3-Personal Transition Mode. 5 / WPA3_PER / WPA3-Personal. 6 / WPA2_ENT_MM / WPA2-Enterprise Mixed Mode. 7 / WPA2_ENT / WPA2-Enterprise. 8 / WPA3_ENT_TM / WPA3-Enterprise Transition Mode. 9 / WPA3_ENT / WPA3-Enterprise.
<CHANNEL>	INTEGER	The channel number of the detected device
<BSSID>	STRING	The BSSID of detected device
<SSID>	STRING	SSID of detected device

 ©2025-2026

WINC_AEC_ID_WSCNDONE

Description

Scan completed.

AEC Syntax

AEC	Description
<NUM_RESULTS>	Scan complete

AEC Element Syntax

Element Name	Type	Description
<NUM_RESULTS>	INTEGER	Number of results returned in this scan operation

WINC_AEC_ID_WSCNPDONE

Description

Periodic scanning stopped on request. Sent once when the run ends, after the final sweep has reported its results, and is the only report of the run-wide counters.

AEC Syntax

AEC	Description
<NUM_SWEEPS>	Periodic scanning stopped

AEC Element Syntax

Element Name	Type	Description
<NUM_SWEEPS>	INTEGER	Number of sweeps started over the periodic scanning run which has just ended

 ©2025-2026

WINC_AEC_ID_WSCNPERR

Description

A periodic scan ended on a fault rather than by completing.

AEC Syntax

AEC	Description
<ERROR_CODE>	Periodic scan error

AEC Element Syntax

Element Name	Type	Description
<ERROR_CODE>	INTEGER	Error code See Status Response Codes

Examples:

 ©2025-2026

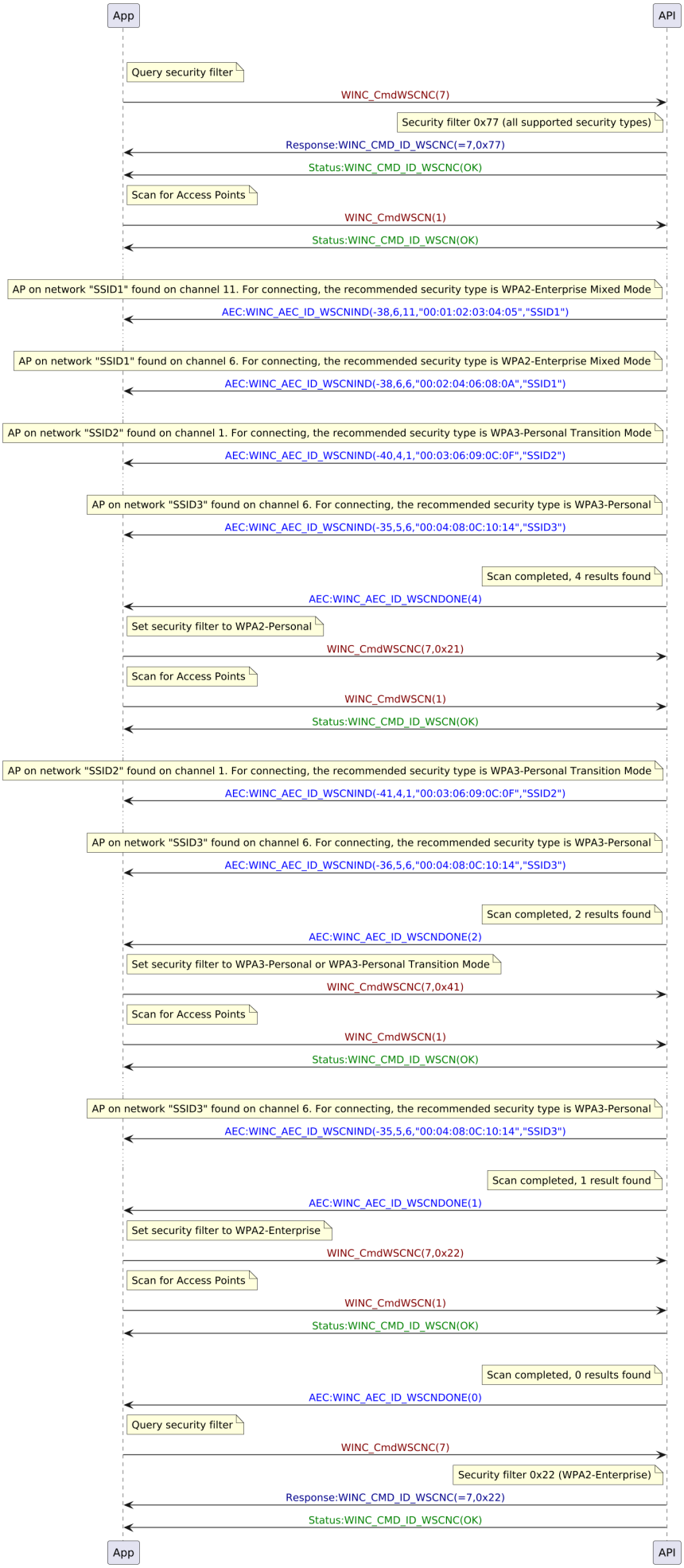


Figure 53. Scan for Access Points and filter by security type

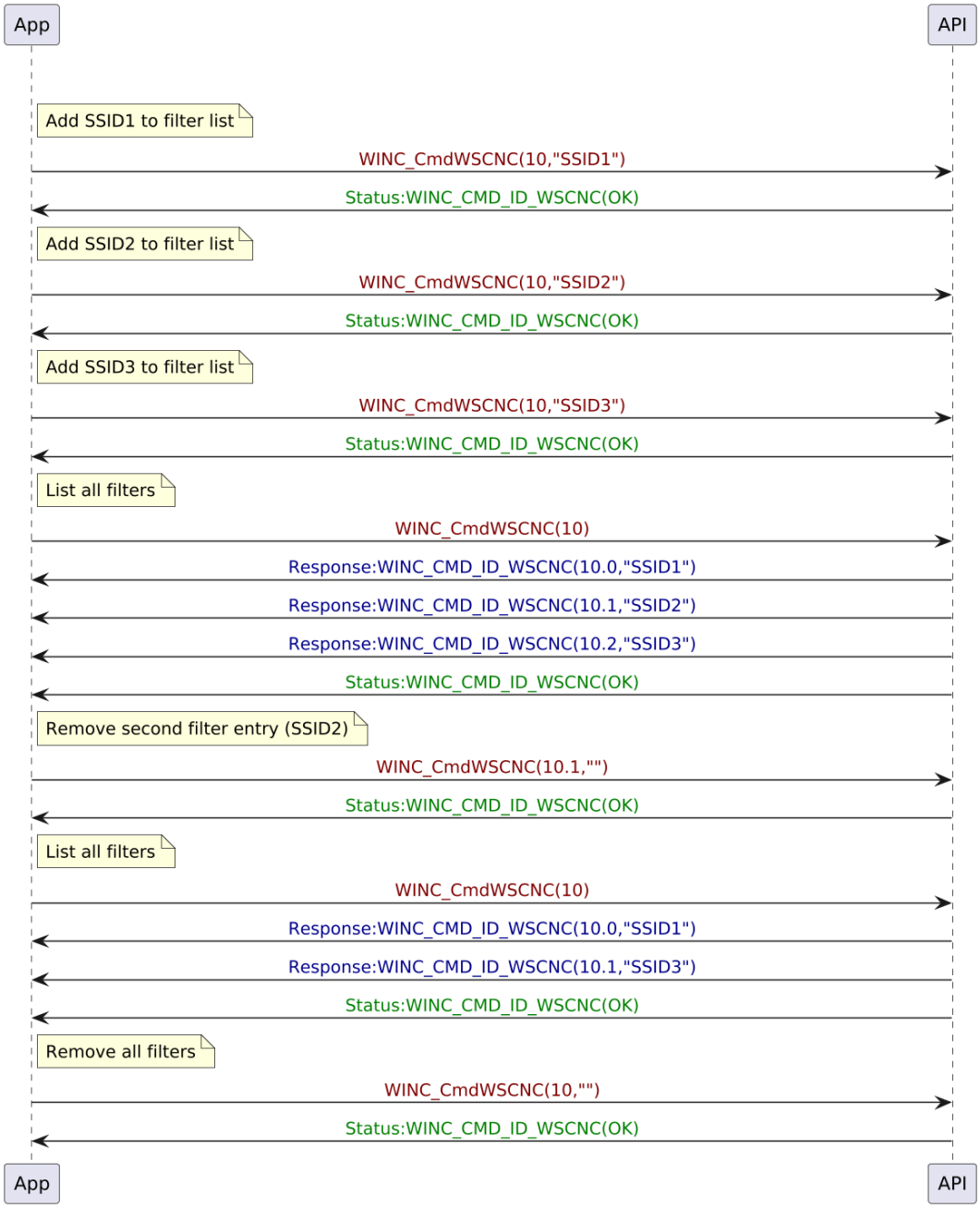


Figure 54. Setting SSID filter with WINC_CmdWSCNC

 ©2025-2026

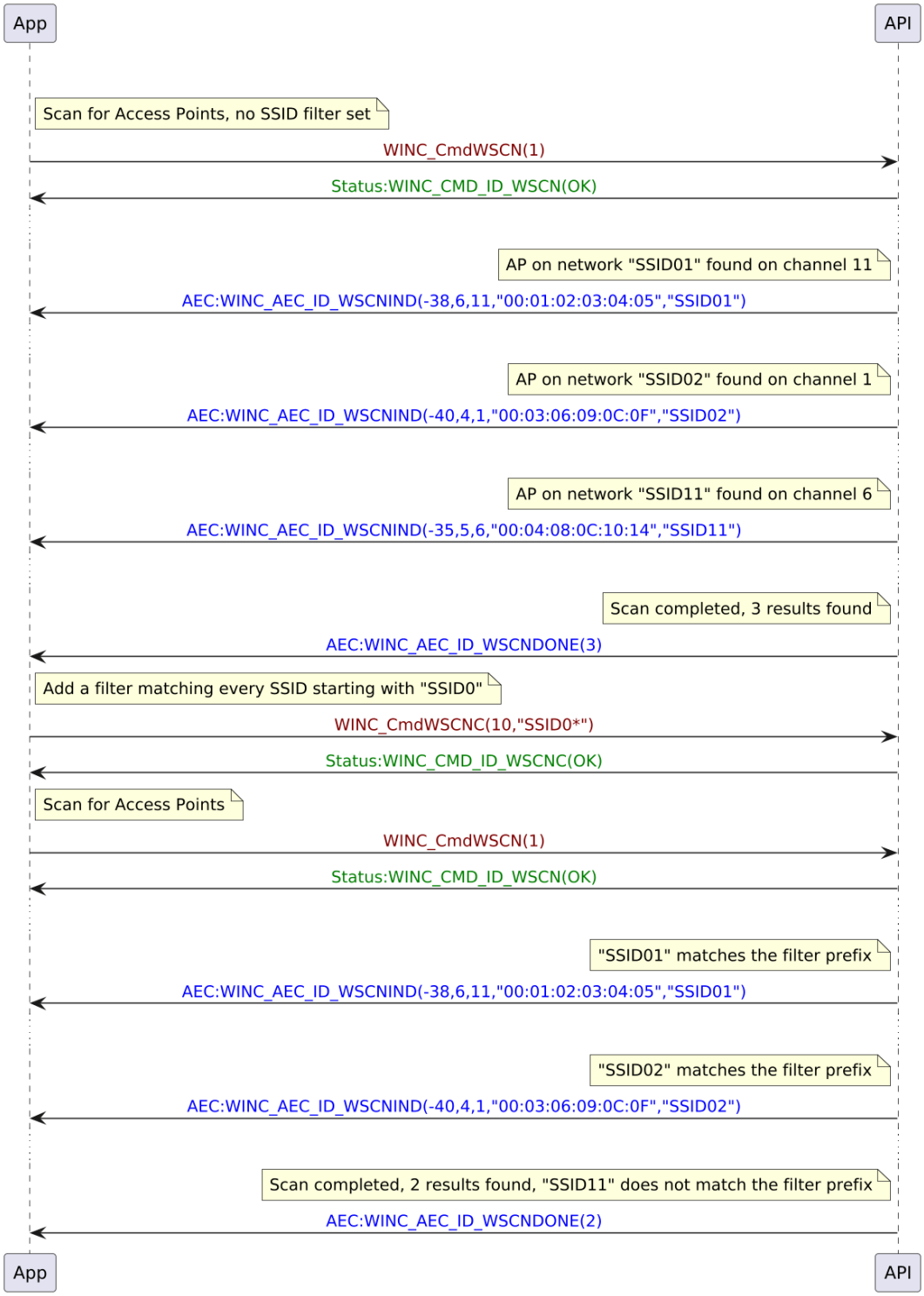


Figure 55. Setting SSID prefix filters with WINC_CmdWSCNC

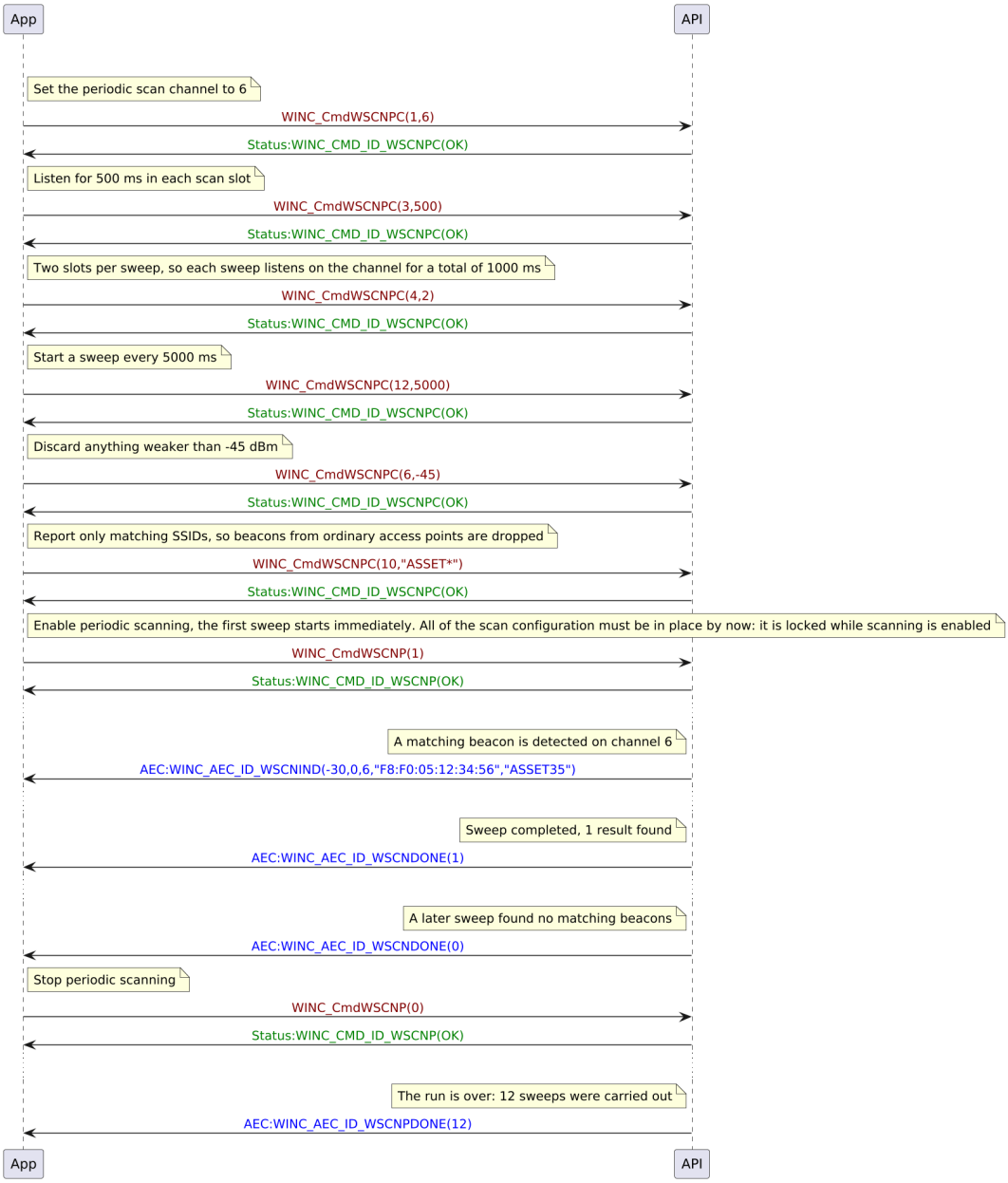


Figure 56. Periodic passive scanning with an SSID prefix filter

 ©2025-2026



Figure 57. Periodic passive scanning with an SSID prefix filter

WSTA (Module ID = 20)

Introduction

The WINCS02 supports a wide range of features when operating as a Wi-Fi station (STA). It is designed for robust connectivity, security, and seamless network integration in a variety of applications.

The WSTA module is used to configure the STA, control STA connection and get information about the STA.

Quick Start: Connecting to an Access Point

1. Configure the Access Point's SSID, security type and credentials with WINC_CmdWSTAC, see example WPA Personal. The configuration can only be changed while the STA is disconnected, except ROAMING.
2. Select how the STA's network interface (index 0 by default, see NETIF) obtains its IP address with WINC_CmdNETIFC: the DHCP client (DHCPC_EN, enabled by default), or a static address set with IP_MASK and GATEWAY. If DHCP is disabled and no address is set, a link-local (ZeroConf) address is used.
3. Start the connection with WINC_CmdWSTA. A successful command status indicates the connection attempt has started, not that the STA is connected.
4. The WINC_AEC_ID_WSTALU AEC reports that the link is up, with the association ID, the BSSID of the Access Point and the channel.
5. The WINC_AEC_ID_WSTAAIP AEC reports each address assigned automatically, by DHCP or ZeroConf for IPv4 and by SLAAC for IPv6. It is sent again when the DHCP lease is renewed or the address changes. A static IPv4 address is not reported.

See example Connect at Wi-Fi and IP layers.

Connection Refused

Starting the connection fails with an error status, and no AEC is sent, if:

- the STA is already connecting or connected, or the soft AP is running: STA Connection Not Permitted;
- a scan or periodic scan is in progress: Scan In Progress or Periodic Scan is Running;
- the security configuration is not valid, for example an invalid passphrase, an unsupported security type or a missing Enterprise TLS context: Invalid Parameter.

Connection Failed

If the Access Point refuses or does not respond to the connection, or the connection has not completed within CONN_TIMEOUT (10 seconds by default), the WINC_AEC_ID_WSTAERR AEC is sent with the status STA Connection Failed. The cause is not reported; an incorrect password, an Access Point which cannot be found and an authentication failure all appear the same.

 ©2025-2026

After WINC_AEC_ID_WSTAERR the STA is disconnected and the WINCS02 does not retry. To retry, start the connection again.

Link Loss and Disconnection

The WINC_AEC_ID_WSTALD AEC reports that the link has been lost. The WINCS02 does not reconnect automatically, start the connection again to reconnect. Roaming between Access Points within the same ESSID is handled automatically while connected and is reported by the WINC_AEC_ID_WSTAROAM AEC, followed by WINC_AEC_ID_WSTAAIP when roaming is configured for layer 3.

Disconnecting with WINC_CmdWSTA completes asynchronously: if the STA was connected the WINC_AEC_ID_WSTALD AEC follows, if it was still connecting no AEC follows. Querying the STA state returns 1 from the start of the connection attempt until the disconnection has completed.

Configuring the STA

Set STA parameters using the WINC_CmdWSTAC function.

In order to connect the STA to an Access Point (AP), the following parameters must be set:

- The AP's network name (SSID): SSID
- The security type (Open, WPA Personal or WPA Enterprise): SEC_TYPE
- The security credentials (depending on security type)

Other parameters are optional, with default behaviour if they are not set:

- The AP's Wi-Fi channel (1-13, or 0 for any; defaults to 0): CHANNEL
- The AP's BSSID: BSSID
- Management Frame Protection configuration (disabled, enabled or required; defaults to enabled): MFP_TYPE
- Disabling of WPA3 Transition modes (defaults to "not modified"): WPA3_TD
- Connection attempt timeout in milliseconds (defaults to 10000 ms; 0 disables the timeout): CONN_TIMEOUT
- Roaming configuration (disabled, Wi-Fi only, or layer 3; defaults to layer 3): ROAMING

Security types and credentials

Open security provides no authentication or encryption. There are no security credentials to set.

WPA Personal security provides authentication and encryption via a password. WPA2 and WPA3 are supported, including transition and mixed modes. The credentials parameter must be set to the AP's WPA Personal password: CREDENTIALS

WPA Enterprise security provides encryption and various methods for authentication. WPA2 and WPA3 are supported, including transition and mixed modes. See the WPA Enterprise section below for further information about how to set authentication methods and security credentials.

Roaming

The WINCS02 STA supports Simple Roaming, which enables faster handover between Access Points within an ESSID for maintained connectivity.

Roaming can be configured as:

- Wi-Fi only - the STA moves to another AP within the same ESSID; the IP configuration (layer 3 and above) is unchanged
- Layer 3 - as above, and the DHCP lease is also renewed after each roam

Controlling STA connection

Once the STA has been configured appropriately, start and stop the STA connection using the WINC_CmdWSTA function.

STA information

Query the STA parameters using the WINC_CmdWSTAC function.

Query the STA status using the WINC_CmdWSTA function.

During connection:

- The WINC_AEC_ID_WSTALU and WINC_AEC_ID_WSTALD AECs indicate the Wi-Fi link being established ("up") or lost ("down") respectively. The WINC_AEC_ID_WSTAROAM AEC indicates a layer 2 roam event (connection lost and regained, possibly to a different AP).
- The WINC_AEC_ID_WSTAAIP AEC indicates the IP address assigned to the STA. Layer 3 roam events may cause this AEC to be received multiple times, whenever the IP address changes.

 ©2025-2026

WPA Enterprise

WPA Enterprise significantly enhances Wi-Fi security by using an external authentication server (typically a RADIUS server) to perform user or device authentication dynamically. This process utilises the Extensible Authentication Protocol (EAP), enabling flexible authentication mechanisms beyond entering a simple shared password.

WPA Enterprise typically uses a **two-step process**:

1. **Establish a secure encrypted tunnel between the Wi-Fi client (STA) and the authentication server.** A TLS (Transport Layer Security) handshake allows the client to authenticate the server. The resulting TLS session provides an encrypted tunnel to protect subsequent communications.
2. **Authenticate the client securely inside the encrypted tunnel.** After the tunnel is established, a second method (such as MSCHAPv2 or another TLS handshake) allows the server to authenticate the client via the client's credentials (such as an MSCHAPv2 username and password or a TLS certificate and private key). This client authentication is protected by the encrypted tunnel.

Upon successful authentication, the client and server generate a Pairwise Master Key (PMK). This is used to establish a secure Wi-Fi association between the STA and the Access Point.

Supported WPA Enterprise methods

To maximise compatibility with a broad range of Enterprise networks, the WINCS02 supports the following tunnel and credential methods:

Method	Description
EAP-TTLSv0/MSCHAPv2	TLS tunnel method is TTLSv0; client authentication is done using MSCHAPv2
EAP-PEAPv0/MSCHAPv2	TLS tunnel method is PEAPv0; client authentication method is MSCHAPv2
EAP-PEAPv1/MSCHAPv2	TLS tunnel method is PEAPv1; client authentication method is MSCHAPv2
EAP-TLS	Single-step process with mutual authentication in the TLS handshake
EAP-PEAPv0/TLS	TLS tunnel method is PEAPv0; client authentication method is another TLS handshake
EAP-PEAPv1/TLS	TLS tunnel method is PEAPv1; client authentication method is another TLS handshake

Configuring WPA Enterprise methods and security credentials

In order to use WPA Enterprise to connect the STA to an AP, via an authentication server, the following parameters must be set:

- The AP's network name (SSID): SSID
- The security type (WPA Enterprise): SEC_TYPE
- The unprotected user identity (many servers accept an anonymous identity): ENT_IDENT_PLAIN
- The protected user identity: ENT_IDENT_PROT

NOTE The protected user identity parameter may be left unset if the tunnel method parameter is set to "None" or "TTLSv0".

- The client credential type (TLS or MSCHAPv2): ENT_CRED_TYPE
- The TLS configuration index for the tunnel method: ENT_TUNN_TLS_CONF

NOTE The TLS configuration index for the tunnel method may be left unset if the tunnel method parameter is set to "None".

In addition:

- If the client credential type is TLS, the TLS configuration index to use for client credentials must be set: ENT_CRED_TLS_CONF
- If the client credential type is MSCHAPv2, the MSCHAPv2 username and password parameters must be set: ENT_MSCHV2_UN and ENT_MSCHV2_PW

Other WPA Enterprise credentials are optional, with default behaviours if they are not set:

- The WPA Enterprise tunnel method (TTLSv0, PEAPv0, PEAPv1, "none" or "any"; defaults to "any"): ENT_TUNN_METHOD

NOTE

TLS configurations must be configured using the TLSC module. In particular:

- If the TLS configuration is to be used for client credentials, the client's certificate and private key must be specified via CERT_NAME and PRI_KEY_NAME
- For EAP-PEAPv0 tunnel method, TLS session caching is not supported and SESSION_CACHING is ignored
- For TLS credential method, TLS session caching is not used and SESSION_CACHING is ignored
- For WPA3 (and WPA2/WPA3 transition mode) Enterprise, server domain name verification must be enabled via PEER_DOMAIN_VERIFY and DOMAIN_NAME

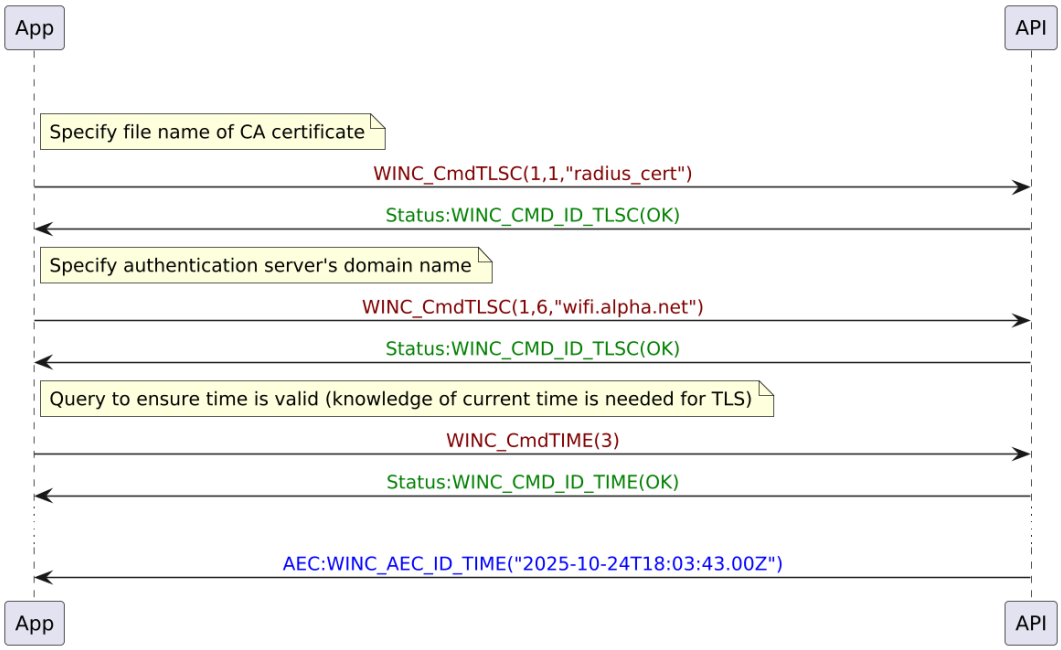
NOTE For EAP-PEAP/TLS methods, it may be appropriate and convenient to use the same TLS configuration for both the tunnel method and the client credentials.

 ©2025-2026

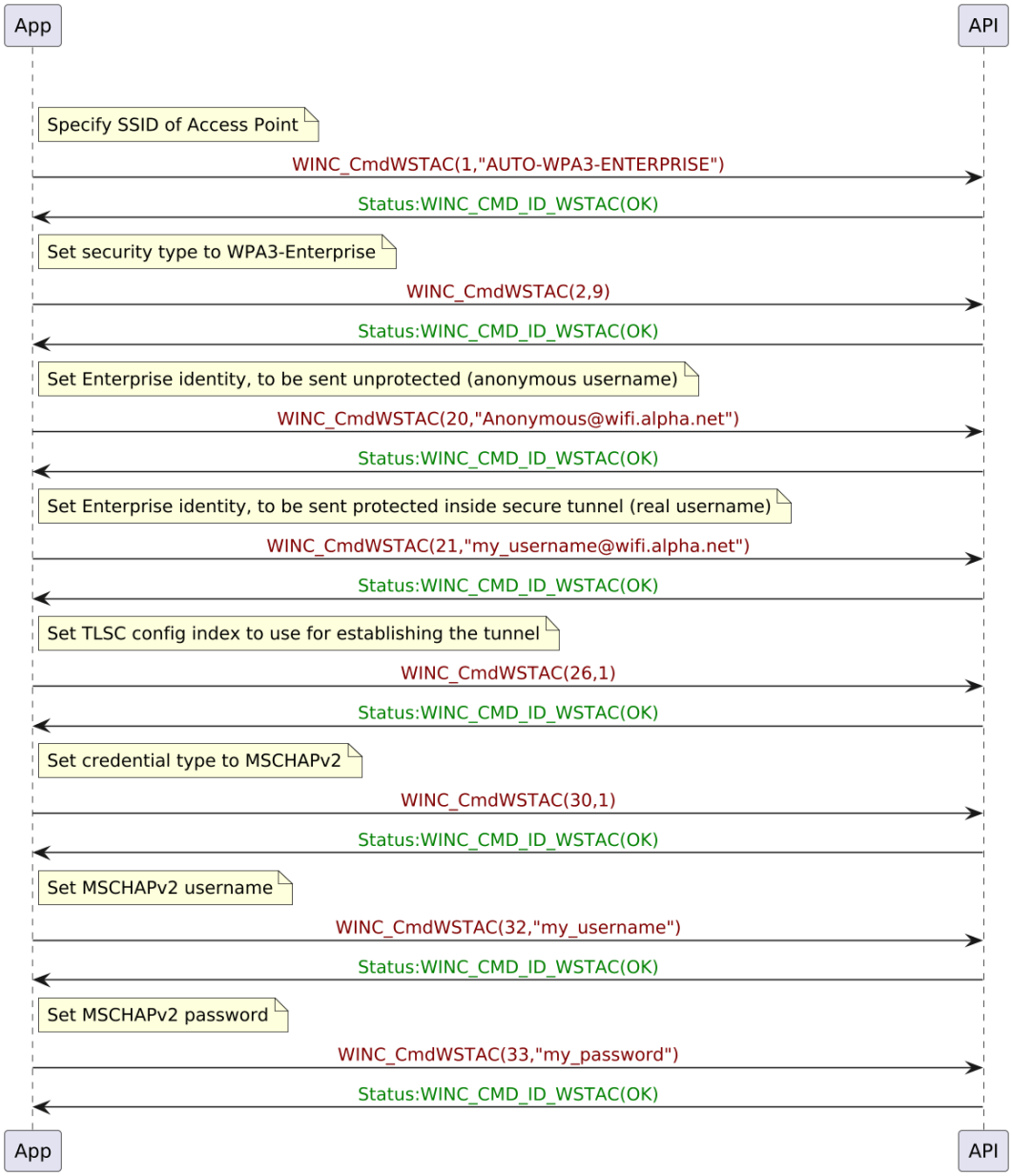
Example WPA Enterprise Configurations

Example 1: WPA3 Enterprise with MSCHAPv2 credentials

First configure a TLS context for establishing a secure encrypted tunnel:



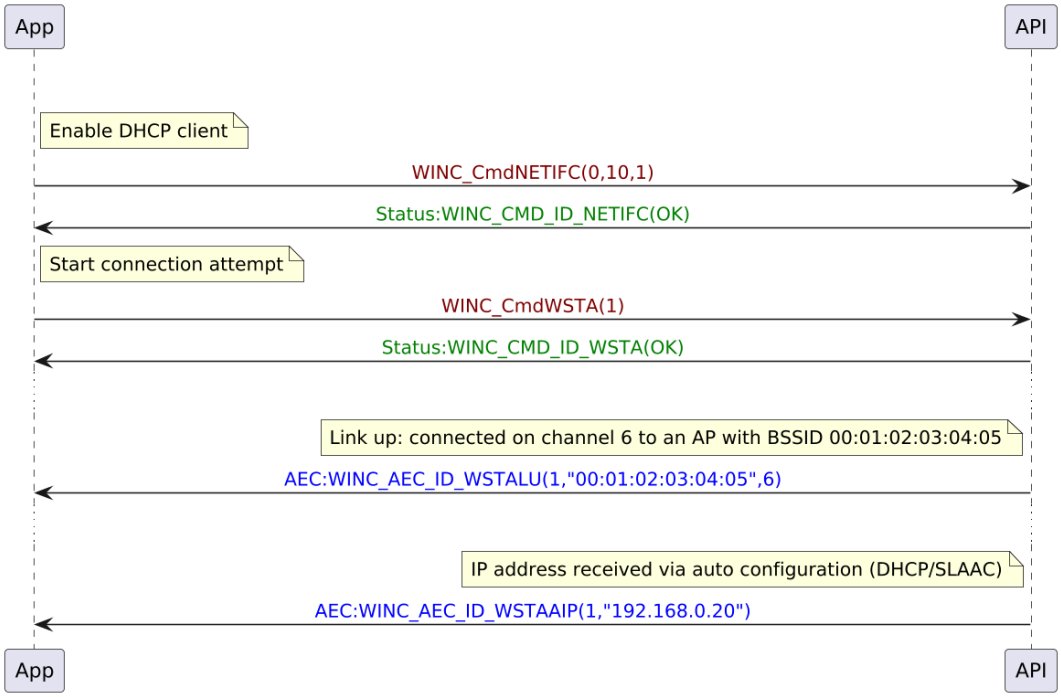
Then configure the STA connection parameters, making use of the above TLS context:



Note that the tunnel method has not been specified here. Either EAP-TTLS or EAP-PEAP may be used, depending on the server's request.

Now connect:

 ©2025-2026


App
API
Enable DHCP client
WINC_CmdNETIFC(0,10,1)
Status:WINC_CMD_ID_NETIFC(OK)
Start connection attempt
WINC_CmdWSTA(1)
Status:WINC_CMD_ID_WSTA(OK)
Link up: connected on channel 6 to an AP with BSSID 00:01:02:03:04:05
AEC:WINC_AEC_ID_WSTALU(1,"00:01:02:03:04:05",6)
IP address received via auto configuration (DHCP/SLAAC)
AEC:WINC_AEC_ID_WSTAAIP(1,"192.168.0.20")
App
API

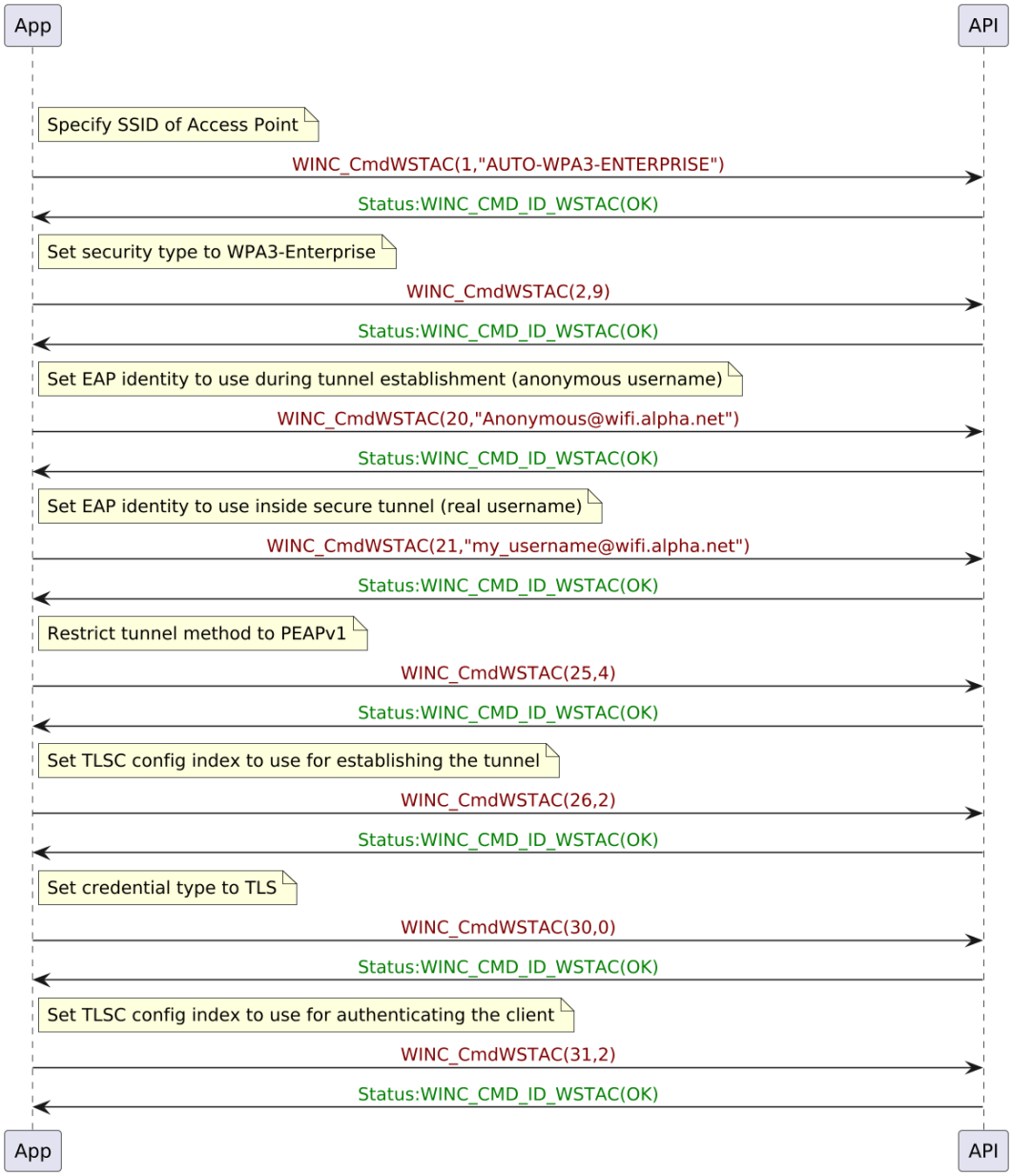
Example 2: WPA3 Enterprise with EAP-PEAPv1/TLS method

First configure a TLS context, including the user's certificate and private key:

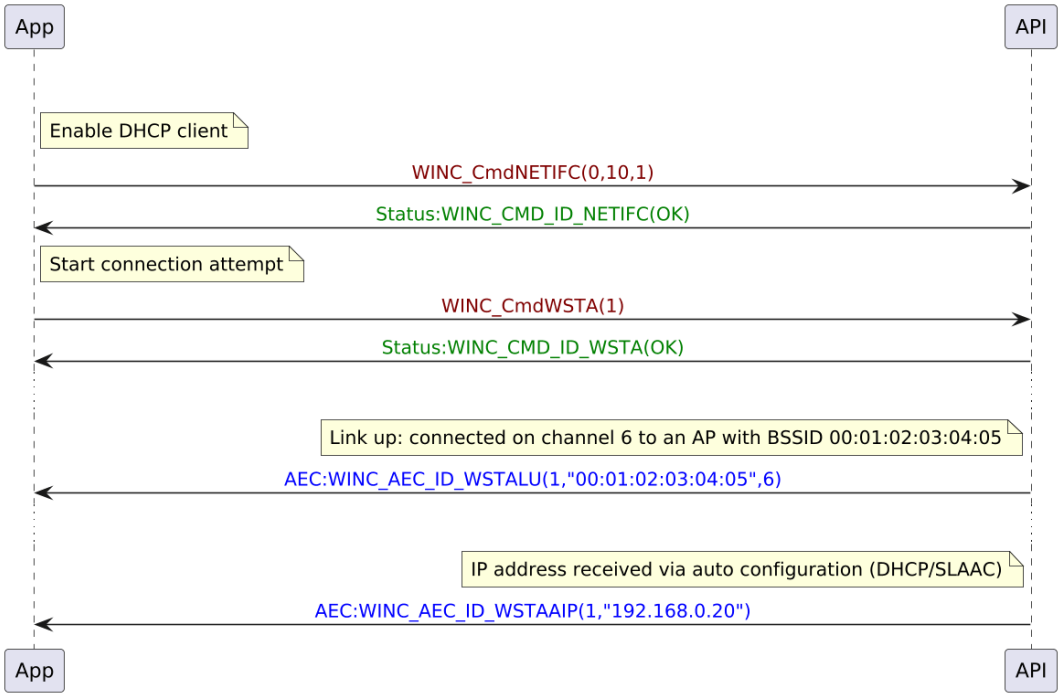


Then configure the STA connection parameters, making use of the above TLS context for both establishing the tunnel and authenticating the client:

 ©2025-2026


App
API
Specify SSID of Access Point
WINC_CmdWSTAC(1,"AUTO-WPA3-ENTERPRISE")
Status:WINC_CMD_ID_WSTAC(OK)
Set security type to WPA3-Enterprise
WINC_CmdWSTAC(2,9)
Status:WINC_CMD_ID_WSTAC(OK)
Set EAP identity to use during tunnel establishment (anonymous username)
WINC_CmdWSTAC(20,"Anonymous@wifi.alpha.net")
Status:WINC_CMD_ID_WSTAC(OK)
Set EAP identity to use inside secure tunnel (real username)
WINC_CmdWSTAC(21,"my_username@wifi.alpha.net")
Status:WINC_CMD_ID_WSTAC(OK)
Restrict tunnel method to PEAPv1
WINC_CmdWSTAC(25,4)
Status:WINC_CMD_ID_WSTAC(OK)
Set TLSC config index to use for establishing the tunnel
WINC_CmdWSTAC(26,2)
Status:WINC_CMD_ID_WSTAC(OK)
Set credential type to TLS
WINC_CmdWSTAC(30,0)
Status:WINC_CMD_ID_WSTAC(OK)
Set TLSC config index to use for authenticating the client
WINC_CmdWSTAC(31,2)
Status:WINC_CMD_ID_WSTAC(OK)
App
API

Now connect:


App
API
Enable DHCP client
WINC_CmdNETIFC(0,10,1)
Status:WINC_CMD_ID_NETIFC(OK)
Start connection attempt
WINC_CmdWSTA(1)
Status:WINC_CMD_ID_WSTA(OK)
Link up: connected on channel 6 to an AP with BSSID 00:01:02:03:04:05
AEC:WINC_AEC_ID_WSTALU(1,"00:01:02:03:04:05",6)
IP address received via auto configuration (DHCP/SLAAC)
AEC:WINC_AEC_ID_WSTAAIP(1,"192.168.0.20")
App
API

 ©2025-2026

Function Reference:

WINC_CmdWSTAC

Description

This function is used to read or set the device's Wi-Fi station mode configuration.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdWSTAC
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDWSTAC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_WSTA_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset **Value** / **Label** / **Description** 0 / DEFAULT / Default.
1	SSID	STRING	Network name Maximum length of string is 32
2	SEC_TYPE	INTEGER	Security type **Value** / **Label** / **Description** 0 / OPEN / Open. 2 / WPA2_PERS_MIXED / WPA2-Personal Mixed Mode. 3 / WPA2_PERS / WPA2-Personal. 4 / WPA3_PERS_TRANS / WPA3-Personal Transition Mode. 5 / WPA3_PERS / WPA3-Personal. 6 / WPA2_ENT_MIXED / WPA2-Enterprise Mixed Mode. 7 / WPA2_ENT / WPA2-Enterprise. 8 / WPA3_ENT_TRANS / WPA3-Enterprise Transition Mode. 9 / WPA3_ENT / WPA3-Enterprise.
3	CREDENTIALS	STRING	Credentials for connecting to the network Maximum length of string is 128
4	CHANNEL	INTEGER	Current channel of BSS (or 0 for any) Valid range is 0 to 13
5	BSSID	MACADDR	The BSSID of the network to connect to
7	CONN_TIMEOUT	INTEGER	Connection timeout in milliseconds. 0 disables the timeout. Default is 10000 Valid range is 0 to 0x7FFFFFFF
8	NETIF_IDX	INTEGER_UNSIGNED	Network interface index Valid range is 0 to 1

 ©2025-2026

<table>
<tr><th>ID</th><th>Name</th><th>Type</th><th>Description</th></tr>
<tr><td>9</td><td>ROAMING</td><td>INTEGER</td><td>Roaming state
<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>DISABLED</td><td>Roaming is disabled.</td></tr><tr><td>1</td><td>WIFI</td><td>Roaming is enabled on WiFi only.</td></tr><tr><td>2</td><td>LAYER3</td><td>Roaming is enabled on layer 3.</td></tr></table></td></tr>
<tr><td>20</td><td>ENT_IDENT_PLAIN</td><td>STRING</td><td>Identity for Enterprise, to be sent unprotected
Maximum length of string is 255</td></tr>
<tr><td>21</td><td>ENT_IDENT_PROT</td><td>STRING</td><td>Identity for Enterprise, to be sent protected (PEAP tunnels only)
Maximum length of string is 255</td></tr>
<tr><td>25</td><td>ENT_TUNN_METHOD</td><td>INTEGER</td><td>Tunnel method for Enterprise
<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>ANY</td><td>Any.</td></tr><tr><td>1</td><td>NONE</td><td>None.</td></tr><tr><td>2</td><td>TTLSV0</td><td>TTLSv0.</td></tr><tr><td>3</td><td>PEAPV0</td><td>PEAPv0.</td></tr><tr><td>4</td><td>PEAPV1</td><td>PEAPv1.</td></tr></table></td></tr>
<tr><td>26</td><td>ENT_TUNN_TLS_CONF</td><td>INTEGER</td><td>TLS configuration index (see WINC_CmdTLSCDA) to use for Enterprise tunnel. 0 if not configured

Valid range is 0 to 4</td></tr>
<tr><td>30</td><td>ENT_CRED_TYPE</td><td>INTEGER</td><td>Type of credentials for Enterprise
<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>TLS</td><td>TLS.</td></tr><tr><td>1</td><td>MSCHAPV2</td><td>MSCHAPv2.</td></tr></table></td></tr>
<tr><td>31</td><td>ENT_CRED_TLS_CONF</td><td>INTEGER</td><td>TLS configuration index (see WINC_CmdTLSCDA) to use for Enterprise TLS credentials. 0 if not configured

Valid range is 0 to 4</td></tr>
<tr><td>32</td><td>ENT_MSCHV2_UN</td><td>STRING</td><td>Username element of Enterprise MSCHAPv2 credentials
Maximum length of string is 255</td></tr>
<tr><td>33</td><td>ENT_MSCHV2_PW</td><td>STRING</td><td>Password element of Enterprise MSCHAPv2 credentials
Maximum length of string is 255</td></tr>
</table>

<table>
<tr><th>ID</th><th>Name</th><th>Type</th><th>Description</th></tr>
<tr><td>50</td><td>MFP_TYPE</td><td>INTEGER</td><td>Management Frame Protection configuration
<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>DISABLED</td><td>MFP Disabled.</td></tr><tr><td>1</td><td>ENABLED</td><td>MFP Enabled.</td></tr><tr><td>2</td><td>REQUIRED</td><td>MFP Required.</td></tr></table></td></tr>
<tr><td>51</td><td>WPA3_TD</td><td>BOOL</td><td>WPA3 Transition disable modifier
<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>MODIFIED</td><td>WPA3 Transition algorithms not modified.</td></tr><tr><td>1</td><td>DISABLED</td><td>WPA3 Transition algorithms are disabled.</td></tr></table></td></tr>
</table>

 ©2025-2026

WINC_CmdWSTA

Description

This function is used to control or query the device's station mode functionality.

```
bool WINC_CmdWSTA
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optState
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optState	int32_t	State of the Wi-Fi station feature. **Value** / **Label** / **Description** 0 / DISABLE / Disable. 1 / ENABLE / Use configuration from WINC_CmdWSTAC. optState will be ignored if its value is WINC_CMDWSTA_STATE_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<STATE>	State

Response Element Description

<table>
<tr><th>Element Name</th><th>Type</th><th>Description</th></tr>
<tr><td><STATE></td><td>INTEGER</td><td>Station state

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>NOT_CONNECTED</td><td>Not connected.</td></tr><tr><td>1</td><td>CONNECTED</td><td>Connecting, connected or disconnecting.</td></tr></table></td></tr>
</table>

 ©2025-2026

AEC Reference:

WINC_AEC_ID_WSTAAIP

Description

Indication of STA automatic address assignment.

AEC Syntax

AEC	Description
<ASSOC_ID>,<IP_ADDRESS>	Assignment of IP address to association

AEC Element Syntax

Element Name	Type	Description
<ASSOC_ID>	INTEGER	Association ID
<IP_ADDRESS>	STRING	IP address assigned

WINC_AEC_ID_WSTALD

Description

Link lost.

AEC Syntax

AEC	Description
<ASSOC_ID>	Link down

AEC Element Syntax

Element Name	Type	Description
<ASSOC_ID>	INTEGER	Association ID

 ©2025-2026

WINC_AEC_ID_WSTAERR

Description

Connection error.

AEC Syntax

AEC	Description
<ERROR_CODE>	Link error

AEC Element Syntax

Element Name	Type	Description
<ERROR_CODE>	INTEGER	Error code See Status Response Codes

WINC_AEC_ID_WSTALU

Description

Link established.

AEC Syntax

AEC	Description
<ASSOC_ID>,<BSSID>,<CHANNEL>	Link up

AEC Element Syntax

Element Name	Type	Description
<ASSOC_ID>	INTEGER	Association ID
<BSSID>	STRING	The BSSID of the Access Point the device has connected to
<CHANNEL>	INTEGER	The channel number of network

 ©2025-2026

WINC_AEC_ID_WSTAROAM

Description

Roaming occurred.

AEC Syntax

AEC	Description
<ASSOC_ID>	Roaming occurred

AEC Element Syntax

Element Name	Type	Description
<ASSOC_ID>	INTEGER	Association ID

Examples:

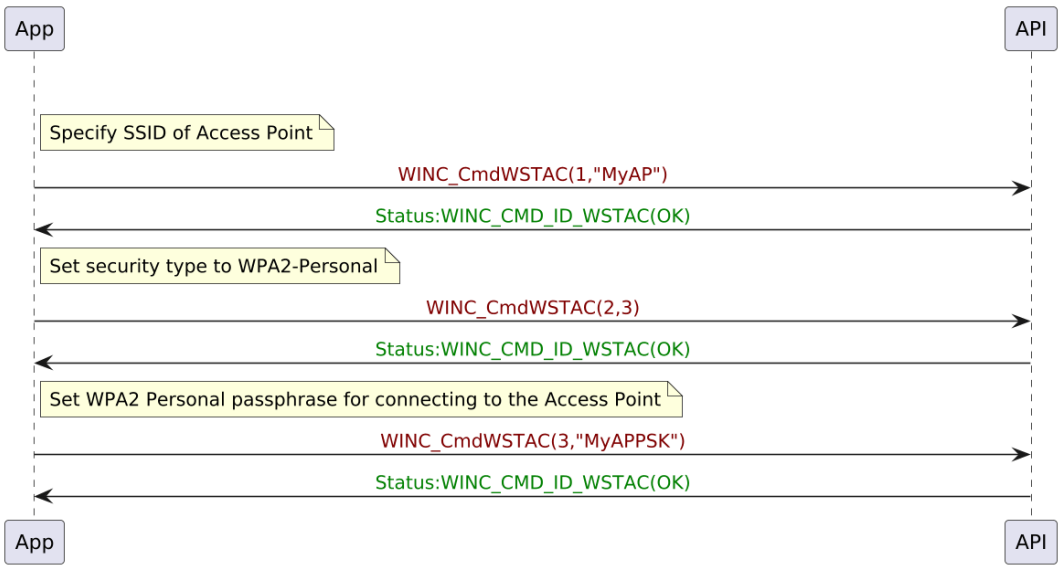


Figure 58. WPA Personal: Configure STA connection parameters

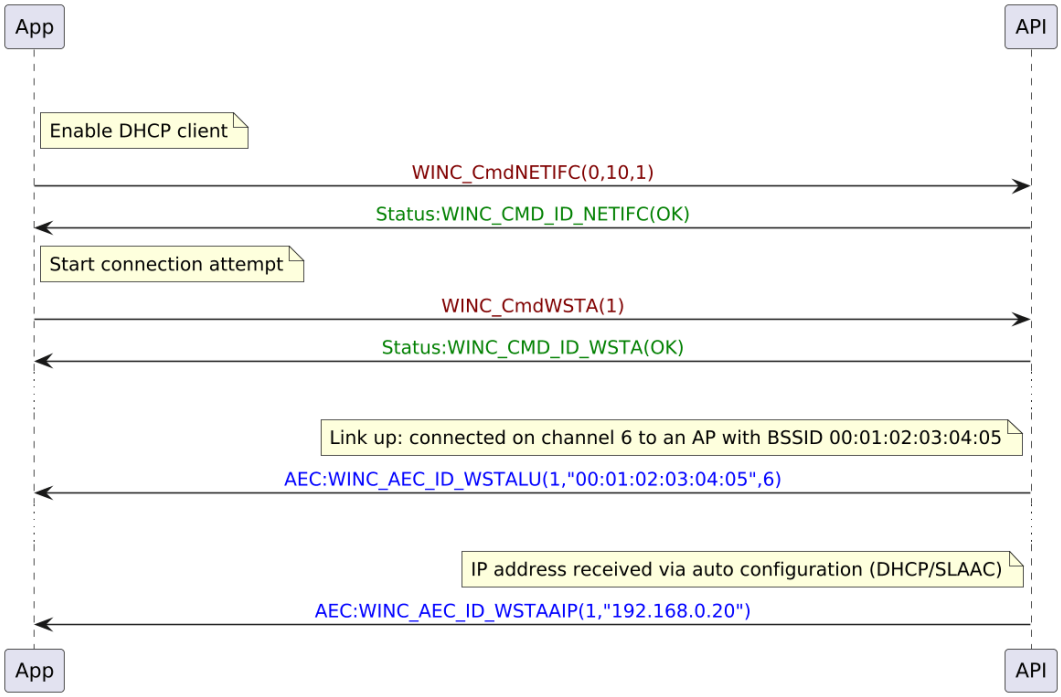


Figure 59. Connect at Wi-Fi and IP layers

 ©2025-2026

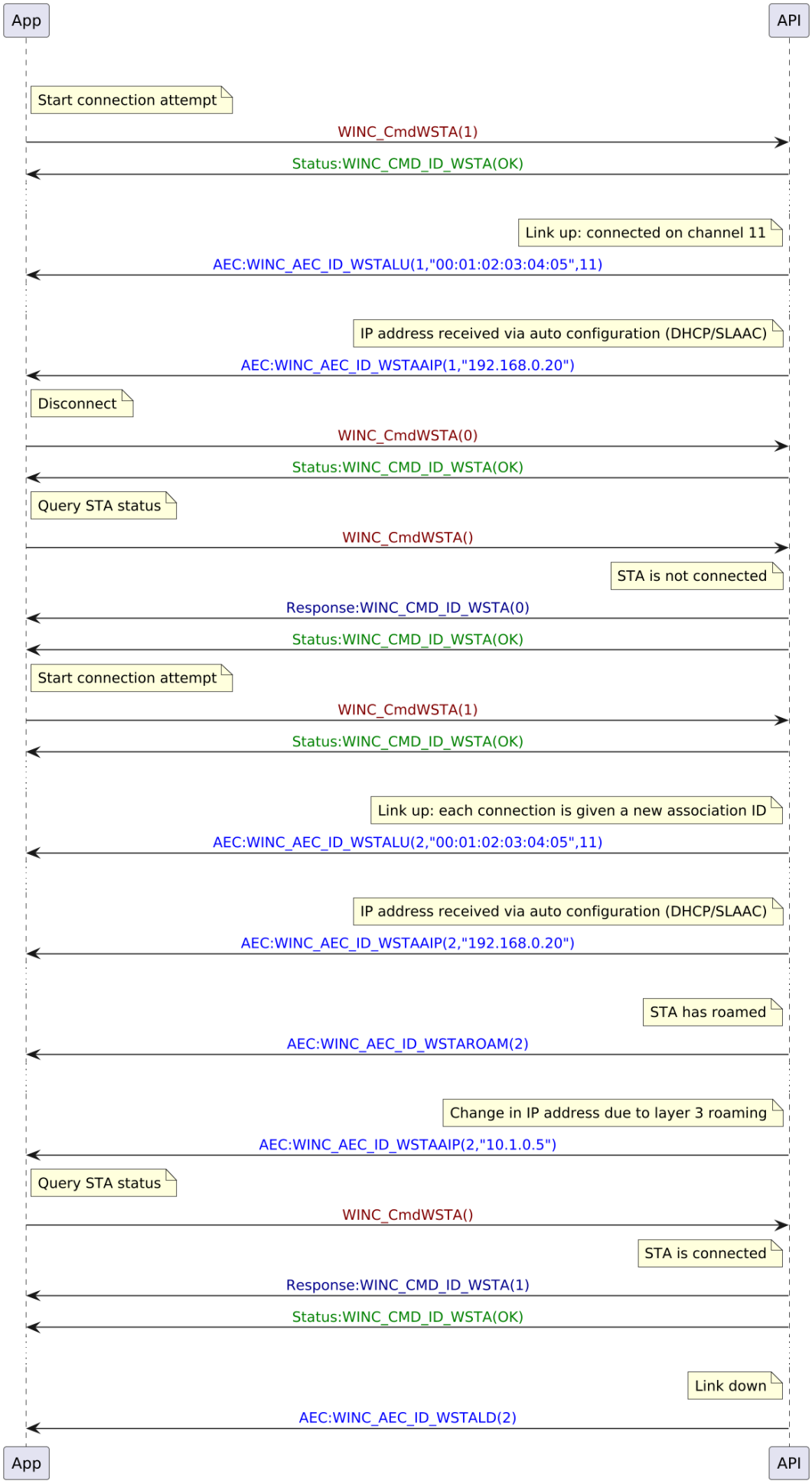


Figure 60. Some disconnections, reconnections and roaming

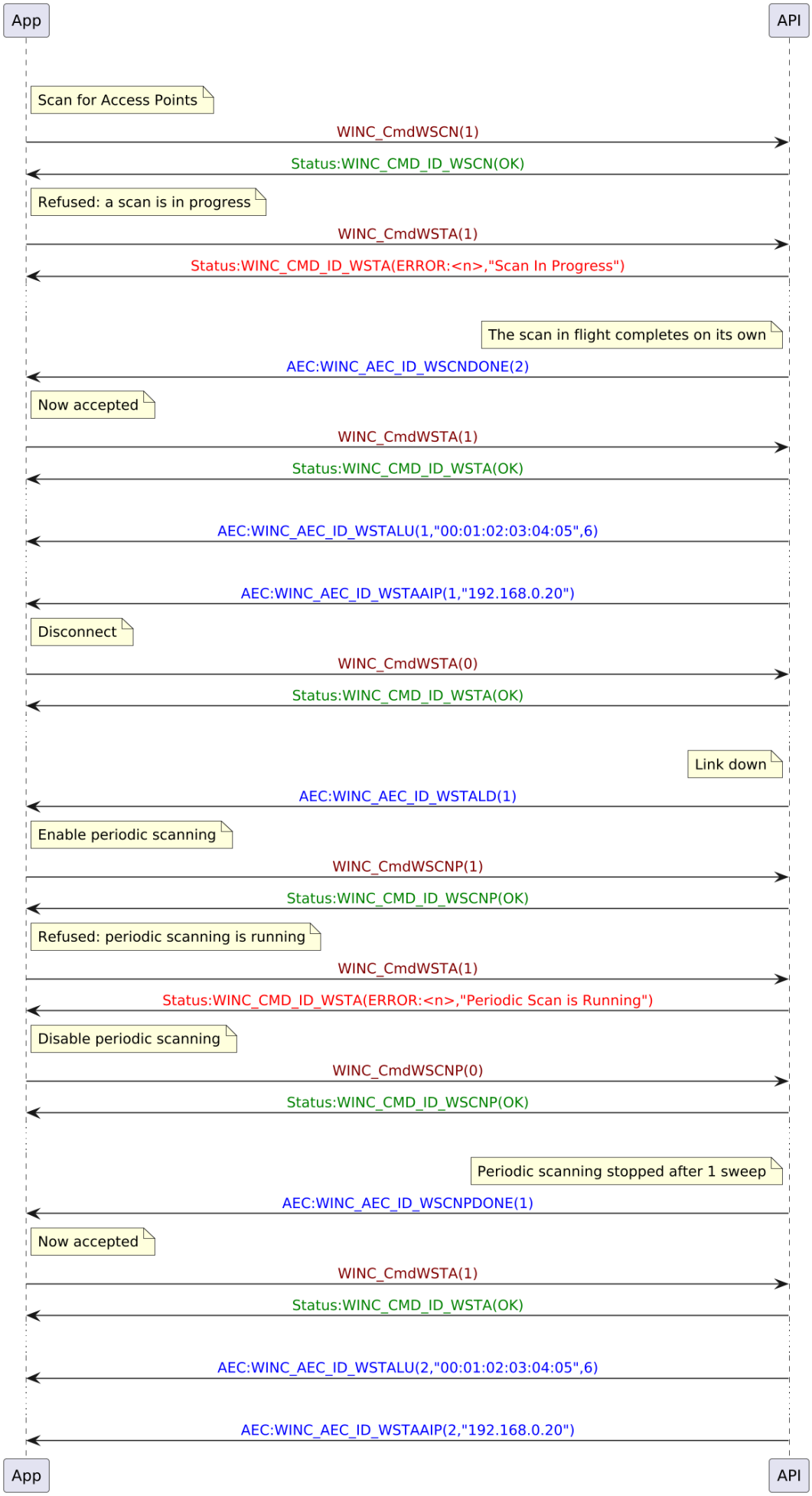


Figure 61. WINC_CmdWSTA refused while a scan or periodic

 ©2025-2026

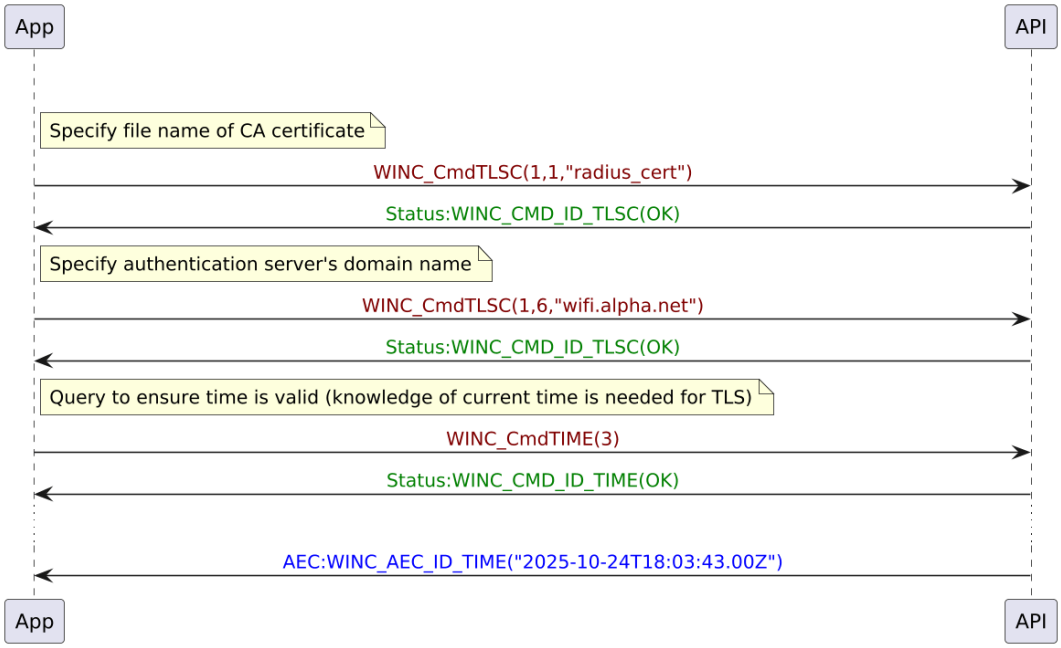


Figure 62. WPA Enterprise: Configure a TLS context for establishing a tunnel

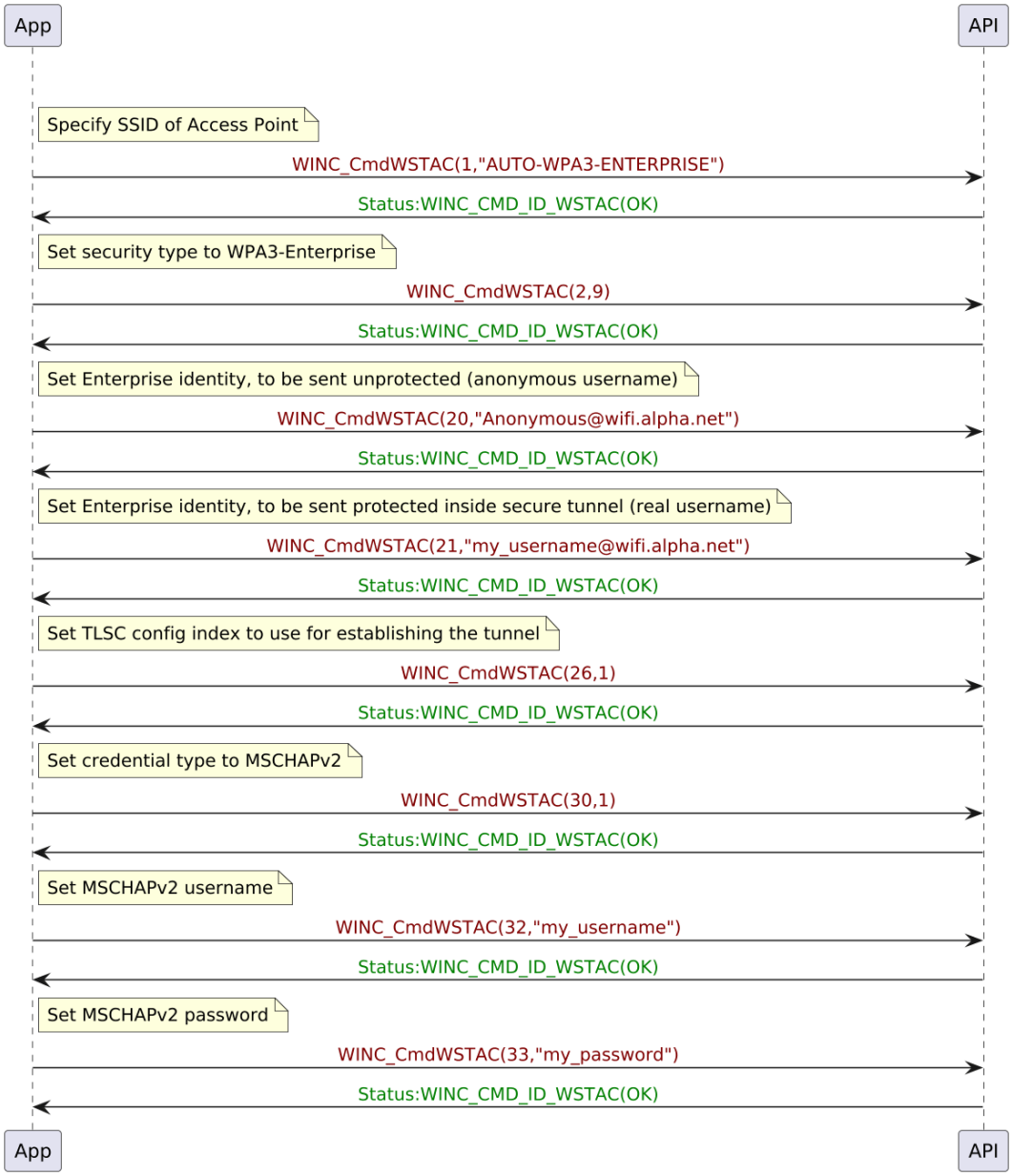


Figure 63. WPA Enterprise: Configure STA connection parameters for MSCHAPv2

 ©2025-2026



Figure 64. WPA Enterprise: Configure a TLS context for EAP-PEAPv1/TLS

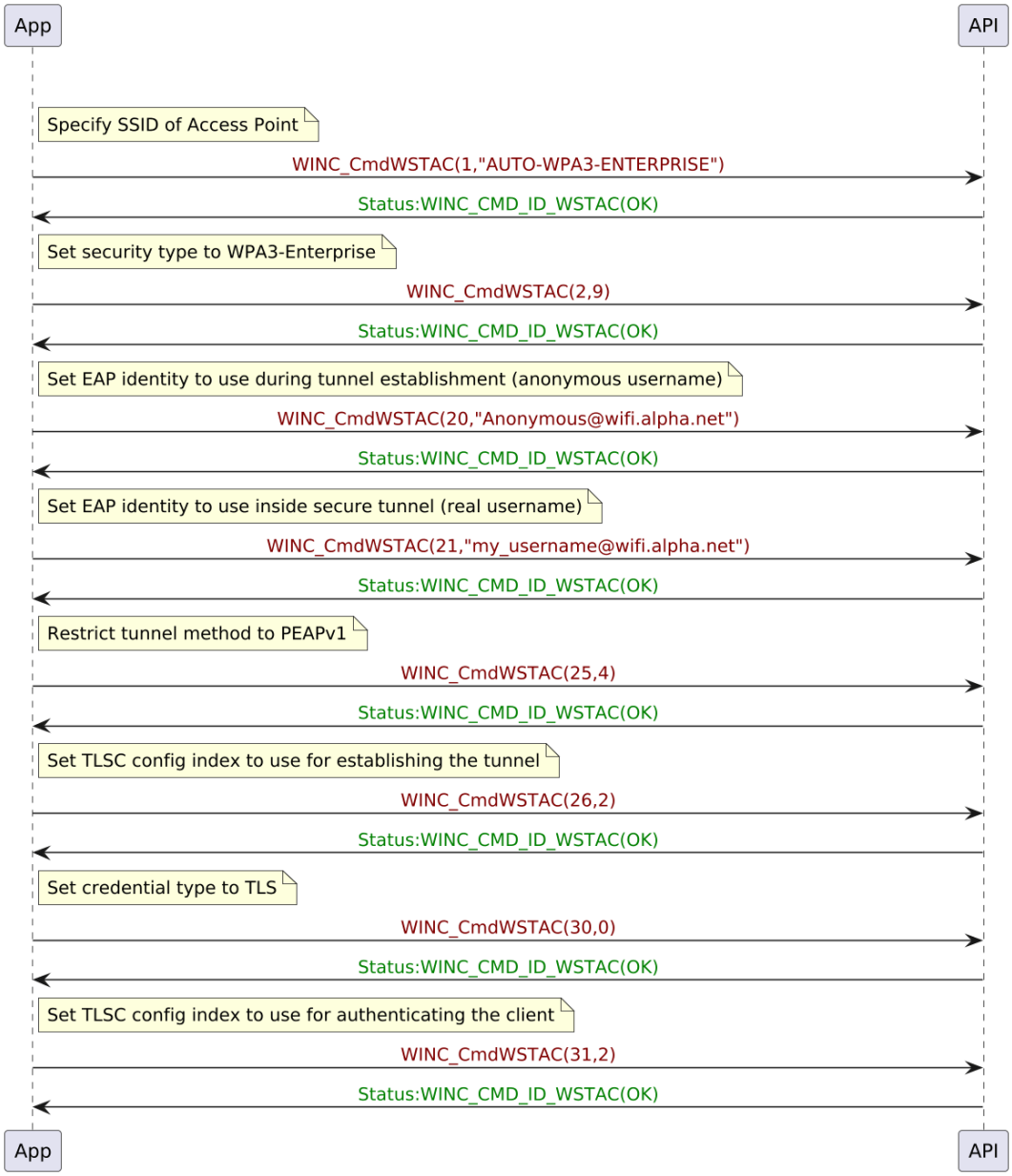


Figure 65. WPA Enterprise: Configure STA connection parameters for EAP-PEAPv1/TLS

 ©2025-2026

ASSOC (Module ID = 22)

Introduction

The ASSOC module lists the current Wi-Fi associations of the WINCS02. These are the station's association with its access point, reported with AP_STA set to 0, and the soft AP's connected clients, reported with AP_STA set to 1.

Each association is reported by an WINC_AEC_ID_ASSOC AEC which includes its current RSSI. WINC_CmdASSOC can also disconnect an association.

The association IDs are the same as those reported by the WSTA and WAP modules.

Function Reference:

WINC_CmdASSOC

Description

This function is used to query or disconnect WiFi associations.

```
bool WINC_CmdASSOC
(
    WINC_CMD_REQ_HANDLE handle
    uint16_t optAssocId
    int32_t optState
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optAssocId	uint16_t	Association ID. optAssocId will be ignored if its value is WINC_CMDASSOC_ASSOC_ID_IGNORE_VAL.
optState	int32_t	State to set. **Value** / **Label** / **Description** 0 / DISCONNECT / Disconnect the association. optState will be ignored if its value is WINC_CMDASSOC_STATE_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

AEC Reference:

WINC_AEC_ID_ASSOC

Description

WiFi association report.

AEC Syntax

AEC	Description
<ASSOC_ID>,<AP_STA>,<BSSID>,<RSSI>	WiFi association report

AEC Element Syntax

Element Name	Type	Description
<ASSOC_ID>	INTEGER	Association ID Positive unsigned 16-bit value
<AP_STA>	INTEGER STRING BYTE_ARRAY	AP or STA **Value** / **Label** / **Description** 0 / AP / AP. 1 / STA / STA.
<BSSID>	STRING	The BSSID of association peer
<RSSI>	INTEGER	RSSI

SI (Module ID = 23)

Introduction

The SI module reports system information. WINC_CmdSI returns the system uptime in its response. The filter parameter selects additional statistics which are reported as AECs:

- Wi-Fi memory pool usage, WINC_AEC_ID_SIMERR and WINC_AEC_ID_SIMSTAT.
- Wi-Fi packet counts, WINC_AEC_ID_SIWPKTS.
- Heap usage, WINC_AEC_ID_SIHERR and WINC_AEC_ID_SIHEAP.
- Stack usage, WINC_AEC_ID_SISTACK.

If no filter is given all statistics are reported, a filter of 0 reports only the uptime.

 ©2025-2026

Function Reference:

WINC_CmdSI

Description

This function is used to query system information.

```
bool WINC_CmdSI
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t optFilter
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optFilter	uint8_t	System information filter bitmask. **Value / Label / Description** 0x01 / WIFI_MEM_ALLOC / WiFi memory allocation stats. 0x02 / WIFI_PKTS / WiFi packet stats. 0x04 / HEAP_ALLOC / Heap allocation stats. 0x08 / STACK_USAGE / Stack usage stats. optFilter will be ignored if its value is WINC_CMDSI_FILTER_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<UPTIME>	Response

Response Element Description

Element Name	Type	Description
<UPTIME>	INTEGER	System uptime in seconds

AEC Reference:

WINC_AEC_ID_SIMSTAT

Description

WiFi memory statistics for each type of memory pool.

AEC Syntax

AEC	Description
<TYPE>,<ALLOC>,<FREE>,<ALLOC_SZ>,<FREE_SZ>	WiFi memory statistic

AEC Element Syntax

Element Name	Type	Description
<TYPE>	INTEGER	Type Value / Label / Description: 0 / MEM / Memory. 1 / CFG / Config. 2 / HPTX / High priority TX. 3 / HPRX / High priority RX. 4 / NRMTX / Normal priority TX. 5 / NRMRX / Normal priority RX.
<ALLOC>	INTEGER	Number of memory allocations made
<FREE>	INTEGER	Number of memory frees
<ALLOC_SZ>	INTEGER	Total bytes allocated
<FREE_SZ>	INTEGER	Total bytes freed

 ©2025-2026

WINC_AEC_ID_SIMERR

Description

WiFi Memory errors.

AEC Syntax

AEC	Description
<ALLOC_ERR>	WiFi memory errors

AEC Element Syntax

Element Name	Type	Description
<ALLOC_ERR>	INTEGER	Memory allocation errors

WINC_AEC_ID_SIWPKTS

Description

WiFi packets statistics.

AEC Syntax

AEC	Description
<TX>,<RX>	WiFi packets

AEC Element Syntax

Element Name	Type	Description
<TX>	INTEGER	WiFi packets transmitted
<RX>	INTEGER	WiFi packets received

 ©2025-2026

WINC_AEC_ID_SIHEAP

Description

Heap allocation statistics.

AEC Syntax

AEC	Description
<ALLOC>,<FREE>,<ALLOC_SZ>,<FREE_SZ>,<PEAK_ALLOC_SZ>	Heap statistics

AEC Element Syntax

Element Name	Type	Description
<ALLOC>	INTEGER	Number of memory allocations made
<FREE>	INTEGER	Number of memory frees
<ALLOC_SZ>	INTEGER	Total bytes allocated
<FREE_SZ>	INTEGER	Total bytes freed
<PEAK_ALLOC_SZ>	INTEGER	Peak bytes allocated

WINC_AEC_ID_SIHERR

Description

Heap allocation errors.

AEC Syntax

AEC	Description
<ALLOC_ERR>	Heap errors

AEC Element Syntax

Element Name	Type	Description
<ALLOC_ERR>	INTEGER	Memory allocation errors

 ©2025-2026

WINC_AEC_ID_SISTACK

Description

Stack usage.

AEC Syntax

AEC	Description
<STACK_MIN_RESERVED>,<STACK_RESERVED>,<STACK_USAGE>	Stack statistics

AEC Element Syntax

Element Name	Type	Description
<STACK_MIN_RESERVED>	INTEGER	Minimum stack reserved memory
<STACK_RESERVED>	INTEGER	Stack reserved memory
<STACK_USAGE>	INTEGER	Stack usage

WPROV (Module ID = 25)

Introduction

The WPROV module provides a TCP server which allows a remote client to send commands to the WINCS02 over the network, using either the ASCII or binary command format. It may be used, for example, to configure the station from a phone connected to the soft AP.

1. Bring up a network interface with WAP or WSTA.
2. Configure the service with WINC_CmdWPROVC. The configuration cannot be changed while the service is running.
3. Start the service with WINC_CmdWPROV.
4. When a client connects the WINC_AEC_ID_WPROVAT AEC is sent; the client can then send commands, and receives their responses, over the connection.
5. When the client disconnects the WINC_AEC_ID_WPROVDT AEC is sent.

WINC_CmdWPROV also stops the service.

 ©2025-2026

Function Reference:

WINC_CmdWPROVC

Description

This function is used to read or set the provisioning service configuration.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdWPROVC
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDWPROVC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_WPROV_*)

ID	Name	Type	Description
1	PORT	INTEGER_UNSIGNED	Service listening port Positive unsigned 16-bit value
2	ASCII	BOOL	ASCII or binary protocol **Value** / **Label** / **Description** 0 / BINARY / Binary protocol. 1 / ASCII / ASCII protocol.
3	PROTOCOL_VERSION	INTEGER_UNSIGNED	IP protocol version **Value** / **Label** / **Description** 4 / IPV4 / IPv4. 6 / IPV6 / IPv6.

 ©2025-2026

WINC_CmdWPROV

Description

This function is used to control or query the provisioning service.

```
bool WINC_CmdWPROV
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optState
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optState	int32_t	State of the provisioning service. **Value** / **Label** / **Description** 0 / DISABLE / Disable. 1 / ENABLE / Use configuration from WINC_CmdWPROVC. optState will be ignored if its value is WINC_CMDWPROV_STATE_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

AEC Reference:

WINC_AEC_ID_WPROVAT

Description

Indication of client attach to provisioning service.

AEC Syntax

AEC	Description
<ID>,<IP_ADDRESS>,<PORT>	Attachment of client to provisioning service

AEC Element Syntax

Element Name	Type	Description
<ID>	INTEGER	Client ID
<IP_ADDRESS>	STRING	IP address assigned
<PORT>	INTEGER	Port

 ©2025-2026

WINC_AEC_ID_WPROVDT

Description

Indication of client detach from provisioning service.

AEC Syntax

AEC	Description
<ID>	Detachment of client from provisioning service

AEC Element Syntax

Element Name	Type	Description
<ID>	INTEGER	Client ID

DI (Module ID = 26)

Function Reference:

WINC_CmdDI

Description

This function is used to query the device information.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdDI
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optId
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDDI_ID_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

 ©2025-2026

Configuration Parameters (WINC_CFG_PARAM_ID_DI_*)

ID	Name	Type	Description
1	DEVICE_ID	INTEGER_UNSIGNED (Read Only)	Device ID
14	NUM_IMAGES	INTEGER (Read Only)	Number of images
15	IMAGE_INFO	Complex Value[1] (Read Only)	Image information This is a multiple value parameter with an ID range 15.0 to 15.1

Complex Value Syntax

No.	Complex Value	Description
1	<SEQ_NUM>,<VERSION>,<SRC_ADDR>,<IMG_STATE>	Image information

Complex Value Element Syntax

Element Name	Type	Description
<SEQ_NUM>	Integer	Sequence number
<VERSION>	Integer	Version information
<SRC_ADDR>	Integer	Source address
<IMG_STATE>	Integer	Image state

Value	Label	Description
0	LATENT	The image has not yet been activated.
1	ACTIVATED	The image will run on next boot (subject to verification).
2	CURRENT	The image is currently running.
3	ROLLBACK	The image will run on next boot (subject to verification) if the current image is invalidated.

Examples:

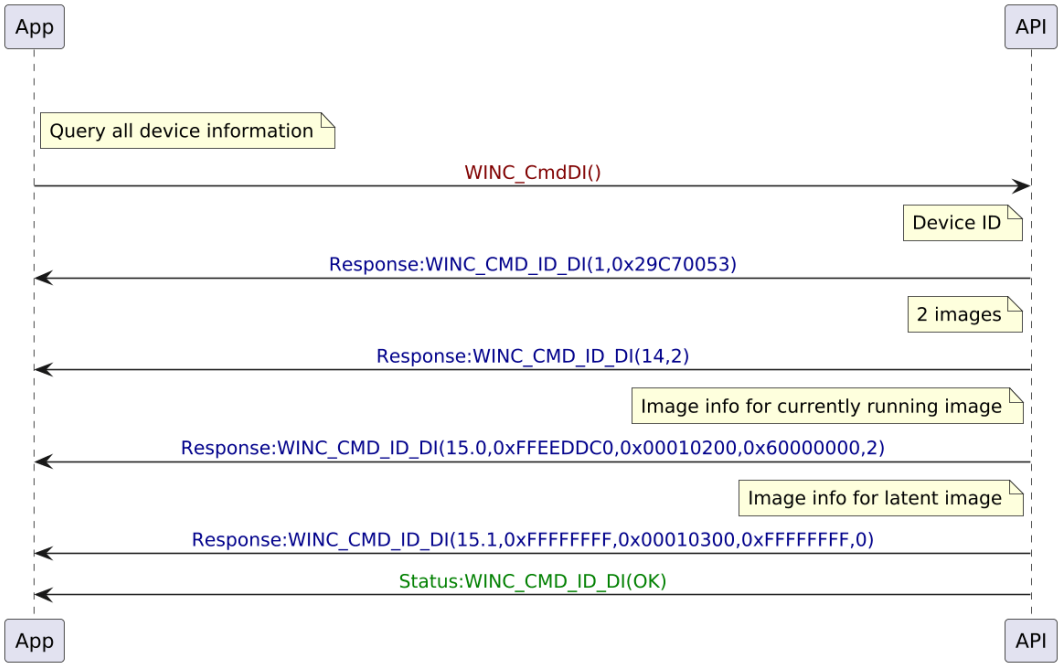


Figure 66. Query device information with WINC_CmdDI

 ©2025-2026

EXTCRYPTO (Module ID = 27)

Introduction

Cryptographic operations can be delegated to an external crypto device controlled by the host application, providing secure key storage and hardware-accelerated public key operations. This can be a Microchip ATECC608 Trust&GO (TNGTLS) secure element, which is fitted on some module variants (see the module data sheet), or an equivalent third-party device.

The ATECC608 Trust&GO is pre-provisioned, featuring ECDSA signing, ECC private key secure storage and X.509 compressed certificate storage. The host application accesses the secure element directly through its I2C or SWI interface; the WINCS02 does not communicate with it.

The WINC_AEC_ID_EXTCRYPTO AEC allows the application to receive a crypto operation request from the WINCS02. The WINC_CmdEXTCRYPTO function is used to send the operation result back to the WINCS02.

Crypto operation requests are enabled via the EXTCRYPTO_OPS parameter of the WINC_CmdTLSC function. The only supported operation is ECDSA signing.

Using External Crypto for Secure Signing

To leverage the crypto device for secure signing, the application should follow these steps:

1. Retrieve from the crypto device the X.509 certificate that includes the public key corresponding to the securely stored private key.
2. Send the X.509 certificate to the WINCS02 using the WINC_CmdFSSTORE function.
3. Enable secure signing via the external crypto device and specify the certificate in the TLS configuration.

   See example configuring TLS with external crypto.

4. Respond to signing requests from the WINC_AEC_ID_EXTCRYPTO AEC with the WINC_CmdEXTCRYPTO function.

   See example handling signing request and calling response function.

NOTE	The ECC curve used for signing is selected according to the certificate public key. Refer to the CURVE element in the WINC_AEC_ID_EXTCRYPTO AEC for the list of supported curves.

NOTE	The application code is responsible for interacting with the external crypto device to retrieve the certificate and forward signing requests and responses.

Function Reference:

WINC_CmdEXTCRYPTO

Description

This function is used to provide the result of an external crypto operation.

```
bool WINC_CmdEXTCRYPTO
(
    WINC_CMD_REQ_HANDLE handle
    uint16_t opId
    uint8_t status
    const uint8_t* pOptSignature
    size_t lenOptSignature
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
opId	uint16_t	Operation identifier, matching <OP_ID> in corresponding AEC.
status	uint8_t	Operation success or failure. **Value** / **Label** / **Description** 0 / SUCCESS / Operation succeeded. 1 / FAILURE / Operation failed.
pOptSignature	const uint8_t*	Signature (big endian). For ECDSA signatures: R then S, each the size of the curve.. pOptSignature will be ignored if its value is NULL.
lenOptSignature	size_t	Length of pOptSignature in bytes.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

AEC Reference:

WINC_AEC_ID_EXTCRYPTO

Description

This event is used to request an external crypto operation.

AEC Syntax

AEC	Description
<OP_ID>,<OP_TYPE>,<OP_SOURCE_TYPE>,<OP_SOURCE_ID>,<SIGN_TYPE>,<CURVE>,<SIGN_VALUE>	Request an external signing operation

AEC Element Syntax

Element Name	Type	Description
<OP_ID>	INTEGER	Operation identifier, to be included in EXTCRYPTO CMD Positive unsigned 16-bit value
<OP_TYPE>	INTEGER	The type of operation **Value** / **Label** / **Description** 1 / SIGN / Signing.
<OP_SOURCE_TYPE>	INTEGER	The type of source requesting the operation **Value** / **Label** / **Description** 1 / TLSC / A TLSC configuration.
<OP_SOURCE_ID>	INTEGER	The ID of the source requesting the operation (the TLS configuration index for TLSC) Valid range is 1 to 4
<SIGN_TYPE>	INTEGER	The type of signing operation **Value** / **Label** / **Description** 1 / ECDSA / ECDSA.
<CURVE>	INTEGER	The ECDSA curve **Value** / **Label** / **Description** 1 / SECP256R1 / Curve secp256r1. 2 / SECP384R1 / Curve secp384r1. 3 / SECP521R1 / Curve secp521r1.
<SIGN_VALUE>	BYTE_ARRAY	The value to be signed (big endian)

 ©2025-2026

Examples:



Figure 67. Example of TLS configuration for secure signing via external crypto operations

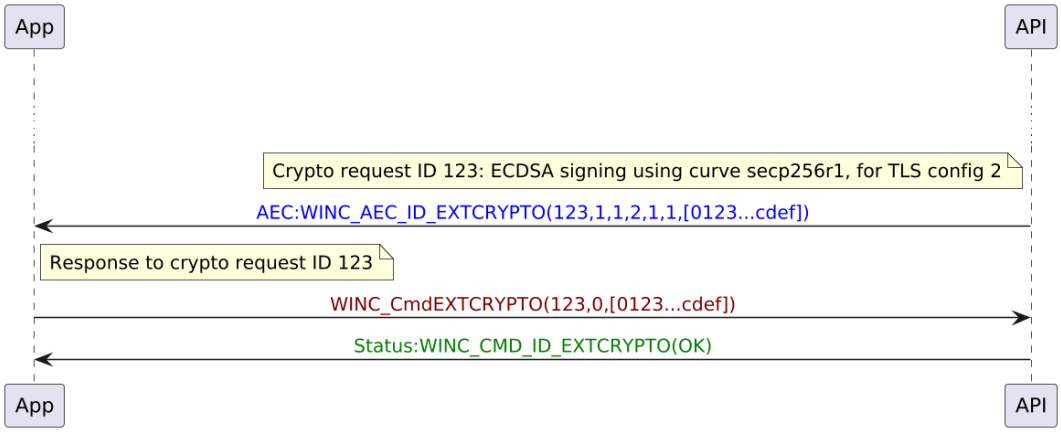


Figure 68. Example of signing request/response using WINC_CmdEXTCRYPTO

WIFI (Module ID = 28)

Introduction

The Wi-Fi module provides configuration settings for:

- Wi-Fi power save
- Bluetooth/Wi-Fi coexistence
- Regulatory domain
- Vendor-specific information elements (IEs)

Configure these settings with the WIFIC function.

Vendor-specific IEs are supported in management frames in both station (STA) and SoftAP modes. The transmit and receive settings are independent.

Transmitting vendor-specific IEs

Configure transmission with VSIE_TX_MASK, VSIE_TX_DATA, and VSIE_TX_DATA_COMMIT.

- VSIE_TX_MASK selects the frame types in which the IEs are transmitted. The IEs are added to frames of the selected types that the WINCS02 transmits. In SoftAP mode these are beacon and probe response frames; in STA mode, probe request frames.
- VSIE_TX_DATA holds up to four IE payloads. Each payload starts with a three-byte organizationally unique identifier (OUI), followed by up to 252 bytes of vendor-specific data.
- VSIE_TX_DATA_COMMIT activates the staged IE payloads.

Changes to VSIE_TX_DATA are applied to a staged set and have no effect until VSIE_TX_DATA_COMMIT is set. Any change to the staged set clears the commit state; the previously committed IEs remain active until the new set is committed. Reading VSIE_TX_DATA returns the active entries when VSIE_TX_DATA_COMMIT is set and the staged entries otherwise.

Clearing VSIE_TX_MASK or committing an empty VSIE_TX_DATA set disables vendor-specific IE transmission.

Both VSIE_TX_MASK and the IE payloads can be updated at run time.

See example configuring vendor-specific IEs for transmission.

 ©2025-2026

Receiving vendor-specific IEs

Configure reception with VSIE_RX_MASK and VSIE_RX_OUI.

- VSIE_RX_MASK selects the frame types from which IEs are received. Set it to `0` to stop receiving IEs.
- VSIE_RX_OUI is the three-byte OUI used to filter received IEs. Matching IEs from the selected frame types are reported by the WINC_AEC_ID_WIFIVSIERX AEC.

In SoftAP mode, IEs can be received from all three supported frame types: beacons, probe requests, and probe responses. In STA mode, reception additionally depends on the connection and scan state:

- While connected, beacon IEs are received only from the associated AP. Beacons from other APs are ignored.
- While disconnected and not scanning, all beacons are ignored.
- During a passive scan, beacon IEs are received from any AP that transmits a beacon within the scan interval.
- During an active scan, IEs are received in probe responses (scan responses).

See example configuring vendor-specific IE reception.

Function Reference:

WINC_CmdWIFIC

Description

This function is used to read or set the device's Wi-Fi configuration.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdWIFIC
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDWIFIC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response

 ©2025-2026

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_WIFI_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset **Value** / **Label** / **Description** 0 / DEFAULT / Default.
10	REGDOMAIN_SELECTED	STRING	Regulatory domain selected Maximum length of string is 6
11	REGDOMAIN_AVAILABLE	STRING (Read Only)	Regulatory domains available Maximum length of string is 6 This is a multiple value parameter with an ID range 11.0 to 11.5
12	REGDOMAIN_CHANMASK24	INTEGER_UNSIGNED (Read Only)	Regulatory channel mask (2.4 GHz) 15 14 13 12 11 10 9 8 - - CH14 CH13 CH12 CH11 CH10 CH9 7 6 5 4 3 2 1 0 CH8 CH7 CH6 CH5 CH4 CH3 CH2 CH1 **Value** / **Label** / **Description** 0x0001 / CH1 / Channel 1 - 2412 MHz. 0x0002 / CH2 / Channel 2 - 2417 MHz. 0x0004 / CH3 / Channel 3 - 2422 MHz. 0x0008 / CH4 / Channel 4 - 2427 MHz. 0x0010 / CH5 / Channel 5 - 2432 MHz. 0x0020 / CH6 / Channel 6 - 2437 MHz. 0x0040 / CH7 / Channel 7 - 2442 MHz. 0x0080 / CH8 / Channel 8 - 2447 MHz. 0x0100 / CH9 / Channel 9 - 2452 MHz. 0x0200 / CH10 / Channel 10 - 2457 MHz. 0x0400 / CH11 / Channel 11 - 2462 MHz. 0x0800 / CH12 / Channel 12 - 2467 MHz. 0x1000 / CH13 / Channel 13 - 2472 MHz. 0x2000 / CH14 / Channel 14 - 2485 MHz.

<table>
<tr><th>ID</th><th>Name</th><th>Type</th><th>Description</th></tr>
<tr><td>20</td><td>POWERSAVE</td><td>BOOL</td><td>Power Save Mode

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>OFF</td><td>Off.</td></tr><tr><td>1</td><td>ON</td><td>On.</td></tr></table></td></tr>
<tr><td>21</td><td>POWERSAVE_LISTEN_INTERVAL</td><td>INTEGER_UNSIGNED</td><td>Listen Interval in units of beacon period

Valid range is 3 to 100</td></tr>
<tr><td>22</td><td>POWERSAVE_INACT_LIMIT</td><td>INTEGER_UNSIGNED</td><td>Inactivity Time Limit - the sleep inactivity threshold/limit for power-save operation in seconds

Valid range is 10 to 6000</td></tr>
<tr><td>30</td><td>COEX_ENABLED</td><td>BOOL</td><td>BT/Wi-Fi coexistence arbiter

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>DISABLED</td><td>Disabled.</td></tr><tr><td>1</td><td>ENABLED</td><td>Enabled.</td></tr></table></td></tr>
<tr><td>31</td><td>COEX_INTERFACE_TYPE</td><td>BOOL</td><td>BT/Wi-Fi coexistence arbiter interface type

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>3WIRE</td><td>3-wire interface (BT_Act, BT_Prio, WLAN_Act).</td></tr><tr><td>1</td><td>2WIRE</td><td>2-wire interface (BT_Prio, WLAN_Act).</td></tr></table></td></tr>
<tr><td>32</td><td>COEX_WLAN_RX_VS_BT</td><td>BOOL</td><td>BT/Wi-Fi coexistence arbiter WLAN Rx priority over BT Low Priority

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>WLAN_LOW</td><td>WLAN Rx priority lower than BT Low Priority.</td></tr><tr><td>1</td><td>WLAN_HIGH</td><td>WLAN Rx priority higher than BT Low Priority.</td></tr></table></td></tr>
<tr><td>33</td><td>COEX_WLAN_TX_VS_BT</td><td>BOOL</td><td>BT/Wi-Fi coexistence arbiter WLAN Tx priority over BT Low Priority

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>WLAN_LOW</td><td>WLAN Tx priority lower than BT Low Priority.</td></tr><tr><td>1</td><td>WLAN_HIGH</td><td>WLAN Tx priority higher than BT Low Priority.</td></tr></table></td></tr>
</table>

 ©2025-2026

<table>
<tr><th>ID</th><th>Name</th><th>Type</th><th>Description</th></tr>
<tr><td>34</td><td>COEX_ANTENNA_MODE</td><td>BOOL</td><td>BT/Wi-Fi coexistence arbiter antenna mode

Value | Label | Description
0 | DEDICATED | Dedicated antenna.
1 | SHARED | Shared antenna.</td></tr>
<tr><td>40</td><td>AMPDU_TX_ENABLED</td><td>BOOL</td><td>Wi-Fi A-MPDU Tx Enable

Value | Label | Description
0 | TX_AMPDU_DISABLED | Disabled.
1 | TX_AMPDU_ENABLED | Enabled.</td></tr>
<tr><td>50</td><td>VSIE_TX_MASK</td><td>INTEGER_UNSIGNED</td><td>Frame types carrying the transmitted vendor-specific IEs, bitmask of:

Value | Label | Description
0x01 | BEACON | Beacon.
0x02 | PROBE_REQ | Probe request.
0x04 | PROBE_RSP | Probe response.</td></tr>
<tr><td>51</td><td>VSIE_TX_DATA</td><td>BYTE_ARRAY</td><td>Vendor-specific IE data
This is a multiple value parameter
with an ID range 51.0 to 51.3</td></tr>
<tr><td>52</td><td>VSIE_TX_DATA_COMMIT</td><td>BOOL</td><td>Commit the vendor-specific IE transmit data to the device. Modifying the transmit data clears this element, it must be set again for the new data to be transmitted

Value | Label | Description
0 | UNCOMMITTED | Transmit IE data has changed since it was last committed.
1 | COMMITTED | Transmit IE data is live.</td></tr>
<tr><td>55</td><td>VSIE_RX_MASK</td><td>INTEGER_UNSIGNED</td><td>Frame types filtered for vendor-specific IE reception, bitmask of:

Value | Label | Description
0x01 | BEACON | Beacon.
0x02 | PROBE_REQ | Probe request.
0x04 | PROBE_RSP | Probe response.</td></tr>
<tr><td>56</td><td>VSIE_RX_OUI</td><td>BYTE_ARRAY</td><td>Receive vendor-specific IE OUI filter</td></tr>
</table>

AEC Reference:

WINC_AEC_ID_WIFIVSIERX

Description

Received vendor-specific IE.

AEC Syntax

AEC	Description
<FRAME_TYPE>,<SOURCE_MAC>,<RSSI>,<DATA>	Received vendor-specific IE

AEC Element Syntax

Element Name	Type	Description
<FRAME_TYPE>	INTEGER	Management frame type **Value** / **Label** / **Description** 1 / BEACON / Beacon. 2 / PROBE_REQ / Probe request. 4 / PROBE_RSP / Probe response.
<SOURCE_MAC>	STRING	Source MAC address
<RSSI>	INTEGER	Received signal strength in dBm
<DATA>	BYTE_ARRAY	Vendor-specific IE data

 ©2025-2026

Examples:



Figure 69. Settings with WINC_CmdWIFIC

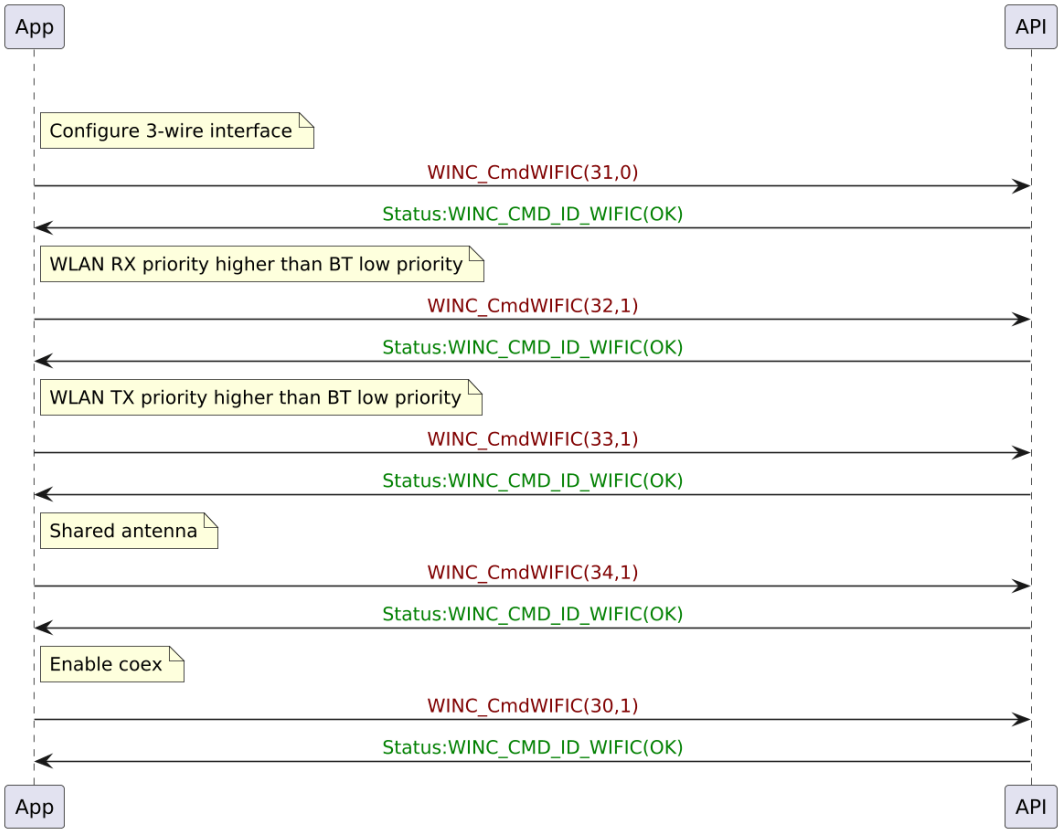


Figure 70. WINC_CmdWIFIC coexistence settings for 3-wire PTA interface



Figure 71. Configure vendor-specific information elements for transmission

 ©2025-2026

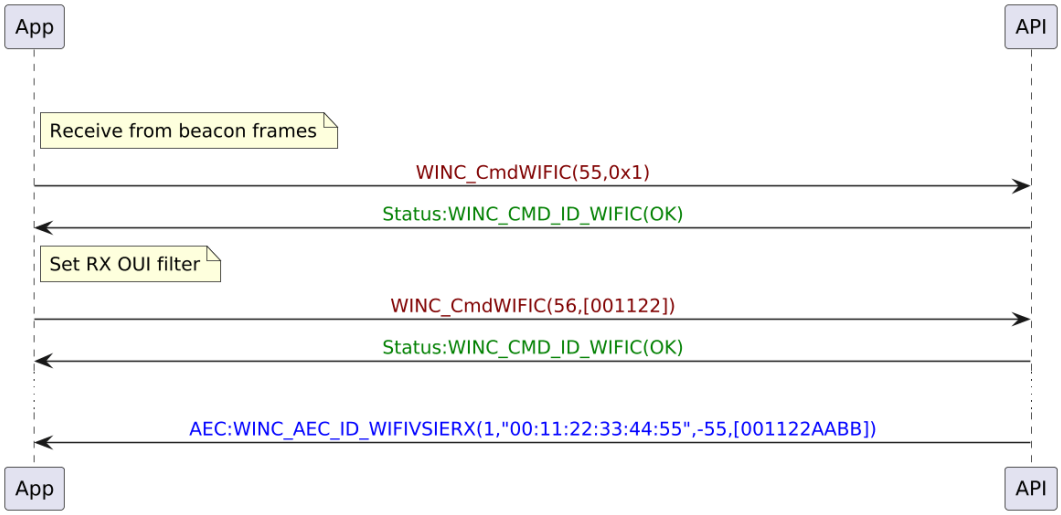


Figure 72. Configure vendor-specific information element reception

NVM (Module ID = 29)

Introduction

The NVM module provides access to the alternate firmware partition, the partition which does not contain the running firmware image. All offsets are relative to the start of the partition, its size is reported by the NUM_SECTORS and SECTOR_SZ configuration values.

- WINC_CmdNVMER erases whole 4 KB sectors. The erase runs in the background and completes with the WINC_AEC_ID_NVMER AEC, or WINC_AEC_ID_NVMERR on failure.
- WINC_CmdNVMWR writes up to 1024 bytes and completes immediately. A write must not cross a sector boundary, and the area must have been erased first. The data is read back and verified after writing.
- WINC_CmdNVMRD reads up to 1024 bytes, which are returned in the command response.
- WINC_CmdNVMCHK calculates a CRC16 or SHA digest over an area of the partition, as selected by CHECK_MODE. The calculation runs in the background and the result is reported by the WINC_AEC_ID_NVMCHK AEC, or WINC_AEC_ID_NVMERR on failure.

Only one erase, write, read or check operation may run at a time, a new operation fails while an erase or check is in progress. After WINC_CmdOTAINV erase and write are locked until the next reset.

 ©2025-2026

Function Reference:

WINC_CmdNVMC

Description

This function is used to read or set the NVM configuration.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdNVMC
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDNVMC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_NVM_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset Value: 0 — Label: DEFAULT — Description: Default.
1	START_OFFSET	INTEGER_UNSIGNED (Read Only)	Starting address offset
2	NUM_SECTORS	INTEGER_UNSIGNED (Read Only)	Number of sectors
3	SECTOR_SZ	INTEGER_UNSIGNED (Read Only)	Sector size
4	CHECK_MODE	INTEGER_UNSIGNED	Checking mode Value: 2 — Label: CRC16 — Description: CRC16. Value: 10 — Label: SHA — Description: SHA. Value: 11 — Label: SHA256 — Description: SHA256. Value: 12 — Label: SHA224 — Description: SHA224. Value: 13 — Label: SHA512 — Description: SHA512. Value: 14 — Label: SHA384 — Description: SHA384.

 ©2025-2026

WINC_CmdNVMER

Description

This function allows sector erase within the alternate firmware partition.

```
bool WINC_CmdNVMER
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t sectorOffset
    uint8_t sectors
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
sectorOffset	uint8_t	Sector offset to be used for the NVM operation.
sectors	uint8_t	The number of sectors to erase during NVM operation.

Return Values

Value	Description
true	Success.
false	Failure.

WINC_CmdNVMWR

Description

This function allows data to be written to the alternate firmware partition. Writes are performed as normal for flash devices, therefore it is recommended to erase the sector of data before performing a write. Write operations cannot cross a sector boundary.

```
bool WINC_CmdNVMWR
(
    WINC_CMD_REQ_HANDLE handle
    uint32_t offset
    uint16_t length
    const uint8_t* pData
    size_t lenData
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
offset	uint32_t	Byte offset to be used for the NVM operation.
length	uint16_t	The number of bytes for the NVM operation.
pData	const uint8_t*	The data to write in hexadecimal string format.
lenData	size_t	Length of pData in bytes.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

WINC_CmdNVMCHK

Description

This function allows checking of the alternate firmware partition.

```
bool WINC_CmdNVMCHK
(
    WINC_CMD_REQ_HANDLE handle
    uint32_t offset
    uint32_t length
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
offset	uint32_t	Byte offset to be used for the NVM operation.
length	uint32_t	The number of bytes for the NVM operation.

Return Values

Value	Description
true	Success.
false	Failure.

WINC_CmdNVMRD

Description

This function allows reading of the alternate firmware partition.

```
bool WINC_CmdNVMRD
(
    WINC_CMD_REQ_HANDLE handle
    uint32_t offset
    uint16_t length
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
offset	uint32_t	Byte offset to be used for the NVM operation.
length	uint16_t	The number of bytes for the NVM operation.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<OFFSET>,<LENGTH>,<DATA>	Read response

Response Element Description

Element Name	Type	Description
<OFFSET>	INTEGER	Byte offset Valid range is 0 to 0x000EFFFF
<LENGTH>	INTEGER	The number of bytes Valid range is 1 to 1024
<DATA>	STRING BYTE_ARRAY	Data bytes Valid range is 1 to 1024

 ©2025-2026

AEC Reference:

WINC_AEC_ID_NVMER

Description

NVM Erase.

AEC Syntax

AEC	Description
No Elements	NVM erase success

WINC_AEC_ID_NVMCHK

Description

NVM Check.

AEC Syntax

AEC	Description
<OFFSET>,<LENGTH>,<CHECK_MODE>,<HASH>	Hash response

AEC Element Syntax

Element Name	Type	Description
<OFFSET>	INTEGER	Byte offset Valid range is 0 to 0x000EFFFF
<LENGTH>	INTEGER	The number of bytes Valid range is 1 to 0x000F0000
<CHECK_MODE>	INTEGER	Checking mode used **Value** / **Label** / **Description** 2 / CRC16 / CRC16. 10 / SHA / SHA. 11 / SHA256 / SHA256. 12 / SHA224 / SHA224. 13 / SHA512 / SHA512. 14 / SHA384 / SHA384.
<HASH>	STRING BYTE_ARRAY	Hash response

 ©2025-2026

WINC_AEC_ID_NVMERR

Description

NVM error.

AEC Syntax

AEC	Description
<ERROR_CODE>	NVM error

AEC Element Syntax

Element Name	Type	Description
<ERROR_CODE>	INTEGER	Error code See Status Response Codes

Examples:

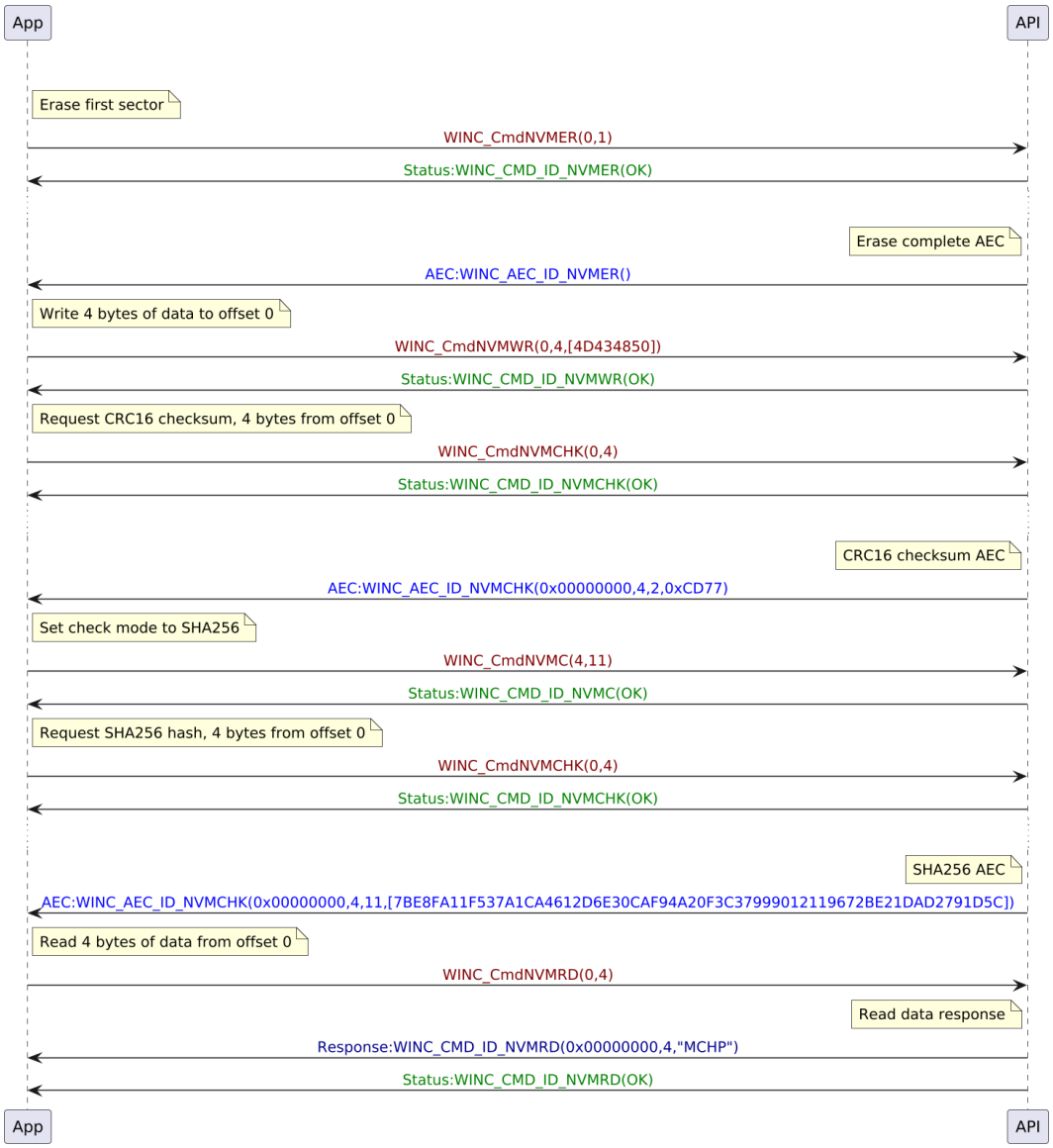


Figure 73. Simple example of NVM usage

 ©2025-2026

DFU (Module ID = 30)

Introduction

Secure Device Firmware Update (DFU) is a process for upgrading the firmware of a secured device.

The WINCS02 device has non-volatile memory (NVM) with two partitions which may be used to store firmware images.

The partitions can be referred to as "low" and "high" based on their physical addresses. They can also be referred to as "current" and "alternate" based on their usage at a given time; in normal operation, the "current" partition contains a copy of the currently running firmware image, while the "alternate" partition may be blank or may contain a rollback image, a latent image or any other data.

Region	Address	Size
Low partition	0x60000000	960 KB (0xF0000)
High partition	0x600F0000	960 KB (0xF0000)
File system	0x601E0000	Not affected by DFU

The DFU process should involve the following steps:

1. Determine the NVM partition in which to program the firmware image.
2. Prepare the firmware image.
3. Boot the device in DFU mode.
4. Erase and program the NVM partition.
5. Reset the device.

Determine the NVM partition in which to program the firmware image

The appropriate NVM partition ("alternate", "current", "low" or "high") depends on the user's upgrade policy. Two common upgrade policies are:

- A "flip-flop" policy in which the new firmware image is always programmed to the "alternate" partition.
- A "golden image" policy in which the initial firmware image is programmed to one partition (based on physical address), and subsequent firmware images are always programmed to the other partition.

Prepare the firmware image

Locate the released wincs02_dfu_high.bin file for the firmware version to which you want to upgrade.

Read the firmware image from the file, then update two values in the image header. Both are 32-bit little endian values:

Offset	Length	Field
0	4	Sequence Number
20	4	Firmware Image Source Address

- Sequence Number: The Sequence Number is the boot priority of the firmware image. Use the WINC_CmdDFUSEQ function to get the offset, length and value for updating it.
- Firmware Image Source Address: The Firmware Image Source Address must match the physical address of the NVM partition in which the firmware image is to be programmed. Use the WINC_CmdDFUADR function, with the desired partition, to get the offset, length and value for updating it.

Neither field is covered by the image signatures, so updating them does not invalidate the image. No other part of the image may be changed.

The image with the lowest Sequence Number has the highest boot priority. The Sequence Number of a latent image is 0xFFFFFFFF; a latent image is never booted until it is activated. For an activated image, the value returned is 0x10 lower than the lowest Sequence Number of any valid image already in the NVM, so the new image boots on the next reset. The lowest four bits of the Sequence Number are used by the boot ROM and are always returned as 0.

NOTE A status code of WINC_STATUS_DFU_ADDRESS_WARNING is a warning that the other partition does not contain a bootable firmware image. The offset, length and value in the response are still valid and should be used. If the user proceeds in spite of this warning and step 4 does not complete for any reason, the device will reach a state in which it contains no bootable firmware images. In that situation, repeating step 4 should recover the device, so long as the user still has the firmware image available.

NOTE Save the value of the NVM partition address for use during step 4.

 ©2025-2026

Boot the device in DFU mode

Boot the device in DFU mode using the following steps:

1. Assert WINCS02 pin NMCLR (low).
2. Write to WINCS02 pins PB0 (Clock) and PB1 (Data) to send the DFU Test Mode Entry pattern "MCHP".
3. Deassert WINCS02 pin NMCLR.

In DFU mode the device runs the Programming Executive (PE), which communicates over a UART at 230400 baud, 8 data bits, no parity, 1 stop bit.

Erase and program the NVM partition

Configure pins PB0 and PB1 for UART Rx and Tx respectively, from the WINCS02's point of view: the host transmits to PB0 and receives from PB1.

Use Programming Executive (PE) commands to erase the NVM partition, at the address obtained during step 2. Erase only the number of pages needed for the image.

WARNING	Do not use the PE chip erase command. It erases both partitions and the file system.

Use Programming Executive (PE) commands to program the new firmware image into the NVM partition, at the address obtained during step 2.

Reset the device

Reset the device to leave DFU mode. If the image was prepared as activated it is booted; if it was prepared as latent, the previous image continues to run.

The state of the image in each partition can be read with WINC_CmdDI. A latent image in the alternate partition can be activated later with WINC_CmdOTAACT.

Function Reference:

WINC_CmdDFUADR

Description

This function is used to get the parameters for modifying the little endian address field of a DFU image.

```
bool WINC_CmdDFUADR
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t partition
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
partition	uint8_t	The partition into which the DFU image will be programmed. **Value** / **Label** / **Description** 0 / ALTERNATE / Alternate partition (recommended). 1 / CURRENT / Current partition. 2 / LOW / Low partition. 3 / HIGH / High partition.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<OFFSET>,<LENGTH>,<VALUE>	Parameters for modifying the little endian address field of a DFU image

Response Element Description

Element Name	Type	Description
<OFFSET>	INTEGER	Offset (in bytes) of field in DFU image
<LENGTH>	INTEGER	Length (in bytes) of field in DFU image
<VALUE>	INTEGER	Value to write to field in DFU image

 ©2025-2026

WINC_CmdDFUSEQ

Description

This function is used to get the parameters for modifying the little endian sequence number field of a DFU image.

```
bool WINC_CmdDFUSEQ
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t imageState
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
imageState	uint8_t	The desired state of the DFU image after programming. **Value** / **Label** / **Description** 0 / LATENT / Latent image (will not run until activated). 1 / ACTIVATED / Activated image.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<OFFSET>,<LENGTH>,<VALUE>	Parameters for modifying the little endian sequence number field of a DFU image

Response Element Description

Element Name	Type	Description
<OFFSET>	INTEGER	Offset (in bytes) of field in DFU image
<LENGTH>	INTEGER	Length (in bytes) of field in DFU image
<VALUE>	INTEGER	Value to write to field in DFU image

Examples:

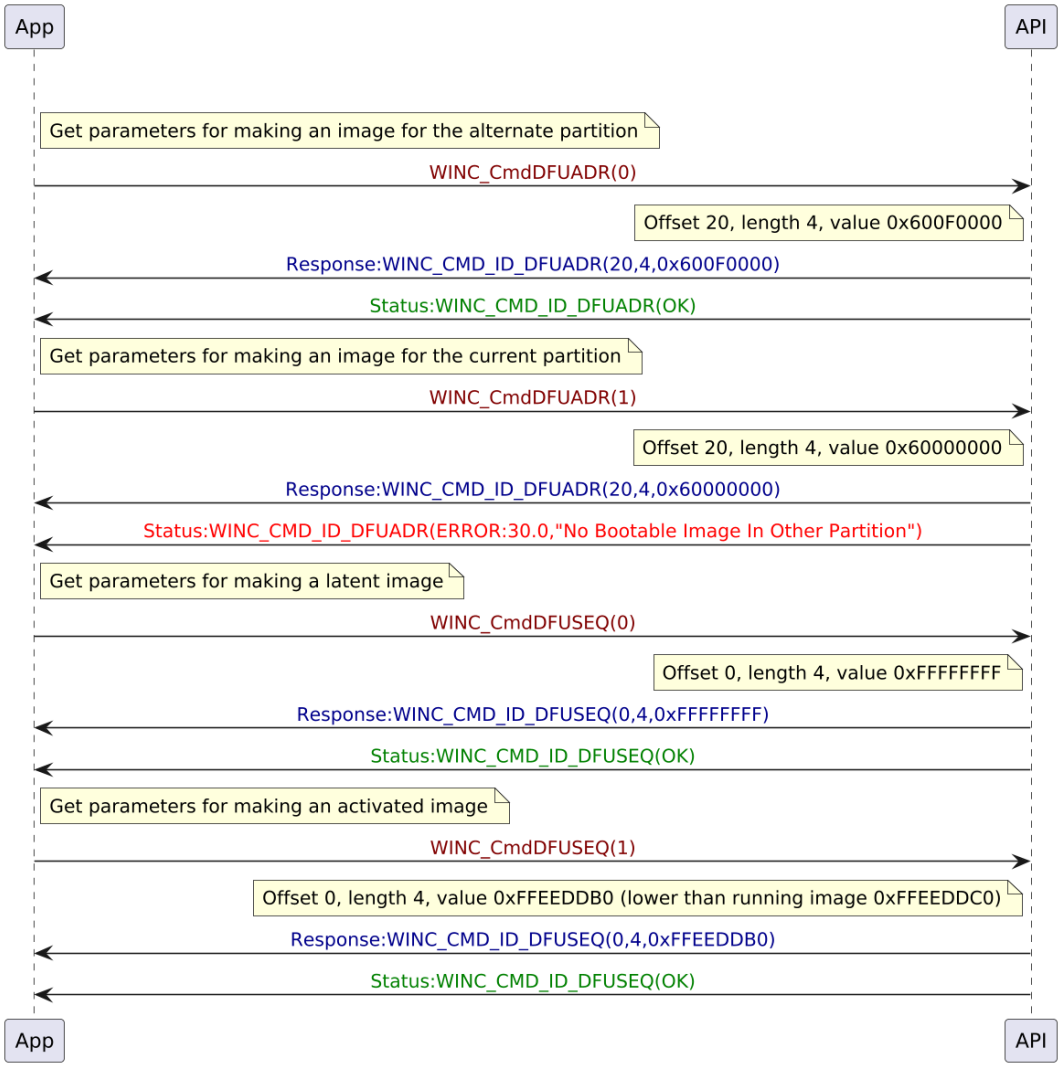


Figure 74. Example of using WINC_CmdDFUADR and WINC_CmdDFUSEQ

 ©2025-2026

PPS (Module ID = 31)

Introduction

The WINCS02 device supports two advanced power-saving modes designed to reduce current consumption in typical IoT and embedded applications:

- **Platform Power-Save (PPS)** mode places the CPU and hardware peripherals into sleep modes - most clocks and peripherals are powered down, leaving only essential hardware blocks active. This drastically reduces power consumption while preserving Wi-Fi connectivity and system context, enabling rapid wake-up and seamless operation.
- **EXtreme Deep Sleep (XDS)** enters the device's lowest power state, with all peripherals powered down and system context lost. This mode achieves maximum power savings for long standby periods where Wi-Fi or network connections are not required. Waking from XDS triggers a full system reboot.

PPS Operation Details

When **PPS** is enabled, the system automatically transitions into PPS mode during idle periods, minimising power while maintaining system and Wi-Fi state. This includes active protocol connections, Wi-Fi association, and buffered data queues.

Wake-Up Triggers

There are a number of events that will cause the WINCS02 to wake when PPS is active:

- Internal timers or internal scheduled events
  - These will be accounted for when PPS calculates its sleep duration for the next period, ensuring it wakes up in time to process the next event/timer
- Host MCU activity over the SPI interface
  - Attempts to use the SPI interface will wake the WINCS02 immediately
- In the absence of internal timers or external triggers, PPS will wake the WINCS02 after 10 seconds

Once awake, the WINCS02 will run until the next idle period, whereby it will transition into PPS mode again.

Wi-Fi Power Save Mode during PPS

When PPS is enabled and Wi-Fi is connected, the WINCS02 operates Wi-Fi Legacy Powersave to minimise power consumption while maintaining network connectivity.

If Wi-Fi power save (POWERSAVE) is not already enabled, enabling PPS enables it. It remains enabled after PPS is disabled. If Wi-Fi power save cannot be enabled, enabling PPS fails with WINC_STATUS_PPS_WIFI_PS_NOT_ENABLED.

Listen Interval

The WINCS02 wakes periodically based on the configured Wi-Fi Listen Interval, which determines how often it wakes to listen for buffered traffic from the AP. By default, this interval is 10 beacon periods, roughly equivalent to 1 second (typical beacon interval is 102.4 ms). The range is 3 to 100 beacon periods.

The listen interval is configured with POWERSAVE_LISTEN_INTERVAL.

Traffic buffered by the AP may be delayed by up to one listen interval. A longer interval saves more power but increases this delay.

Each sleep period lasts between 40 ms and 20 seconds; the WINCS02 does not sleep if the next event is less than 40 ms away.

DTIM (Delivery Traffic Indication Message) Awareness

To efficiently receive broadcast and multicast traffic buffered by the AP, the device aims to wake on the closest DTIM period aligned with the configured listen interval. The DTIM is a special beacon that signals buffered broadcast/multicast data transmission by the AP. For example, if the Listen Interval is set to 10 beacon periods (default) and the AP's DTIM period is 3, the WINCS02 will wake after 9 beacon periods (i.e. on the 3rd DTIM) instead of exactly after 10 beacons. This alignment maximises power savings by minimising unnecessary wake events while ensuring timely reception of important broadcast traffic.

Sleep Timing Accuracy

To optimise power saving and accuracy of wake intervals, it is strongly recommended to connect and enable a secondary oscillator (SOSC) clocked at 32.768 kHz. The SOSC provides a high-accuracy timing source during sleep, allowing for more precise wake-up timing when using longer sleep durations. The crystal is connected to pins PB3 and PB4.

The SOSC shares pin PB4 with the BT_ACT input of 3-wire Wi-Fi/BT coexistence. If SEC_OSC_ENABLED is set while coexistence is enabled in 3-wire mode (COEX_ENABLED and COEX_INTERFACE_TYPE), enabling PPS fails with WINC_STATUS_CONFIG_CONFLICT. Use 2-wire coexistence, or do not enable the SOSC.

 ©2025-2026

PPS Pause

PPS Pause allows temporarily suspending Platform Power-Save operation without fully disabling it. This is useful during periods where continuous device responsiveness is required, such as during critical command sequences, firmware updates or other high-activity times when entering sleep state would be undesirable.

How PPS Pause Works

- When a pause is active, the system remains fully awake, bypassing PPS sleep transitions.
- Multiple pause commands can be issued sequentially, and the pause durations extend the total pause period.
- After all active pause timers expire, the WINCS02 automatically sends the WINC_AEC_ID_PPS AEC to indicate that PPS will now resume.
- Once the pause period has expired, the system will return to normal PPS operation, entering low-power sleep during idle periods.

Hardware Connections

Signal	Pin	Function
SOSC	PB3, PB4	Optional 32.768 kHz crystal, see Sleep Timing Accuracy. PB4 is also the BT_ACT input of 3-wire coexistence.
MCLR	MCLR	Reset. Also wakes the WINCS02 from XDS.

No dedicated wake pins are needed; the WINCS02 is woken by a SPI transaction.

Wake-Up via SPI

- The SPI core supports **clockless operation**, so no additional hardware signals or protocol changes are needed to wake the device.
- The host MCU simply starts a SPI transaction to wake the WINCS02.

NOTE

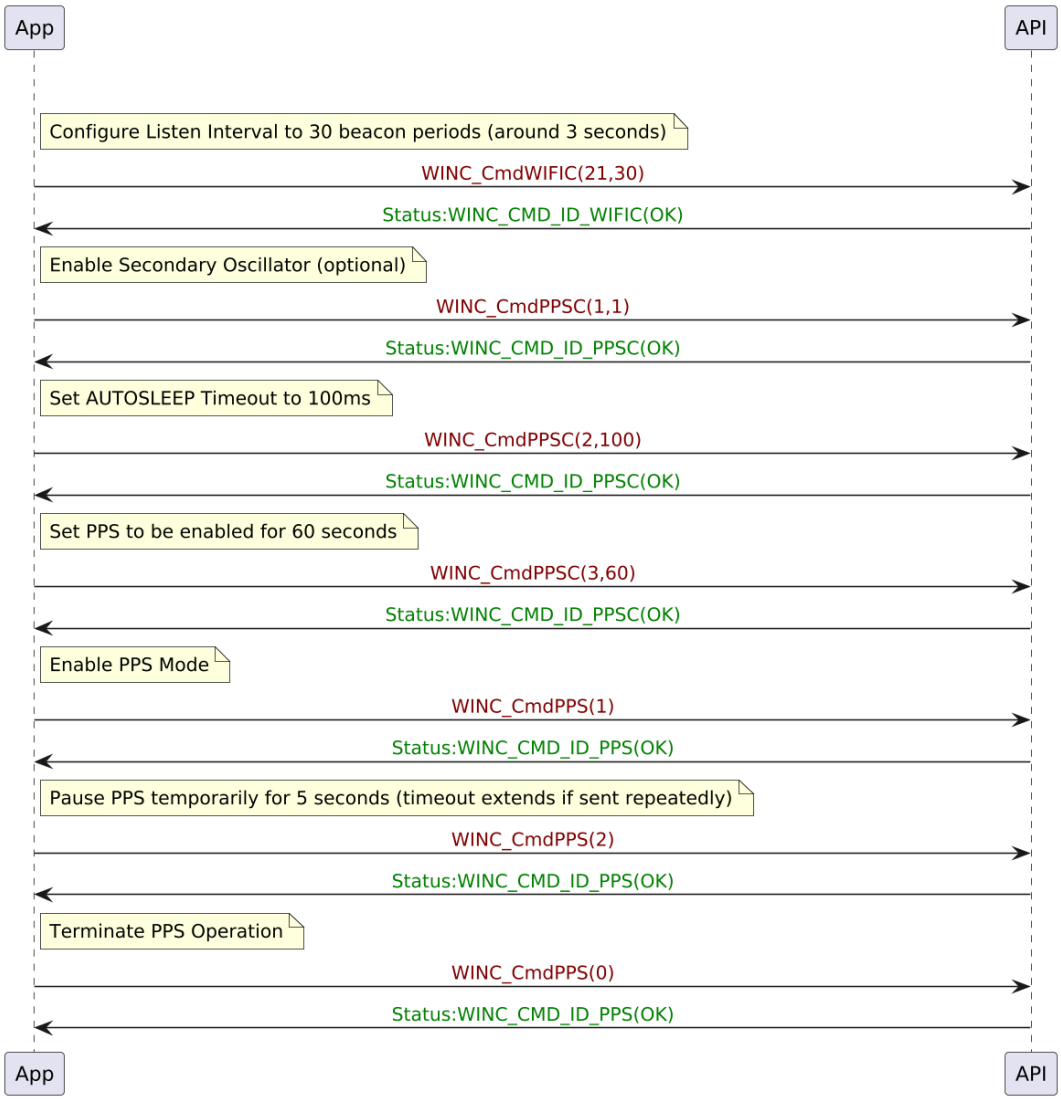
- Wake-up latency from PPS mode is approximately **4 ms**.

Command Transmission and AUTOSLEEP

- The WINCS02 remains awake while processing the SPI transactions and any corresponding Wi-Fi or protocol activity.
- Once no further commands or traffic are outstanding, the system returns to PPS sleep.
- The AUTOSLEEP parameter controls how long the WINCS02 remains awake, allowing multiple transactions to be sent without re-waking each time:
  - Configured with `WINC_CmdPPSC(handle, WINC_CFG_PARAM_ID_PPS_AUTO_SLEEP_TIMEOUT, WINC_TYPE_INTEGER_UNSIGNED, <ms>, 0)`
  - Extends the time WINCS02 stays awake after waking for a SPI transaction, providing the host extra time to send multiple SPI transactions.

Configuration and Usage

See example Configuring and enabling PPS



 ©2025-2026

EXtreme Deep Sleep (XDS)

XDS mode puts the WINCS02 into the absolute lowest power state, with all but the bare minimum clocks and peripherals powered off. Use this mode when long idle periods are expected and Wi-Fi connectivity/state retention is not required.

XDS characteristics

- The system loses all context and network state (no Wi-Fi connection or open sockets preserved).
- The only wake-up methods available are hardware signals:
  - Pull **MCLR** pin low (hardware reset)

NOTE	Wake-up triggers a **full system reboot**.

XDS Operation details

To enter XDS mode, call WINC_CmdPPS:

```
WINC_CmdPPS(handle, WINC_CONST_PPS_PPS_STATE_XDS)
```

The WINCS02 enters XDS when it is next idle after the command completes.

Summary

PPS (Platform Power Save)

Balances power saving with connectivity. Maintains Wi-Fi and protocol state. Supports low-latency wake-up. Recommended to enable SOSC for precise timing.

XDS (EXtreme Deep Sleep)

Maximises power saving by powering down all but the bare minimum device functions. Requires hardware wake-up and full reboot on wake. Used for extended idle periods with no connectivity requirements.

Function Reference:

WINC_CmdPPSC

Description

This function is used to read or set the PPS configuration.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdPPSC
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDPPSC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response

 ©2025-2026

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_PPS_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset **Value** / **Label** / **Description** 0 / DEFAULT / Default.
1	SEC_OSC_ENABLED	BOOL	Use secondary oscillator **Value** / **Label** / **Description** 0 / OFF / Off. 1 / ON / On.
2	AUTO_SLEEP_TIMEOUT	INTEGER_UNSIGNED	Timeout between commands before entering sleep measured in ms Valid range is 10 to 3000
3	DURATION	INTEGER_UNSIGNED	Time to maintain PPS mode in seconds (0 for infinite) Valid range is 0 to 86400

WINC_CmdPPS

Description

This function allows the firmware to change the state of platform power save.

```
bool WINC_CmdPPS
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t ppsState
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
ppsState	uint8_t	PPS State. **Value** / **Label** / **Description** 0 / DISABLE / Disable. 1 / PPS / PPS mode. 10 / XDS / XDS mode.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

WINC_CmdPPSPAUSE

Description

This function allows the firmware to change the state of platform power save.

```
bool WINC_CmdPPSPAUSE
(
    WINC_CMD_REQ_HANDLE handle
    uint16_t pauseTime
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
pauseTime	uint16_t	PPS pause time in sec.

Return Values

Value	Description
true	Success.
false	Failure.

AEC Reference:

WINC_AEC_ID_PPS

Description

Indication of PPS event.

AEC Syntax

AEC	Description
<PPS_EVENT>	PPS Event

AEC Element Syntax

Element Name	Type	Description
<PPS_EVENT>	INTEGER	PPS status code One of the PPS status codes WINC_STATUS_PPS_TIMEOUT (PPS duration expired) or WINC_STATUS_PPS_PAUSE_EXPIRED (PPS pause ended), see Status Response Codes.

 ©2025-2026

Examples:

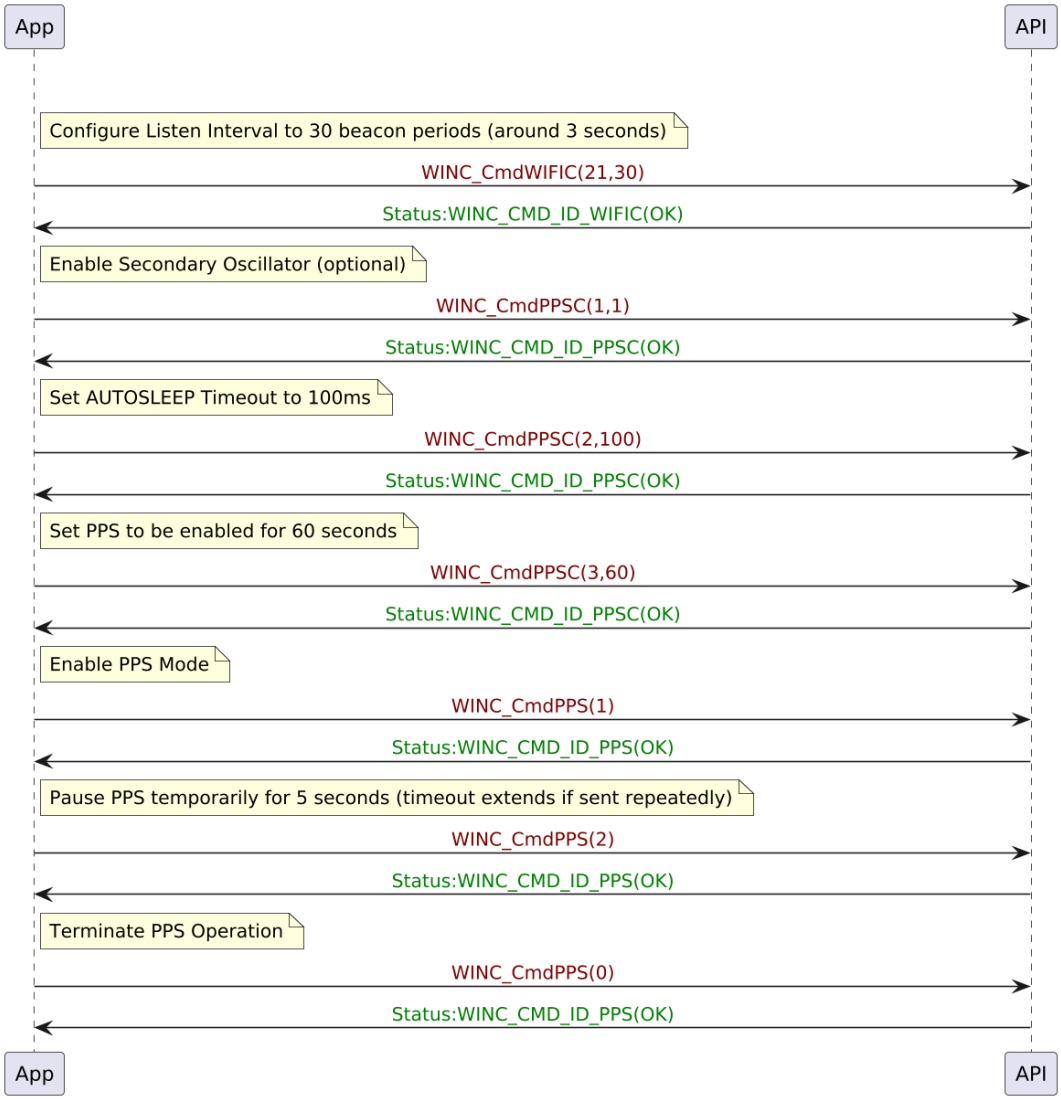


Figure 75. Configuring and enabling PPS

SYSLOG (Module ID = 32)

Introduction

When enabled with ENABLED, the SYSLOG module reports internal log events as WINC_AEC_ID_SYSLOG AECs.

Each event carries a timestamp in milliseconds, the ID of the module which raised it, a severity and a message of up to 127 characters. All events are reported, there is no filtering by module or severity.

Logging is disabled by default and the setting is not stored in the configuration archive. Currently only the TLS module reports events.

 ©2025-2026

Function Reference:

WINC_CmdSYSLOGC

Description

This function is used to read or set the SYSLOG configuration.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdSYSLOGC
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDSYSLOGC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_SYSLOG_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset **Value** / **Label** / **Description** 0 / DEFAULT / Default.
1	ENABLED	BOOL	device's SYSLOG **Value** / **Label** / **Description** 0 / DISABLED / Disabled. 1 / ENABLED / Enabled.

 ©2025-2026

AEC Reference:

WINC_AEC_ID_SYSLOG

Description

SYSLOG Event.

AEC Syntax

AEC	Description
<TIMESTAMP>,<MODULE_ID>,<EVENT_SEVERITY>,<MESSAGE>	Log message

AEC Element Syntax

Element Name	Type	Description
<TIMESTAMP>	INTEGER	System time in milliseconds when the message was logged Unsigned 32-bit value
<MODULE_ID>	INTEGER	ID of the module that logged the message Unsigned 8-bit value Module IDs are listed in the headings of the Status Response Codes tables.
<EVENT_SEVERITY>	INTEGER	Event severity **Value** / **Label** / **Description** 0 / EMERGENCY / Emergency. 1 / ALERT / Alert. 2 / CRITICAL / Critical. 3 / ERROR / Error. 4 / WARNING / Warning. 5 / NOTICE / Notice. 6 / INFO / Info. 7 / DEBUG / Debug.
<MESSAGE>	STRING	Log message

ARB (Module ID = 33)

Introduction

The WINCS02 has an anti-rollback counter (ARB) which can be used to prevent vulnerable firmware from running on the device.

The value in the ARB is between 0 and 127 and can be increased but never decreased. If a firmware image has a security level which is lower than the value in the ARB, then that image will not boot.

Use the WINC_CmdARB function to get the current value in the device's ARB and the current firmware's security level. If the device's ARB is lower than the firmware's security level, then consider increasing the value in the device's ARB to improve the security of the device.

For information about the security benefits of increasing the device's ARB, refer to WINCS02 device firmware release notes at microchip.com.

To increase the value in the device's ARB, use the WINC_CmdARB function.

CAUTION	If the device has an image present in the alternate NVM partition, be aware that increasing the device's ARB may invalidate that image.

 ©2025-2026

Function Reference:

WINC_CmdARB

Description

This function is used to get or increase the device's ARB value, and to get the current firmware's security level.

```
bool WINC_CmdARB
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t optValue
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optValue	uint8_t	The value to which to increase the device's ARB. optValue will be ignored if its value is WINC_CMDARB_VALUE_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<VALUE>,<FW_SECURITY>	The device's ARB value and the current firmware's security level

Response Element Description

Element Name	Type	Description
<VALUE>	INTEGER	device's ARB value
<FW_SECURITY>	INTEGER	Current firmware's security level

Examples:

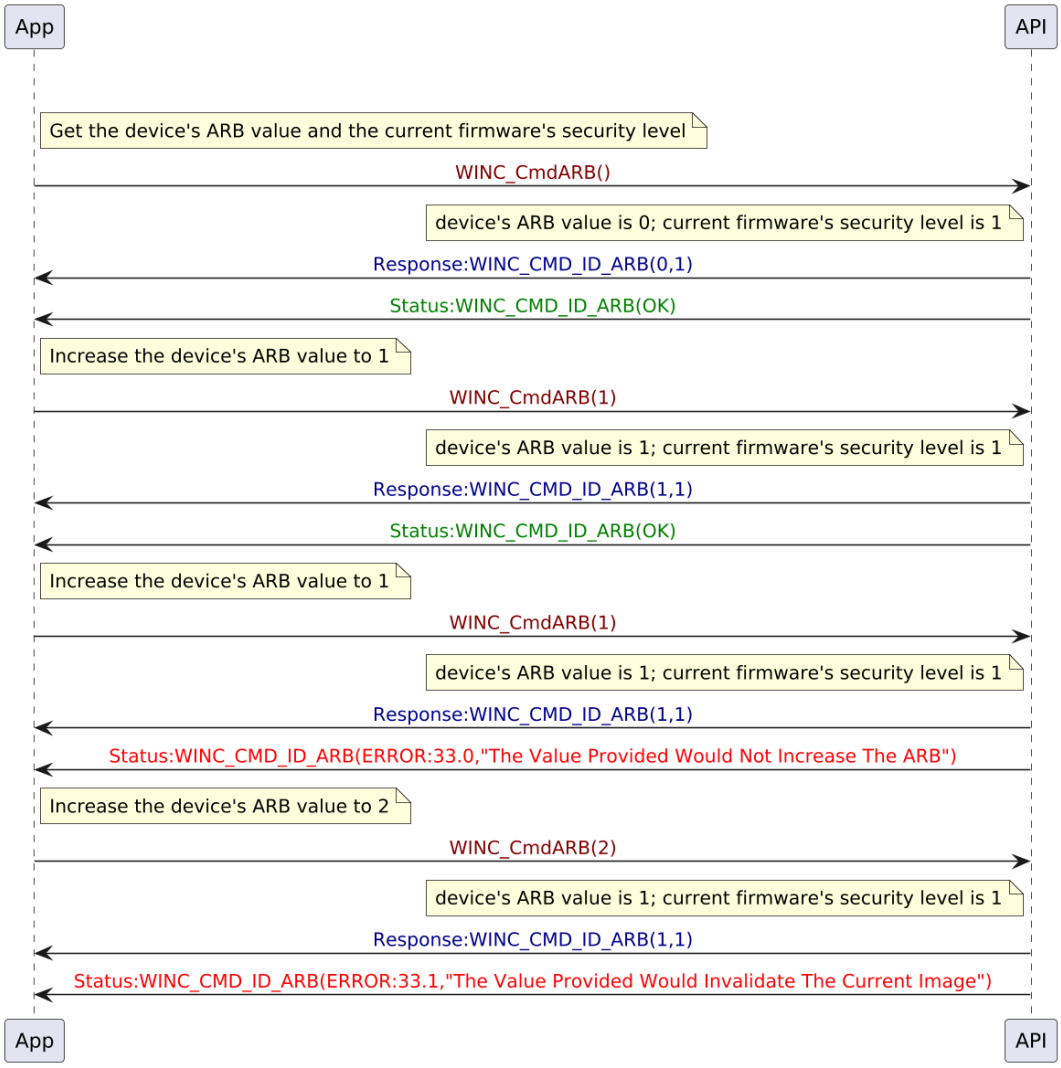


Figure 76. Example of using WINC_CmdARB

 ©2025-2026

HTTP (Module ID = 34)

Introduction

The WINCS02 supports comprehensive HTTP client capabilities via its API, enabling easy integration with web servers for a variety of applications such as firmware updates, data retrieval, and cloud communication.

Key Features

HTTP Configuration Management

Configure HTTP connection parameters including the server address (hostname or IP), port number, request path, TLS settings, timeouts, and more. These settings ensure flexible connection setups to suit diverse network environments and security needs.

HTTP GET Requests

Perform HTTP GET operations to request resources from a web server. You can specify the resource path, enabling the retrieval of files, JSON data, or other web content.

Direct File Downloads to Flash Filesystem

Download HTTP response data directly into the device's flash filesystem. By configuring a local filename and file type, the downloaded content is automatically stored in the flash filesystem without requiring manual data reading. This is particularly useful for downloading certificates, configuration files, firmware images, or other persistent data. The filesystem supports multiple file types including USER files, certificates (CERT), private keys (PRIKEY), Diffie-Hellman parameters (DHPARAM), and configuration files (CFG).

Data Reading

Efficiently read HTTP response data in chunks of defined length. This allows controlled data transfer suitable for devices with limited memory. Manual data reading can be performed using WINC_CmdHTTPRD when not downloading directly to the filesystem.

Response Handling

Receive detailed response information such as HTTP status codes, response headers, and body content, including full notification when the response body is completely received.

Data Integrity Verification

Optional hash calculation (SHA, SHA256, SHA224, SHA512, SHA384) during download to verify data integrity. The hash is computed automatically and reported in the WINC_AEC_ID_HTTPBODYEND asynchronous event.

Security Support

Integrated TLS configuration allows HTTPS connections ensuring data privacy and integrity.

How It Works

Manual Data Retrieval

See examples HTTPS download and HTTPS download from TCP socket.

- Configure the HTTP client using WINC_CmdHTTPC to set the hostname, port, path, and other parameters.
- Optionally configure TLS with WINC_CmdTLSC for encrypted HTTPS connections.
- Initiate an HTTP GET request via WINC_CmdHTTPGET.
- Monitor response status and headers through AECs (WINC_AEC_ID_HTTPRSPCODE, WINC_AEC_ID_HTTPRSPFIELD).
- Read the HTTP response body in manageable blocks using WINC_CmdHTTPRD.
- Close the connection cleanly with WINC_CmdHTTPCL after data transfer completes.

Direct Filesystem Download

See example File download to local filesystem.

- Configure the HTTP client using WINC_CmdHTTPC to set the hostname, port, path, and other parameters.
- Set the local filename using WINC_CmdHTTPC ID 20 (FILENAME parameter).
- Set the local file type using WINC_CmdHTTPC ID 21 (FILETYPE parameter):
  - 0 = USER (user files)
  - 1 = CERT (certificates)
  - 2 = PRIKEY (private keys)
  - 3 = DHPARAM (Diffie-Hellman parameters)
  - 20 = CFG (configuration files)
- Optionally configure hash verification using WINC_CmdHTTPC ID 22 (CHECK_MODE parameter):
  - 0 = NONE (no hash verification)
  - 10 = SHA
  - 11 = SHA256
  - 12 = SHA224
  - 13 = SHA512
  - 14 = SHA384
- Optionally configure TLS with WINC_CmdTLSC for encrypted HTTPS connections.
- Initiate an HTTP GET request via WINC_CmdHTTPGET.
- Monitor response status and headers through AECs (WINC_AEC_ID_HTTPRSPCODE, WINC_AEC_ID_HTTPRSPFIELD).

 ©2025-2026

- The downloaded data is automatically written to the flash filesystem.
- When complete, WINC_AEC_ID_HTTPBODYEND is sent with total bytes downloaded and optional hash value.
- A filesystem update notification WINC_AEC_ID_FSUP confirms the file has been stored.
- The connection closes automatically (non-persistent connection mode).

NOTE	When a local filename is configured, the HTTP client operates in non-persistent connection mode and automatically writes received data to the filesystem. Manual data reading via WINC_CmdHTTPRD is not available in this mode.

Advanced Features

Using an Existing Socket Connection

The HTTP client can utilize an existing TCP socket connection instead of creating its own. This is useful when you need more control over the connection setup, such as pre-configuring socket options or managing connection pooling.

- Create and connect a TCP socket using socket functions (WINC_CmdSOCKO, WINC_CmdSOCKBR).
- Optionally upgrade the socket to TLS using WINC_CmdSOCKTLS.
- Assign the socket ID to the HTTP configuration using WINC_CmdHTTPC ID 10 (SOCKET_ID parameter).
- Perform HTTP GET operations as normal using WINC_CmdHTTPGET.

NOTE	When using an existing socket, do not configure the HOST (ID 2) or PORT (ID 3) parameters in the HTTP configuration.

Path Override with Query Parameters

When issuing an HTTP GET request, you can specify a complete path including query parameters and fragments as the second parameter to WINC_CmdHTTPGET. This overrides the PATH (ID 4) and FILE (ID 5) configuration parameters for that specific request. See example HTTPGET with path and query/fragments.

HTTP Authentication

HTTP basic authentication can be implemented by providing additional request headers using WINC_CmdHTTPC ID 8 (REQ_HEADERS parameter). The authorization header should contain the Base64-encoded credentials.

See example HTTP basic authentication using additional request headers.

Reading Configuration Values

The read-only configuration parameter **CONTENT_LENGTH** reports the Content-Length value from the HTTP response headers after they are received.

Error Handling

The HTTP client reports errors via two mechanisms:

- **HTTP status codes** - The server's response status is reported via WINC_AEC_ID_HTTPRSPCODE. Non-2xx status codes (e.g., 401 Unauthorized, 404 Not Found, 500 Internal Server Error) indicate the server rejected the request.
- **Transport errors** - Connection-level errors are reported via WINC_AEC_ID_HTTPERR. These indicate failures before or during HTTP communication.

Common transport errors:

- **DNS Failed** - The server hostname could not be resolved
- **TLS Failed** - TLS handshake failed (wrong CA certificate, SNI mismatch, expired certificate)
- **Connect Failed** - TCP connection to server failed
- **Timeout** - No response received within the configured timeout period
- **Receive Failed** - Error receiving data from server
- **Send Failed** - Error sending request to server

 ©2025-2026

Function Reference:

WINC_CmdHTTPC

Description

This function is used to read or set the HTTP configuration.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdHTTPC
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t optConf
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optConf	uint8_t	Configuration number. optConf will be ignored if its value is WINC_CMDHTTPC_CONF_IGNORE_VAL.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDHTTPC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<MIN_IDX>,<MAX_IDX>	Valid index range
<ID>,<VAL>	Read response

Response Element Description

Element Name	Type	Description
<MIN_IDX>	INTEGER	Lowest valid index
<MAX_IDX>	INTEGER	Highest valid index
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters (WINC_CFG_PARAM_ID_HTTP_*)

ID	Name	Type	Description
0	PRESET	INTEGER	Configuration preset Value: 0 — Label: DEFAULT — Description: Default.
2	HOST	STRING IPV4ADDR IPV6ADDR	Host domain name or IP address Maximum length of string is 255 Format of IPv4 address is A1|A2|A3|A4 = A1.A2.A3.A4 Format of IPv6 address is A1|...|A16 = A1A2::A15A16
3	PORT	INTEGER_UNSIGNED	Server port (e.g. 80 for HTTP, 443 for HTTPS); must be set
4	PATH	STRING	Path to remote file Maximum length of string is 1024
5	FILE	STRING	Remote filename Maximum length of string is 1024
6	TIMEOUT	INTEGER_UNSIGNED	Timeout in seconds Positive unsigned 8-bit value
7	TLS_CONF	INTEGER	TLS configuration index (see WINC_CmdTLSCDA). 0 disables TLS Valid range is 0 to 4
8	REQ_HEADERS	STRING	Additional request headers Maximum length of string is 1024
10	SOCKET_ID	INTEGER_UNSIGNED	Socket ID
20	FILENAME	STRING	Local filename Maximum length of string is 32

 ©2025-2026

<table>
<tr><th>ID</th><th>Name</th><th>Type</th><th>Description</th></tr>
<tr><td>21</td><td>FILETYPE</td><td>INTEGER_UNSIGNED</td><td>Local file type
<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>USER</td><td>User.</td></tr><tr><td>1</td><td>CERT</td><td>Certificate.</td></tr><tr><td>2</td><td>PRIKEY</td><td>Private Key.</td></tr><tr><td>3</td><td>DHPARAM</td><td>Diffie-Hellman parameters.</td></tr><tr><td>20</td><td>CFG</td><td>Configuration File.</td></tr></table></td></tr>
<tr><td>22</td><td>CHECK_MODE</td><td>INTEGER_UNSIGNED</td><td>Checking mode
<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>NONE</td><td>None.</td></tr><tr><td>10</td><td>SHA</td><td>SHA.</td></tr><tr><td>11</td><td>SHA256</td><td>SHA256.</td></tr><tr><td>12</td><td>SHA224</td><td>SHA224.</td></tr><tr><td>13</td><td>SHA512</td><td>SHA512.</td></tr><tr><td>14</td><td>SHA384</td><td>SHA384.</td></tr></table></td></tr>
<tr><td>40</td><td>CONTENT_LENGTH</td><td>INTEGER_UNSIGNED (Read Only)</td><td>Content length</td></tr>
<tr><td>100</td><td>ASYNC_NEXT_OFFSET</td><td>INTEGER_UNSIGNED</td><td>Async next data offset</td></tr>
<tr><td>101</td><td>ASYNC_WIN_SZ</td><td>INTEGER_UNSIGNED</td><td>Async window size</td></tr>
<tr><td>102</td><td>ASYNC_MAX_FRM_SZ</td><td>INTEGER_UNSIGNED</td><td>Async maximum frame size</td></tr>
</table>

WINC_CmdHTTPGET

Description

This function performs an HTTP GET operation.

```
bool WINC_CmdHTTPGET
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t conf
    const uint8_t* pOptPath
    size_t lenOptPath
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
conf	uint8_t	Configuration number.
pOptPath	const uint8_t*	Resource path. pOptPath will be ignored if its value is NULL.
lenOptPath	size_t	Length of pOptPath in bytes, strings are not NUL terminated so the length must not include a terminator.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

WINC_CmdHTTPRD

Description

This function performs an HTTP data read operation.

```
bool WINC_CmdHTTPRD
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t conf
    int32_t length
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
conf	uint8_t	Configuration number.
length	int32_t	Number of bytes.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<CONF>,<DATA_LEN>,<DATA_OFFSET>,<DATA>	HTTP body data read

Response Element Description

Element Name	Type	Description
<CONF>	INTEGER	HTTP Configuration number Value is 1
<DATA_LEN>	INTEGER	Length of data
<DATA_OFFSET>	INTEGER	Offset of data bytes
<DATA>	BYTE_ARRAY	HTTP data

WINC_CmdHTTPCL

Description

This function performs an HTTP close operation.

```
bool WINC_CmdHTTPCL
(
    WINC_CMD_REQ_HANDLE handle
    uint8_t conf
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
conf	uint8_t	Configuration number.

Return Values

Value	Description
true	Success.
false	Failure.

 ©2025-2026

AEC Reference:

WINC_AEC_ID_HTTPRSPCODE

Description

HTTP response code.

AEC Syntax

AEC	Description
<CONF>,<RSP_PROTOCOL>,<RSP_CODE>	HTTP response code

AEC Element Syntax

Element Name	Type	Description
<CONF>	INTEGER	HTTP Configuration number Value is 1
<RSP_PROTOCOL>	INTEGER	HTTP response protocol version Value is 1.1
<RSP_CODE>	INTEGER	HTTP response code Valid range is 100 to 599

WINC_AEC_ID_HTTPRSPFIELD

Description

HTTP response field.

AEC Syntax

AEC	Description
<CONF>,<RSP_FIELD_NAME>,<RSP_FIELD_VALUE>	HTTP response field

AEC Element Syntax

Element Name	Type	Description
<CONF>	INTEGER	HTTP Configuration number Value is 1
<RSP_FIELD_NAME>	STRING	HTTP response field name
<RSP_FIELD_VALUE>	STRING	HTTP response field value

 ©2025-2026

WINC_AEC_ID_HTTPBODYRX

Description

HTTP body.

AEC Syntax

AEC	Description
<CONF>,<PEND_DATA>	HTTP body data notify
<CONF>,<DATA_LEN>,<DATA_OFFSET>,<DATA>	HTTP body data received

AEC Element Syntax

Element Name	Type	Description
<CONF>	INTEGER	HTTP Configuration number Value is 1
<PEND_DATA>	INTEGER	Number of pending bytes
<DATA_LEN>	INTEGER	Length of data
<DATA_OFFSET>	INTEGER	Offset of data bytes
<DATA>	BYTE_ARRAY	HTTP data

WINC_AEC_ID_HTTPBODYEND

Description

HTTP body end.

AEC Syntax

AEC	Description
<CONF>,<DATA_LEN>	HTTP body data end
<CONF>,<DATA_LEN>,<HASH_TYPE>,<HASH>	HTTP body data end

AEC Element Syntax

Element Name	Type	Description
<CONF>	INTEGER	HTTP Configuration number Value is 1
<DATA_LEN>	INTEGER	Length of data
<HASH_TYPE>	INTEGER	Hash algorithm **Value** / **Label** / **Description** 10 / SHA / SHA. 11 / SHA256 / SHA256. 12 / SHA224 / SHA224. 13 / SHA512 / SHA512. 14 / SHA384 / SHA384.
<HASH>	BYTE_ARRAY	Hash

 ©2025-2026

WINC_AEC_ID_HTTPCL

Description

HTTP close.

AEC Syntax

AEC	Description
<CONF>	HTTP close

AEC Element Syntax

Element Name	Type	Description
<CONF>	INTEGER	HTTP Configuration number Value is 1

WINC_AEC_ID_HTTPERR

Description

HTTP error.

AEC Syntax

AEC	Description
<CONF>,<ERROR_CODE>	HTTP error

AEC Element Syntax

Element Name	Type	Description
<CONF>	INTEGER	HTTP Configuration number Value is 1
<ERROR_CODE>	INTEGER	Error code See Status Response Codes

 ©2025-2026

Examples:

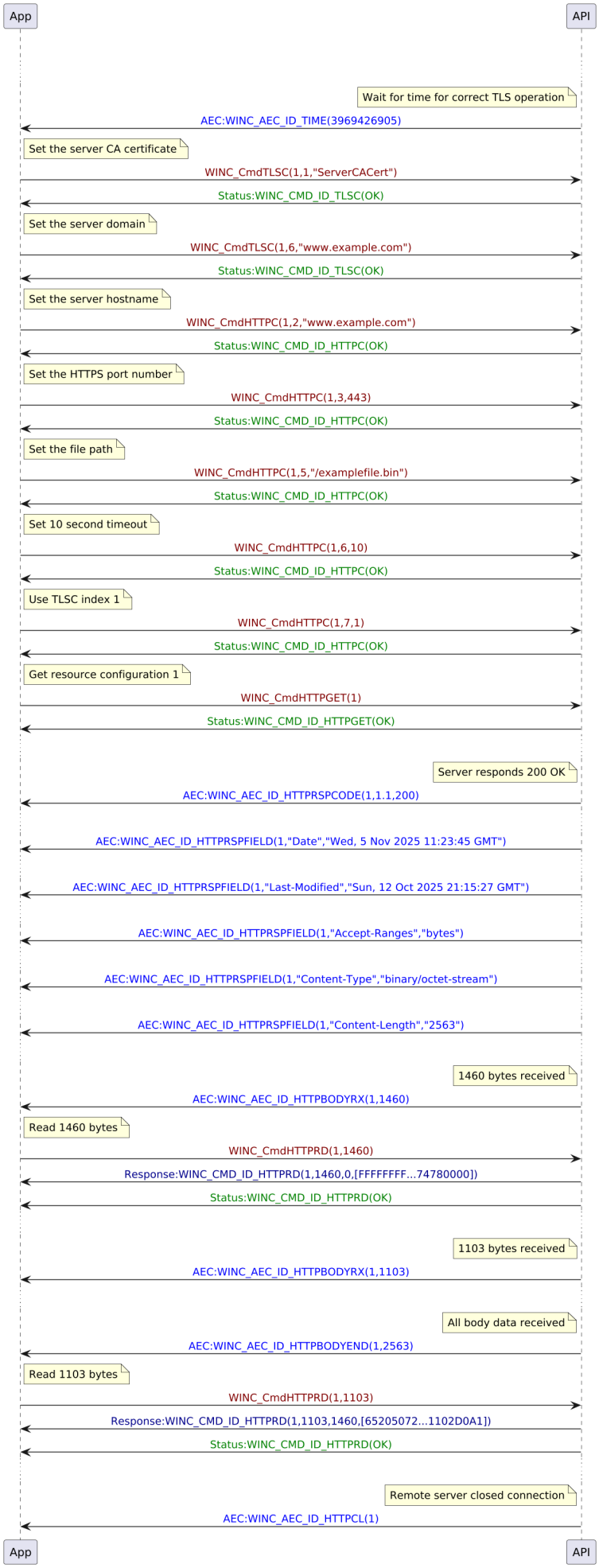


Figure 77. HTTPS download

 ©2025-2026

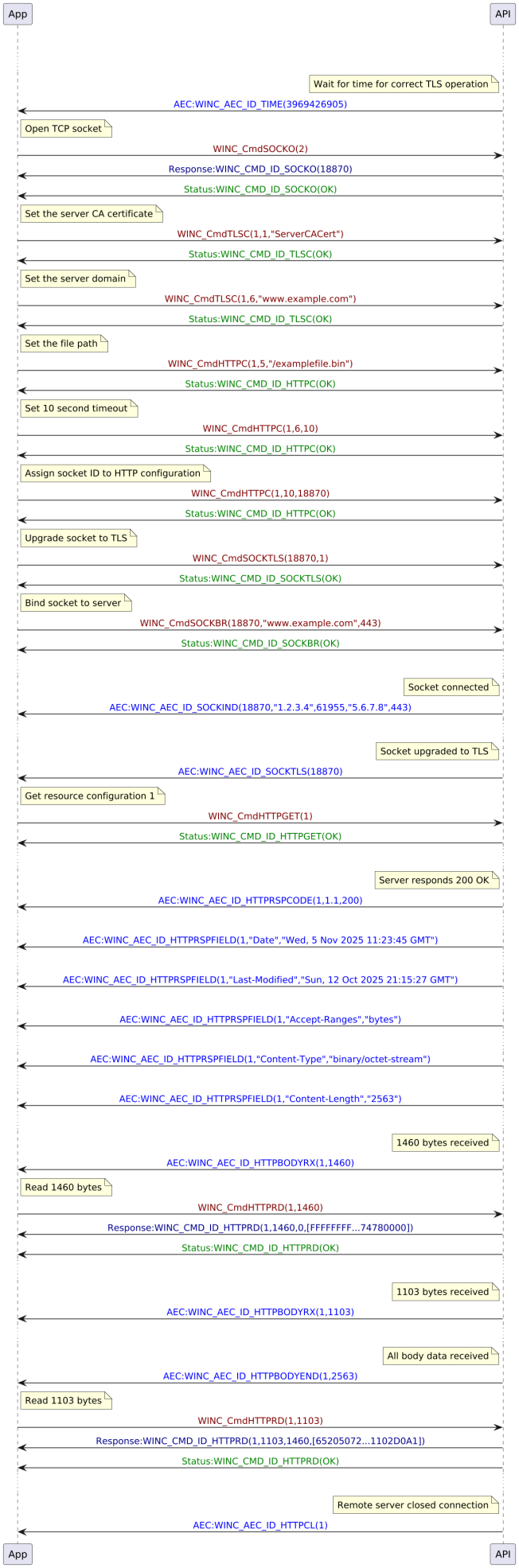


Figure 78. HTTPS download from TCP socket

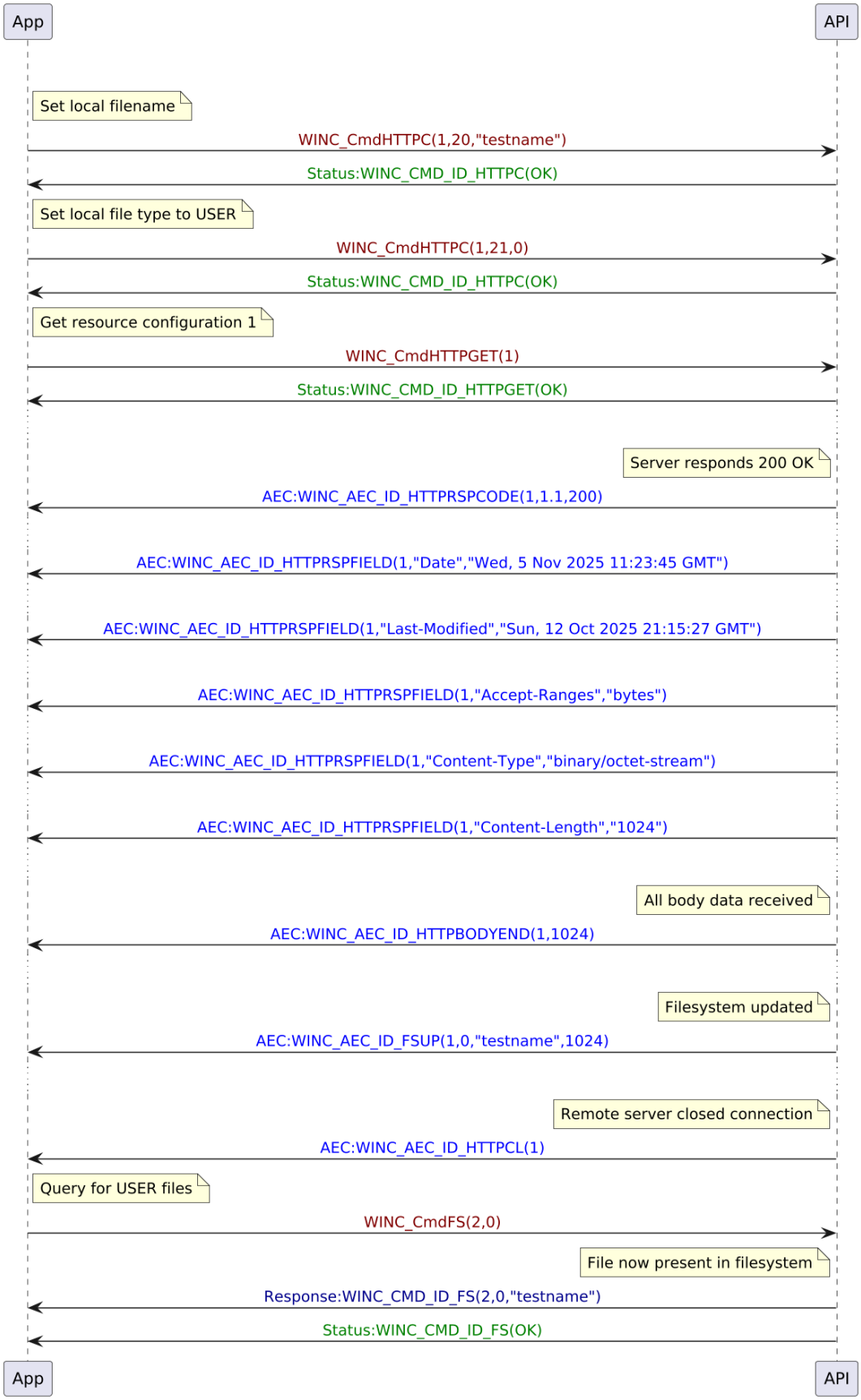


Figure 79. File download to local filesystem

 ©2025-2026

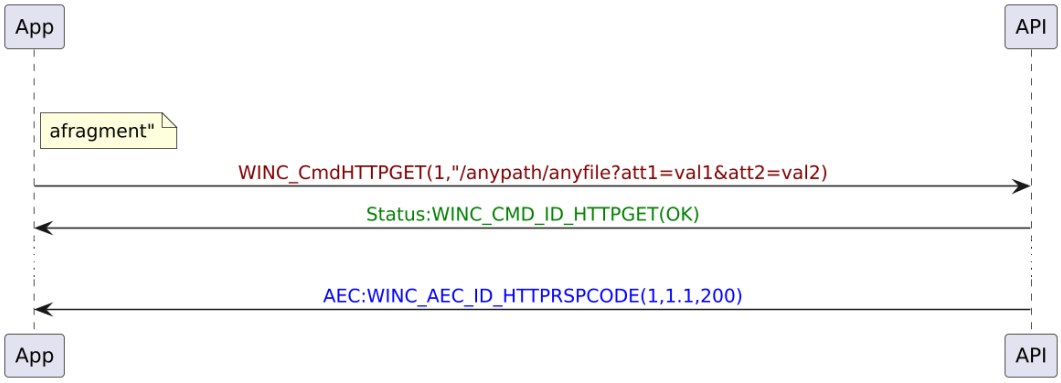


Figure 80. HTTPGET with path and query/fragments

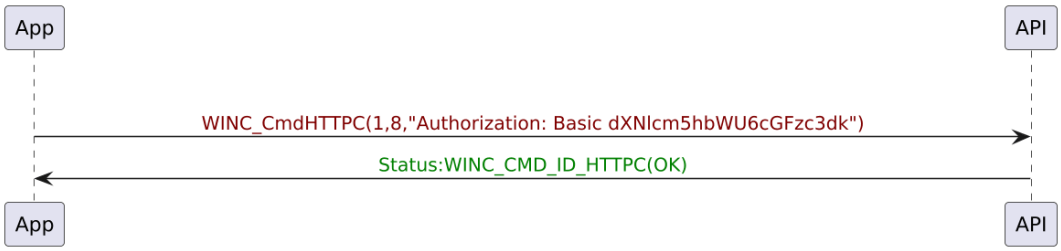


Figure 81. HTTP basic authentication using additional request headers

WBCN (Module ID = 35)

Introduction

The WBCN module transmits Wi-Fi beacon frames. Together with periodic passive scanning, it supports application-defined beacon advertising and discovery.

Information is carried in the **beacon SSID**: the payload is an application-defined string of at most 32 characters. The format of the payload is left to the application. A common leading string allows the SSID prefix filter of the WSCN module to select matching beacons from those of ordinary access points.

Additional application information can be carried alongside the SSID in one or more vendor-specific IEs. Each IE contains up to 252 bytes of vendor-specific data.

Configure vendor-specific IEs with the WIFIC function.

Beacons are transmitted with the WINC_CmdWBCN function described below; beacons from other devices are detected using the WINC_CmdWSCNP function.

Configuring the beacon

Set the beacon parameters using the WINC_CmdWBCNC function.

Five parameters control the beacon. Three of them decide what is transmitted:

- The channel to transmit the beacon on: CHANNEL. A non-zero channel selects that one channel; zero takes the channel set from CHANMASK24.
- The 2.4 GHz channel bit mask used when CHANNEL is zero: CHANMASK24
- The SSID carried in the beacon, which holds the payload: SSID

The other two decide how often the beacon is repeated, and how many times:

- The time between beacons, in milliseconds: INTERVAL
- The number of beacons to transmit, or zero to beacon until stopped: NUM_BEACONS

NOTE

The selected channel or channel mask must lie within the active regulatory domain's channel mask, otherwise the value is rejected. CHANNEL zero with an empty CHANMASK24 is refused when beaconing starts with WINC_STATUS_WBCN_NO_CHANNELS. INTERVAL can be configured from 100 ms to 600000 ms. When beaconing starts, INTERVAL must be strictly greater than 20 ms multiplied by the number of selected channels; otherwise the start is refused with WINC_STATUS_WBCN_INTERVAL_TOO_SHORT.

The SSID may be at most 32 characters. A payload longer than that is rejected.

 ©2025-2026

Transmitting beacons

Once the beacon has been configured, start beaconing using the WINC_CmdWBCN function, and stop it with the same function.

Beaconing takes the form of a run rather than a single transmission: the configured beacon is transmitted on the configured channel set once every INTERVAL milliseconds, until NUM_BEACONS transmissions have completed, or indefinitely if NUM_BEACONS is zero. One transmission covers every selected channel, so NUM_SENT counts channel sets, not individual beacon frames.

A run which cannot continue ends with WINC_AEC_ID_WBCNERR.

A run always ends with one of two AECs:

- WINC_AEC_ID_WBCNDONE - the run ended as asked, either because NUM_BEACONS channel-set transmissions completed, or because it was stopped.
- WINC_AEC_ID_WBCNERR - the run was abandoned before completion.

Stopping a run is a request rather than an immediate halt: success is returned straight away, the run ends at the next point at which nothing is outstanding, and WINC_AEC_ID_WBCNDONE follows. Stopping when no run is in progress succeeds and does nothing.

The beacon configuration is locked while a run is in progress: a write with WINC_CmdWBCNC is rejected with WINC_STATUS_STORE_UPDATE_BLOCKED, and a second attempt to start a run with WINC_STATUS_WBCN_IN_PROGRESS. Reads are permitted throughout. Changing the payload therefore means ending the current run first: stop it, wait for WINC_AEC_ID_WBCNDONE, write the new SSID, then start a new run. The module goes idle before that AEC is sent, so the configuration is always writable by the time the host sees it.

Beaconing can overlap periodic passive scanning with WINC_CmdWSCNP, which allows a device to transmit and scan at the same time.

NOTE	When beaconing and periodic passive scanning overlap, scanning takes precedence. Beaconing resumes after the scan sweep completes.

NOTE	Beaconing is unavailable while soft-AP mode is running; a start is refused with WINC_STATUS_WBCN_AP_MODE_ACTIVE.

Detecting beacons

Beacons transmitted by other devices are detected using periodic passive scanning.

Set the periodic scan channel (CHANNEL) to match the beacon channel, optionally adjust the passive scan slot time (PASV_SLOT_TIME) and interval (INTERVAL), then start periodic passive scanning using the WINC_CmdWSCNP function.

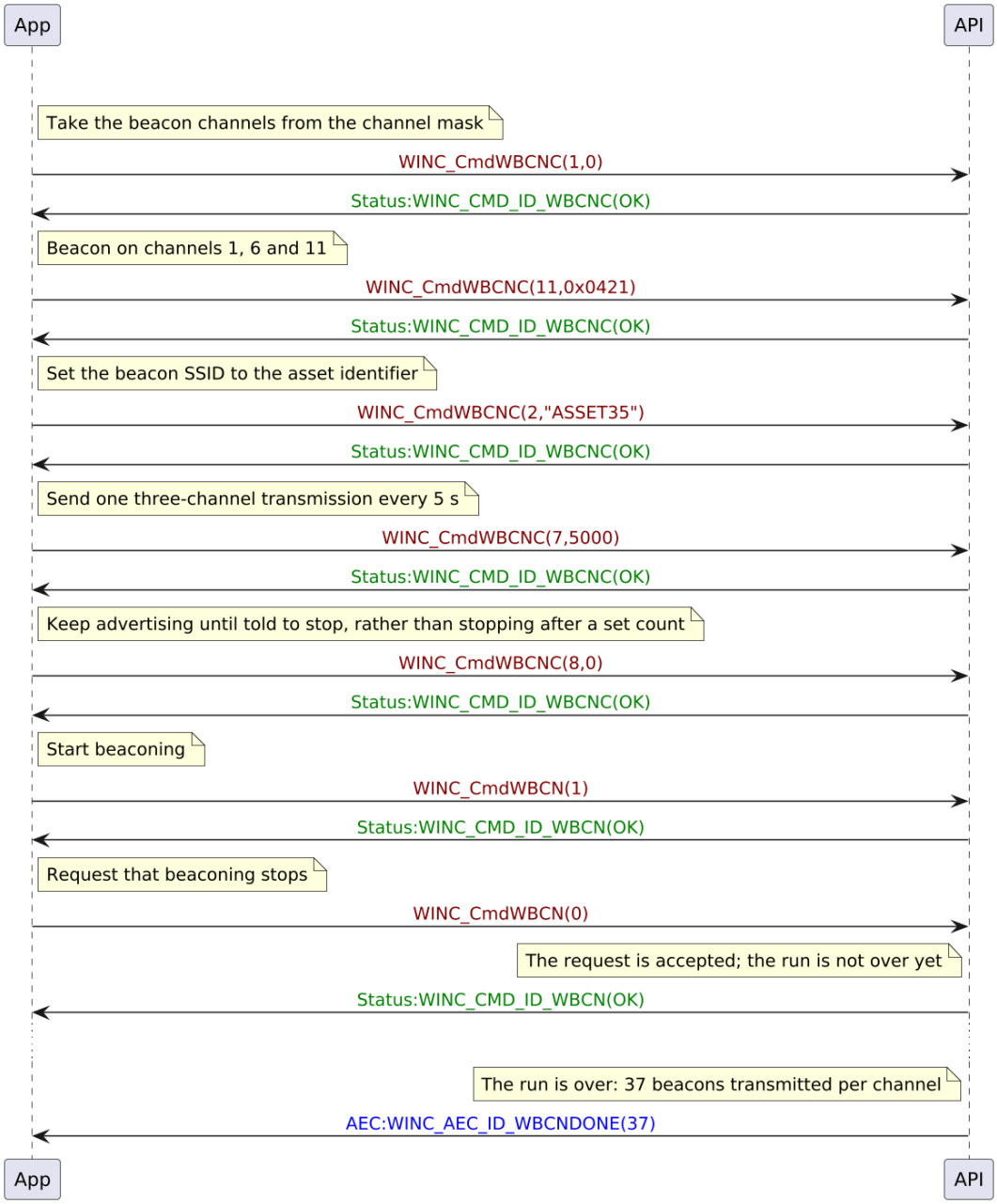
Examples

Asset Tracking

Asset tracking is one application of beacon advertising and periodic passive scanning. A target asset advertises its identifier and listens for commands; a tracker asset detects target assets and delivers commands. The examples below show the exchange one stage at a time.

 ©2025-2026

A target asset advertises its location by beaconing its asset identifier:

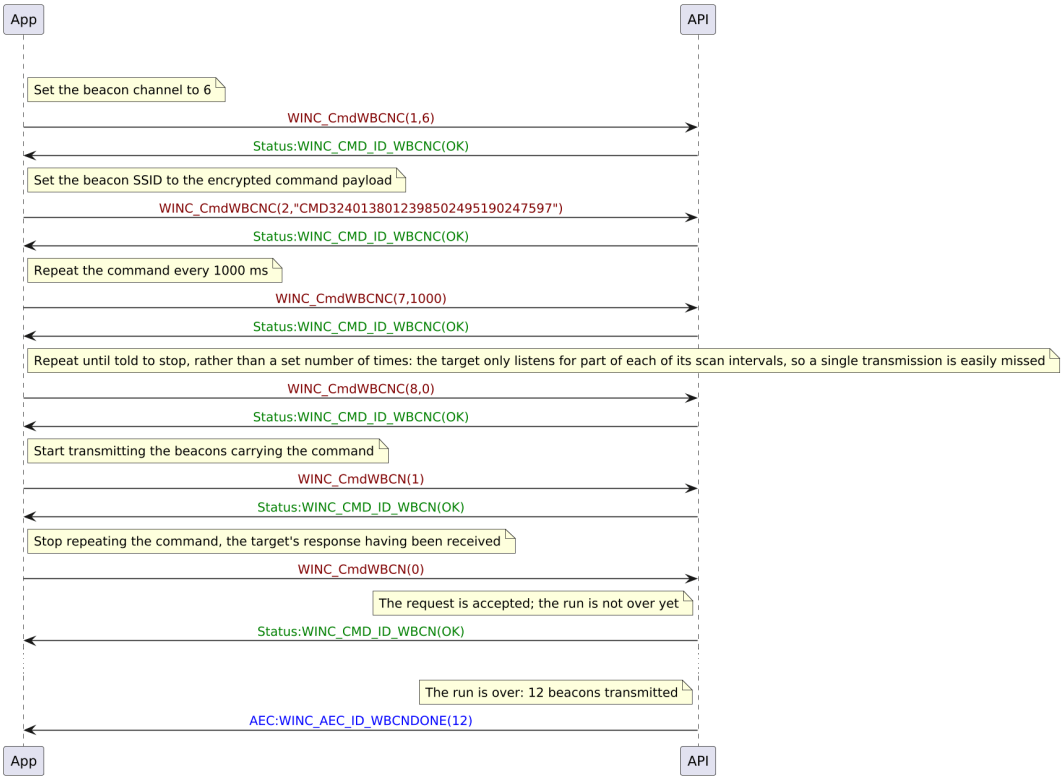


A tracker asset runs a periodic passive scan and detects the target asset:



 ©2025-2026

A tracker asset delivers an encoded command to a target asset by beaconing the command payload:



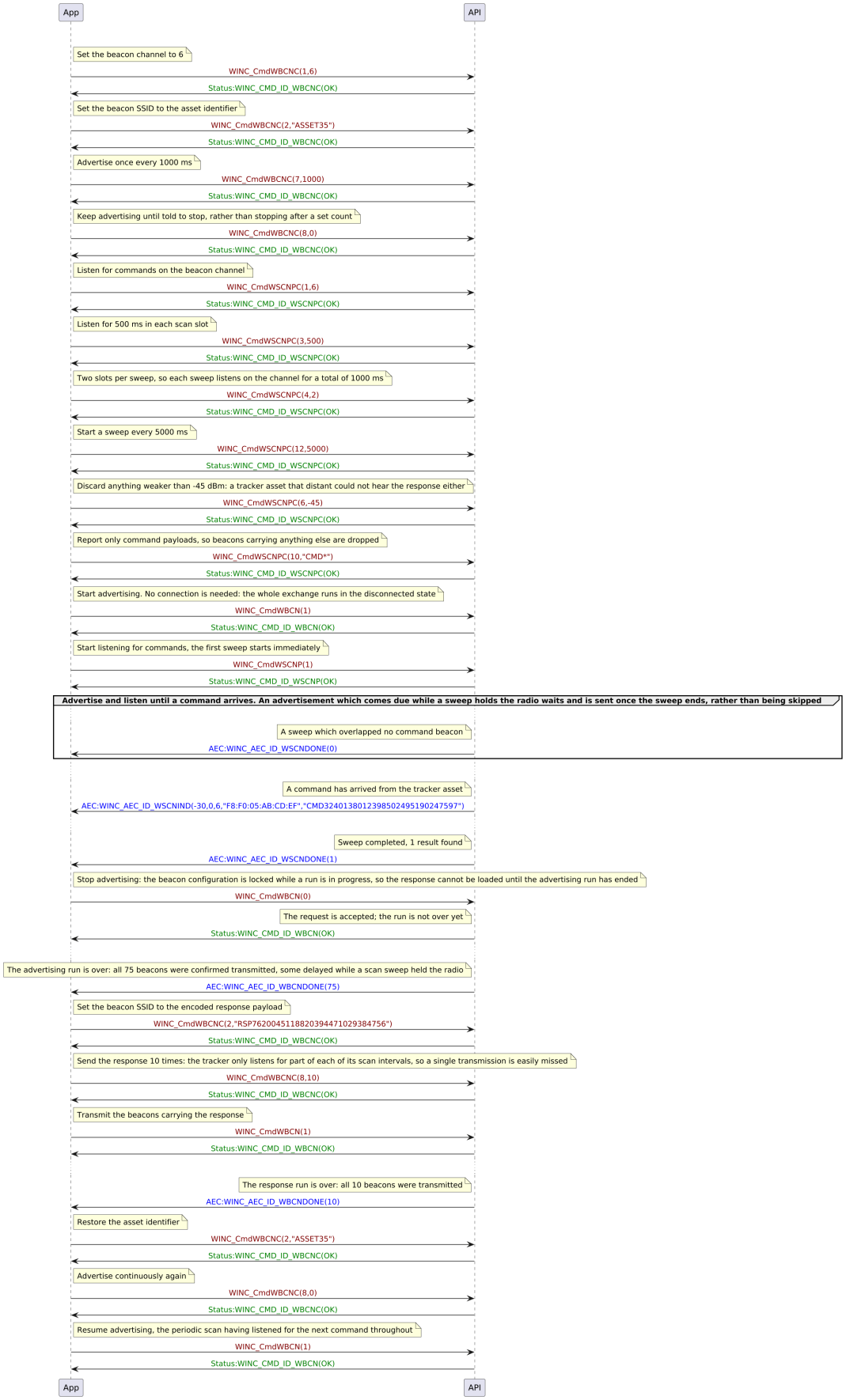
The target asset runs a periodic passive scan and receives the command:



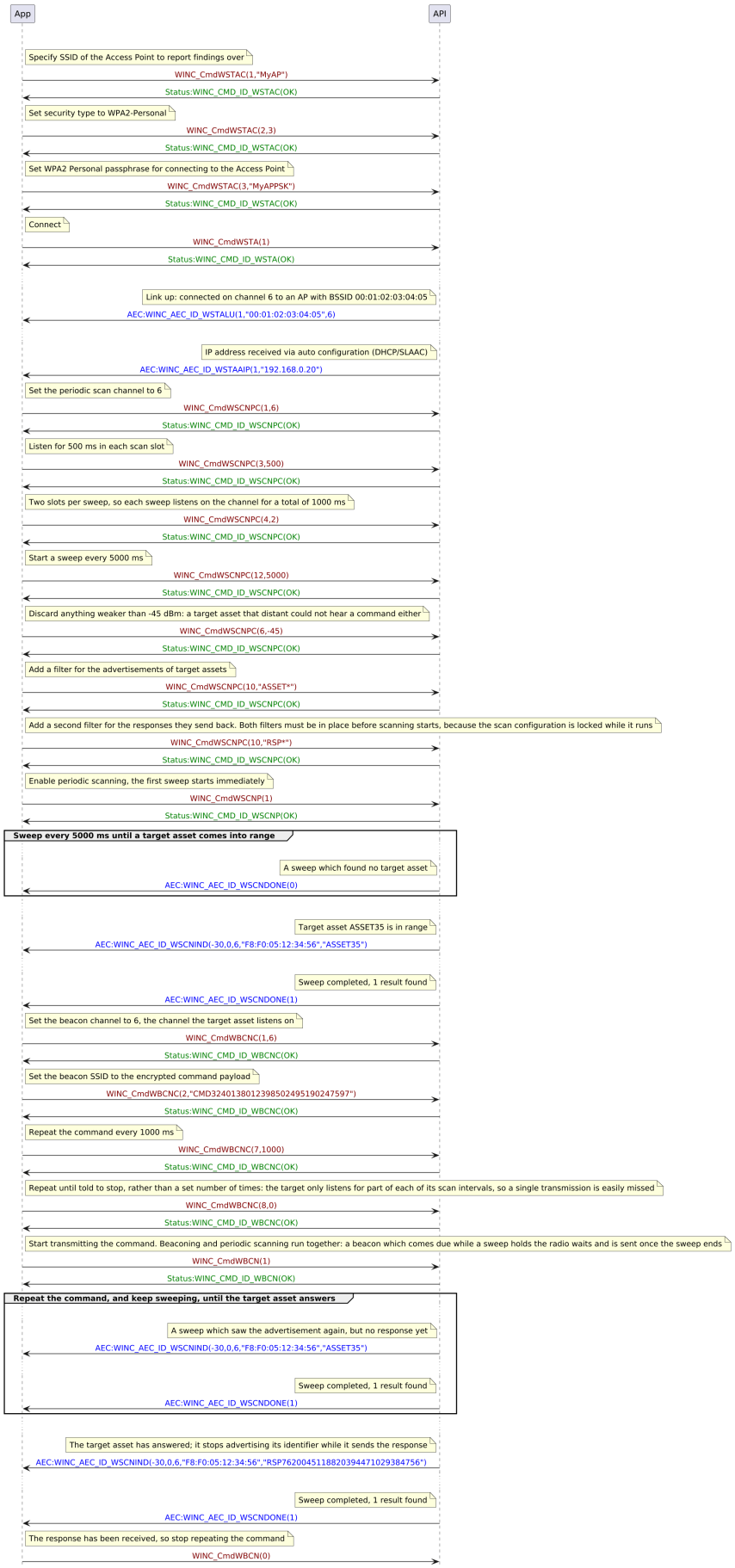
The two examples below then show each role's complete exchange, from the two sides of the same conversation.

 ©2025-2026

The target asset advertises and listens at the same time, answers a command, and resumes advertising:



The tracker asset connects, discovers the target asset, commands it and collects the response:



 ©2025-2026

Function Reference:

WINC_CmdWBCNC

Description

This function is used to modify or query the behavior of the beacon function.

This function is a configuration function which supports setting and getting parameter values. The behaviour of configuration functions is described in general in the Configuration Functions section.

```
bool WINC_CmdWBCNC
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optId
    WINC_TYPE typeOptVal
    uintptr_t optVal
    size_t lenOptVal
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optId	int32_t	Parameter ID number. A fractional ID formed with WINC_CmdFormFractInt selects a single value of a multiple value parameter. optId will be ignored if its value is WINC_CMDWBCNC_ID_IGNORE_VAL.
typeOptVal	WINC_TYPE	Type of optVal, optVal is not sent if the type is WINC_TYPE_INVALID.
optVal	uintptr_t	Parameter value. For integer types optVal holds the value, for other types optVal is a pointer to the data.
lenOptVal	size_t	Length in bytes of the data optVal points to, ignored for integer types.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<ID>,<VAL>	Read response

Response Element Description

Element Name	Type	Description
<ID>	INTEGER	Parameter ID number
<VAL>	Any	Parameter value

 ©2025-2026

Configuration Parameters (WINC_CFG_PARAM_ID_WBCN_*)

<table>
<tr><th>ID</th><th>Name</th><th>Type</th><th>Description</th></tr>
<tr><td>0</td><td>PRESET</td><td>INTEGER</td><td>Configuration preset

<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0</td><td>DEFAULT</td><td>Default.</td></tr></table></td></tr>
<tr><td>1</td><td>CHANNEL</td><td>INTEGER_UNSIGNED</td><td>The channel to send the beacon on (or 0 to use CHANMASK24)

Valid range is 0 to 13</td></tr>
<tr><td>2</td><td>SSID</td><td>STRING</td><td>The SSID advertised in the beacon
Maximum length of string is 32</td></tr>
<tr><td>7</td><td>INTERVAL</td><td>INTEGER_UNSIGNED</td><td>The interval between beacon transmissions, in milliseconds

Valid range is 100 to 600000</td></tr>
<tr><td>8</td><td>NUM_BEACONS</td><td>INTEGER_UNSIGNED</td><td>The number of beacons to send per channel; 0 to send continuously until a stop request

Unsigned 16-bit value</td></tr>
<tr><td>11</td><td>CHANMASK24</td><td>INTEGER_UNSIGNED</td><td>Beacon channel mask (2.4 GHz), used when CHANNEL is 0

<table><tr><td>15
-</td><td>14
-</td><td>13
CH14</td><td>12
CH13</td><td>11
CH12</td><td>10
CH11</td><td>9
CH10</td><td>8
CH9</td></tr><tr><td>7
CH8</td><td>6
CH7</td><td>5
CH6</td><td>4
CH5</td><td>3
CH4</td><td>2
CH3</td><td>1
CH2</td><td>0
CH1</td></tr></table>
<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0x0001</td><td>CH1</td><td>Channel 1 - 2412 MHz.</td></tr><tr><td>0x0002</td><td>CH2</td><td>Channel 2 - 2417 MHz.</td></tr><tr><td>0x0004</td><td>CH3</td><td>Channel 3 - 2422 MHz.</td></tr><tr><td>0x0008</td><td>CH4</td><td>Channel 4 - 2427 MHz.</td></tr><tr><td>0x0010</td><td>CH5</td><td>Channel 5 - 2432 MHz.</td></tr><tr><td>0x0020</td><td>CH6</td><td>Channel 6 - 2437 MHz.</td></tr><tr><td>0x0040</td><td>CH7</td><td>Channel 7 - 2442 MHz.</td></tr><tr><td>0x0080</td><td>CH8</td><td>Channel 8 - 2447 MHz.</td></tr><tr><td>0x0100</td><td>CH9</td><td>Channel 9 - 2452 MHz.</td></tr><tr><td>0x0200</td><td>CH10</td><td>Channel 10 - 2457 MHz.</td></tr><tr><td>0x0400</td><td>CH11</td><td>Channel 11 - 2462 MHz.</td></tr><tr><td>0x0800</td><td>CH12</td><td>Channel 12 - 2467 MHz.</td></tr><tr><td>0x1000</td><td>CH13</td><td>Channel 13 - 2472 MHz.</td></tr><tr><td>0x2000</td><td>CH14</td><td>Channel 14 - 2485 MHz.</td></tr></table></td></tr>
</table>

WINC_CmdWBCN

Description

This function is used to start or stop periodic beacon transmission. Exactly one of WINC_AEC_ID_WBCNDONE or WINC_AEC_ID_WBCNERR closes out every accepted start request. A stop request only requests the stop: its OK does not mark the end of the run, the WINC_AEC_ID_WBCNDONE which follows it does.

```
bool WINC_CmdWBCN
(
    WINC_CMD_REQ_HANDLE handle
    int32_t optState
)
```

Function Parameter Syntax

Name	Type	Description
handle	WINC_CMD_REQ_HANDLE	Command request session handle.
optState	int32_t	State of the beacon function. **Value** / **Label** / **Description** 0 / DISABLE / Disable. 1 / ENABLE / Enable. optState will be ignored if its value is WINC_CMDWBCN_STATE_IGNORE_VAL.

Return Values

Value	Description
true	Success.
false	Failure.

Response Format

Response	Description
<STATE>	Query state

 ©2025-2026

Response Element Description

Element Name	Type	Description
<STATE>	INTEGER	State of the beacon function **Value** / **Label** / **Description** 0 / DISABLE / Disabled. 1 / ENABLE / Enabled.

AEC Reference:

WINC_AEC_ID_WBCNDONE

Description

Beacon transmission completed.

AEC Syntax

AEC	Description
<NUM_SENT>	Beaconing complete

AEC Element Syntax

Element Name	Type	Description
<NUM_SENT>	INTEGER	Number of beacon transmitted. Each transmission covers every channel selected by CHANNEL/CHANMASK24, so this counts channel sets aired, not individual frames. A value below the configured NUM_BEACONS means the run was cut short by a stop request, or that a transmission could not be made

 ©2025-2026

WINC_AEC_ID_WBCNERR

Description

Beacon run abandoned because the driver rejected a transmit request.

AEC Syntax

AEC	Description
<ERROR_CODE>	Beaconing abandoned

AEC Element Syntax

Element Name	Type	Description
<ERROR_CODE>	INTEGER	Error code See Status Response Codes

Examples:

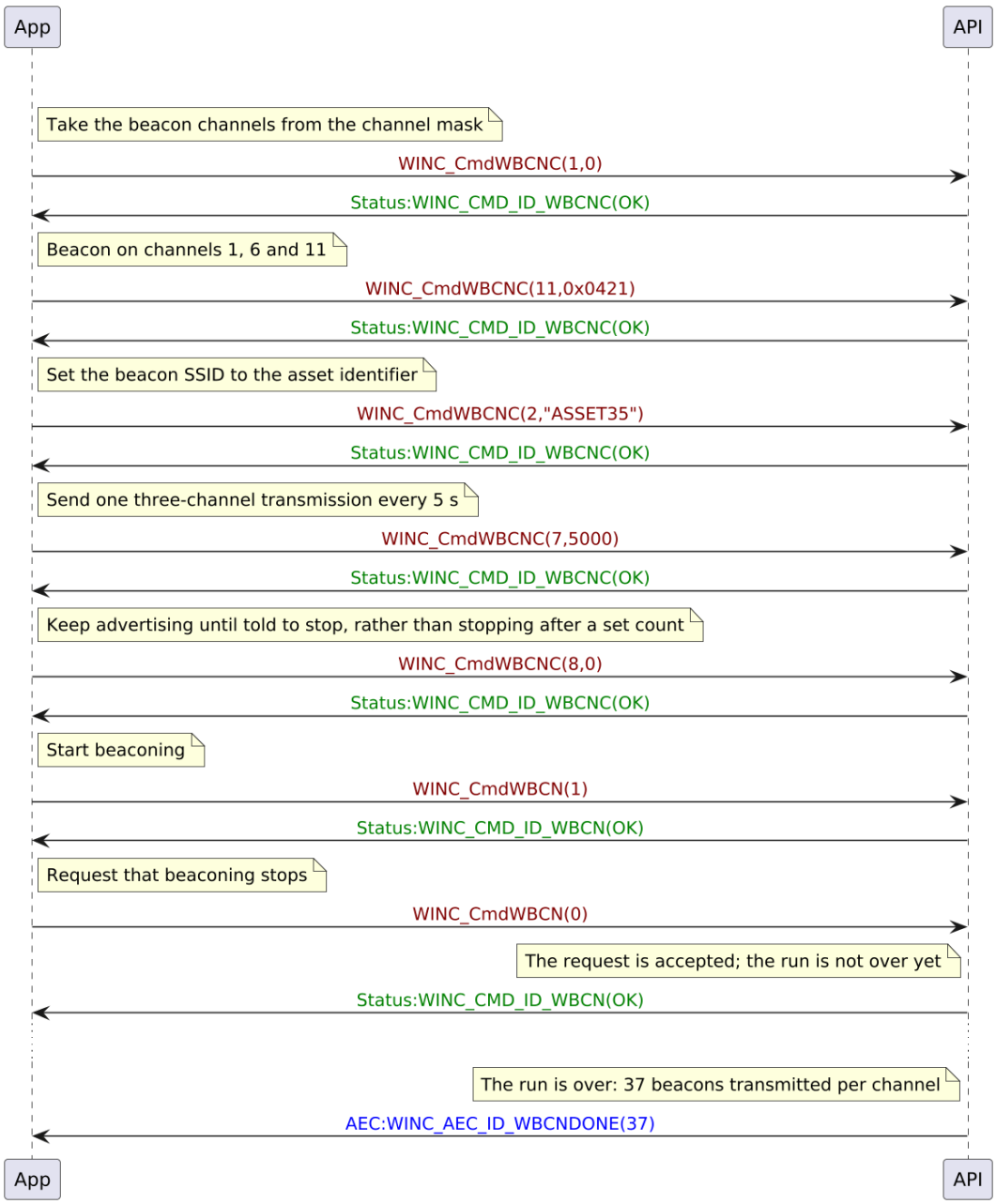


Figure 82. Target asset: advertise its location

 ©2025-2026

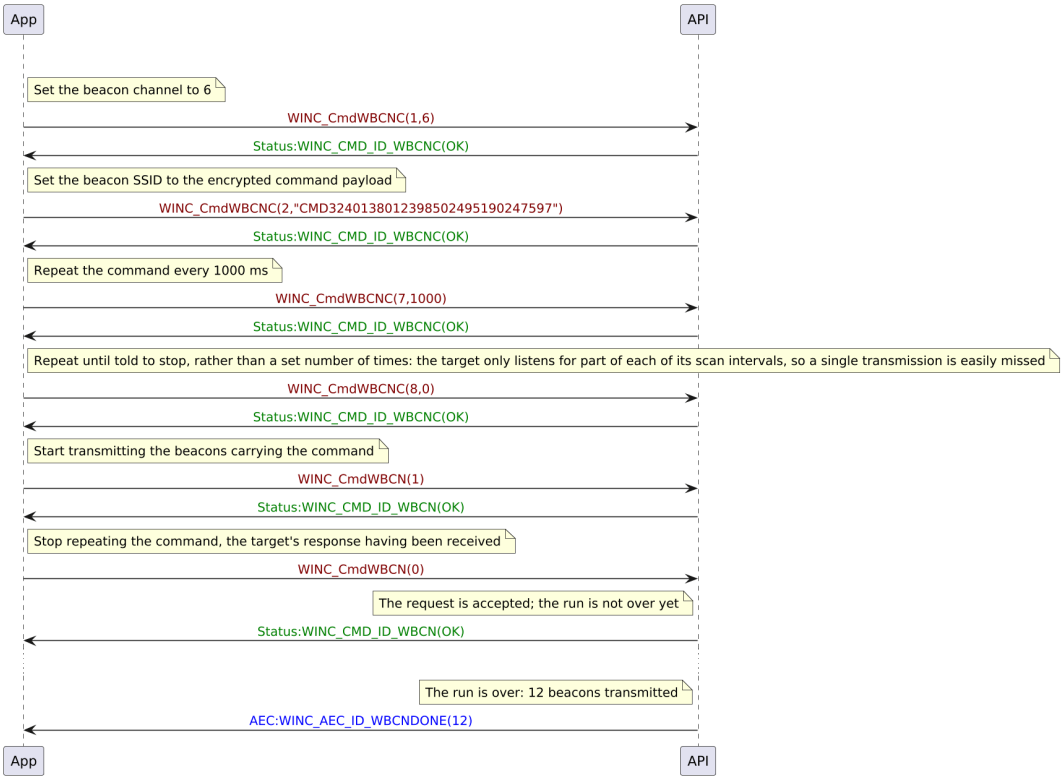


Figure 83. Tracker asset: send a command to a target asset

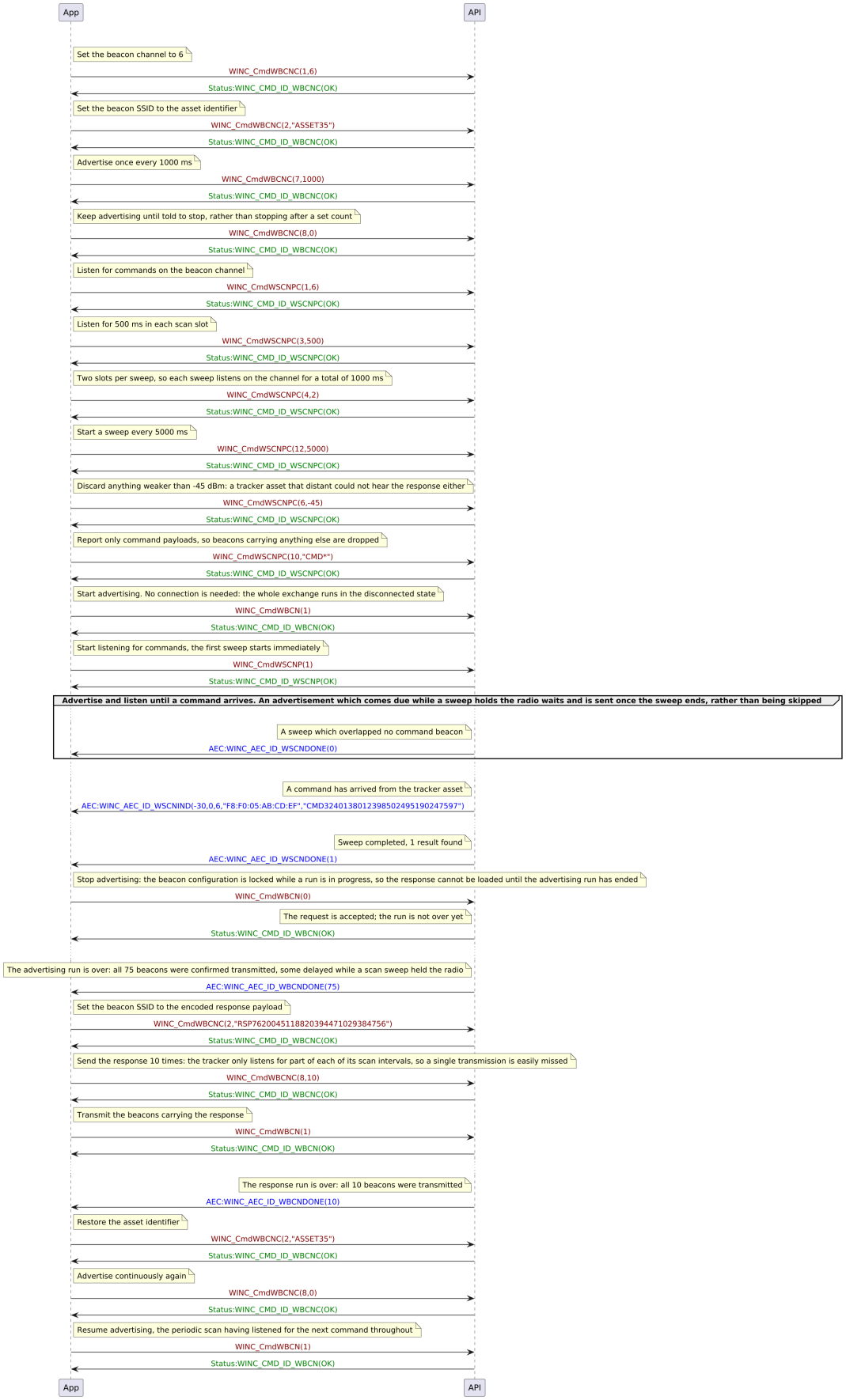


Figure 84. Target asset: a complete exchange with a tracker asset

 ©2025-2026

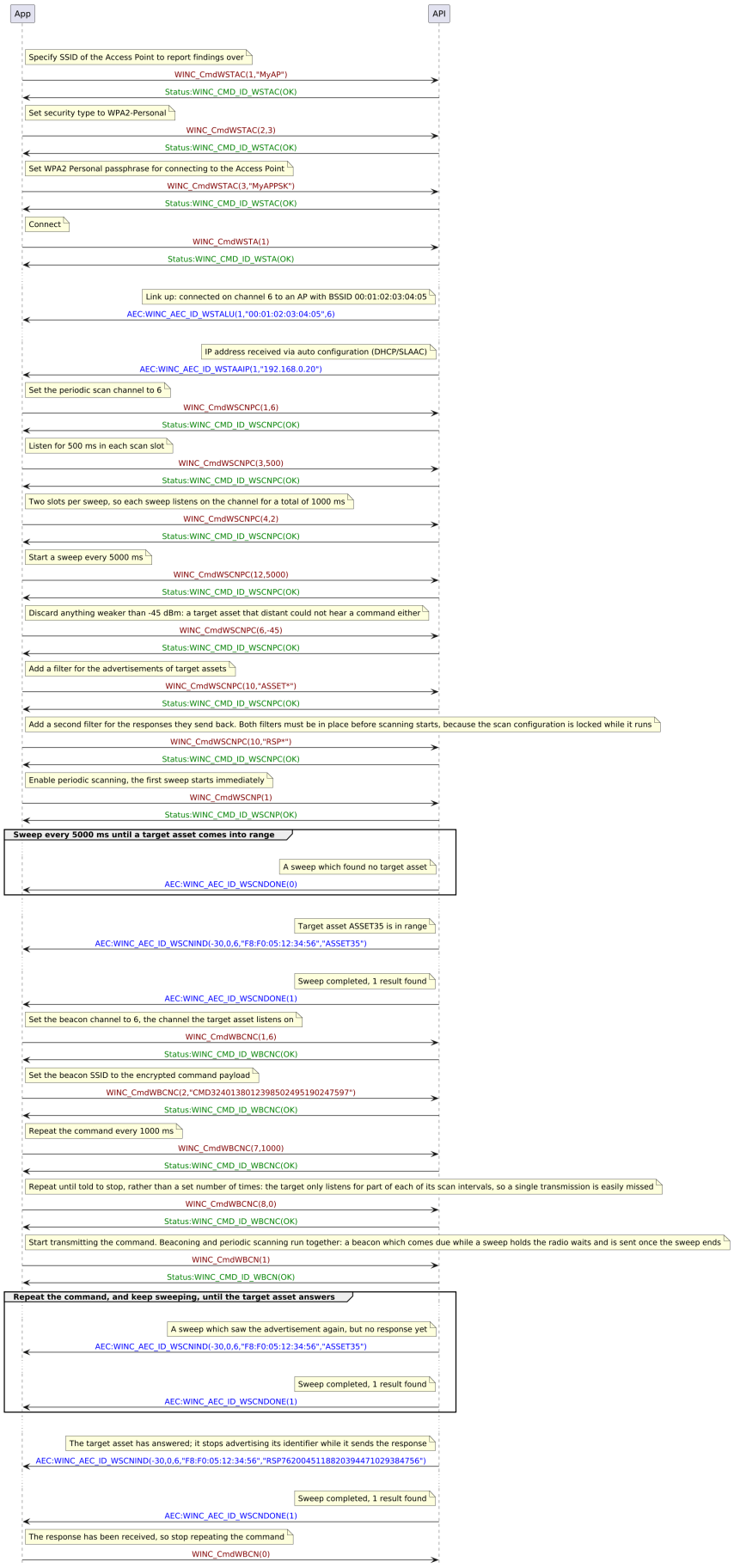


Figure 85. Tracker asset: a complete exchange with a target asset

Berkeley Sockets APIs

Introduction

This interface provides a Berkeley style sockets API. While closely matching the Berkeley standard API this interface does differ due to the architecture of the WINCS02. Only asynchronous sockets are supported; no function will block.

This API supports:

- IPv4 and IPv6 sockets.
- Datagram (UDP) and streaming (TCP) sockets.
- TLS for TCP sockets.
- Server and client operations.

Socket Initialisation

The sockets interface must be initialised before any other socket functions or resources can be used. The function WINC_SockInit is used to initialise the interface. During initialisation the function will allocate a main slab memory block and ensure all state is reset.

Slab Memory Allocation

To avoid causing heap fragmentation the sockets interface uses a preallocated slab architecture to reduce the number of `malloc` and `free` operation. There are two options for how this slab is allocate and managed.

Slab Alignment

To assist with architectures where memory alignment is important, the define `WINC_SOCK_SLAB_MEM_ALIGNMENT` can be set to specify the alignment granularity. The starting address of a slab will be aligned to this define, and the size of allocations will be rounded up to the nearest aligned size.

Slab Allocation Mode 1

When `WINC_SOCK_SLAB_ALLOC_MODE` is defined to be 1 the sockets interface will use a single slab allocation, created when WINC_SockInit is called and destroyed when WINC_SockDeinit is called.

The memory requirement for this slab is defined in the WINC_SOCKET_INIT_TYPE structure passed to WINC_SockInit. This works out to approximately:

```
(WINC_SOCKET_INIT_TYPE.slabSize * WINC_SOCKET_INIT_TYPE.numSlabs) + Overhead
```

All socket allocations will made from this single slab, therefore the size and number of slabs should be chosen to support the expected memory use of the system for all active sockets.

 ©2025-2026

Slab Allocation Mode 2

When `WINC_SOCK_SLAB_ALLOC_MODE` is defined to be 2 the sockets interface will use a main slab allocation for management tasks and then a slab per socket created. The management slab is created the same as the single slab from allocation mode 1, however as this slab does not contain any socket allocations it does not need to be sized to cope with the expect number of active sockets.

In addition to the management slab, when each socket is created it will allocate an additional slab dedicated solely to that socket. The size of this slab will be dictated by the buffer dimensions of the sockets.

Socket Memory Requirements

The memory required for each created socket is determined based on the size of the receive and transmit data buffers as well as the number of receive and transmit datagrams supported, plus some management overhead for the socket. Each type of allocation (receive/transmit data buffer or datagram structure) will be rounded to occupy a whole slab size.

For allocation mode 1, the memory for each type of allocation will be allocated from the single slab as a contiguous allocation, however the four types of allocation will not need to be contiguous, so the transmit buffer can be located separately from the receive buffer allocation.

For allocation mode 2, all four types of allocation are allocated as a single block from the provided heap.

Datagram allocations occupy approximately 8 + (32 * number of datagrams) bytes.

```
If the slab size is 512 bytes:

A socket with a receive buffer of 1600, transmit buffer of 1300 and supporting up to 4 datagrams for both transmit and
receive would require:

2048 (1600 rounded to 512) + 1536 (1300 round to 512) + 512 (4 datagrams) + 512 (4 datagrams) = 4608 bytes
```

Socket Event Callback

A socket event callback of type WINC_SOCKET_EVENT_CALLBACK may be registered with the socket API by calling `WINC_SockRegisterEventCallback`. The purpose of this callback is to provide low level socket events to the application. It is also possible to poll the Berkeley API to ascertain status without using the callback.

Creating Sockets

Sockets are created by calling the function socket.

```
/* Creating IPv4 TCP socket. */
int sockfd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
```

```
/* Creating IPv6 TLS socket. */
int sockfd = socket(AF_INET6, SOCK_STREAM, IPPROTO_TLS);
```

```
/* Creating IPv4 UDP socket. */
int sockfd = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
```

Once created the WINCS02 device will need to allocate resources.

Callback: Completion of the creation process will be signalled via the callback using event `WINC_SOCKET_EVENT_OPEN`.

Polled: Completion of TCP/UDP client sockets will be signalled via connect, this will return -1 for error and the `errno` value `EINPROGRESS`. TCP server listening sockets can successfully bind during completion, then listen will return -1 with `errno` value `EAGAIN`. UDP server listening sockets will return -1 from bind with a value of `EAGAIN` until complete.

Attempting to call another socket function before the socket creation process has completed will normally result in that call failing with `errno` value `ENOTSOCK`.

 ©2025-2026

Server Listening Sockets

Once created a socket can be bound to a local endpoint to create a server listening socket. Multicast sockets are also bound to a local endpoint.

```
struct sockaddr_in addr;

addr.sin_family      = AF_INET;
addr.sin_port        = htons(80);
addr.sin_addr.s_addr = htonl(INADDR_ANY);

/* Bind IPv4 TCP socket to local port 80. */
bind(sockfd, (struct sockaddr*)&addr, sizeof(struct sockaddr_in));
```

```
struct sockaddr_in6 addr;

memset(&addr, 0, sizeof(addr));
addr.sin6_family = AF_INET6;
addr.sin6_port   = htons(80);
memcpy(&addr.sin6_addr, &in6addr_any, sizeof(in6addr_any));

/* Bind IPv6 TCP socket to local port 80. */
bind(sockfd, (struct sockaddr*)&addr, sizeof(struct sockaddr_in6));
```

If the socket supports TLS, the socket must be associated with a TLS configuration. TLS configurations are set up using WINC_CmdTLSC; the configuration number (optConf) is then passed to the socket via the setsockopt function. The WINC_CmdTLSC command request must have completed before setsockopt is called.

```
int tlsIndex = 2;

/* Configure TLS configuration 2 with the server certificate and private key. */
WINC_CmdTLSC(cmdReqHandle, tlsIndex, WINC_CFG_PARAM_ID_TLS_CTX_CERT_NAME, WINC_TYPE_STRING, (uintptr_t
)"MyServerCertChain", strlen("MyServerCertChain"));
WINC_CmdTLSC(cmdReqHandle, tlsIndex, WINC_CFG_PARAM_ID_TLS_CTX_PRI_KEY_NAME, WINC_TYPE_STRING, (uintptr_t
)"MyServerPrivateKey", strlen("MyServerPrivateKey"));
WINC_CmdTLSC(cmdReqHandle, tlsIndex, WINC_CFG_PARAM_ID_TLS_CTX_PEER_AUTH, WINC_TYPE_BOOL, 0, 0);

/* Once the configuration is complete, associate it with the socket. */
setsockopt(sockfd, IPPROTO_TLS, TLS_CONF_IDX, &tlsIndex, sizeof(tlsIndex));
```

TCP server listening sockets will then need to be set to a listening state by calling listen. A backlog can be specified which sets the number of simultaneous sockets supported, the maximum supported is specified by SOMAXCONN.

> ***NOTE:*** The backlog value 0 means default which, to conserve resources, will be a single socket.

```
listen(sockfd, 0);
```

Callback: Once a TCP socket has been configured for listening the event callback will receive the event WINC_SOCKET_EVENT_LISTEN. A UDP socket will generate this event in response to the earlier bind request.

Polled: A TCP socket will return -1 and `errno` value `EAGAIN` until the listen process has completed. A UDP socket is the same in response to the bind call.

 ©2025-2026

Client Sockets

If the socket supports TLS, the socket must be associated with a TLS configuration. TLS configurations are set up using WINC_CmdTLSC; the configuration number (optConf) is then passed to the socket via the setsockopt function. The WINC_CmdTLSC command request must have completed before setsockopt is called.

```
int tlsIndex = 1;

/* Configure TLS configuration 1 to authenticate the server. */
WINC_CmdTLSC(cmdReqHandle, tlsIndex, WINC_CFG_PARAM_ID_TLS_CTX_CA_CERT_NAME, WINC_TYPE_STRING, (uintptr_t
)"ServerRootCACert", strlen("ServerRootCACert"));
WINC_CmdTLSC(cmdReqHandle, tlsIndex, WINC_CFG_PARAM_ID_TLS_CTX_DOMAIN_NAME, WINC_TYPE_STRING, (uintptr_t)
"ServerDomain", strlen("ServerDomain"));

/* Once the configuration is complete, associate it with the socket. */
setsockopt(sockfd, IPPROTO_TLS, TLS_CONF_IDX, &tlsIndex, sizeof(tlsIndex));
```

TCP sockets will then need to be connected to a remote endpoint by calling connect.

```
struct sockaddr_in addr;

addr.sin_family      = AF_INET;
addr.sin_port        = htons(80);
addr.sin_addr.s_addr = htonl(0xC0A80064);

/* Connect an IPv4 socket to 192.168.0.100:80. */
connect(sockfd, (struct sockaddr*)&addr, sizeof(struct sockaddr_in));
```

Multicast Sockets

Multicast sockets start like server listening sockets by being bound to a local endpoint. The socket then needs to be joined into the appropriate multicast group.

```
struct ip_mreqn group;

memset(&group, 0, sizeof(group));
group.imr_multiaddr.s_addr = htonl(0xE0000001U);
group.imr_address.s_addr   = htonl(INADDR_ANY);
group.imr_ifindex          = 0;

/* Join the IPv4 multicast group 224.0.0.1. */
setsockopt(sockfd, IPPROTO_IP, IP_ADD_MEMBERSHIP, &group, sizeof(group));
```

> ***NOTE:*** IPv6 multicast membership uses option level IPPROTO_IP. IPPROTO_IPV6 is not supported.

```
struct ipv6_mreq group;

memset(&group, 0, sizeof(group));
/* IPv6 multicast group ff02::1. */
group.ipv6mr_multiaddr.s6_addr[0]  = 0xff;
group.ipv6mr_multiaddr.s6_addr[1]  = 0x02;
group.ipv6mr_multiaddr.s6_addr[15] = 0x01;
group.ipv6mr_interface             = 0;
```

```
/* Join the IPv6 multicast group, the option level is IPPROTO_IP. */
setsockopt(sockfd, IPPROTO_IP, IPV6_ADD_MEMBERSHIP, &group, sizeof(group));
```

 ©2025-2026

Incoming Server Socket Connections

Callback: For TCP sockets a new incoming connection request is signalled via the callback event `WINC_SOCKET_EVENT_CONNECT_REQ`.

Polled: New connection requests will be signalled through the poll function. Selecting the event `POLLIN` on a server listening socket will result in the `revent` value `POLLIN` being signalled.

```
struct pollfd fd;

fd.fd     = sockh;
fd.events = POLLIN;

if (-1 == poll(&fd, 1, 0))
{
    printf("error\n");
}
else if (0 != (fd.revents & POLLIN))
{
    union sockaddr_union sockAddr;
    socklen_t sockAddrLen = sizeof(union sockaddr_union);
    int newSockh;

    newSockh = accept(sockh, (struct sockaddr*)&sockAddr, &sockAddrLen);
}
```

In response to a connection attempt the application may accept the connection by calling accept.

```
struct sockaddr_in addr;
socklen_t addrLen = sizeof(struct sockaddr_in);

/* Accept an incoming IPv4 connection request. */
new_sockfd = accept(sockfd, (struct sockaddr*)&addr, &addrLen);
```

```
struct sockaddr_in6 addr;
socklen_t addrLen = sizeof(struct sockaddr_in6);

/* Accept an incoming IPv6 connection request. */
new_sockfd = accept(sockfd, (struct sockaddr*)&addr, &addrLen);
```

Client Socket Connections

Callback: Completion of TCP client socket connections are indicated be reception of the `WINC_SOCKET_EVENT_CONNECT` event. If TLS has been configured on the socket a second event, `WINC_SOCKET_EVENT_TLS_CONNECT` will indicate that TLS has also connected and that data is ready to flow.

Polled: While connections are in progress connect will return -1 and `errno` value `EINPROGRESS`. Calling poll with event `POLLOUT` will signal the `revent` for `POLLOUT` when connected.

```
struct pollfd fd;

fd.fd     = sockh;
fd.events = POLLOUT;

if (-1 == poll(&fd, 1, 0))
{
    printf("error\n");
}
else if (0 != (fd.revents & POLLOUT))
{
    printf("socket connected\n");
}
```

 ©2025-2026

Sending Data

To send data the application can call either send for streaming sockets (TCP/TLS) or sendto for datagram sockets (UDP).

```
/* Send data via TCP socket. */
send(sockfd, dataPtr, dataLength, 0);
```

```
struct sockaddr_in addr;

addr.sin_family      = AF_INET;
addr.sin_port        = htons(1000);
addr.sin_addr.s_addr = htonl(0xC0A80064);

/* Send data via UDP socket to 192.168.0.100:1000. */
sendto(sockfd, dataPtr, dataLength, 0, (struct sockaddr*)&addr, sizeof(struct sockaddr_in));
```

If the socket is available for sending the call to send or sendto will return the number of byte written. If the socket is not available the call will return -1 with `errno` value `EWOULDBLOCK` / `EAGAIN`.

Callback: There is an `WINC_SOCKET_EVENT_SEND` event delivered to the event callback, however this does not correspond directly to the calling of send or sendto. The event corresponds to the low level sending of socket data to the WINCS02. Calling send or sendto places the application data into an internal buffer within the WINCS02 driver, the driver then chooses when and how to deliver this data to the WINCS02 device.

Polled: The ability to send data through a socket will be signalled through poll event `POLLOUT`. When the `revent` for the socket signals `POLLOUT` the application may send data.

```
struct pollfd fd;

fd.fd     = sockh;
fd.events = POLLOUT;

if (-1 == poll(&fd, 1, 0))
{
    printf("error\n");
}
else if (0 != (fd.revents & POLLOUT))
{
    send(sockh, buffer, numBytes, 0);
}
```

Receiving Data

To receive data the application can call either recv for streaming sockets (TCP/TLS) or recvfrom for datagram sockets (UDP).

```
uint8_t buffer[128];

/* Receive data from a TCP socket. */
recv(sockfd, buffer, sizeof(buffer), 0);
```

```
uint8_t buffer[128];
struct sockaddr_in addr;
socklen_t fromLen = sizeof(struct sockaddr_in);

/* Receive data from a UDP socket. */
recvfrom(sockfd, buffer, sizeof(buffer), 0, (struct sockaddr*)&addr, &fromLen);
```

Callback: There is an WINC_SOCKET_EVENT_RECV event delivered to the event callback, however this does not correspond directly to data being available through recv or recvfrom. The event corresponds to the low level reception of data from the WINCS02. When data is received from the WINCS02 device it is placed in an internal buffer within the WINCS02 driver, the driver then chooses when and how to deliver this data to the application.

Polled: The ability to receive data from a socket will be signalled through poll event POLLIN. When the revent for the socket signals POLLIN the application may receive data.

```
struct pollfd fd;

fd.fd     = sockh;
fd.events = POLLIN;

if (-1 == poll(&fd, 1, 0))
{
    printf("error\n");
}
else if (0 != (fd.revents & POLLIN))
{
    recv(sockh, buffer, numBytes, 0);
}
```

 ©2025-2026

TCP Socket Termination

Remote socket termination can be detected when attempting to send or receive data, or when polling using poll.

recv: When a stream socket peer has performed an orderly shutdown, the return value of recv will be `0`, the traditional 'end-of-file' value.

send: Attempting to send on a terminated socket will result in a return value of -1 and the `errno` value of `ENOTCONN`.

poll: The value `POLLHUP` will be signalled in `revents` when a connection is terminated. `POLLIN` can be used to determine if the socket has remaining data to receive.

```
struct pollfd fd;

fd.fd     = sockh;
fd.events = POLLIN|POLLOUT;

if (-1 == poll(&fd, 1, 0))
{
    printf("error\n");
}
else if (0 != (fd.revents & POLLHUP))
{
    printf("socket disconnected\n");

    if (0 == (fd.revents & POLLIN))
    {
        /* No pending data to receive, socket can be closed. */
    }
}
```

Closing Sockets

To close a socket the application should use the function shutdown.

DNS Resolution.

This API includes the function getaddrinfo which can be used to resolve hostnames. Resolved names result in a list of `struct addrinfo` structures which are dynamically created, therefore the application must call freeaddrinfo when finished with the results.

For IPv6 AAAA record resolution a hints structure can be included.

```
struct addrinfo *pAddrInfo;
int result;

result = getaddrinfo("www.example.com", NULL, NULL, &pAddrInfo);

if (0 == result)
{
    /* Result is available. */
}
else if (EAI_AGAIN == result)
{
    /* Result is not available yet, try again. */
}
else
{
    /* An error occurred. */
}

if (NULL != pAddrInfo)
{
    /* Clean up allocation results structures. */
    freeaddrinfo(pAddrInfo);
}
```

```
struct addrinfo hints;
struct addrinfo *pAddrInfo;
int result;

/* Indicate IPv6 AAAA records are required. */
memset(&hints, 0, sizeof(hints));
hints.ai_family = AF_INET6;

result = getaddrinfo("www.example.com", NULL, &hints, &pAddrInfo);
```

 ©2025-2026

API Reference

WINC_SOCKET_EVENT

Description

Socket event callback types.

```
typedef enum
{
    WINC_SOCKET_EVENT_OPEN,
    WINC_SOCKET_EVENT_LISTEN,
    WINC_SOCKET_EVENT_CONNECT_REQ,
    WINC_SOCKET_EVENT_CONNECT,
    WINC_SOCKET_EVENT_SEND,
    WINC_SOCKET_EVENT_RECV,
    WINC_SOCKET_EVENT_CLOSE,
    WINC_SOCKET_EVENT_TLS_CONNECT,
    WINC_SOCKET_EVENT_DISCONNECT,
    WINC_SOCKET_EVENT_DNS_RESOLV,
    WINC_SOCKET_EVENT_ERROR
} WINC_SOCKET_EVENT;
```

Members

Member	Description
WINC_SOCKET_EVENT_OPEN	Socket create/open has completed.
WINC_SOCKET_EVENT_LISTEN	Socket listen on local endpoint completed.
WINC_SOCKET_EVENT_CONNECT_REQ	Connection request received.
WINC_SOCKET_EVENT_CONNECT	Connect to remote endpoint completed.
WINC_SOCKET_EVENT_SEND	Data send operation completed.
WINC_SOCKET_EVENT_RECV	Data receive operation completed.
WINC_SOCKET_EVENT_CLOSE	Socket close completed.
WINC_SOCKET_EVENT_TLS_CONNECT	TLS connection to remote endpoint completed.
WINC_SOCKET_EVENT_DISCONNECT	A remote disconnection has occurred.
WINC_SOCKET_EVENT_DNS_RESOLV	A DNS resolve event has completed.
WINC_SOCKET_EVENT_ERROR	An error has occurred.

Remarks

None.

WINC_SOCKET_STATUS

Description

Socket status types.

```
typedef enum
{
    WINC_SOCKET_STATUS_OK,
    WINC_SOCKET_STATUS_ERROR
} WINC_SOCKET_STATUS;
```

Members

Member	Description
WINC_SOCKET_STATUS_OK	The status is OK.
WINC_SOCKET_STATUS_ERROR	The status is an error.

Remarks

None.

 ©2025-2026

WINC_SOCKET_INIT_TYPE

Description

Socket module initialisation structure.

```
typedef struct
{
    void*           (*pfMemAlloc)(size_t size);
    void            (*pfMemFree)(void *pData);
    size_t          slabSize;
    int8_t          numSlabs;
} WINC_SOCKET_INIT_TYPE;
```

Members

Member	Description
pfMemAlloc	Function pointer for allocating memory.
pfMemFree	Function pointer for freeing memory.
slabSize	Size, in bytes, of a single slab allocations.
numSlabs	Number of slabs in main allocation.

Remarks

None.

WINC_SOCKET_EVENT_CALLBACK

Description

Socket event callback function type.

```
typedef void (*WINC_SOCKET_EVENT_CALLBACK)
(
    uintptr_t context,
    int socket,
    WINC_SOCKET_EVENT event,
    WINC_SOCKET_STATUS status
);
```

Parameters

Parameter	Type	Description
context	uintptr_t	User supplied callback context.
socket	int	Socket handle.
event	WINC_SOCKET_EVENT	Socket event, see WINC_SOCKET_EVENT.
status	WINC_SOCKET_STATUS	Event status, see WINC_SOCKET_STATUS.

Returns

None.

Remarks

None.

 ©2025-2026

WINC_SockDeinit

Description

De-initialise the socket module.

```
bool WINC_SockDeinit(WINC_DEVICE_HANDLE devHandle)
```

Parameters

Parameter	Type	Description
devHandle	WINC_DEVICE_HANDLE	WINC device handle

Returns

true or false indicating success or failure

Remarks

None.

WINC_SockInit

Description

Initialise the socket module.

```
bool WINC_SockInit(WINC_DEVICE_HANDLE devHandle, WINC_SOCKET_INIT_TYPE *pInitData)
```

Parameters

Parameter	Type	Description
devHandle	WINC_DEVICE_HANDLE	WINC device handle
pInitData	WINC_SOCKET_INIT_TYPE*	Pointer to initialisation data

Returns

true or false indicating success or failure

Remarks

If `pInitData` is not `NULL` this function initialises the socket interface, if the pointer is `NULL` this function will reset an already initialised socket interface back to it's initial state.

 ©2025-2026

accept

Description

Accept a connection on a socket.

```
int accept(int fd, struct sockaddr *addr, socklen_t *len)
```

Parameters

Parameter	Type	Description
fd	int	Socket file descriptor.
addr	struct sockaddr*	Pointer to a structure to filled in with the address of the peer socket.
len	socklen_t*	Length of address structure.

Returns

Socket file descriptor or -1 for error (errno set).

Remarks

errno	Description
EBADF	The argument `fd` is not a valid file descriptor.
ENOMEM	Insufficient memory is available.
EWOULDBLOCK	The socket is marked nonblocking and no connections are present to be accepted.

bind

Description

Bind a name to a socket.

```
int bind(int fd, const struct sockaddr *addr, socklen_t len)
```

Parameters

Parameter	Type	Description
fd	int	Socket file descriptor.
addr	struct sockaddr*	Pointer to socket address.
len	socklen_t	Length of socket address.

Returns

- 0 - Success.
- -1 - Error, `errno` set.

Remarks

errno	Description
EFAULT	System fault.
EAFNOSUPPORT	The implementation does not support the specified address family.
ENOMEM	Insufficient memory is available.
EBADMSG	Bad message.
EAGAIN	The socket is nonblocking, and the binding cannot be completed immediately.

 ©2025-2026

connect

Description

Initiate a connection on a socket.

```
int connect(int fd, const struct sockaddr *addr, socklen_t len)
```

Parameters

Parameter	Type	Description
fd	int	Socket file descriptor.
addr	struct sockaddr*	Pointer to socket address.
len	socklen_t	Length of socket address.

Returns

- 0 - Success.
- -1 - Error, `errno` set.

Remarks

errno	Description
EFAULT	System fault.
ENOTSOCK	The file descriptor `fd` does not refer to a socket.
EAFNOSUPPORT	The implementation does not support the specified address family.
EAGAIN	The socket is nonblocking, and the connection cannot be completed immediately.
ENOMEM	Insufficient memory is available.
EBADMSG	Bad message.
EINPROGRESS	The socket is nonblocking and the connection cannot be completed immediately. It is possible to poll for completion by selecting the socket for writing. After poll indicates writability.
EISCONN	The socket is already connected.

inet_ntop

Description

Convert IPv4 and IPv6 addresses from binary to text form.

```
const char *inet_ntop(int af, const void *a0, char *s, socklen_t l)
```

Parameters

Parameter	Type	Description
af	int	Address family, either `AF_INET` or `AF_INET6`.
a0	void*	Pointer to IP address in network byte order.
s	char*	Pointer to string buffer to output to.
l	socklen_t	Size of buffer supplied.

Returns

Pointer to converted string within supplied buffer or 0 for error (errno set).

Remarks

errno	Description
EAFNOSUPPORT	`af` was not a valid address family.
ENOSPC	The converted address string would exceed the size given by size.

 ©2025-2026

inet_pton

Description

Convert IPv4 and IPv6 addresses from text to binary form.

```
int inet_pton(int af, const char *s, void *a0)
```

Parameters

Parameter	Type	Description
af	int	Address family, either `AF_INET` or `AF_INET6`.
s	char*	Pointer to input IP address string.
a0	void*	Pointer to output binary form IP address.

Returns

- -1 - Error, `errno` is set.
- 0 - No valid IP address string found.
- 1 - Success.

Remarks

errno	Description
EAFNOSUPPORT	`af` was not a valid address family.

listen

Description

Listen for connections on a socket.

```
int listen(int fd, int backlog)
```

Parameters

Parameter	Type	Description
fd	int	Socket file descriptor.
backlog	int	Maximum number of pending connections.

Returns

- 0 - Success.
- -1 - Error, `errno` set.

Remarks

errno	Description
EADDRINUSE	The referred to by `fd` is not bound to a local port.
EBADF	The argument `fd` is not a valid file descriptor.
ENOTSOCK	The file descriptor `fd` does not refer to a socket.
EOPNOTSUPP	The socket is not of a type that supports the listen() operation.
EAGAIN	The socket is nonblocking, and the listening cannot be completed immediately.

 ©2025-2026

poll

Description

Wait for some event on a file descriptor.

```
int poll(struct pollfd *fds, nfds_t nfds, int timeout)
```

Parameters

Parameter	Type	Description
fds	struct pollfd*	Set of file descriptors to be monitored. Entries with a negative fd are ignored and their revents set to 0.
nfds	nfds_t	Number of items in the fds array.
timeout	int	Must be 0; poll() does not block.

Returns

On success, `poll()` returns a nonnegative value which is the number of elements in the pollfds whose revents fields have been set to a nonzero value (indicating an event or an error).

A return value of zero indicates that no file descriptors are ready.

On error, -1 is returned, and `errno` is set to indicate the error.

Remarks

errno	Description
EFAULT	System fault.
EINVAL	The nfds value exceeds the WINC_SOCK_NUM_SOCKETS value, or timeout is not 0.

fds events/revents	Description
POLLIN	There is data to read.
POLLOUT	Writing is now possible.
POLLERR	An error has been reported for the socket (only returned in revents; ignored in events). Any requested POLLIN or POLLOUT bits are also set, so that the error is returned by the next recv or send call.
POLLHUP	Hang up (only returned in revents; ignored in events). Subsequent reads from the channel will return 0 (end of file) only after all outstanding data in the channel has been consumed.
POLLNVAL	Invalid request: fd not open (only returned in revents; ignored in events).

recv

Description

Receive a message from a socket.

```
ssize_t recv(int fd, void *buf, size_t len, int flags)
```

Parameters

Parameter	Type	Description
fd	int	Socket file descriptor.
buf	void*	Pointer to buffer to receive data.
len	size_t	Length of data to receive.
flags	int	Option for reception, currently ignored.

Returns

The number of bytes received or -1 for error, see `errno`. If the return value is 0, the socket has performed an orderly shutdown.

Remarks

errno	Description
EBADF	The argument `fd` is not a valid file descriptor.
EFAULT	System fault.
ENOTSOCK	The file descriptor `fd` does not refer to a socket.
EWOULDBLOCK	The socket is marked nonblocking and the receive operation would block.

 ©2025-2026

recvfrom

Description

Receive a message from a socket.

```
ssize_t recvfrom(int fd, void *buf, size_t len, int flags, struct sockaddr *addr, socklen_t *alen)
```

Parameters

Parameter	Type	Description
fd	int	Socket file descriptor.
buf	void*	Pointer to buffer to receive data.
len	size_t	Length of data to receive.
flags	int	Option for reception, currently ignored.
addr	struct sockaddr*	Pointer to a structure to receive the peer address.
alen	ocklen_t*	Pointer to length of address structure.

Returns

The number of bytes received or -1 for error, see `errno`. If the return value is 0, the socket has performed an orderly shutdown.

Remarks

errno	Description
EBADF	The argument `fd` is not a valid file descriptor.
EFAULT	System fault.
ENOTSOCK	The file descriptor `fd` does not refer to a socket.
EWOULDBLOCK	The socket is marked nonblocking and the receive operation would block.

send

Description

Send a message on a socket.

```
ssize_t send(int fd, const void *buf, size_t len, int flags)
```

Parameters

Parameter	Type	Description
fd	int	Socket file descriptor.
buf	void*	Pointer to buffer containing data to send.
len	size_t	Length of data to send.
flags	int	Option for transmission, currently ignored.

Returns

The number of bytes sent or -1 for error, see `errno`.

Remarks

errno	Description
EBADF	The argument `fd` is not a valid file descriptor.
EFAULT	System fault.
ENOTSOCK	The file descriptor `fd` does not refer to a socket.
EWOULDBLOCK	The socket is marked nonblocking and the send operation would block.
ENOMEM	Insufficient memory is available.
EDESTADDRREQ	The socket is not connection-mode, and no peer address is set.
ENOTCONN	The socket is not connected, and no target has been given.

 ©2025-2026

sendto

Description

Send a message on a socket.

```
ssize_t sendto(int fd, const void *buf, size_t len, int flags, const struct sockaddr *addr, socklen_t alen)
```

Parameters

Parameter	Type	Description
fd	int	Socket file descriptor.
buf	void*	Pointer to buffer containing data to send.
len	size_t	Length of data to send.
flags	int	Option for transmission, currently ignored.
addr	struct sockaddr*	Pointer to address structure.
alen	socklen_t	Length of address structure.

Returns

The number of bytes sent or -1 for error, see `errno`.

Remarks

errno	Description
EBADF	The argument `fd` is not a valid file descriptor.
EFAULT	System fault.
ENOTSOCK	The file descriptor `fd` does not refer to a socket.
EWOULDBLOCK	The socket is marked nonblocking and the send operation would block.
ENOMEM	Insufficient memory is available.
EMSGSIZE	The argument data is too long to pass atomically through the underlying protocol.
EDESTADDRREQ	The socket is not connection-mode, and no peer address is set.
ENOTCONN	The socket is not connected, and no target has been given.

setsockopt

Description

Set options on sockets.

```
int setsockopt(int fd, int level, int optname, const void *optval, socklen_t optlen)
```

Parameters

Parameter	Type	Description
fd	int	Socket file descriptor.
level	int	Option level.
optname	int	Option name.
optval	void*	Option value.
optlen	socklen_t	Length of option value.

Returns

- 0 - Success.
- -1 - Error, `errno` set.

Remarks

errno	Description
EBADF	The argument `fd` is not a valid file descriptor.
ENOPROTOOPT	The option is unknown at the level indicated.
EINVAL	`optlen` invalid.
ENOMEM	Insufficient memory is available.
EBADMSG	Bad message.

Socket options IP:

IP supports some protocol-specific socket options that can be set with setsockopt. The socket option level for IP is `IPPROTO_IP`. A boolean integer flag is zero when it is false, otherwise true. Unless otherwise noted, `optval` is a pointer to an int.

When an invalid socket option is specified, setsockopt fail with the error `ENOPROTOOPT`.

IP_TOS: Set or receive the Type-Of-Service (TOS) field that is sent with every IP packet originating from this socket. It is used to prioritize packets on the network. TOS is an int. There are some standard TOS flags defined: `IPTOS_LOWDELAY` to minimize delays for interactive traffic, `IPTOS_THROUGHPUT` to optimize throughput, `IPTOS_RELIABILITY` to optimize for reliability. At most one of these TOS values can be specified. Other bits are invalid and shall be cleared.

 ©2025-2026

IP_ADD_MEMBERSHIP: Join a multicast group. Argument is an `ip_mreqn` structure.

```
struct ip_mreqn {
    struct in_addr imr_multiaddr;
    struct in_addr imr_address;
    int            imr_ifindex;
};
```

The structure `imr_multiaddr` contains the address of the multicast group the application wants to join or leave. It must be a valid multicast address (or setsockopt fails with the error `EINVAL`).

IPV6_ADD_MEMBERSHIP: Control membership in multicast groups. Argument is a pointer to a `struct ipv6_mreq`.

Socket options SOL_SOCKET:

The socket options listed below can be set by using setsockopt with the socket level set to `SOL_SOCKET` for all sockets.

SO_ASYNC_MODE: Set the asynchronous read mode. The argument is a integer. A value of 0 turns this mode off, 1 enables a simple protocol and 2 enables an acknowledged protocol.

SO_KEEPALIVE: Enable sending of keep-alive messages on connection-oriented sockets. Expects an integer boolean flag.

SO_LINGER: Sets the SO_LINGER option. The argument is a integer.

SO_RCVBUF: Sets the maximum socket receive buffer in bytes.

SO_SNDBUF: Sets the maximum socket send buffer in bytes.

Socket options IPPROTO_TCP:

To set a TCP socket option, call setsockopt to write the option with the option level argument set to `IPPROTO_TCP`.

TCP_NODELAY: If set, disable the Nagle algorithm. This means that segments are always sent as soon as possible, even if there is only a small amount of data. When not set, data is buffered until there is a sufficient amount to send out, thereby avoiding the frequent sending of small packets, which results in poor utilization of the network.

Socket options IPPROTO_TLS:

To set a TCP(TLS) socket option, call setsockopt to write the option with the option level argument set to `IPPROTO_TLS`.

TLS_CONF_IDX: Sets the TLS configuration index.

shutdown

Description

Shut down part of a full-duplex connection.

```
int shutdown(int fd, int how)
```

Parameters

Parameter	Type	Description
fd	int	Socket file descriptor.
how	int	Shutdown flags, must be `SHUT_RDWR`.

Returns

- 0 - Success.
- -1 - Error, `errno` set.

Remarks

errno	Description
EBADF	`fd` is not a valid file descriptor.
ENOTSOCK	The file descriptor `fd` does not refer to a socket.
ENOMEM	Insufficient memory is available. The socket cannot be created until sufficient resources are freed.
EBADMSG	Bad message.

 ©2025-2026

socket

Description

Create an endpoint for communication.

```
int socket(int domain, int type, int protocol)
```

Parameters

Parameter	Type	Description
domain	int	Communication domain, must be AF_INET or AF_INET6.
type	int	Socket type, must be SOCK_STREAM or SOCK_DGRAM.
protocol	int	Socket protocol. SOCK_STREAM: 0 or IPPROTO_TCP for TCP, IPPROTO_TLS for TLS. SOCK_DGRAM: 0 or IPPROTO_UDP for UDP.

Returns

Socket file descriptor or -1 for error (errno set).

Remarks

errno	Description
EAFNOSUPPORT	The implementation does not support the specified address family.
EINVAL	type is not SOCK_STREAM or SOCK_DGRAM.
EPROTONOSUPPORT	protocol is not supported for the socket type.
EFAULT	System fault
ENFILE	The system-wide limit on the total number of open files has been reached.
ENOMEM	Insufficient memory is available. The socket cannot be created until sufficient resources are freed.
EBADMSG	Bad message.

freeaddrinfo

Description

Frees the memory allocated by getaddrinfo.

```
void freeaddrinfo(struct addrinfo *p)
```

Parameters

Parameter	Type	Description
p	struct addrinfo*	Pointer to address linked list returned by `getaddrinfo()`.

Returns

None.

Remarks

None.

 ©2025-2026

getaddrinfo

Description

Network address and service translation.

```
int getaddrinfo(const char *host, const char *serv, const struct addrinfo *hint, struct addrinfo **res)
```

Parameters

Parameter	Type	Description
host	char*	Pointer to host to translate.
serv	char*	Pointer to service to translate, currently ignored.
hint	struct addrinfo*	Hint to affect the translation.
res	struct addrinfo*	Pointer to results pointer.

Returns

value	Description
EAI_NONAME	The node or service is not known.
EAI_AGAIN	The name server returned a temporary failure indication. Try again later.
EAI_SYSTEM	Other system error; `errno` is set to indicate the error.
EAI_FAMILY	The requested address family is not supported.
EAI_MEMORY	Out of memory.
EAI_SERVICE	The requested service is not available for the requested socket type.

Remarks

errno	Description
EFAULT	System fault.
EBADMSG	Bad message.

The getaddrinfo function allocates and initializes a linked list of `addrinfo` structures, one for each network address that matches node and service, subject to any restrictions imposed by hints, and returns a pointer to the start of the list in `res`. The items in the linked list are linked by the `ai_next` field.

The freeaddrinfo function frees the memory that was allocated for the dynamically allocated linked list `res`.

getpeername

Description

Returns the address of the peer connected to the socket fd.

```
int getpeername(int fd, struct sockaddr *addr, socklen_t *addrlen)
```

Parameters

Parameter	Type	Description
fd	int	Socket file descriptor.
addr	sockaddr*	Pointer to address structure.
addrlen	socklen_t*	Length of address structure.

Returns

On success, zero is returned.

On error, -1 is returned, and `errno` is set to indicate the error.

Remarks

errno	Description
EFAULT	System fault.
EINVAL	addrlen is invalid (e.g., is negative).
ENOTSOCK	The file descriptor fd does not refer to a socket.
ENOTCONN	The socket is not connected, and no target has been given.

 ©2025-2026

getsockname

Description

Returns the current address to which the socket fd is bound.

```
int getsockname(int fd, struct sockaddr *addr, socklen_t *addrlen)
```

Parameters

Parameter	Type	Description
fd	int	Socket file descriptor.
addr	sockaddr*	Pointer to address structure.
addrlen	socklen_t*	Length of address structure.

Returns

On success, zero is returned.

On error, -1 is returned, and `errno` is set to indicate the error.

Remarks

errno	Description
EFAULT	System fault.
EINVAL	addrlen is invalid (e.g., is negative).
ENOTSOCK	The file descriptor fd does not refer to a socket.
ENOTCONN	The socket is not connected, and no target has been given.

htonl

Description

Convert 32-bit value from host to network byte order.

```
uint32_t htonl(uint32_t n)
```

Parameters

Parameter	Type	Description
n	uint32_t	Value to convert.

Returns

Converted value.

Remarks

None.

 ©2025-2026

htons

Description

Convert 16-bit value from host to network byte order.

```
uint16_t htons(uint16_t n)
```

Parameters

Parameter	Type	Description
n	uint16_t	Value to convert.

Returns

Converted value.

Remarks

None.

ntohl

Description

Convert 32-bit value from network to host byte order.

```
uint32_t ntohl(uint32_t n)
```

Parameters

Parameter	Type	Description
n	uint32_t	Value to convert.

Returns

Converted value.

Remarks

None.

 ©2025-2026

ntohs

Description

Convert 16-bit value from network to host byte order.

```
uint16_t ntohs(uint16_t n)
```

Parameters

Parameter	Type	Description
n	uint16_t	Value to convert.

Returns

Converted value.

Remarks

None.

Examples

These are examples of using the Berkeley sockets API.

- TCP Client
- TCP Server
- UDP Client
- UDP Server
- DNS lookup

Prerequisites

The examples assume that the driver has been initialized, the transport is active and the event loop is being run (see Event Processing). The sockets interface must also have been initialized by calling WINC_SockInit.

```
WINC_SOCKET_INIT_TYPE sockInitData;

sockInitData.pfMemAlloc = malloc;
sockInitData.pfMemFree  = free;
sockInitData.slabSize   = 1472;
sockInitData.numSlabs   = 20;

WINC_SockInit(wincDevHandle, &sockInitData);
```

Socket descriptors start at 0, so the examples use -1 to indicate that no socket is open.

 ©2025-2026

DNS Lookup

```
#include <string.h>

#define BSOCK_GET_ADDR_MAX_HOSTNAME_SZ      (256+1)

typedef enum
{
    MODULE_STATE_INIT,
    MODULE_STATE_WAIT,
    MODULE_STATE_END,
    MODULE_STATE_ERROR,
} MODULE_STATE;

typedef struct
{
    sa_family_t     sin_family;
    char            hostName[BSOCK_GET_ADDR_MAX_HOSTNAME_SZ];
} BSOCK_GET_ADDR_INIT_TYPE;

static MODULE_STATE                 state;
static BSOCK_GET_ADDR_INIT_TYPE     initData = {0};
static struct addrinfo              *pAddrInfo;
static struct addrinfo              *pHints, hints;

void bsock_get_addr_init(void *pInitData)
{
    if (NULL != pInitData)
    {
        memcpy(&initData, pInitData, sizeof(BSOCK_GET_ADDR_INIT_TYPE));
    }

    state = MODULE_STATE_INIT;
}

void bsock_get_addr_run(void)
{
    switch (state)
    {
        case MODULE_STATE_INIT:
        {
            int result;

            if (AF_INET == initData.sin_family)
            {
                memset(&hints, 0, sizeof(hints));
                hints.ai_family = AF_INET;
                pHints = &hints;
            }
            else if (AF_INET6 == initData.sin_family)
            {
                memset(&hints, 0, sizeof(hints));
                hints.ai_family = AF_INET6;
                pHints = &hints;
            }
            else if (AF_UNSPEC == initData.sin_family)
            {
                pHints = NULL;
            }
            else
            {
                printf("invalid address family %d\n", initData.sin_family);
                state = MODULE_STATE_ERROR;
            }

            pAddrInfo = NULL;
```

```
            errno = 0;
            result = getaddrinfo(initData.hostName, NULL, pHints, &pAddrInfo);

            printf("getaddrinfo = %d (%d)\n", result, errno);

            if (0 == result)
            {
                printf("result found\n");

                state = MODULE_STATE_END;
            }
            else if (EAI_AGAIN == result)
            {
                printf("result pending\n");
                state = MODULE_STATE_WAIT;
            }
            else
            {
                printf("result error\n");

                state = MODULE_STATE_ERROR;
            }

            break;
        }

        case MODULE_STATE_WAIT:
        {
            int result;

            pAddrInfo = NULL;

            errno = 0;
            result = getaddrinfo(initData.hostName, NULL, pHints, &pAddrInfo);

            printf("getaddrinfo = %d (%d)\n", result, errno);

            if (0 == result)
            {
                printf("result found\n");

                state = MODULE_STATE_END;
            }
            else if (EAI_AGAIN != result)
            {
                printf("result error\n");

                state = MODULE_STATE_ERROR;
            }

            break;
        }

        case MODULE_STATE_END:
        {
            if (NULL != pAddrInfo)
            {
                struct addrinfo *pRes = pAddrInfo;

                while (NULL != pRes)
                {
                    if (NULL != pRes->ai_addr)
                    {
                        char ipStr[64];
                        union sockaddr_union *pAddr = (union sockaddr_union*)pRes->ai_addr;

                        if (AF_INET == pAddr->sin_family)
                        {
                            inet_ntop(pAddr->sin_family, &pAddr->in4.sin_addr, ipStr, sizeof(ipStr));
```

```
                        }
                        else if (AF_INET6 == pAddr->sin_family)
                        {
                            inet_ntop(pAddr->sin_family, &pAddr->in6.sin6_addr, ipStr, sizeof(ipStr));
                        }

                        printf("result is %s\n", ipStr);
                    }

                    pRes = pRes->ai_next;
                }

                freeaddrinfo(pAddrInfo);
                pAddrInfo = NULL;
            }

            break;
        }

        case MODULE_STATE_ERROR:
        {
            break;
        }

        default:
        {
            break;
        }
    }
}
```

TCP Client

```
#include <string.h>

#include "winc_socket.h"

typedef enum
{
    MODULE_STATE_INIT,
    MODULE_STATE_SOCKET_WAITING,
    MODULE_STATE_CONNECTING,
    MODULE_STATE_CONNECTED,
    MODULE_STATE_DISCONNECTED,
    MODULE_STATE_ERROR,
} MODULE_STATE;

typedef struct
{
    union sockaddr_union sockAddr;
} BSOCK_TCP_CLIENT_INIT_TYPE;

static MODULE_STATE                 state;
static int                          sockh = -1;
static BSOCK_TCP_CLIENT_INIT_TYPE   initData = {0};

void bsock_tcp_client_init(void *pInitData)
{
    if (NULL != pInitData)
    {
        memcpy(&initData, pInitData, sizeof(BSOCK_TCP_CLIENT_INIT_TYPE));
    }

    state = MODULE_STATE_INIT;
}

void bsock_tcp_client_run(void)
{
    switch (state)
    {
        case MODULE_STATE_INIT:
        {
            state = MODULE_STATE_ERROR;

            sockh = socket(initData.sockAddr.sin_family, SOCK_STREAM, IPPROTO_TCP);

            if (sockh < 0)
            {
                break;
            }

            printf("opening socket %d\n", sockh);

            /* Fall through */
        }

        case MODULE_STATE_SOCKET_WAITING:
        {
            errno = 0;
            if (-1 == connect(sockh, (struct sockaddr*)&initData.sockAddr, sizeof(initData.sockAddr)))
            {
                if (EINPROGRESS == errno)
                {
                    printf("connection in progress\n");
                    state = MODULE_STATE_CONNECTING;
                }
                else if (EAGAIN == errno)
```

 ©2025-2026

```
            {
                printf("socket not ready try again\n");
                state = MODULE_STATE_SOCKET_WAITING;
            }
            else
            {
                printf("connect failed, errno is %d\n", errno);
                state = MODULE_STATE_ERROR;
            }
        }
        else
        {
            printf("socket connected\n");
            state = MODULE_STATE_CONNECTED;
        }

        break;
    }

    case MODULE_STATE_CONNECTING:
    {
        struct pollfd fd;

        fd.fd     = sockh;
        fd.events = POLLOUT;

        if (-1 == poll(&fd, 1, 0))
        {
            state = MODULE_STATE_ERROR;
            break;
        }

        if (0 != (fd.revents & POLLERR))
        {
            printf("connect failed\n");
            state = MODULE_STATE_ERROR;
        }

        if (0 != (fd.revents & POLLOUT))
        {
            printf("socket connected\n");
            state = MODULE_STATE_CONNECTED;
        }

        break;
    }

    case MODULE_STATE_CONNECTED:
    {
        struct pollfd fd;
        ssize_t numBytes = 0;
        uint8_t buffer[128];

        fd.fd     = sockh;
        fd.events = POLLIN|POLLOUT;

        if (-1 == poll(&fd, 1, 0))
        {
            state = MODULE_STATE_ERROR;
            break;
        }

        if (0 != (fd.revents & POLLIN))
        {
            errno = 0;
            numBytes = recv(sockh, NULL, 0, MSG_PEEK|MSG_TRUNC);

            if ((numBytes <= 0) && (errno != EWOULDBLOCK))
            {
                state = MODULE_STATE_DISCONNECTED;
```

```
                break;
            }

            if (numBytes >= sizeof(buffer))
            {
                numBytes = sizeof(buffer);
            }

            errno = 0;
            numBytes = recv(sockh, buffer, numBytes, 0);
            if (numBytes <= 0)
            {
                if (errno != EWOULDBLOCK)
                {
                    printf("recv failed, errno is %d\n", errno);
                    state = MODULE_STATE_DISCONNECTED;
                    break;
                }
            }

            printf("recv: %d\n", numBytes);
        }

        if (0 != (fd.revents & POLLHUP))
        {
            printf("socket disconnected\n");

            if (0 == (fd.revents & POLLIN))
            {
                state = MODULE_STATE_DISCONNECTED;
            }
        }

        if (0 != (fd.revents & POLLOUT))
        {
            if (numBytes > 0)
            {
                errno = 0;
                numBytes = send(sockh, buffer, numBytes, 0);

                if ((numBytes < 0) && (errno != EWOULDBLOCK) && (errno != EAGAIN))
                {
                    printf("send failed, errno is %d\n", errno);
                    state = MODULE_STATE_DISCONNECTED;
                    break;
                }
            }
        }

        break;
    }

    case MODULE_STATE_DISCONNECTED:
    {
        if (sockh >= 0)
        {
            shutdown(sockh, SHUT_RDWR);
            sockh = -1;
        }
        break;
    }

    case MODULE_STATE_ERROR:
    {
        break;
    }

    default:
    {
        break;
```

 ©2025-2026

```
        }
    }
}
```

TCP Server

```
#include <string.h>

#include "winc_socket.h"

#define TCP_SERVER_MAX_CLIENTS  2

typedef enum
{
    MODULE_STATE_INIT,
    MODULE_STATE_SOCKET_WAITING,
    MODULE_STATE_LISTEN,
    MODULE_STATE_LISTENING,
    MODULE_STATE_ERROR,
} MODULE_STATE;

typedef struct
{
    union sockaddr_union sockAddr;
} BSOCK_TCP_SERVER_INIT_TYPE;

static MODULE_STATE                state;
static int                         sockh = -1;
static int                         sockhClient[TCP_SERVER_MAX_CLIENTS];
static BSOCK_TCP_SERVER_INIT_TYPE  initData = {0};

void bsock_tcp_server_init(void *pInitData)
{
    int i;

    if (NULL != pInitData)
    {
        memcpy(&initData, pInitData, sizeof(BSOCK_TCP_SERVER_INIT_TYPE));
    }

    for (i=0; i<TCP_SERVER_MAX_CLIENTS; i++)
    {
        sockhClient[i] = -1;
    }

    state = MODULE_STATE_INIT;
}

void bsock_tcp_server_run(void)
{
    switch (state)
    {
        case MODULE_STATE_INIT:
        {
            state = MODULE_STATE_ERROR;

            sockh = socket(initData.sockAddr.sin_family, SOCK_STREAM, IPPROTO_TCP);

            if (sockh < 0)
            {
                break;
            }

            printf("opening socket %d\n", sockh);

            /* Fall through */
        }

        case MODULE_STATE_SOCKET_WAITING:
        {
```

 ©2025-2026

```
            errno = 0;
            if (-1 == bind(sockh, (struct sockaddr*)&initData.sockAddr, sizeof(initData.sockAddr)))
            {
                printf("bind failed, errno is %d\n", errno);
                state = MODULE_STATE_ERROR;
                break;
            }

            printf("socket bound\n");

            /* Fall through */
        }

        case MODULE_STATE_LISTEN:
        {
            errno = 0;
            if (-1 == listen(sockh, TCP_SERVER_MAX_CLIENTS))
            {
                if (EAGAIN == errno)
                {
                    printf("socket not ready try again\n");
                    state = MODULE_STATE_LISTEN;
                }
                else
                {
                    printf("listen failed, errno is %d\n", errno);
                    state = MODULE_STATE_ERROR;
                }
            }
            else
            {
                printf("socket listening\n");
                state = MODULE_STATE_LISTENING;
            }

            break;
        }

        case MODULE_STATE_LISTENING:
        {
            int i;

            struct pollfd fd[1+TCP_SERVER_MAX_CLIENTS] = {0};

            fd[0].fd     = sockh;
            fd[0].events = POLLIN;

            for (i=0; i<TCP_SERVER_MAX_CLIENTS; i++)
            {
                /* Unused entries have fd -1 and are ignored by poll. */
                fd[i+1].fd     = sockhClient[i];
                fd[i+1].events = POLLIN|POLLOUT;
            }

            if (-1 == poll(fd, 1+TCP_SERVER_MAX_CLIENTS, 0))
            {
                state = MODULE_STATE_ERROR;
                break;
            }

            if (0 != (fd[0].revents & POLLIN))
            {
                for (i=0; i<TCP_SERVER_MAX_CLIENTS; i++)
                {
                    union sockaddr_union sockAddr;
                    socklen_t sockAddrLen = sizeof(union sockaddr_union);
                    char ipStr[64];

                    if (-1 == sockhClient[i])
                    {
```

```
                sockhClient[i] = accept(sockh, (struct sockaddr*)&sockAddr, &sockAddrLen);

                if (sockhClient[i] >= 0)
                {
                    if (AF_INET == sockAddr.sin_family)
                    {
                        inet_ntop(sockAddr.sin_family, &sockAddr.in4.sin_addr, ipStr, sizeof(ipStr));
                    }
                    else if (AF_INET6 == sockAddr.sin_family)
                    {
                        inet_ntop(sockAddr.sin_family, &sockAddr.in6.sin6_addr, ipStr, sizeof(ipStr));
                    }

                    printf("accept %d from %s:%d\n", sockhClient[i], ipStr, ntohs(sockAddr.sin_port));
                }

                break;
            }
        }
    }

    for (i=1; i<=TCP_SERVER_MAX_CLIENTS; i++)
    {
        if ((fd[i].fd >= 0) && (0 != fd[i].revents))
        {
            ssize_t numBytes = 0;
            uint8_t buffer[128];

            if (0 != (fd[i].revents & POLLIN))
            {
                errno = 0;
                numBytes = recv(fd[i].fd, NULL, 0, MSG_PEEK|MSG_TRUNC);

                if ((numBytes <= 0) && (errno != EWOULDBLOCK))
                {
                    shutdown(fd[i].fd, SHUT_RDWR);
                    sockhClient[i-1] = -1;
                    break;
                }

                if (numBytes >= sizeof(buffer))
                {
                    numBytes = sizeof(buffer);
                }

                errno = 0;
                numBytes = recv(fd[i].fd, buffer, numBytes, 0);
                if (numBytes <= 0)
                {
                    if (errno != EWOULDBLOCK)
                    {
                        printf("recv failed, errno is %d\n", errno);
                        shutdown(fd[i].fd, SHUT_RDWR);
                        sockhClient[i-1] = -1;
                        break;
                    }
                }

                printf("recv: %d\n", numBytes);
            }

            if (0 != (fd[i].revents & POLLHUP))
            {
                printf("socket disconnected\n");

                if (0 == (fd[i].revents & POLLIN))
                {
                    shutdown(fd[i].fd, SHUT_RDWR);
                    sockhClient[i-1] = -1;
                    break;
```

 ©2025-2026

```
                    }
                }

                if (0 != (fd[i].revents & POLLOUT))
                {
                    if (numBytes > 0)
                    {
                        errno = 0;
                        numBytes = send(fd[i].fd, buffer, numBytes, 0);

                        if ((numBytes < 0) && (errno != EWOULDBLOCK) && (errno != EAGAIN))
                        {
                            printf("send failed, errno is %d\n", errno);
                            shutdown(fd[i].fd, SHUT_RDWR);
                            sockhClient[i-1] = -1;
                            break;
                        }
                    }
                }
            }
        }

        break;
    }

    case MODULE_STATE_ERROR:
    {
        break;
    }

    default:
    {
        break;
    }
  }
}
```

UDP Client

```
#include <string.h>

#include "winc_socket.h"

typedef enum
{
    MODULE_STATE_INIT,
    MODULE_STATE_SOCKET_WAITING,
    MODULE_STATE_CONNECTING,
    MODULE_STATE_CONNECTED,
    MODULE_STATE_DISCONNECTED,
    MODULE_STATE_ERROR,
} MODULE_STATE;

typedef struct
{
    union sockaddr_union sockAddr;
} BSOCK_UDP_CLIENT_INIT_TYPE;

static MODULE_STATE                 state;
static int                          sockh = -1;
static BSOCK_UDP_CLIENT_INIT_TYPE   initData = {0};

void bsock_udp_client_init(void *pInitData)
{
    if (NULL != pInitData)
    {
        memcpy(&initData, pInitData, sizeof(BSOCK_UDP_CLIENT_INIT_TYPE));
    }

    state = MODULE_STATE_INIT;
}

void bsock_udp_client_run(void)
{
    switch (state)
    {
        case MODULE_STATE_INIT:
        {
            state = MODULE_STATE_ERROR;

            sockh = socket(initData.sockAddr.sin_family, SOCK_DGRAM, IPPROTO_UDP);

            if (sockh < 0)
            {
                break;
            }

            printf("opening socket %d\n", sockh);

            /* Fall through */
        }

        case MODULE_STATE_SOCKET_WAITING:
        {
            errno = 0;
            if (-1 == connect(sockh, (struct sockaddr*)&initData.sockAddr, sizeof(initData.sockAddr)))
            {
                if (EINPROGRESS == errno)
                {
                    printf("connection in progress\n");
                    state = MODULE_STATE_CONNECTING;
                }
                else if (EAGAIN == errno)
```

 ©2025-2026

```
            {
                printf("socket not ready try again\n");
                state = MODULE_STATE_SOCKET_WAITING;
            }
            else
            {
                printf("connect failed, errno is %d\n", errno);
                state = MODULE_STATE_ERROR;
            }
        }
        else
        {
            printf("socket connected\n");
            state = MODULE_STATE_CONNECTED;
        }

        break;
    }

    case MODULE_STATE_CONNECTING:
    {
        struct pollfd fd;

        fd.fd     = sockh;
        fd.events = POLLOUT;

        if (-1 == poll(&fd, 1, 0))
        {
            state = MODULE_STATE_ERROR;
            break;
        }

        if (0 != (fd.revents & POLLERR))
        {
            printf("connect failed\n");
            state = MODULE_STATE_ERROR;
        }

        if (0 != (fd.revents & POLLOUT))
        {
            printf("socket connected\n");
            state = MODULE_STATE_CONNECTED;

            send(sockh, "Hello\n", 6, 0);
        }

        break;
    }

    case MODULE_STATE_CONNECTED:
    {
        struct pollfd fd;
        ssize_t numBytes = 0;
        uint8_t buffer[128];

        fd.fd     = sockh;
        fd.events = POLLIN|POLLOUT;

        if (-1 == poll(&fd, 1, 0))
        {
            state = MODULE_STATE_ERROR;
            break;
        }

        if (0 != (fd.revents & POLLIN))
        {
            union sockaddr_union sockAddr;
            socklen_t fromLen = sizeof(sockAddr);
            char ipStr[64];
```

```
            errno = 0;
            numBytes = recvfrom(sockh, NULL, 0, MSG_PEEK|MSG_TRUNC, NULL, NULL);

            if ((numBytes <= 0) && (errno != EWOULDBLOCK))
            {
                state = MODULE_STATE_DISCONNECTED;
                break;
            }

            if (numBytes >= sizeof(buffer))
            {
                numBytes = sizeof(buffer);
            }

            errno = 0;
            numBytes = recvfrom(sockh, buffer, numBytes, 0, (struct sockaddr*)&sockAddr, &fromLen);
            if (numBytes <= 0)
            {
                if (errno != EWOULDBLOCK)
                {
                    printf("recv failed, errno is %d\n", errno);
                    state = MODULE_STATE_DISCONNECTED;
                    break;
                }
            }

            if (AF_INET == sockAddr.sin_family)
            {
                inet_ntop(sockAddr.sin_family, &sockAddr.in4.sin_addr, ipStr, sizeof(ipStr));
            }
            else if (AF_INET6 == sockAddr.sin_family)
            {
                inet_ntop(sockAddr.sin_family, &sockAddr.in6.sin6_addr, ipStr, sizeof(ipStr));
            }

            printf("recv: %d from %s:%d\n", numBytes, ipStr, ntohs(sockAddr.sin_port));
        }

        if (0 != (fd.revents & POLLOUT))
        {
            if (numBytes > 0)
            {
                errno = 0;
                numBytes = send(sockh, buffer, numBytes, 0);

                if ((numBytes < 0) && (errno != EWOULDBLOCK) && (errno != EAGAIN))
                {
                    printf("send failed, errno is %d\n", errno);
                    state = MODULE_STATE_DISCONNECTED;
                    break;
                }
            }
        }

        break;
    }

    case MODULE_STATE_DISCONNECTED:
    {
        if (sockh >= 0)
        {
            shutdown(sockh, SHUT_RDWR);
            sockh = -1;
        }
        break;
    }

    case MODULE_STATE_ERROR:
    {
        break;
```

 ©2025-2026

```
        }

        default:
        {
            break;
        }
    }
}
```

UDP Server

```
#include <string.h>

#include "winc_socket.h"

#define UDP_SERVER_MAX_CLIENTS  2

typedef enum
{
    MODULE_STATE_INIT,
    MODULE_STATE_SOCKET_WAITING,
    MODULE_STATE_LISTENING,
    MODULE_STATE_DISCONNECTED,
    MODULE_STATE_ERROR,
} MODULE_STATE;

typedef struct
{
    union sockaddr_union sockAddr;
} BSOCK_UDP_SERVER_INIT_TYPE;

static MODULE_STATE                 state;
static int                          sockh = -1;
static BSOCK_UDP_SERVER_INIT_TYPE   initData = {0};

void bsock_udp_server_init(void *pInitData)
{
    if (NULL != pInitData)
    {
        memcpy(&initData, pInitData, sizeof(BSOCK_UDP_SERVER_INIT_TYPE));
    }

    state = MODULE_STATE_INIT;
}

void bsock_udp_server_run(void)
{
    switch (state)
    {
        case MODULE_STATE_INIT:
        {
            state = MODULE_STATE_ERROR;

            sockh = socket(initData.sockAddr.sin_family, SOCK_DGRAM, IPPROTO_UDP);

            if (sockh < 0)
            {
                break;
            }

            printf("opening socket %d\n", sockh);

            /* Fall through */
        }

        case MODULE_STATE_SOCKET_WAITING:
        {
            errno = 0;
            if (-1 == bind(sockh, (struct sockaddr*)&initData.sockAddr, sizeof(initData.sockAddr)))
            {
                if (EAGAIN == errno)
                {
                    printf("socket not ready try again\n");
                    state = MODULE_STATE_SOCKET_WAITING;
                }
```

 ©2025-2026

```
            else
            {
                printf("bind failed, errno is %d\n", errno);
                state = MODULE_STATE_ERROR;
            }
        }
        else
        {
            printf("socket bound\n");
            state = MODULE_STATE_LISTENING;
        }

        break;
    }

    case MODULE_STATE_LISTENING:
    {
        struct pollfd fd;
        ssize_t numBytes = 0;
        uint8_t buffer[128];
        union sockaddr_union sockAddr;

        fd.fd     = sockh;
        fd.events = POLLIN|POLLOUT;

        if (-1 == poll(&fd, 1, 0))
        {
            state = MODULE_STATE_ERROR;
            break;
        }

        if (0 != (fd.revents & POLLIN))
        {
            socklen_t fromLen = sizeof(sockAddr);
            char ipStr[64];

            errno = 0;
            numBytes = recvfrom(sockh, NULL, 0, MSG_PEEK|MSG_TRUNC, NULL, NULL);

            if ((numBytes <= 0) && (errno != EWOULDBLOCK))
            {
                state = MODULE_STATE_DISCONNECTED;
                break;
            }

            if (numBytes >= sizeof(buffer))
            {
                numBytes = sizeof(buffer);
            }

            errno = 0;
            numBytes = recvfrom(sockh, buffer, numBytes, 0, (struct sockaddr*)&sockAddr, &fromLen);
            if (numBytes <= 0)
            {
                if (errno != EWOULDBLOCK)
                {
                    printf("recv failed, errno is %d\n", errno);
                    state = MODULE_STATE_DISCONNECTED;
                    break;
                }
            }

            if (AF_INET == sockAddr.sin_family)
            {
                inet_ntop(sockAddr.sin_family, &sockAddr.in4.sin_addr, ipStr, sizeof(ipStr));
            }
            else if (AF_INET6 == sockAddr.sin_family)
            {
                inet_ntop(sockAddr.sin_family, &sockAddr.in6.sin6_addr, ipStr, sizeof(ipStr));
            }
```

```
                printf("recv: %d from %s:%d\n", numBytes, ipStr, ntohs(sockAddr.sin_port));
            }

            if (0 != (fd.revents & POLLOUT))
            {
                if (numBytes > 0)
                {
                    errno = 0;
                    numBytes = sendto(sockh, buffer, numBytes, 0, (struct sockaddr*)&sockAddr, sizeof(sockAddr));

                    if ((numBytes < 0) && (errno != EWOULDBLOCK) && (errno != EAGAIN))
                    {
                        printf("send failed, errno is %d\n", errno);
                        state = MODULE_STATE_DISCONNECTED;
                        break;
                    }
                }
            }

            break;
        }

        case MODULE_STATE_DISCONNECTED:
        {
            if (sockh >= 0)
            {
                shutdown(sockh, SHUT_RDWR);
                sockh = -1;
            }
            break;
        }

        case MODULE_STATE_ERROR:
        {
            break;
        }

        default:
        {
            break;
        }
    }
}
```

 ©2025-2026

List of Examples

Module	Description
Configuration Functions	Reading all parameter values
	To read a single value
	To set an additional value
	To replace an existing value
	Range of IDs
	Simple ID/Value
	Complex ID/Value
	Using preset 0
CFG	Saving active configuration to storage
	Restoring configuration from storage
	Storing a partial configuration
DNS	A record query
	NS record query
	CNAME record query
	SOA record query
	PTR record query
	MX record query
	TXT record query
	AAAA record query
	SRV record query
	DS record query
	NSEC record query
	DNSKEY record query
	ANY record query
	A record query With DNSSEC awareness
	NSEC3PARAM (unknown type) record query
	NSEC3 (unknown type) response to invalid A record query
	Multicast A record query
	MDNS-SD of local SSH services
	List and delete a file
	Get filesystem information
	File load using FS-TSFR protocol
	File store using FS-TSFR protocol
MQTT	MQTT connection directly over TLS
	MQTT connection over TLS socket
	Subscribe to MQTT topic and receive data (asynchronously)
	Subscribe to MQTT topic and receive data (polled)
OTA	Internal OTA HTTP Configuration

	Internal OTA HTTPS Configuration
	Internal OTA
	NVM OTA
SOCKET	Opening a TCP server socket
	Basic socket read
	In-band socket read
	Simple asynchronous socket read
	Reliable asynchronous socket read
	Set TLS configuration 1 for TLS client
	Open a TLS client socket
	Set TLS configuration 2 for TLS server
	Open a TLS server socket
	Reading partial datagram
TLS	Peer certificate digest pinning with WINC_CmdTLSCDA
WSCN	Scan for Access Points and filter by security type
	Setting SSID filter with WINC_CmdWSCNC
	Setting SSID prefix filters with WINC_CmdWSCNC
	Periodic passive scanning with an SSID prefix filter
	Periodic passive scanning with an SSID prefix filter
WSTA	WPA Personal: Configure STA connection parameters
	Connect at Wi-Fi and IP layers
	Some disconnections, reconnections and roaming
	WINC_CmdWSTA refused while a scan or periodic
	WPA Enterprise: Configure a TLS context for establishing a tunnel
	WPA Enterprise: Configure STA connection parameters for MSCHAPv2
	WPA Enterprise: Configure a TLS context for EAP-PEAPv1/TLS
	WPA Enterprise: Configure STA connection parameters for EAP-PEAPv1/TLS
DI	Query device information with WINC_CmdDI
EXTCRYPTO	Example of TLS configuration for secure signing via external crypto operations
	Example of signing request/response using WINC_CmdEXTCRYPTO
WIFI	Settings with WINC_CmdWIFIC
	WINC_CmdWIFIC coexistence settings for 3-wire PTA interface
	Configure vendor-specific information elements for transmission
	Configure vendor-specific information element reception
NVM	Simple example of NVM usage
DFU	Example of using WINC_CmdDFUADR and WINC_CmdDFUSEQ
PPS	Configuring and enabling PPS
ARB	Example of using WINC_CmdARB
HTTP	HTTPS download
	HTTPS download from TCP socket
	File download to local filesystem
	HTTPGET with path and query/fragments

 ©2025-2026

HTTP basic authentication using additional request headers

WBCN

Target asset: advertise its location

Tracker asset: send a command to a target asset

Target asset: a complete exchange with a tracker asset

Tracker asset: a complete exchange with a target asset

Index

A

accept, 447
ARB - WINC_CmdARB, 385
ASSOC (AEC), 319
ASSOC - WINC_CmdASSOC, 318

B

bind, 448
BOOT (AEC), 55

C

CFG - WINC_CmdCFG, 59
CFGCP - WINC_CmdCFGCP, 62
connect, 449

D

DFUADR - WINC_CmdDFUADR, 366
DFUSEQ - WINC_CmdDFUSEQ, 367
DHCPSC - WINC_CmdDHCPSC, 69
DI - WINC_CmdDI, 334
DNS Lookup, 471
DNSC - WINC_CmdDNSC, 74
DNSERR (AEC), 81
DNSRESOLV (AEC), 78
DNSRESOLV - WINC_CmdDNSRESOLV, 77

E

EXTCRYPTO (AEC), 339
EXTCRYPTO - WINC_CmdEXTCRYPTO, 338

F

freeaddrinfo, 462
FSLOAD - WINC_CmdFSLOAD, 91
FSOP - WINC_CmdFSOP, 88
FSRECV - WINC_CmdFSRECV, 97
FSSTORE - WINC_CmdFSSTORE, 93
FSTSFR - WINC_CmdFSTSFR, 95
FSUP (AEC), 99

G

getaddrinfo, 463
getpeername, 464
getsockname, 465
GMI - WINC_CmdGMI, 56
GMM - WINC_CmdGMM, 57
GMR - WINC_CmdGMR, 58

H

htonl, 466
htons, 467
HTTPBODYEND (AEC), 400
HTTPBODYRX (AEC), 399
HTTPC - WINC_CmdHTTPC, 391
HTTPCL (AEC), 401
HTTPCL - WINC_CmdHTTPCL, 396
HTTPERR (AEC), 402
HTTPGET - WINC_CmdHTTPGET, 394
HTTPRD - WINC_CmdHTTPRD, 395
HTTPRSPCODE (AEC), 397
HTTPRSPFIELD (AEC), 398

I

inet_ntop, 450
inet_pton, 451

L

listen, 452

M

MQTTC - WINC_CmdMQTTC, 113
MQTTCONN (AEC), 141
MQTTCONN - WINC_CmdMQTTCONN, 116
MQTTCONNACK (AEC), 142
MQTTDISCONN - WINC_CmdMQTTDISCONN, 127
MQTTLWT - WINC_CmdMQTTLWT, 126
MQTTPROPRX (AEC), 150
MQTTPROPRX - WINC_CmdMQTTPROPRX, 133
MQTTPROPRXPRESET - WINC_CmdMQTTPROPRXPRESET, 136
MQTTPROPTX - WINC_CmdMQTTPROPTX, 128
MQTTPROPTXKV - WINC_CmdMQTTPROPTXKV, 131
MQTTPROPTXS - WINC_CmdMQTTPROPTXS, 138
MQTTPUB - WINC_CmdMQTTPUB, 122
MQTTPUBACK (AEC), 144
MQTTPUBCOMP (AEC), 145
MQTTPUBERR (AEC), 146
MQTTPUBFRAG - WINC_CmdMQTTPUBFRAG, 124

 ©2025-2026

MQTTPUBSENT (AEC), 151
MQTTSUB (AEC), 147
MQTTSUB - WINC_CmdMQTTSUB, 117
MQTTSUBLST - WINC_CmdMQTTSUBLST, 118
MQTTSUBRD - WINC_CmdMQTTSUBRD, 119
MQTTSUBRX (AEC), 149
MQTTSUBRXERR (AEC), 152
MQTTUNSUB (AEC), 148
MQTTUNSUB - WINC_CmdMQTTUNSUB, 121

N

NETIFC - WINC_CmdNETIFC, 159
NETIFRX (AEC), 163
NETIFTX - WINC_CmdNETIFTX, 162
ntohl, 468
ntohs, 469
NVMC - WINC_CmdNVMC, 353
NVMCHK (AEC), 360
NVMCHK - WINC_CmdNVMCHK, 357
NVMER (AEC), 359
NVMER - WINC_CmdNVMER, 355
NVMERR (AEC), 361
NVMRD - WINC_CmdNVMRD, 358
NVMWR - WINC_CmdNVMWR, 356

O

OTA (AEC), 176
OTAACT - WINC_CmdOTAACT, 174
OTAC - WINC_CmdOTAC, 170
OTADL - WINC_CmdOTADL, 172
OTAERR (AEC), 177
OTAINV - WINC_CmdOTAINV, 175
OTAVFY - WINC_CmdOTAVFY, 173

P

PING (AEC), 183
PING - WINC_CmdPING, 182
PINGERR (AEC), 184
poll, 453
PPS (AEC), 378
PPS - WINC_CmdPPS, 376
PPSC - WINC_CmdPPSC, 374
PPSPAUSE - WINC_CmdPPSPAUSE, 377

R

recv, 454
recvfrom, 455
RST - WINC_CmdRST, 186

S

send, 456
sendto, 457
setsockopt, 458
shutdown, 460
SI - WINC_CmdSI, 321
SIHEAP (AEC), 325
SIHERR (AEC), 326
SIMERR (AEC), 323
SIMSTAT (AEC), 322
SISTACK (AEC), 327
SIWPKTS (AEC), 324
SNTPC - WINC_CmdSNTPC, 188
SNTPERR (AEC), 190
SOCKBL - WINC_CmdSOCKBL, 200
SOCKBM - WINC_CmdSOCKBM, 202
SOCKBR - WINC_CmdSOCKBR, 201
SOCKC - WINC_CmdSOCKC, 214
SOCKCL (AEC), 220
SOCKCL - WINC_CmdSOCKCL, 211
SOCKERR (AEC), 222
socket, 461
SOCKIND (AEC), 217
SOCKLST - WINC_CmdSOCKLST, 212
SOCKO - WINC_CmdSOCKO, 198
SOCKRD - WINC_CmdSOCKRD, 207
SOCKRDBUF - WINC_CmdSOCKRDBUF, 209
SOCKRXT (AEC), 218
SOCKRXU (AEC), 219
SOCKTLS (AEC), 221
SOCKTLS - WINC_CmdSOCKTLS, 203
SOCKWR - WINC_CmdSOCKWR, 204
SOCKWRTO - WINC_CmdSOCKWRTO, 205
SYSLOG (AEC), 383
SYSLOGC - WINC_CmdSYSLOGC, 381

T

TCP Client, 474
TCP Server, 478
TIME (AEC), 235
TIME - WINC_CmdTIME, 231
TIMERFC - WINC_CmdTIMERFC, 234
TIMEUTCSEC - WINC_CmdTIMEUTCSEC, 233
TLSC - WINC_CmdTLSC, 248
TLSCDA - WINC_CmdTLSCDA, 252
TLSCSC - WINC_CmdTLSCSC, 254
TLSERR (AEC), 256

U

UDP Client, 482
UDP Server, 486

W

WAP - WINC_CmdWAP, 262
WAPAIP (AEC), 263
WAPC - WINC_CmdWAPC, 259
WAPERR (AEC), 266
WAPSC (AEC), 264
WAPSD (AEC), 265
WBCN - WINC_CmdWBCN, 420
WBCNC - WINC_CmdWBCNC, 417
WBCNDONE (AEC), 422
WBCNERR (AEC), 423
WIFIC - WINC_CmdWIFIC, 344
WIFIVSIERX (AEC), 348
WINC_CmdReadParamElem, 33
WINC_CmdReqDiscard, 27
WINC_CmdReqInit, 25
WINC_CmdReqSend, 28
WINC_DEV_AEC_RSP_CB, 47
WINC_DEV_BUS_STATE_TYPE, 46
WINC_DEV_CMD_RSP_CB, 47
WINC_DEV_CMDREQ_EVENT_TYPE, 44
WINC_DEV_ERROR_TYPE, 46
WINC_DEV_EVENT_CHECK_FP, 47
WINC_DEV_EVENT_COMPLETE_ARGS, 45
WINC_DEV_EVENT_RSP_ELEMS, 44
WINC_DEV_EVENT_SRC_CMD, 45
WINC_DEV_EVENT_STATUS_ARGS, 45
WINC_DEV_EXTENDED_ERROR, 46
WINC_DEV_INIT, 44
WINC_DEV_RX_INTERCEPT_CB, 47
WINC_DevAECCallbackDeregister, 30
WINC_DevAECCallbackRegister, 29
WINC_DevBusStateGet, 34
WINC_DevBusStateSet, 35
WINC_DevDeinit, 18
WINC_DevErrorToString, 38
WINC_DevEventPending, 24
WINC_DevGetLastError, 36
WINC_DevGetLastErrorEx, 37
WINC_DevHandleEvent, 22
WINC_DevInit, 17
WINC_DevInterceptCallbackRegister, 31
WINC_DevReset, 19
WINC_DevSetDebugPrintf, 20
WINC_DevTransmitCmdReq, 26
WINC_DevUnpackElements, 32
WINC_DevUpdateEvent, 23
WINC_SDIO_SEND_RECEIVE_FP, 48
WINC_SDIO_STATE_TYPE, 48
WINC_SDIODeviceInit, 20
WINC_SockDeinit, 445
WINC_SOCKET_EVENT, 441
WINC_SOCKET_EVENT_CALLBACK, 444
WINC_SOCKET_INIT_TYPE, 443
WINC_SOCKET_STATUS, 442
WINC_SockInit, 446
WINC_TablesAECIterate, 39
WINC_TablesAECLookupByID, 41
WINC_TablesAECLookupByString, 42
WINC_TablesCommandIterate, 39
WINC_TablesCommandLookupByID, 41
WINC_TablesCommandLookupByString, 42
WINC_TablesModuleIDListGet, 43
WINC_TablesModuleStatusIterate, 40
WINC_TablesStatusIterate, 39
WINC_TablesStatusLookupByID, 41
WPROV - WINC_CmdWPROV, 331
WPROVAT (AEC), 332
WPROVC - WINC_CmdWPROVC, 329
WPROVDT (AEC), 333
WSCN - WINC_CmdWSCN, 272
WSCNC - WINC_CmdWSCNC, 268
WSCNDONE (AEC), 279
WSCNIND (AEC), 278
WSCNP - WINC_CmdWSCNP, 277
WSCNPC - WINC_CmdWSCNPC, 273
WSCNPDONE (AEC), 280
WSCNPERR (AEC), 281
WSTA - WINC_CmdWSTA, 303
WSTAAIP (AEC), 305
WSTAC - WINC_CmdWSTAC, 299
WSTAERR (AEC), 307
WSTALD (AEC), 306
WSTALU (AEC), 308
WSTAROAM (AEC), 309

 ©2025-2026